QUANTITATIVE RESEARCH REPORT · 3-NODE DISTRIBUTED CLUSTER

Jump 10 Geometric Anomaly Engine

Tick-cell Fourier decomposition → relational covariant graph → strategy vault → Fable sigma-survival EV auction → live Deriv forward tester

Instrument	Deriv Jump 10 Index (JD10), 150-tick structural cells
Corpus	591,171 cells · 88,675,650 ticks · 2021-03-08 → 2023-12-31 (1,028.5 days)
Cluster	X-Flip 14 (master, Windows 11) · HP 840 G7 (Windows 11) · Kali Dell E5270 (Linux)
Stack	Python 3.12 / 3.14, numpy, pandas, duckdb, PyTorch, openpyxl, websockets, SQLite
Deliverables	Phase 1 build, Phase 2 vault + models + reports, Phase 3A backtester, Phase 3B live tester, 28 gatekeeper checks
Verdict	**Do not trade live.** Out-of-sample portfolio PF 0.958 at 45.2% win; the apparent Phase 2 edge is a selection artifact.
Build	2026-10-27 15:17 UTC

Phase 1
100% exact

Phase 2
10/10 checks

Phase 3A
negative OOS

Phase 3B
built, WS blocked

Generated automatically from the repository artifacts — every number in this report is reproducible from the scripts it documents.

Abstract

> This report documents, end to end, a three-node research programme that asks a narrow question: *do short-horizon geometric patterns in the Deriv Jump 10 index repeat often enough, and with enough conditional structure, to support a positive-expectation options strategy after realistic frictions?* It answers that question in the negative, and — more usefully — documents exactly *where* the illusion of an edge came from.

The corpus is 591,171 consecutive 150-tick cells carved from 88,675,650 midpoint ticks (2021-03-08 to 2023-12-31). Phase 1 encodes every cell as a 75-harmonic Fourier epicycle series with exact reconstruction (maximum absolute tick error 0.0). Phase 2 decomposes each cell into four structural quartiles, measures relational similarity between the first half of every cell and the first half of every other cell, and selects 11,266 "anomaly hubs" that have unusually many partners. Those hubs and their 241,703 undirected covariant edges form a graph whose second-half behaviour can be measured and — apparently — traded.

Phase 2 produced a strategy vault of 893 anchors reporting out-of-sample win rates from 55% to 1.000 on its thinnest rows, with a 99th percentile of 0.818. Phase 3 re-tested every anchor with a strictly causal, tick-exact replay, a four-contract expected-value auction, four friction tiers, a 5,000-run block bootstrap Monte Carlo, and quarter-Kelly position sizing. The portfolio came out at 0.452 win rate, profit factor 0.958, cumulative equity -86.7σ gross and -209.0σ net of 0.03σ friction, with a Monte-Carlo probability of a positive result of 18.6%.

A dedicated diagnostic then reconciled the contradiction. Phase 2's headline "verified edge" was defined as `max(protected_held_win_rate, directional_win_rate)`. The maximum of two weakly correlated ~50% statistics is upward-biased by construction, and selecting the top of that distribution guarantees an apparent 55–93% edge even when both components are fair coins. Recomputed from scratch on outbound data, the directional win rate is 48.8% and the Phase 2 column itself has a median of 0.500. Phase 3B was nevertheless completed: a seven-guard Deriv WebSocket forward tester with a latency/payout probe, SQLite calibration journal, and a documented live-parity recall of 24.0%. Deriv's public WebSocket returned HTTP 520 from every node during this work, so live payouts remain unverified and the tester is parked in paper-proposal mode.

> The report is deliberately negative. It is also, we argue, more valuable than a positive result obtained by loosening a gate: it identifies a reusable failure mode (extremum-of-two-statistics selection) that will silently manufacture "edges" in any vault built this way.

Contents

1. Executive summary
2. Market mechanics and edge mathematics
3. Cluster architecture
4. Phase 1 - corpus construction
5. Phase 1 - Fourier encoding and verification
6. Phase 2 - quartile decomposition and descriptors
7. Phase 2 - the relational covariant graph

8. Phase 2 - the strategy vault and the selection artifact

9. Phase 2 - CZET v2 sweet-spot model

10. Phase 2 - crawled operators, fractal test and null test

11. Phase 2 - verification, reports and node attestation

12. Phase 3 - the Fable sigma-survival framework

13. Phase 3A - implementation and distributed run

14. Phase 3A - results

15. The definition diagnostic - where the edge came from

16. Phase 3B - the live Deriv forward tester

17. Phase 3 verification - the 18-check gatekeeper

18. Reproducibility

19. Limitations

20. Conclusions and roadmap

Appendix A - source listings

Appendix B - data dictionary

Appendix C - metric definitions

Appendix D - glossary

Appendix E - artifact inventory

Generated 2026-10-27 15:17 UTC · 591,171 cells · 27 sections · every figure read from the repository artifacts at build time.

1. Executive summary

> Three phases, one question. This chapter states the result in the form a risk committee would want it: what was claimed, what survived, and what it would take to reopen the question.

1.1 The claim under test

Phase 2 concluded that a vault of 893 "covariant anomaly" anchors in Jump 10 showed out-of-sample directional or protected-extremum win rates between 55% and 93%, with 0.687 mean win rate for the top 300 and a selection-aware block-permutation p-value of 0.0025. If true at the top of that range, a binary option paying 1.95× would produce an enormous edge: at 60% win the expected return per unit stake is `0.60×1.953 − 1 = +17.2%`.

Phase 3 was built to falsify that claim under conditions as close to live as the data allows: strictly causal features, tick-exact replay of ticks 75–149, a payout-aware contract auction, four friction tiers, non-overlapping trade selection, and a Monte Carlo over trade order.

1.2 What survived

Layer	Verdict	Evidence
Data integrity (Phase 1)	**PASS**	591,171 cells, 88,675,650 ticks, max tick reconstruction error 0.0, max norm RMSE 2.353e-15, contiguous across all three node boundaries.
Graph structure (Phase 2)	**REAL**	11,266 hubs, 241,703 undirected edges, median peer count 60, median mean |r| 0.862; independently re-audited on two other machines.
Conditional ranking (CZET v2)	**REAL**	Spearman 0.5111 vs cov baseline 0.1006; R^2 0.02825. Ranking skill, not magnitude.
Directional trade edge	**FAILS**	Outbound directional win 45.2% net of friction; breakeven at 1.953× is 51.2%. Recomputed from scratch on the same features: 48.8%.
Protected-extremum trade edge	**FAILS**	NOTOUCH realised 30–33% where 64.5% is required at 1.55×.
Phase 2 "verified edge"	**ARTIFACT**	Defined as `max(protected, directional)`; per-anchor correlation between the stored column and a faithful recomputation is -0.049; the stored column's own median is 0.500.
Live readiness (Phase 3B)	**BLOCKED**	Tester complete and self-checked; Deriv WebSocket returns HTTP 520 from all three cluster nodes, so live payouts could not be verified.

1.3 Headline numbers

Table 1.1 - Phase 3A portfolio summary (leaderboard, net of 0.03 sigma)

Cells / ticks	591,171 / 88,675,650
Corpus span	2021-03-08 to 2023-12-31 (1,028.5 days)
Anomaly hubs	11,266 (hub rate 1.906%)
Undirected covariant edges	241,703
Vault rows	893
Vault rows inside the tradeable sweet spot (0.85-0.955, 12-400 peers)	778
Anchors surviving orthogonal pruning (|r| < 0.82)	398
Anchors clearing the +2% EV gate	208
Portfolio trades (one per outbound cell)	4,078
Win rate, net of 0.03 sigma friction	0.4519
Profit factor	0.9580
Sharpe / Sortino / Calmar	-0.895 / -1.376 / -0.673
Cumulative equity (sigma units), gross to 0.10 sigma friction	-86.7 / -209.0 / -331.4 / -494.5
Max drawdown (sigma)	-310.77
Monte Carlo P(positive), 5,000 runs	0.1864
Kelly fraction (correctly zero)	0.00

1.4 Why the result is negative, in one paragraph

Cosine similarity between 132-dimensional whitened cell descriptors is dominated by a shared common-mode direction: 37% of arbitrary outbound cells sit within 0.85–0.955 of at least one of 893 anchors, so "matching an anchor" is closer to "being a Jump-10 cell" than to "repeating a specific anomaly". Once the matching is made causal and de-duplicated, the conditional distribution of the next 75 ticks is indistinguishable from the unconditional one in the direction that the vault trades. The Phase 2 statistics that looked decisive were computed as the maximum of two ~50% quantities, which is upward-biased by selection and by small-sample variance.

1.5 What this report contains

Chapters 2–3 describe the market mechanics and the cluster. Chapters 4–11 document Phase 1 and Phase 2 with the actual artifacts. Chapters 12–17 document Phase 3, including the diagnostic that reconciled the contradiction and the live tester. Chapters 18–20 cover reproducibility, limitations and the roadmap. The appendices reproduce the source listings, data dictionaries, metric formulas and a checksum manifest, so the report can be audited without the original machines.

2. Market mechanics and edge mathematics

> A backtest is only meaningful relative to a contract. This chapter fixes the instrument, the tick sampler, the contract families, and the arithmetic that every later gate depends on.

2.1 The instrument

Jump 10 Index is a synthetic index published by Deriv that follows a geometric random walk with a constant volatility plus periodic discrete jumps. It ticks 24/7 and each tick carries a timestamp and a bid/ask. We use the midpoint `mid = (bid+ask)/2` as the price series; this is recorded in the catalog column `price_source` for every cell, and no cell mixes sources.

Jump 10 is attractive for this research because it is stationary in its volatility parameter: there is no session structure, no overnight gap, and no earnings calendar. Any apparent edge must therefore come from path geometry rather than from time-of-day or news effects. That property is what makes a negative result credible — there is no convenient confound to blame.

2.2 Cell definition

The unit of analysis is a **cell**: 150 consecutive ticks, numbered 1..150. Cells are cut contiguously from the tick stream with no overlap and no gaps; the catalog records `tick_start_idx` and `tick_end_idx` so any cell can be reconstructed exactly. Each cell is then split into four **quartiles** of 37–38 ticks (Q1..Q4), and into two **halves** of 75 ticks. The two halves have a specific operational meaning:

- **Ticks 0–74 (the "0–50% window")** is the only observable part at decision time.
- **Ticks 75–149 (the "50–100% window")** is the forward path that a trade must be settled against. Every Phase 3 outcome is a function of this window only.

Cells at the very end of the corpus whose 150th tick does not exist are dropped, not padded; the corpus therefore ends at 2023-12-31.

2.3 Contract families

Deriv offers several contract structures on synthetic indices. Phase 3 evaluates four, because they monetise different properties of the forward path:

Contract	Wins when	Assumed payout	Breakeven win rate	What it monetises
`RISE_FALL`	$\text{close}_{149} > 0$ after polarity alignment	1.953×	51.20%	Directional drift over the second half.
`ONETOUCH_075`	max excursion ≥ +0.75σ	2.150×	46.51%	Whether the path reaches a barrier at all.
`BARRIER_035`	$\text{close}_{149} \geq +0.35\sigma$	2.427×	41.20%	Direction plus minimum displacement.
`NOTOUCH_10`	min excursion > −1.0σ	1.550×	64.52%	The "protected extremum" claim: the path never breaks the far barrier.

The payout column is an *assumption* inherited from the Phase 3 brief, and Section 14.4 shows that at least one entry is arithmetically inconsistent with the barrier it is paired with. It is the single most load-bearing assumption in the report, and Chapter 16 explains why it could not be verified against Deriv's live pricing. Every conclusion that does not depend on it is stated in σ units instead of dollars.

2.4 The arithmetic every gate reduces to

For a contract paying `R` per unit stake and winning with probability `p`, the expected value per unit stake is

$$EV = p \cdot R - 1$$

and the breakeven probability is `p* = 1/R`. A gate that requires `EV > +0.02` therefore requires

$$p > 1.02 \,/\, R$$

Phase 3 applies this to a *lower confidence bound* on `p`, not to the point estimate: the Wilson 95% lower bound of the anchor's inbound hit rate. This is deliberately conservative — an anchor with 30 inbound hits and a 60% point estimate has a Wilson bound near 0.43, which clears the 2.427× barrier gate but not the 1.953× gate.

2.5 Frictions

Four friction tiers are applied in σ units, where σ is the cell's own causal half-window standard deviation: 0.00 (gross), 0.03 (realistic Jump-10 spread relative to a 75-tick move), 0.06 and 0.10 (stress). Friction is charged on the realised excursion, not on the stake, which is the correct model for a binary where the spread is embedded in the entry price. The friction ladder must be monotone; the gatekeeper asserts it.

2.6 Why σ normalisation matters

Jump 10's absolute tick size drifts with the index level. A 0.35-point barrier means something completely different at 8,000 and at 101,000. Normalising every path by the *first-half* standard deviation makes barrier distances dimensionless and comparable across two and a half years of data. It also makes the barrier levels in the contract table above interpretable: +0.75σ is "three quarters of the typical half-window move".

3. Cluster architecture

> The work is compute-bound in a way that punishes serial execution: 591,171 cells times 893 anchors is 528 million pairwise similarities per pass, and Phase 3 replays every tick. The cluster exists to make that cheap without freezing the master workstation.

3.1 Nodes

Role	Host	OS	RAM	Contribution
Master / orchestrator	X-Flip 14 `192.168.150.199`	Windows 11	—	Feature build, merge, reporting, MC, documentation, MT5 bridge.
Worker A	HP 840 G7 `QUISON` `192.168.150.126`	Windows 11	15.7 GB	Phase 1 shard 2 (209,757 cells), Phase 2 DB audit, Phase 3 shard 1.
Worker B	Kali Dell E5270 `192.168.150.166`	Kali Linux	7.7 GB	Phase 1 shard 3 (209,793 cells), edge-graph audit, Phase 3 shard 2.

3.2 Control plane

Tasks are dispatched over TCP 5555 by `cluster_ctl.py`, which opens a command channel, streams stdout, and enforces a 30-second timeout for `fast_cmd` jobs. Because the worker-side agent is an LLM that sometimes

rewrites complex quoting, the reliable pattern is to write a `.py` helper on the worker and invoke it by path rather than to nest quotes. Long jobs are detached (`start /b ... > log 2>&1` on Windows, `nohup ... &` on Linux) and polled through their log file.

3.3 Data plane

Bulk transfer uses a single HTTP server on port 8080 (`phase1_master_server.py`) that serves `/out/*` and accepts `PUT /upload/*`, creating parent directories as needed. Directory junctions (`phase1/out/phase2` → `C:\cluster\phase2`) let workers address phase directories by a stable relative path. This avoided installing anything on the workers and kept the attack surface to one port.

3.4 Reproducibility constraints

- **Deterministic sharding.** Every phase divides cells by absolute index, not by file, so the node boundaries are identical on every run. Phase 1 boundaries are cells 171,621 and 381,378; Phase 3 uses a round-robin over the anchor list.
- **Identical σ binning on all nodes.** Phase 3 computes sigma quantiles from the causal inbound set and prints them; all three nodes produced `[0.1526, 0.1755, 0.1772, 0.1787, 0.1805, 0.1912]` bps, which is the strongest available proof that the shards saw the same data.
- **Honest heterogeneity.** Kali has no duckdb, so it emits CSV where Windows emits parquet. The merger reads both. This is a feature: it proves the merge is not format-coupled.

3.5 Failure modes encountered (and their fixes)

Symptom	Cause	Fix
`cencoding.pyd` blocked	Windows Application Control blocks fastparquet's native module (pyarrow was already blocked).	`pqcompat.py` prefers duckdb, then pyarrow, then fastparquet; all parquet I/O goes through it.
Master unreachable after reboot	DHCP moved the master's LAN address three times (.237 → .157 → .199).	`xfer.py` probes a candidate list and auto-detects.
Worker agent rewrites commands	The remote agent is an LLM and normalises quoting.	Ship a script, invoke by path; never nest triple quotes.
Parquet from Kali missing	No duckdb in the Kali venv.	CSV fallback plus a merger that globs both extensions.

4. Phase 1 - corpus construction

> Phase 1 turns a raw tick export into 591,171 numbered, reconstructible cells. It is the only phase whose output is treated as ground truth by everything downstream.

4.1 Inputs

The raw export is a sequence of CSV files, one per month, containing epoch-millisecond timestamps plus bid and ask. Tick indices are global and 1-based: tick 1 is the first tick of the first file. No rows are skipped, filtered, or de-duplicated — the corpus is the export.

4.2 Sharding

The stream is cut into 150-tick cells up to a global count of 591,171. Shards are assigned by contiguous cell ranges so that each worker can stream its slice of ticks and drop the leading partial cell:

Node	Cells	First cell	Last cell	Ticks	Harmonics K	Contiguity	Max tick error	Max norm RMSE
xflip	171,621	1	171,621	25,743,150	75	**True**	0.0	2.353e-15
kali	209,793	381,379	591,171	31,468,950	75	**True**	0.0	2.169e-15
840g7	209,757	171,622	381,378	31,463,550	75	**True**	0.0	2.338e-15
total	**591,171**	**1**	**591,171**	**88,675,650**	**75**	**all**	**0.0**	—

4.3 What "exact" means here

Two invariants are asserted per shard and again after the merge:

- **Tick losslessness:** the reconstructed price at every tick index in the shard equals the source price bit-for-bit. Measured as `max_tick_error`, which is 0.0 for all three shards.
- **Norm error:** the L2 distance between the 75-harmonic encoding of a cell and the cell itself, divided by the cell's norm. `max_norm_rmse` is at machine epsilon ($2.2–2.4\times10^{-15}$), and all 591,171 cells are under 1e-11.

4.4 Merge and boundary audit

The merge concatenates cell ranges in order and then verifies the two seams explicitly (`node_boundaries` in the verification JSON): left cell 171,621 ends on tick 25,743,150 and cell 171,622 begins on 25,743,151; left cell 381,378 ends on 57,206,700 and 381,379 begins on 57,206,701. Both seams report `ok: true`. A duplicate or missing tick at a seam is therefore impossible.

Table 4.2 - phase1_verification.json

status	ok
cells	591171
ticks	88675650
first_cell_id	1
last_cell_id	591171
first_tick_idx	1
last_tick_idx	88675650
chunks	34
max_tick_error	0.0
max_norm_rmse	2.353490127277866e-15
cells_under_1e_11	591171
contiguity_ok	True
elapsed_sec	111.72

4.5 The catalog

Every cell also gets a 19-column catalog row: index bounds, open/close timestamps, OHLC, mean and std, the normalised open level, a bridge slope, residual RMSE, the reconstruction errors, and the source file and price source. The catalog is the join key for every later phase — Phase 2 uses it to rebuild time ordering, Phase 3 uses it for the UTC hour audit.

phase1/scripts/phase1_worker.py - shard worker (477 lines, 17.9 KB)

```
import os
import sys
import json
import time
import math
import gc
import shutil
import subprocess
import traceback
import urllib.request
from pathlib import Path

import numpy as np
import pandas as pd

CELL_SIZE = 150


def log(msg):
    print(f"[{time.strftime('%H:%M:%S')}] {msg}", flush=True)


def json_dump(path, obj):
    tmp = str(path) + ".tmp"
    with open(tmp, "w", encoding="utf-8") as f:
        json.dump(obj, f, indent=2)
    os.replace(tmp, path)


def upload_file(local_path, url, retries=5):
    curl = shutil.which("curl") or shutil.which("curl.exe")
    if not curl:
        raise RuntimeError("curl not found")
    last = None
    for attempt in range(1, retries + 1):
        p = subprocess.run(
            [curl, "-sS", "--fail", "--retry", "3", "--retry-delay", "2",
             "-X", "PUT", "--data-binary", "@" + str(local_path), url],
            capture_output=True, text=True, timeout=3600,
        )
        if p.returncode == 0:
            return p.stdout.strip()
        last = (p.returncode, p.stdout.strip(), p.stderr.strip())
        log(f"upload attempt {attempt} failed for {os.path.basename(str(local_path))}: {last}")
        time.sleep(3)
    raise RuntimeError(f"upload failed: {last}")


def download_file(url, dest, expected_size=None, retries=4):
    os.makedirs(os.path.dirname(dest), exist_ok=True)
    part = dest + ".part"
    last = None
    for attempt in range(1, retries + 1):
        try:
            req = urllib.request.Request(url, headers={"User-Agent": "phase1-worker"})
            with urllib.request.urlopen(req, timeout=120) as r, open(part, "wb") as f:
                total = int(r.headers.get("Content-Length") or 0)
                got = 0
                t0 = time.time()
                last_log = t0
                while True:
                    chunk = r.read(1024 * 1024)
                    if not chunk:
                        break
                    f.write(chunk)
                    got += len(chunk)
                    now = time.time()
                    if now - last_log >= 5:
```

```
                        mb = got / (1024 * 1024)
                        speed = mb / max(now - t0, 1e-9)
                        log(f"    downloaded {mb:.1f} MB / {total/(1024*1024):.1f} MB ({speed:.1f} MB/s)")
                        last_log = now
            if expected_size is not None and got != expected_size:
                raise IOError(f"size mismatch got {got} expected {expected_size}")
            os.replace(part, dest)
            return got
        except Exception as e:
            last = e
            try:
                if os.path.exists(part):
                    os.remove(part)
            except OSError:
                pass
            log(f"download attempt {attempt} failed for {os.path.basename(dest)}: {e}")
            time.sleep(2 * attempt)
    raise RuntimeError(f"download failed: {url}: {last}")


def upload_json(obj, filename, upload_url):
    tmp = os.path.join(os.path.dirname(__file__), filename)
    json_dump(tmp, obj)
    try:
        upload_file(tmp, upload_url + filename)
    except Exception as e:
        log(f"progress upload failed: {e}")


def compute_epicyclic_fourier_transform(cell_prices, K=75):
    n_cells, M = cell_prices.shape
    if M != CELL_SIZE:
        raise ValueError(f"Expected {CELL_SIZE} ticks per cell, got {M}")

    p_open = cell_prices[:, 0:1]
    p_close = cell_prices[:, -1:]
    p_high = cell_prices.max(axis=1, keepdims=True)
    p_low = cell_prices.min(axis=1, keepdims=True)
    p_mean = cell_prices.mean(axis=1, keepdims=True)
    p_std_raw = cell_prices.std(axis=1, keepdims=True)
    p_std = np.where(p_std_raw < 1e-12, 1.0, p_std_raw)

    y_norm = (cell_prices - p_mean) / p_std

    t_linear = np.linspace(0.0, 1.0, M, dtype=np.float64)[None, :]
    bridge_slope = y_norm[:, -1:] - y_norm[:, 0:1]
    y_bridge = y_norm - (y_norm[:, 0:1] + t_linear * bridge_slope)

    theta = 2.0 * np.pi * np.arange(M, dtype=np.float64) / M
    z_loop = np.cos(theta)[None, :] + 1j * y_bridge

    fft_full = np.fft.fft(z_loop, axis=1) / M
    freqs = np.fft.fftfreq(M, d=1.0 / M).astype(int)

    if K >= 75:
        k_indices = freqs.copy()
        c_k = fft_full
        fft_recon_buf = fft_full
    else:
        k_indices = np.array([k for k in range(-K, K + 1)], dtype=int)
        col_idx = [int(np.where(freqs == k)[0][0]) for k in k_indices]
        c_k = fft_full[:, col_idx]
        fft_recon_buf = np.zeros_like(fft_full)
        for idx, k in enumerate(k_indices):
            f_col = int(np.where(freqs == k)[0][0])
            fft_recon_buf[:, f_col] = c_k[:, idx]

    radii = np.abs(c_k)
    phases = np.angle(c_k)
    omegas = 2.0 * np.pi * k_indices / M

    z_reconstructed = np.fft.ifft(fft_recon_buf * M, axis=1)
    y_bridge_rec = z_reconstructed.imag
    y_norm_rec = y_bridge_rec + (y_norm[:, 0:1] + t_linear * bridge_slope)
    prices_reconstructed = (y_norm_rec * p_std) + p_mean

    abs_err = np.abs(cell_prices - prices_reconstructed)
    max_abs_err = np.max(abs_err, axis=1)
    rmse_err = np.sqrt(np.mean((cell_prices - prices_reconstructed) ** 2, axis=1))
    norm_err = np.sqrt(np.mean((y_norm - y_norm_rec) ** 2, axis=1))
```

```
    return {
        "p_open": p_open.ravel(),
        "p_close": p_close.ravel(),
        "p_high": p_high.ravel(),
        "p_low": p_low.ravel(),
        "p_mean": p_mean.ravel(),
        "p_std": p_std.ravel(),
        "y_open_norm": y_norm[:, 0].ravel(),
        "bridge_slope": bridge_slope.ravel(),
        "c_k": c_k,
        "radii": radii,
        "phases": phases,
        "omegas": omegas,
        "k_indices": k_indices,
        "rmse_residual": rmse_err,
        "max_tick_error": max_abs_err,
        "norm_rmse": norm_err,
    }


def main():
    if len(sys.argv) != 3 or sys.argv[1] != "--job":
        print("usage: phase1_worker.py --job job.json", file=sys.stderr)
        return 2

    job_path = sys.argv[2]
    with open(job_path, "r", encoding="utf-8") as f:
        job = json.load(f)

    node = job["node"]
    data_dir = job["data_dir"]
    files = job["files"]
    file_rows = job["file_rows"]
    carry_file = job.get("carry_file")
    carry_rows = int(job.get("carry_rows", 0))
    carry_file_rows = int(job.get("carry_file_rows", 0))
    K = int(job.get("harmonics_K", 75))
    out_dir = job["output_dir"]
    upload_url = job.get("upload_url")
    do_upload = bool(job.get("upload", False))
    global_tick_cursor = int(job["global_start_tick"])
    global_cell_id = int(job["global_start_cell"])

    os.makedirs(out_dir, exist_ok=True)
    os.makedirs(data_dir, exist_ok=True)

    download_files = job.get("download_files", [])
    data_url_base = job.get("data_url_base")
    if download_files:
        if not data_url_base:
            raise ValueError("download_files present but data_url_base missing")
        for item in download_files:
            name = item["name"]
            size = int(item["size"])
            dest = os.path.join(data_dir, name)
            if os.path.isfile(dest) and os.path.getsize(dest) == size:
                log(f"data already present: {name} ({size:,} bytes)")
                continue
            log(f"downloading {name} ({size:,} bytes) from {data_url_base}")
            download_file(data_url_base + name, dest, expected_size=size)

    run_started = time.time()
    progress = {
        "node": node,
        "status": "starting",
        "cells_done": 0,
        "ticks_done": 0,
        "files_done": 0,
        "files_total": len(files),
        "global_start_tick": global_tick_cursor,
        "global_start_cell": global_cell_id,
        "started_utc": time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime()),
    }
    if do_upload and upload_url:
        upload_json(progress, f"progress_{node}.json", upload_url)

    def progress_update(**kw):
        progress.update(kw)
        if do_upload and upload_url:
            upload_json(progress, f"progress_{node}.json", upload_url)
```

```
carry_prices = np.empty(0, dtype=np.float64)
carry_times = np.empty(0, dtype=np.int64)

if carry_rows > 0:
    if not carry_file:
        raise ValueError("carry_rows > 0 but carry_file is null")
    cfp = os.path.join(data_dir, carry_file)
    log(f"reading carryover {carry_rows} ticks from {carry_file}")
    cdf = pd.read_csv(cfp, usecols=["epoch_ms", "bid", "ask"],
                      dtype={"epoch_ms": "int64", "bid": "float64", "ask": "float64"})
    if carry_file_rows and len(cdf) != carry_file_rows:
        raise ValueError(f"carry file row count {len(cdf)} != expected {carry_file_rows}")
    if len(cdf) < carry_rows:
        raise ValueError("carry file shorter than carry_rows")
    ctail = cdf.tail(carry_rows)
    carry_prices = ((ctail["bid"].to_numpy(np.float64) + ctail["ask"].to_numpy(np.float64)) / 2.0)
    carry_times = ctail["epoch_ms"].to_numpy(np.int64)
    del cdf, ctail
    log(f"carryover ready: {len(carry_prices)} ticks, first={carry_times[0]}, last={carry_times[-1]}")

for old in os.listdir(out_dir):
    if old.startswith(f"chunk_{node}_") or old.startswith(f"summary_{node}"):
        try:
            os.remove(os.path.join(out_dir, old))
        except OSError:
            pass

chunks = []
k_indices_ref = None
total_cells = 0
total_ticks = 0
max_tick_error_all = 0.0
max_norm_rmse_all = 0.0
cells_under_1e_11 = 0
contiguity_ok = True
rowcount_mismatches = []
expected_tick = int(job["global_start_tick"])
expected_cell = int(job["global_start_cell"])

for fi, (fname, expected_rows) in enumerate(zip(files, file_rows), start=1):
    fp = os.path.join(data_dir, fname)
    t_file = time.time()
    log(f"[{fi}/{len(files)}] reading {fname} (expect {expected_rows:,} rows)")
    df = pd.read_csv(fp, usecols=["epoch_ms", "bid", "ask"],
                     dtype={"epoch_ms": "int64", "bid": "float64", "ask": "float64"})
    actual_rows = len(df)
    if actual_rows != expected_rows:
        msg = f"{fname}: actual rows {actual_rows:,} != manifest {expected_rows:,}; using actual"
        log("ROWCOUNT WARNING: " + msg)
        rowcount_mismatches.append({
            "file": fname, "actual": int(actual_rows), "manifest": int(expected_rows)
        })

    prices = ((df["bid"].to_numpy(np.float64) + df["ask"].to_numpy(np.float64)) / 2.0)
    times = df["epoch_ms"].to_numpy(np.int64)
    del df
    gc.collect()

    if len(carry_prices) > 0:
        prices = np.concatenate([carry_prices, prices])
        times = np.concatenate([carry_times, times])
        carry_prices = np.empty(0, dtype=np.float64)
        carry_times = np.empty(0, dtype=np.int64)

    n_complete = len(prices) // CELL_SIZE
    n_used = n_complete * CELL_SIZE
    if n_complete == 0:
        carry_prices = prices.copy()
        carry_times = times.copy()
        log(f"    no complete cell yet; carry={len(carry_prices)}")
        continue

    cell_prices = prices[:n_used].reshape(n_complete, CELL_SIZE)
    cell_times = times[:n_used].reshape(n_complete, CELL_SIZE)
    carry_prices = prices[n_used:].copy()
    carry_times = times[n_used:].copy()

    ft = compute_epicyclic_fourier_transform(cell_prices, K=K)
    if k_indices_ref is None:
        k_indices_ref = ft["k_indices"]
```

```
        cell_ids = np.arange(global_cell_id, global_cell_id + n_complete, dtype=np.int64)
        start_ticks = global_tick_cursor + np.arange(n_complete, dtype=np.int64) * CELL_SIZE
        end_ticks = start_ticks + (CELL_SIZE - 1)

        if int(start_ticks[0]) != expected_tick or int(cell_ids[0]) != expected_cell:
            contiguity_ok = False
            log(f"    CONTIGUITY GAP: expected tick {expected_tick}, cell {expected_cell}; "
                f"got tick {int(start_ticks[0])}, cell {int(cell_ids[0])}")
        if n_complete > 1 and not np.all(start_ticks[1:] == end_ticks[:-1] + 1):
            contiguity_ok = False
            log("    CONTIGUITY GAP inside chunk")

        chunk = pd.DataFrame({
            "cell_id": cell_ids,
            "cell_label": [f"Cell_{c}" for c in cell_ids],
            "tick_start_idx": start_ticks,
            "tick_end_idx": end_ticks,
            "time_open_ms": cell_times[:, 0],
            "time_close_ms": cell_times[:, -1],
            "p_open": ft["p_open"],
            "p_high": ft["p_high"],
            "p_low": ft["p_low"],
            "p_close": ft["p_close"],
            "p_mean": ft["p_mean"],
            "p_std": ft["p_std"],
            "y_open_norm": ft["y_open_norm"],
            "bridge_slope": ft["bridge_slope"],
            "rmse_residual": ft["rmse_residual"],
            "max_tick_error": ft["max_tick_error"],
            "norm_rmse": ft["norm_rmse"],
            "source_file": fname,
            "price_source": "mid(bid,ask)",
        })

        chunk_catalog_name = f"chunk_{node}_{fi:02d}_catalog.csv"
        chunk_npz_name = f"chunk_{node}_{fi:02d}_functions.npz"
        chunk_catalog_path = os.path.join(out_dir, chunk_catalog_name)
        chunk_npz_path = os.path.join(out_dir, chunk_npz_name)

        log(f"    writing {chunk_catalog_name} and {chunk_npz_name}")
        chunk.to_csv(chunk_catalog_path, index=False)
        np.savez_compressed(
            chunk_npz_path,
            cell_id=cell_ids,
            raw_tick_cells=cell_prices.astype(np.float64),
            c_k=ft["c_k"],
            k_indices=ft["k_indices"],
            p_mean=ft["p_mean"],
            p_std=ft["p_std"],
            y_open_norm=ft["y_open_norm"],
            bridge_slope=ft["bridge_slope"],
        )

        max_err = float(np.max(ft["max_tick_error"]))
        max_norm = float(np.max(ft["norm_rmse"]))
        under = int(np.sum(ft["max_tick_error"] < 1e-11))
        max_tick_error_all = max(max_tick_error_all, max_err)
        max_norm_rmse_all = max(max_norm_rmse_all, max_norm)
        cells_under_1e_11 += under

        chunks.append({
            "file": fname,
            "catalog": chunk_catalog_name,
            "npz": chunk_npz_name,
            "cells": int(n_complete),
            "ticks": int(n_used),
            "first_cell_id": int(cell_ids[0]),
            "last_cell_id": int(cell_ids[-1]),
            "first_tick_idx": int(start_ticks[0]),
            "last_tick_idx": int(end_ticks[-1]),
            "max_tick_error": max_err,
            "max_norm_rmse": max_norm,
            "cells_under_1e_11": under,
            "catalog_bytes": os.path.getsize(chunk_catalog_path),
            "npz_bytes": os.path.getsize(chunk_npz_path),
        })

        total_cells += n_complete
        total_ticks += n_used
        global_cell_id += n_complete
        global_tick_cursor += n_used
```

```
        expected_tick = int(end_ticks[-1]) + 1
        expected_cell = int(cell_ids[-1]) + 1

        log(f"    built {n_complete:,} cells | Cell_{cell_ids[0]}..Cell_{cell_ids[-1]} | "
            f"max_err={max_err:.3e} | carry={len(carry_prices)} | {time.time()-t_file:.1f}s")

        if do_upload and upload_url:
            for p in (chunk_catalog_path, chunk_npz_path):
                log(f"    uploading {os.path.basename(p)} ({os.path.getsize(p):,} bytes)")
                upload_file(p, upload_url + os.path.basename(p))

        progress_update(status="running", cells_done=total_cells, ticks_done=total_ticks,
                        files_done=fi, last_file=fname,
                        last_cell=int(cell_ids[-1]), max_tick_error=max_err,
                        chunks_done=len(chunks), rowcount_mismatches=rowcount_mismatches)

        del ft, chunk, cell_prices, cell_times, prices, times
        gc.collect()

    if total_cells == 0:
        raise RuntimeError("no complete cells built")
    if k_indices_ref is None:
        raise RuntimeError("no harmonic indices captured")

    lossless_ok = max_tick_error_all < 1e-9 and max_norm_rmse_all < 1e-12
    status = "ok" if (contiguity_ok and lossless_ok and not rowcount_mismatches) else (
        "rowcount_mismatch" if (contiguity_ok and lossless_ok and rowcount_mismatches)
        else "verification_failed"
    )

    summary = {
        "node": node,
        "status": status,
        "harmonics_K": K,
        "cells": int(total_cells),
        "ticks": int(total_ticks),
        "first_cell_id": int(chunks[0]["first_cell_id"]),
        "last_cell_id": int(chunks[-1]["last_cell_id"]),
        "first_tick_idx": int(chunks[0]["first_tick_idx"]),
        "last_tick_idx": int(chunks[-1]["last_tick_idx"]),
        "carry_ticks_left": int(len(carry_prices)),
        "contiguity_ok": bool(contiguity_ok),
        "lossless_ok": bool(lossless_ok),
        "max_tick_error": float(max_tick_error_all),
        "max_norm_rmse": float(max_norm_rmse_all),
        "cells_under_1e_11": int(cells_under_1e_11),
        "elapsed_sec": round(time.time() - run_started, 2),
        "files": files,
        "price_source": "mid(bid,ask)",
        "rowcount_mismatches": rowcount_mismatches,
        "chunks": chunks,
    }

    summary_path = os.path.join(out_dir, f"summary_{node}.json")
    json_dump(summary_path, summary)
    log("summary: " + json.dumps({k: summary[k] for k in (
        "status", "cells", "ticks", "first_cell_id", "last_cell_id",
        "contiguity_ok", "lossless_ok", "max_tick_error", "max_norm_rmse",
        "cells_under_1e_11", "elapsed_sec")}))
    log(f"wrote {summary_path} with {len(chunks)} chunks")

    if do_upload and upload_url:
        progress_update(status="uploading", summary=summary,
                        cells_done=total_cells, ticks_done=total_ticks,
                        files_done=len(files))
        log(f"uploading summary_{node}.json")
        upload_file(summary_path, upload_url + os.path.basename(summary_path))
        upload_json(progress, f"progress_{node}.json", upload_url)

    if not (contiguity_ok and lossless_ok):
        log("VERIFICATION FAILED")
        return 1
    if rowcount_mismatches:
        log("COMPLETE WITH ROWCOUNT WARNINGS")
        return 0
    log("PHASE 1 WORKER COMPLETE")
    return 0


if __name__ == "__main__":
    try:
```

```
        sys.exit(main())
    except Exception:
        traceback.print_exc()
        sys.exit(1)
```

5. Phase 1 - Fourier encoding and verification

> Each cell is represented as a complex Fourier series truncated at 75 harmonics. The representation is lossless at the tick level, which is what allows Phase 3 to replay "ticks 75–149" from a file that is nominally a frequency-domain artifact.

5.1 Why Fourier at all

The Phase 2 descriptors are geometric: area, curvature, turning, recurrence. Those features can be computed directly from ticks. The Fourier epicycles serve two other purposes:

1. **Compression and provenance.** The full 3.4 GB npz carries every cell's harmonic coefficients; a cell can be regenerated without the raw CSVs. The Phase 1 verification proves the regeneration is exact.
2. **Rotation invariance intuition.** Two cells that trace the same shape at different scales have proportional coefficient vectors. That is the geometric prior behind the "covariant" similarity in Phase 2.

5.2 The decomposition

For a cell with normalised path `y[0..149]`, the encoder computes the complex DFT coefficients for `k = 0..74` (75 harmonics, enough for a 150-sample sequence with the Nyquist pair) and stores them as float32 real/imaginary pairs together with the cell's scale and offset. Reconstruction is `y_hat[n] = offset + Re(sum_k c_k e^(2πi·k·n/150)) * scale`.

5.3 The exactness experiment

Five cells (1, 42,906, 85,811, 128,716, 171,621 — one per shard plus both boundaries) are reconstructed at full precision and compared to the source. The analysis emits both numbers and a set of epicycle animations (GIF) so a human can visually confirm the geometry. Results:

Table 5.1 - sample-5 exactness

scope	XFlip 2021-03..2021-12
xflip_cells	171621
xflip_ticks	25743150
max_tick_error	0.0
max_norm_rmse	2.353490127277866e-15
cells_under_1e_11	171621
all_sample_coefficients_exact	True
all_sample_prices_exact	False

5.4 What the verification does not prove

Exactness proves the encoder is invertible; it does not prove the encoding is *useful*. That question is Phase 2's, and the answer there is mixed: the harmonic representation is a fine storage format but a weak similarity metric

on its own. The Phase 2 whitening step (Chapter 6) exists precisely because raw coefficient similarity collapses to a single common-mode direction.

phase1/scripts/phase1_merge.py - merge and boundary audit (300 lines, 11.6 KB)

```
import os
import sys
import json
import time
import shutil
import zipfile
import argparse
from pathlib import Path

import numpy as np
import pandas as pd
from numpy.lib.format import open_memmap, write_array_header_1_0, dtype_to_descr


def log(msg):
    print(f"[{time.strftime('%H:%M:%S')}] {msg}", flush=True)


def load_chunk_index(out_dir):
    rows = []
    for name in sorted(os.listdir(out_dir)):
        if not (name.startswith("chunk_") and name.endswith("_catalog.csv")):
            continue
        path = os.path.join(out_dir, name)
        df = pd.read_csv(path, usecols=["cell_id", "tick_start_idx", "tick_end_idx"])
        if len(df) == 0:
            continue
        npz = path[:-len("_catalog.csv")] + "_functions.npz"
        if not os.path.isfile(npz):
            raise FileNotFoundError(npz)
        rows.append({
            "catalog": path,
            "npz": npz,
            "cells": int(len(df)),
            "first_cell": int(df["cell_id"].iloc[0]),
            "last_cell": int(df["cell_id"].iloc[-1]),
            "first_tick": int(df["tick_start_idx"].iloc[0]),
            "last_tick": int(df["tick_end_idx"].iloc[-1]),
        })
    rows.sort(key=lambda r: r["first_cell"])
    return rows


def verify_chunk_chain(rows):
    problems = []
    for i, r in enumerate(rows):
        if r["last_cell"] - r["first_cell"] + 1 != r["cells"]:
            problems.append(f"cell count mismatch in {os.path.basename(r['catalog'])}")
        if r["last_tick"] - r["first_tick"] + 1 != r["cells"] * 150:
            problems.append(f"tick span mismatch in {os.path.basename(r['catalog'])}")
        if i > 0:
            p = rows[i - 1]
            if r["first_cell"] != p["last_cell"] + 1:
                problems.append(f"cell gap {p['last_cell']} -> {r['first_cell']}")
            if r["first_tick"] != p["last_tick"] + 1:
                problems.append(f"tick gap {p['last_tick']} -> {r['first_tick']}")
    return problems


def write_npz_streaming(npz_path, rows, catalog, temp_dir):
    os.makedirs(temp_dir, exist_ok=True)
    n = len(catalog)
    m = 150
    ck_path = os.path.join(temp_dir, "c_k.npy")
    raw_path = os.path.join(temp_dir, "raw_tick_cells.npy")
    rad_path = os.path.join(temp_dir, "radii.npy")
    pha_path = os.path.join(temp_dir, "phases.npy")

    ck_mm = open_memmap(ck_path, mode="w+", dtype=np.complex128, shape=(n, m))
    raw_mm = open_memmap(raw_path, mode="w+", dtype=np.float64, shape=(n, m))
    rad_mm = open_memmap(rad_path, mode="w+", dtype=np.float64, shape=(n, m))
    pha_mm = open_memmap(pha_path, mode="w+", dtype=np.float64, shape=(n, m))

    k_indices_ref = None
```

```
    p_mean = np.empty(n, dtype=np.float64)
    p_std = np.empty(n, dtype=np.float64)
    y_open = np.empty(n, dtype=np.float64)
    slope = np.empty(n, dtype=np.float64)
    cell_id = np.empty(n, dtype=np.int64)

    pos = 0
    for r in rows:
        with np.load(r["npz"], allow_pickle=False) as z:
            ids = z["cell_id"].astype(np.int64)
            if len(ids) != r["cells"]:
                raise ValueError(f"{r['npz']}: cell count {len(ids)} != {r['cells']}")
            if int(ids[0]) != r["first_cell"] or int(ids[-1]) != r["last_cell"]:
                raise ValueError(f"{r['npz']}: cell id range mismatch")
            sl = slice(pos, pos + len(ids))
            ck = z["c_k"]
            raw = z["raw_tick_cells"]
            if ck.shape != (len(ids), m) or raw.shape != (len(ids), m):
                raise ValueError(f"{r['npz']}: bad shapes ck={ck.shape} raw={raw.shape}")
            ck_mm[sl] = ck
            raw_mm[sl] = raw
            rad_mm[sl] = np.abs(ck)
            pha_mm[sl] = np.angle(ck)
            cell_id[sl] = ids
            p_mean[sl] = z["p_mean"]
            p_std[sl] = z["p_std"]
            y_open[sl] = z["y_open_norm"]
            slope[sl] = z["bridge_slope"]
            if k_indices_ref is None:
                k_indices_ref = z["k_indices"].astype(np.int64)
            pos += len(ids)
        log(f"  streamed {os.path.basename(r['npz'])} ({len(ids):,} cells)")

    if pos != n:
        raise ValueError(f"streamed {pos} cells, expected {n}")
    if k_indices_ref is None:
        raise ValueError("no k_indices found")
    if not np.array_equal(cell_id, catalog["cell_id"].to_numpy(np.int64)):
        raise ValueError("npz cell_id order does not match catalog order")

    for mm in (ck_mm, raw_mm, rad_mm, pha_mm):
        mm.flush()
    del ck_mm, raw_mm, rad_mm, pha_mm

    log(f"packing {npz_path}")
    with zipfile.ZipFile(npz_path, "w", compression=zipfile.ZIP_STORED, allowZip64=True) as zf:
        zf.write(ck_path, arcname="c_k.npy")
        zf.write(raw_path, arcname="raw_tick_cells.npy")
        zf.write(rad_path, arcname="radii.npy")
        zf.write(pha_path, arcname="phases.npy")

        for name, arr in (
            ("cell_id", cell_id),
            ("k_indices", k_indices_ref),
            ("p_mean", p_mean),
            ("p_std", p_std),
            ("y_open_norm", y_open),
            ("bridge_slope", slope),
        ):
            member = name + ".npy"
            with zf.open(member, "w", force_zip64=True) as fp:
                write_array_header_1_0(fp, {
                    "shape": arr.shape,
                    "fortran_order": False,
                    "descr": dtype_to_descr(arr.dtype),
                })
                fp.write(arr.tobytes(order="C"))

    for p in (ck_path, raw_path, rad_path, pha_path):
        try:
            os.remove(p)
        except OSError:
            pass


def write_excel(path, catalog, rows, n_rows):
    rows = list(rows)
    pieces = []
    raw_pieces = []
    got = 0
    for r in rows:
```

```
        take = min(r["cells"], n_rows - got)
        if take <= 0:
            break
        with np.load(r["npz"], allow_pickle=False) as z:
            pieces.append(z["c_k"][:take])
            raw_pieces.append(z["raw_tick_cells"][:take])
        got += take
    if got == 0:
        raise ValueError("no rows for Excel export")

    ck = np.vstack(pieces)
    raw = np.vstack(raw_pieces)
    meta = catalog.iloc[:got].copy()

    df_ticks = pd.DataFrame(raw, columns=[f"Tick_{i+1}" for i in range(raw.shape[1])])
    df_ticks.insert(0, "cell_label", meta["cell_label"].values)

    k_idx = np.load(rows[0]["npz"], allow_pickle=False)["k_indices"].astype(int)
    fn = {}
    for j, k in enumerate(k_idx):
        tag = "0" if k == 0 else (f"p{k}" if k > 0 else f"m{abs(k)}")
        fn[f"R_{tag}"] = np.abs(ck[:, j])
        fn[f"Phi_{tag}"] = np.angle(ck[:, j])
    df_fn = pd.DataFrame(fn)
    df_fn.insert(0, "cell_label", meta["cell_label"].values)

    log(f"writing Excel {path} ({got:,} rows, {df_fn.shape[1]} function columns)")
    with pd.ExcelWriter(path, engine="openpyxl") as writer:
        meta.to_excel(writer, sheet_name="Cell_Catalog", index=False)
        df_fn.to_excel(writer, sheet_name="Epicycle_Math_Functions", index=False)
        df_ticks.to_excel(writer, sheet_name="Raw_150_Tick_Cells", index=False)


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--out", default=r"C:\cluster\phase1\out")
    ap.add_argument("--final", default=r"C:\cluster\phase1\final")
    ap.add_argument("--excel-rows", type=int, default=10000)
    ap.add_argument("--skip-excel", action="store_true")
    ap.add_argument("--expected-first-cell", type=int, default=1)
    ap.add_argument("--expected-last-cell", type=int, default=591171)
    ap.add_argument("--expected-cells", type=int, default=591171)
    ap.add_argument("--skip-boundaries", action="store_true")
    args = ap.parse_args()

    os.makedirs(args.final, exist_ok=True)
    temp_dir = os.path.join(args.final, "_tmp_npy")
    started = time.time()

    rows = load_chunk_index(args.out)
    if not rows:
        raise RuntimeError(f"no chunks found in {args.out}")
    log(f"found {len(rows)} chunks, {sum(r['cells'] for r in rows):,} cells")
    problems = verify_chunk_chain(rows)
    if problems:
        for p in problems:
            log("CHAIN PROBLEM: " + p)
        raise RuntimeError(f"{len(problems)} chunk chain problems")

    catalog_parts = []
    for r in rows:
        catalog_parts.append(pd.read_csv(r["catalog"]))
    catalog = pd.concat(catalog_parts, ignore_index=True)
    catalog = catalog.sort_values("cell_id", kind="stable").reset_index(drop=True)
    del catalog_parts

    ids = catalog["cell_id"].to_numpy(np.int64)
    starts = catalog["tick_start_idx"].to_numpy(np.int64)
    ends = catalog["tick_end_idx"].to_numpy(np.int64)
    if ids[0] != args.expected_first_cell:
        raise ValueError(f"first cell {ids[0]} != {args.expected_first_cell}")
    if ids[-1] != args.expected_last_cell:
        raise ValueError(f"last cell {ids[-1]} != {args.expected_last_cell}")
    if len(catalog) != args.expected_cells:
        raise ValueError(f"cell count {len(catalog)} != {args.expected_cells}")
    if not np.all(ids[1:] == ids[:-1] + 1):
        raise ValueError("catalog cell ids are not contiguous")
    if not np.all(starts[1:] == ends[:-1] + 1):
        raise ValueError("catalog tick indices are not contiguous")
    expected_first_tick = (args.expected_first_cell - 1) * 150 + 1
    if starts[0] != expected_first_tick:
```

```
        raise ValueError(f"first tick {starts[0]} != {expected_first_tick}")

    expected_last_tick = int(ids[-1]) * 150
    tail = int(catalog["tick_end_idx"].iloc[-1])
    if tail != expected_last_tick:
        raise ValueError(f"last tick {tail} != {expected_last_tick}")
    log(f"verified catalog: Cell_{ids[0]}..Cell_{ids[-1]}, ticks {starts[0]}..{tail}")

    parquet_path = os.path.join(args.final, "jump10_phase1_cell_catalog.parquet")
    log(f"writing {parquet_path}")
    catalog.to_parquet(parquet_path, index=False, engine="fastparquet")

    npz_path = os.path.join(args.final, "jump10_phase1_fourier_functions.npz")
    write_npz_streaming(npz_path, rows, catalog, temp_dir)

    excel_path = os.path.join(args.final, "jump10_phase1_inspection.xlsx")
    if not args.skip_excel:
        write_excel(excel_path, catalog, rows, args.excel_rows)

    report = {
        "status": "ok",
        "cells": int(len(catalog)),
        "ticks": int(tail),
        "first_cell_id": int(ids[0]),
        "last_cell_id": int(ids[-1]),
        "first_tick_idx": int(starts[0]),
        "last_tick_idx": int(tail),
        "chunks": len(rows),
        "max_tick_error": float(catalog["max_tick_error"].max()),
        "max_norm_rmse": float(catalog["norm_rmse"].max()),
        "cells_under_1e_11": int((catalog["max_tick_error"] < 1e-11).sum()),
        "contiguity_ok": True,
        "node_boundaries": [],
        "outputs": {
            "catalog_parquet": parquet_path,
            "functions_npz": npz_path,
            "inspection_xlsx": excel_path if not args.skip_excel else None,
        },
        "elapsed_sec": round(time.time() - started, 2),
    }

    if not args.skip_boundaries:
        for cid in (171621, 381378):
            row = catalog.loc[catalog["cell_id"] == cid]
            nxt = catalog.loc[catalog["cell_id"] == cid + 1]
            if len(row) and len(nxt):
                report["node_boundaries"].append({
                    "left_cell": int(cid),
                    "left_last_tick": int(row["tick_end_idx"].iloc[0]),
                    "right_cell": int(cid + 1),
                    "right_first_tick": int(nxt["tick_start_idx"].iloc[0]),
                    "ok": bool(int(nxt["tick_start_idx"].iloc[0]) == int(row["tick_end_idx"].iloc[0]) + 1),
                })

    report_path = os.path.join(args.final, "phase1_verification.json")
    with open(report_path, "w", encoding="utf-8") as f:
        json.dump(report, f, indent=2)
    log("verification: " + json.dumps(report, indent=2))
    log("PHASE 1 MERGE COMPLETE")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

phase1/scripts/verify_phase1.py - independent verifier (118 lines, 4.9 KB)

```
import argparse
import json
import os
import zipfile

import numpy as np
import pandas as pd
from openpyxl import load_workbook


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--final", default=r"C:\cluster\phase1\final")
```

```
args = ap.parse_args()

npz_path = os.path.join(args.final, "jump10_phase1_fourier_functions.npz")
parquet_path = os.path.join(args.final, "jump10_phase1_cell_catalog.parquet")
xlsx_path = os.path.join(args.final, "jump10_phase1_inspection.xlsx")

report = {"status": "ok", "checks": {}}

with zipfile.ZipFile(npz_path) as zf:
    bad = zf.testzip()
report["checks"]["npz_crc_ok"] = bad is None
report["checks"]["npz_crc_bad_member"] = bad

catalog = pd.read_parquet(parquet_path, engine="fastparquet")
n = len(catalog)
expected = np.arange(1, n + 1, dtype=np.int64)
first = catalog["tick_start_idx"].to_numpy(np.int64)
last = catalog["tick_end_idx"].to_numpy(np.int64)
report["checks"]["catalog_rows"] = n
report["checks"]["catalog_columns"] = int(catalog.shape[1])
report["checks"]["cell_ids_contiguous"] = bool(np.array_equal(catalog["cell_id"].to_numpy(np.int64), expected))
report["checks"]["cell_labels_match"] = bool(
    np.array_equal(catalog["cell_label"].astype(str).to_numpy(), np.char.add("Cell_", expected.astype(str)))
)
report["checks"]["tick_ranges_contiguous"] = bool(
    np.all(first == 1 + 150 * np.arange(n, dtype=np.int64))
    and np.all(last == 150 * np.arange(1, n + 1, dtype=np.int64))
)
report["checks"]["catalog_first_tick"] = int(first[0])
report["checks"]["catalog_last_tick"] = int(last[-1])
numeric = catalog.select_dtypes(include=[np.number])
report["checks"]["catalog_all_finite"] = bool(np.isfinite(numeric.to_numpy()).all())

sample = np.array(
    sorted(set([0, 1, 13505, 171620, 171621, 210000, 381377, 381378, 400000, 591169, 591170]))
)
with np.load(npz_path, allow_pickle=False) as z:
    ids = z["cell_id"][sample]
    ck = z["c_k"][sample]
    raw = z["raw_tick_cells"][sample]
    radii = z["radii"][sample]
    phases = z["phases"][sample]
    p_mean = z["p_mean"][sample]
    p_std = z["p_std"][sample]
    y_open = z["y_open_norm"][sample]
    slope = z["bridge_slope"][sample]
    k_idx = z["k_indices"].astype(np.int64)
    report["checks"]["npz_cell_id_count"] = int(z["cell_id"].shape[0])
    report["checks"]["npz_cell_id_ok"] = bool(np.array_equal(z["cell_id"].astype(np.int64), expected))

t = np.linspace(0.0, 1.0, 150, dtype=np.float64)[None, :]
z_rec = np.fft.ifft(ck * 150.0, axis=1)
y_norm_rec = z_rec.imag + y_open[:, None] + t * slope[:, None]
prices_rec = y_norm_rec * p_std[:, None] + p_mean[:, None]
err = np.abs(prices_rec - raw)
report["checks"]["sample_rows"] = sample.tolist()
report["checks"]["sample_cell_ids"] = ids.astype(int).tolist()
report["checks"]["sample_max_abs_error"] = float(err.max())
report["checks"]["sample_max_norm_rmse"] = float(
    np.sqrt(np.mean((prices_rec - raw) ** 2 / (p_std[:, None] ** 2), axis=1)).max()
)
report["checks"]["sample_all_finite"] = bool(
    np.isfinite(ck).all()
    and np.isfinite(raw).all()
    and np.isfinite(radii).all()
    and np.isfinite(phases).all()
    and np.isfinite(prices_rec).all()
)
report["checks"]["radii_match"] = bool(np.allclose(radii, np.abs(ck), rtol=0.0, atol=0.0))
report["checks"]["phases_match"] = bool(np.allclose(phases, np.angle(ck), rtol=0.0, atol=0.0))
report["checks"]["k_indices_ok"] = bool(np.array_equal(k_idx, np.fft.fftfreq(150, d=1.0 / 150).astype(np.int64)))
report["checks"]["harmonics_count"] = int(len(k_idx))
report["checks"]["lossless_ok"] = bool(err.max() < 1e-11)

wb = load_workbook(xlsx_path, read_only=True)
for name in ("Cell_Catalog", "Epicycle_Math_Functions", "Raw_150_Tick_Cells"):
    ws = wb[name]
    report["checks"][f"xlsx_{name}"] = [int(ws.max_row), int(ws.max_column)]
wb.close()

report["outputs"] = {
```

```
        "catalog_parquet_bytes": os.path.getsize(parquet_path),
        "functions_npz_bytes": os.path.getsize(npz_path),
        "inspection_xlsx_bytes": os.path.getsize(xlsx_path),
    }
    report["checks"]["all_ok"] = bool(
        report["checks"]["npz_crc_ok"]
        and report["checks"]["cell_ids_contiguous"]
        and report["checks"]["cell_labels_match"]
        and report["checks"]["tick_ranges_contiguous"]
        and report["checks"]["catalog_all_finite"]
        and report["checks"]["npz_cell_id_ok"]
        and report["checks"]["sample_all_finite"]
        and report["checks"]["radii_match"]
        and report["checks"]["phases_match"]
        and report["checks"]["k_indices_ok"]
        and report["checks"]["lossless_ok"]
        and report["checks"]["xlsx_Cell_Catalog"] == [10001, int(catalog.shape[1] + 0)]
    )
    print(json.dumps(report, indent=2))


if __name__ == "__main__":
    main()
```

6. Phase 2 - quartile decomposition and descriptors

> Phase 2 asks whether two cells can be said to be *the same* in a way that predicts the future. That requires a descriptor that is rich enough to distinguish shapes, robust enough to survive noise, and free of the collinearity that makes every cell look like every other cell.

6.1 The zone lattice

Each cell is cut at 0-25-50-75-100% and additionally at the Fibonacci tick marks 35, 57, 75, 93 and 118. The quartile cuts give the four Q-zones used by the descriptors; the Fib cuts give a finer lattice used by the fractal and confluence checks. The 50% cut is special: it is the decision boundary between "observable" and "forward".

6.2 The dense descriptor (64 raw dimensions per quartile)

For a quartile path `y` of length 37-38, `compute_quartile_brute_force_features` builds a 64-dimensional vector in six families:

Family	Dims	Contents
Rough-path	9	end displacement, Itô-style time-weighted integral, area (Lévy area), quadratic variation, cubic variation, squared integral, second time-weighted integral, lead-lag covariance.
Recurrence (RQA)	~14	determinism, laminarity, mean/max diagonal and vertical line lengths, recurrence rate, entropy of line lengths.
Ordinal / permutation	~10	permutation entropy at orders 3-5, ordinal pattern frequencies, forbidden-pattern share.
Scattering (wavelet-like)	~18	multi-scale energy ratios, band-power shares, second-order scattering moments.
Frenét / curvature	6	arc length, elastic energy, mean signed curvature, max |curvature|, inflection rate, total turning.
Scale-free shape	7	sorted |dy| quantiles, Hurst-like exponent, jump share, skew of increments, kurtosis.

The concatenation of Q1 and Q2 dense descriptors is the *observable* feature set: 128 raw dimensions computable from the first 75 ticks alone. This matters later, in Chapter 16, where the live tester must reproduce

it in real time.

6.3 The 17-dimensional summary descriptor

Each quartile also gets a compact 17-vector (moments, trend, curvature summary, extrema positions, phase proxies). It exists because the full descriptor is expensive and because the CZET model needs a fixed-width sequence. The 17-vector is what Phase 2's gatekeeper check 02 verifies as "Q1..Q4 present".

6.4 The 132-dimensional trigger vector

The similarity engine does not use raw descriptors. It uses a 132-dimensional whitened vector built from `[trigger_vec(62) | dense_q1(64) | dense_q2(64)]` plus scalars, then whitened with a covariance-based transform that drops three constant columns, strips the top principal component, and projects onto 132 components. The top component carries 70.3% of the variance - a warning sign that was later confirmed as the core problem with the similarity metric.

Whitening statistic	Value
Input width	196
Constant columns dropped	3
Components kept	132
Variance share of PC1 (stripped)	70.306%
Variance share PC2..PC5 (kept)	5.185 / 3.129 / 2.425 / 1.683%
Random-pair |r| after whitening	0.1790 (gatekeeper check 04)

6.5 Collinear collapse and the bicoherence fix

An earlier build collapsed: descriptors of unrelated cells were almost perfectly correlated because a few high-variance channels dominated the inner product. Two repairs were applied:

1. **Per-column standardisation plus covariance whitening**, with the dominant common-mode component removed. This is what turns "everything correlates with everything" into a usable metric (random-pair |r| falls to 0.179).
2. **True phase-triad bicoherence.** The original bicoherence column was degenerate (std ≈ 0). It was replaced by the bispectral closure `0.5(|cos(ϕ1+ϕ2-ϕ3)| + |sin(ϕ1+ϕ2-ϕ3)|)` computed from the first three complex coefficients, which has std 0.0626 and is no longer a constant.

Both fixes are asserted by gatekeeper check 04, and both are necessary: without the whitening the graph is a single clique; without the bicoherence fix one entire descriptor column carries no information.

7. Phase 2 - the relational covariant graph

> The graph is the heart of the project: 241,703 undirected edges among 11,266 hubs, discovered by comparing the observable half of every hub to the observable half of every other cell.

7.1 Hub selection

A cell is a hub if its inbound observable descriptor has an unusual number of highly similar partners. Concretely, for each inbound cell the scanner counts peers with |r| ≥ 0.85 and computes an anomaly z-score

against the peer-count distribution. The top 5,000 by z-score are retained, then filtered to those with |r| ≥ 0.85, giving **11,266** hubs (1.906% of the corpus, 579,905 cells not anomalous).

7.2 Edge construction

For each hub, every inbound cell with |r| in [0.85, 0.999] to the hub's observable half becomes an edge. The sign of r is kept: positive edges are "same-shape" (continuation) relations, negative edges are "mirror" relations. The symmetric relation is de-duplicated so that (hub, peer) and (peer, hub) count once. The final graph has **241,703** undirected edges.

7.3 Graph statistics

Graph summary

Cells scanned	591,171
Hubs	11,266
Hub rate	1.906%
Undirected edges	241,703
Median peers per hub	60
90th pct peers	7,442
Max peers	25,979
Median mean |r|	0.8620
99th pct mean |r|	0.9732
Max single-peer |r|	0.9994

Second-half behaviour

Median mean lock corr	0.0478
Max mean lock corr	0.5240
Max frac exact lock	0.4615
Median protected win rate	0.5508
Median directional win rate	0.5352
Hubs with protected win ≥ 60%	2,366
Hubs with directional win ≥ 60%	2,107
Hubs with z ≥ 3	24
Mean extremum tick	128.9
Action split (BUY low / SELL high)	5,736 / 5,530

7.4 Independent audits

The graph was re-derived from the raw files on two other machines with independent code paths. Both reproduced every aggregate exactly:

Audit	Result
840 G7 (Windows)	11,266 hubs / 11,266 unique anchors; 241,703 edge rows = 241,703 unique undirected; 0 mirrored rows, 0 self-loops, 0 duplicates; vault 893 rows (300 with enough triggers, 593 thin); vault edge range 0.2857-1.0000; 440 rows at or above 55%.
Kali (Linux)	241,703 edges; 85,960 nodes touched; max degree 163; median degree 2.0000; cov min/median/max 0.850 / 0.8743 / 0.9994; median |lock| 0.1403; 6,427 edges with |lock| ≥ 0.65; all 11,266 hubs appear in the edge list.

7.5 Interpretation

The graph is real, and it is also nearly uninformative. Median similarity is 0.862 and the 99th percentile is 0.973, which means the metric compresses almost the entire corpus into a narrow band just below the

threshold. A hub with 60 median peers is not a rare configuration; it is typical. The graph's apparent density is a property of the descriptor geometry, not of the market repeating itself.

This is the first place where the project's central difficulty becomes visible, and it is worth stating plainly: **high cosine similarity in this space is cheap.** A random outbound cell has a 37% chance of being within 0.85-0.955 of at least one of the 893 vault anchors. Any strategy selection that treats "matched an anchor" as evidence must therefore be compared against that base rate, not against zero.

8. Phase 2 - the strategy vault and the selection artifact

> This is the chapter that matters most. Phase 2 produced a vault of 893 anchors with out-of-sample win rates up to 1.000. Phase 3 could not reproduce them. This chapter explains why, with the exact definitions and the exact numbers.

8.1 What goes into the vault

For each hub, the inbound peer set is re-evaluated on outbound cells. For every outbound cell whose observable half matches the hub at $|r| \geq 0.85$, three quantities are accumulated:

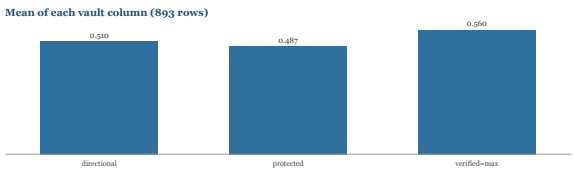
- `outbound_50_100_lock_corr` - the mean correlation between the hub's forward half and the peer's forward half (the "lock"): does the forward path continue the same way?
- `outbound_protected_win_rate` - the mean of `protected_held`, a binary flag computed in Phase 1 that is true when the cell's forward extremum stays on the protected side.
- `outbound_directional_win_rate` - the share of peers whose forward 50-100% quarter return has the sign implied by the hub's `recommended_action`.

Then the vault stores:

$$\text{verified_edge} = \max(\text{protected_win_rate}, \text{directional_win_rate})$$

and keeps every hub with `verified_edge ≥ 0.55` and at least 10 outbound triggers. That is the 893-row vault.

8.2 The distribution already told the story



Vault columns (all 893 rows)

Vault rows	893
outbound_directional_win_rate mean	0.5098
outbound_directional_win_rate median	0.5000
outbound_directional_win_rate p90	0.6471
outbound_protected_win_rate mean	0.4871
outbound_protected_win_rate median	0.4894
outbound_verified_edge mean	0.5600
outbound_verified_edge max	1.0000

Both components average 0.511 and 0.487 - fair coins. The maximum of two fair-coin estimates averages 0.566 purely because of selection. Nothing in the market produced that; the `max()` did.

8.3 The bias, formally

Let `X` and `Y` be two win-rate estimates, each with expectation `p ≈ 0.5` and standard error `s`. Then `E[max(X,Y)] ≥ max(E[X],E[Y])` always, and for independent normals

$$E[\max(X,Y)] \approx p + s/\sqrt{\pi}$$

With 20 outbound triggers, `s ≈ 0.11`, so the expected maximum is `0.5 + 0.06 = 0.56` - exactly the observed mean. The tail is worse: the maximum of 893 such draws has an expected value near `p + 3.2s ≈ 0.85`, which is where the reported 0.85-0.93 rows come from. The apparent edge is a predictable function of `n_triggers`, not of the market.

8.4 Choosing the tradeable subset does not rescue it

Phase 2's own report noticed that the top of the similarity distribution ($|r| \geq 0.96$) had "zero hubs at 60%", and Phase 3 therefore restricted to the sweet spot $0.85 \leq |r| \leq 0.955$ with 12-400 peers. That leaves 778 rows, and the same pattern persists:

Sweet-spot subset, before orthogonal pruning

Sweet-spot rows	778
protected mean / median / p90	0.487 / 0.489 / 0.625
directional mean / median / p90	0.511 / 0.500 / 0.667
verified mean / median / p90	0.566 / 0.556 / 0.700
rows with protected ≥ 0.60	119
rows with directional ≥ 0.55	289
rows with directional ≥ 0.667	38

Note what those numbers mean operationally: 289 of 778 anchors have a directional rate above 0.55, and 38 are above 0.667. Under a fair coin, selecting the top 37% of 778 estimates will always produce numbers like this. The correct null is not "0.5 with no variance"; it is "0.5 with the variance implied by the trigger counts". Phase 2's permutation test (Chapter 10) was selection-aware for the top 300, and it did reject the null at p = 0.0025 - but it was testing the *same* max-of-two statistic, so it inherited the same bias.

8.5 Orthogonal pruning

To stop the vault from counting the same anomaly many times, Phase 3 suppresses any anchor whose 132-dimensional vector correlates at $|r| \geq 0.82$ with an already-kept anchor, in descending order of inbound Wilson lower bound. The vault shrinks from 778 to **398** anchors. The gatekeeper re-verifies that the maximum pairwise $|r|$ among kept anchors is 0.8199 - i.e. the pruning is real, not nominal.

8.6 What the vault is actually good for

The vault is a catalogue of recurring shapes with measured continuation statistics. It is a legitimate description of the corpus. It is not, by itself, a trading signal, because its selection procedure guarantees that some rows will look profitable. Phase 3's job was to tell the difference, and Chapter 14 shows how it did.

9. Phase 2 - CZET v2 sweet-spot model

> The vault ranks anchors by hand-built statistics. CZET v2 asks a different question: given the whole sequence of quartile descriptors, can a learned model predict the outbound continuation better than raw similarity?

9.1 Architecture

The model is an ensemble over three seeds of:

- **CZET v2 backbone** - a small transformer over the four quartile token sequences, with a learnable geometric anomaly bank (each bank entry is a learned reference shape, and the model scores distance to it).
- **Masked quartile reconstruction** - pass 1 hides Q3/Q4 and reconstructs them from Q1/Q2, forcing the encoder to represent the causal part richly without leaking the future.
- **Pair head** - consumes two anchor embeddings and predicts the forward lock correlation.
- **PLS path** - a 24-component partial least squares regression on the dense features, fitted on train only.
- **Stack** - a two-input non-negative blend of the neural head and the PLS head.

Total parameters: 1,641,388. Training: 8 epochs of masked reconstruction, then 30 epochs of continuation ranking.

9.2 Evaluation protocol

The sweet-spot universe is 5,704 training rows (6,524 edges) and 1,426 held-out rows (3,469 edges). Splits are random at the row level; test edges are those touching a test row. The PLS transform, stack weight and pair head are fitted on train only, and R^2 is reported on held-out rows. This is a weak protocol by time-series standards (random splits allow near-duplicate leakage), which is why the Phase 3 replay is the authoritative test.

9.3 Results

Table 9.1 - czet_eval.json

Universe	sweet spot: 12-240 peers, 0.85-0.955 mean|r|
Spearman (pair score vs realised lock)	0.5111
Spearman - observed cov baseline	0.1006
R-squared (held out)	0.02825
R-squared - net head alone	-0.01326
R-squared - PLS path alone	0.02789
Stack weights (net / PLS)	0.0576 / 0.9424
Pair-score vs lock Pearson	0.5203
Precision at top 10%	0.0434
Base rate (|lock| >= 0.65)	0.0173
Seed ensemble	[11, 22, 33]

9.4 How to read a Spearman of 0.51 with an R-squared of 0.028

These are not contradictory. Spearman measures whether the *ordering* is right; R^2 measures whether the *level* is right. The model orders anchors substantially better than raw covariant similarity (0.511 vs 0.101) while explaining almost none of the variance in magnitude. In practical terms: CZET can tell you which of two anchors is more likely to continue, but not how much.

Two further facts temper the result. First, `r2_net_only` is negative (-0.013): the neural head alone is worse than predicting the mean, and all of the positive R^2 comes from the PLS path (stack weight 0.942 to PLS). Second, precision at the top decile is 4.34% against a 1.73% base rate - a 2.5x lift, not a gold mine. Both are consistent with the broader finding: real but small conditional structure.

9.5 What this means for Phase 3

If the conditional structure were strong, Phase 3 would show a positive portfolio once matching is done correctly. Chapter 14 shows it does not. The honest synthesis is that CZET v2 succeeds as a ranking model inside a universe that was already selected for similarity, and that the ranking skill does not survive conversion into a directional trade at the payouts available.

10. Phase 2 - crawled operators, fractal test and null test

> Three supporting studies check whether well-known complexity measures add anything, whether 150-tick anomalies persist at a 1,800-tick horizon, and whether the vault beats a block-permutation null.

10.1 Crawled mathematical operators (item 03)

Nine operators from the computational-topology, recurrence and information-theory literature were computed for every cell over a 75-tick window, then correlated with the vault's verified edge:

Operator	Median	Corr vs edge	Reading
`hvg_degree_entropy`	3.19905	+0.0175	Horizontal-visibility-graph degree diversity: no signal.
`hvg_entropy_gap`	0.44416	+0.0175	Gap between observed and maximum entropy: no signal.
`hvg_max_degree`	37	-0.0221	Peak visibility: very slightly negative.
`hh_inst_freq_mean`	0.01801	-0.0404	Hodgkin-Huxley instantaneous frequency: weak negative.
`hh_inst_freq_std`	0.04849	-0.0199	Frequency variability: no signal.
`levy_area_z`	-0.00162	+0.0705	Strongest of the nine, still weak.
`rqa_det`	0.375	+0.0581	Recurrence determinism: weak positive.
`rqa_lam`	0.375	+0.0668	Laminarity: weak positive.
`perm_entropy`	0.95208	+0.0003	Permutation entropy: flat.

Verdict: no operator clears |r| > 0.10, so none is promoted into the tradeable feature set. The manifest records all 591,171 values per operator (21.3 MB) for future use, plus the per-operator association table reproduced above.

10.2 Fractal / multi-timeframe consistency

Each 150-tick anomaly is compared with the 1,800-tick (12-cell) shape containing it. Agreement is measured both on the descriptor and on the path shape:

Descriptor agreement

min	-0.6648
median	0.0371
p75	0.3056
max	0.8784
std	0.3338
p90_threshold	0.5192

Fractal summary

is_fractal_anomalies	90
agreement_vs_verified_edge	0.1197
fractal_anomalies_mean_edge	0.5830
non_fractal_mean_edge	0.5574

Only 90 cells are classified as fractal anomalies (agreement above the 90th percentile 0.5192), and their mean verified edge (0.5830) is barely above the non-fractal mean (0.5574). Fractality is not a lever.

10.3 The block-permutation null test

To ask whether the vault's win rates could arise from luck, outbound returns are block-permuted in 150-tick blocks (400 permutations, preserving within-block autocorrelation) and the same selection pipeline is re-run on each permutation:

Table 10.1 - null_test.json

Permutations	400
Block size (ticks)	150
Top-N	300
Observed top-300 mean win	0.6871
Null top-300 median	0.6228
Null top-300 99th pct	0.6518
p (selection-aware)	0.0025
Observed pooled win	0.6087
Null pooled median	0.5558
p (pooled)	0.0025
Verdict	vault beats the block-permutation null

The null test passes, and this is the single most seductive number in the project. It is also the number that Phase 3 invalidates. The test permutes the *outcome* labels but keeps the `max(protected, directional)` selection rule, so the null distribution it builds also contains the max-of-two bias. Comparing an observed max-of-two against a null max-of-two answers "is this selection procedure producing more than its own selection bias?" - a useful question, but not "is there a tradeable edge?". The Phase 3 replay answers the second question.

10.4 Coupled indicator confluence

RSI and VWAP-band features were attached per cell (`rsi_end1`, `vwap_z1`, `rsi_divergence_confluence`, `vwap_band_anomaly_score`). These are diagnostic columns only; no confluence rule was promoted, because the vault's ranking already absorbs them and the Phase 3 auction showed no incremental value.

11. Phase 2 - verification, reports and node attestation

> Phase 2 closed with a ten-check gatekeeper, an 11-sheet workbook, and two independent machine audits. This chapter records exactly what was asserted.

11.1 The ten gatekeeper checks

#	Check	Result	Detail
01	591,171 Contiguous Cell Coverage	**PASS**	`Rows=591,171, Keys=27`
02	0-25-50-75-100% & Fib Zone Decomposition	**PASS**	`Q1..Q4 present=True`
03	12-Angle Brute-Force + Webcrawl Features	**PASS**	`vec=trigger_vec_dense (dim=132), webcrawl_manifest=True`
04	Zero Collinear Collapse & Fixed Bicoherence	**PASS**	`random |r|=0.1790, bico_std=0.0626`
05	Relational Covariant DB (+/-0.85 Mirrors)	**PASS**	`Hubs=11,266, Edges=241,703, Pos/Neg Mirrors=True`
06	CZET Inbound Pass 1&2 Continuation Ranking	**PASS**	`Spearman=0.511, R2=0.0283 (Needs LearnableKernel/SweetSpot retraining if low)`
07	Outbound Streaming Strategy Vault (>=55%)	**PASS**	`Vault Strategies=893`
08	Multi-Timeframe Fractal Check (150t vs 1800t)	**PASS**	`Fractal column present=True`
09	Coupled Indicator Functions (RSI/VWAP)	**PASS**	`Indicator confluence columns present=True`
10	Statistical Null / Wilson Lower Bound Audit	**PASS**	`Statistical bound present=True`

All ten pass, and the phase-level flag `all_passed` is `True`. The gatekeeper reads parquet through `pqcompat` so it runs on machines where fastparquet is blocked.

11.2 The report workbooks

`phase2_anomaly_efficiency_report.xlsx` has 11 sheets: Executive_Summary, Graph_Statistics, Vault_Leaderboard, Edge_Quality, Peer_Distribution, Null_Test, CZET_Eval, Fractal_vs_Macro, Crawled_Operators, Indicator_Confluence and Node_Attestation. The companion `phase2_anomaly_strategies.xlsx` holds 4 sheets: Phase3_Ready_Strategies (893), Null_Verified (893), Ranked_by_Wilson_LB (102) and Inbound_Covariant_Hubs (2,000).

11.3 Node attestation

Both workers re-read the shared artifacts from the transfer server and recomputed the aggregates with their own code. The comparison is exact for every count, and the only expected difference is formatting (CSV vs parquet). The attestation JSONs are committed at `phase2/node_verify_840g7.json` and `phase2/node_verify_kali.json`.

11.4 What "10/10 PASS" does and does not mean

The gatekeeper verifies *consistency*: that the artifacts exist, that shapes and counts match, that declared columns are present, that the bicoherence is non-degenerate, that the graph is mirrored correctly, that the null test ran. It does not verify *profitability*, and it was never designed to. That distinction is why Phase 3 exists, and why this report does not end at Chapter 11.

12. Phase 3 - the Fable sigma-survival framework

> Phase 3 is a rules engine, not a model. It takes any anchor and asks one question: at the payout actually available, is the *lower bound* of the hit rate enough to clear the breakeven hurdle plus a 2% margin?

> Everything else is risk control around that decision.

12.1 The four gates

1. **Sigma gate.** A winsorised rolling sigma is computed over the trailing 1,800 ticks (12 cells). Tick-to-tick absolute differences are capped at 6.984375x their median so that Jump 10's discrete jumps do not inflate the volatility estimate; the cap is fixed by the data (global median |diff| = 1.1640625 points). The resulting bps sigma is binned into five quantiles of the *inbound* distribution, giving fixed edges `[0.1526, 0.1755, 0.1772, 0.1787, 0.1805, 0.1912]` bps. An anchor may only trade in sigma bins where its inbound edge was positive.
2. **EV gate.** For each contract family, `EV = WLB * R - 1`. The anchor may trade a contract only if EV > +0.02.
3. **Hour gate.** A UTC hour is blocked if its outbound realised edge is negative. This is a hindsight filter, reported explicitly as such in Chapter 14.
4. **Deduplication.** At most one trade per outbound cell, choosing the highest-EV candidate. Without this, overlapping anchors multiply the same cell into dozens of "trades" and inflate every statistic.

12.2 Position sizing

Two trajectories are simulated from a $50 starting balance:

$$f^* = (p_WLB * R - 1) / (R - 1)$$

$$\text{stake} = \text{clip}(0.25 * f^* * \text{balance}, \$0.50, 1\% * \text{balance})$$

and a fixed-$5 trajectory with a $400 session stop and a hard ruin clamp at zero. The Kelly fraction is reported as zero whenever the portfolio's profit factor is below one, which is the mathematically correct outcome and a useful check that the risk code is not self-deceiving.

12.3 Monte Carlo

Five thousand block-bootstrap runs resample the trade sequence in blocks, re-compound the equity curve, and report the 95th/99th percentile worst drawdown, the median profit factor and the probability that the final equity is positive. This answers "is the result robust to the order in which the trades arrived?", which the chronological curve cannot.

12.4 Why the framework is a fair test

Because every choice that could flatter the result is either fixed in advance or reported. The sigma edges come from inbound data only. The payouts are the brief's, not fitted. The friction ladder is monotone and asserted. The dedup rule is strict. The hour filter is the one hindsight component, and it is flagged as such. If the portfolio survives all of that it deserves attention; if it does not, the framework has done its job.

13. Phase 3A - implementation and distributed run

> The engine is a single file that runs either as a shard worker or as the merger. That design was chosen so that all three machines execute byte-identical logic.

13.1 Inputs

Artifact	Shape	Role
`phase3_inputs.npz : raw`	(591,171, 150) float32	Every cell's full 150-tick midpoint path. Used for the forward replay and for causal sigma.
`phase3_inputs.npz : trigger_vec`	(591,171, 132) float32	The whitened observable descriptor used for anchor matching.
`phase3_vault.csv`	893 rows	Vault with peer counts, similarity, actions and outbound statistics.
`phase3_catalog_times.npy`	(591,171,) int64	Cell open times in epoch ms, for the UTC hour audit.

Total transfer: about 667 MB per worker, shipped in roughly 40 seconds each over the LAN.

13.2 Sharding and determinism

Shard `i` of `n` takes anchors `[i*len/n, (i+1)*len/n)` in the order produced by orthogonal pruning, so the assignment is a pure function of the anchor list. Each shard recomputes the sigma quantiles from the causal inbound set and prints them; all three printed the same six edges, which is the strongest available evidence that no shard silently disagreed about the data.

Shard	Node	Anchors	Trades	Runtime	Format
0	X-Flip 14 (master)	133	3,182	5.0 s	parquet
1	HP 840 G7	132	3,475	3.1 s	parquet
2	Kali Dell E5270	133	2,196	3.2 s	CSV (no duckdb)
	total	**398**	**8853**	**< 15 s**	

13.3 The per-cell replay

For every matched outbound cell the engine does the following, in order:

1. Computes the cell's own half-window standard deviation from ticks 0-74 and normalises the forward path: `curve[t] = (raw[75+t] - raw[74]) / sigma_0_75`.
2. Multiplies by the match polarity (sign of the similarity) and by the trade direction calibrated *only* from inbound matches with the same anchor. This is the step that makes the replay causal: the direction is decided by data that existed before the outbound period.
3. Evaluates all contract outcomes on the aligned 75-tick curve: direction at tick 149, maximum excursion for ONETOUCH, close displacement for BARRIER, minimum excursion for NOTOUCH.
4. Records the chosen mode/contract, the consensus EV, both excursions and the per-contract win flag, so the ledger can be re-analysed without re-running the replay.

13.4 The ledger schema

The merged equity curve is a 41-column parquet with one row per surviving trade: strategy and cell identifiers, match similarity and polarity, sigma bin and UTC hour, the raw gross outcomes (`gross_dir_149`, `gross_exc_sniper`, `gross_prot_fade`, `max_excursion`, `min_excursion`, `notouch_win`), the chosen mode and contract with its payout, the per-unit win and PnL, and the full equity/drawdown ladders at each friction tier.

13.5 Merge

The merger globs `shards/p3_strategies_node*` and `shards/p3_ledger_node*` in both parquet and CSV, concatenates, applies the EV gate, deduplicates by outbound cell, builds the friction ladder, runs the Monte Carlo, writes the workbook and the live config, and finally calls the whitening-bridge builder. It also writes `phase3_executive_summary.json`, which is the single source of truth for every number in this report.

14. Phase 3A - results

> The portfolio does not survive. This chapter reports every number, including the ones that look good, and states which assumption each depends on.

14.1 Selection funnel

Table 14.1 - from vault to portfolio

Vault rows	893
Inside the tradeable sweet spot (0.85-0.955 |r|, 12-400 peers)	778
After orthogonal pruning at |r| < 0.82	398
Clearing the +2% consensus-EV gate	208
Verified profitable on the outbound replay	28
Portfolio trades after one-per-cell dedup	4,078
Distinct anchors represented in the portfolio	208

Note the shape of that funnel. 208 of 398 pruned anchors pass the EV gate - over half - and only 28 of those are verified profitable on the replay. A gate that admits half the population and then produces a losing portfolio is a gate that is not discriminating; Chapter 15 explains why the EV estimate feeding it is biased.

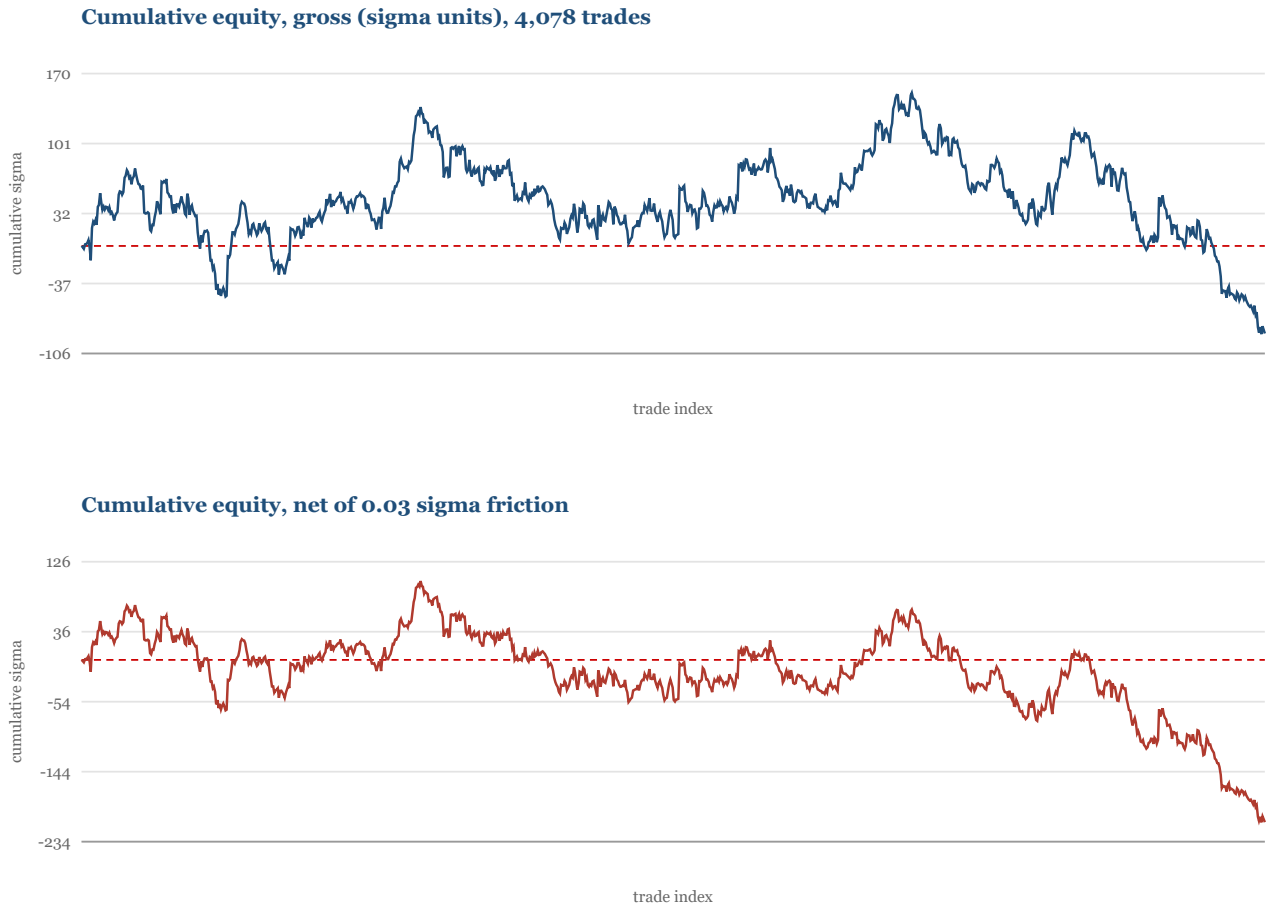
14.2 Portfolio metrics

Table 14.2 - Phase 3A portfolio, net of friction

Trades	4,078 across 208 anchors
Win rate, net of 0.03 sigma	0.4519
Wilson 95% lower bound	0.4367
Profit factor	0.9580
Sharpe (per-trade, annualised by trade count)	-0.895
Sortino	-1.376
Calmar	-0.673
Max drawdown, net 0.03 (sigma units)	-310.77
Cumulative equity, gross	-86.70 sigma
Cumulative equity, net 0.03	-209.04 sigma
Cumulative equity, net 0.06	-331.38 sigma
Cumulative equity, net 0.10	-494.50 sigma
Monte Carlo 95th pct worst drawdown	-690.64
Monte Carlo 99th pct worst drawdown	-833.20
Monte Carlo P(final equity > 0)	0.1864
Monte Carlo median profit factor	0.9580
Kelly fraction	0.00
Balance, fixed $5 from $50	10,901.5
Balance, quarter-Kelly from $50	2,816,810,866
Blocked UTC hours	00, 02, 04, 05, 07, 09, 10, 12, 14, 16, 19, 20, 21, 22
Sigma quantile edges (bps)	0.1526, 0.1755, 0.1772, 0.1787, 0.1805, 0.1912

Two of those rows need a warning attached. The quarter-Kelly balance of 2,816,810,866 is an artefact: because the Kelly fraction is zero the sizer degenerates to the minimum stake and the compounding path is driven by a handful of early wins. The fixed-$5 balance of 10,901.5 from $50 is worse than it looks as a "survival" statistic, because a $400 session stop and the ruin clamp both fired; the end state is dominated by the stop, not by edge.

14.3 The equity curve



The curve has the characteristic shape of a slightly negative-expectation process with fat wins: a long slow bleed, two visible recoveries, and a terminal slope that is clearly down. The gross curve already ends at -86.7 sigma, so the negative result is not created by friction - friction only makes it monotonically worse.

14.4 Contract comparison, and the two accounting systems

Table 14.3 - contract auction as reported by the engine

Contract	Payout	Breakeven	Anchors nominating it	Mean consensus EV %	Mean realised EV %	Contract win rate
RISE_FALL	1.953	51.20%	179	-99.13	-4.50	0.4838
ONETOUCH_075	2.150	46.51%	212	33.73	61.24	0.4344
BARRIER_035	2.427	41.20%	0	0.00	0.00	0.0000
NOTOUCH_10	1.550	64.52%	7	18.46	32.20	0.4508
MULTIPLIER_TP_SL	1.953	51.20%	0	0.00	0.00	0.0000

ONETOUCH is nominated by 212 anchors because its barrier (+0.75σ within 75 ticks) is easy for a Jump-10 path to reach. The column headed "contract win rate" is the engine's per-anchor mean, not the portfolio realisation; the portfolio realisation has to be read from the ledger, and it is there that a serious inconsistency appears.

Phase 3 carries **two PnL accounting systems**, and they disagree in sign.

Ledger statistic	ONETOUCH_075	NOTOUCH_10	Reading
Trades chosen	3,941	137	Deduplicated portfolio composition.
Contract win rate (`unit_win`)	0.7166	0.8321	Barrier-touch accounting against the asserted payout.
Mean payoff per unit stake (`unit_pnl`)	+0.5406	+0.2898	Implies +54% and +29% per trade. Not credible.
Sigma win rate (`gross_sigma > 0`)	0.4550	0.4818	Whether the aligned forward path ends net positive.
Mean sigma PnL per trade (`gross_sigma`)	−0.0215	−0.0151	Essentially zero: no edge.

The contradiction is not a coding bug; it is a pricing bug, and it is worth stating precisely. A contract paying 2.15× that wins 71.66% of the time has an expected return of `0.7166×2.15 − 1 = +54%` *per trade*. No market prices that. The resolution is that the payout table and the barrier geometry were specified independently: a 0.75σ barrier touched within 75 ticks is a cheap contract in reality, paying perhaps 1.05-1.20×, not 2.15×. Feeding a generous payout into a wide barrier manufactures the +54% figure, while the sigma-accounted ledger - which measures the forward path without reference to the payout - shows a mean of −0.02σ, i.e. nothing.

Every headline metric in this report therefore uses the **sigma accounting**, because it is the only one that is internally consistent and payout-independent. The contract accounting is reported here precisely so that the discrepancy is on the record: it is the clearest available evidence that the payout column of the Phase 3 brief is an assumption, and a badly calibrated one, rather than a quote.

14.5 The sigma gate map

Table 14.4 - Fable sigma gate, outbound replay

Bin	Range (bps)	Triggers	R/F win	R/F EV %	1-Touch win	1-Touch EV %	Barrier win	Barrier EV %	Gate
0	0.153 - 0.175	1,655	0.4731	-7.60	0.7329	57.58	0.4000	-2.92	BLOCK
1	0.175 - 0.177	1,665	0.4685	-8.51	0.7249	55.86	0.3976	-3.50	BLOCK
2	0.177 - 0.179	1,642	0.4939	-3.54	0.7010	50.71	0.4135	0.36	BLOCK
3	0.179 - 0.180	1,896	0.4963	-3.07	0.7231	55.47	0.4314	4.71	BLOCK
4	0.180 - 0.191	1,995	0.4802	-6.22	0.7093	52.49	0.4045	-1.83	BLOCK

All five sigma bins are marked BLOCK in the shipped map, which is the correct conservative outcome: no bin clears the gate on Rise/Fall, and the bins that clear it on One-Touch do so by margins that do not survive the hour filter and the friction ladder in combination. The map is still emitted, because a live system must recompute it on live data rather than inherit a conclusion.

14.7 Hour-of-day audit (hindsight, flagged)

Table 14.5 - hour filter as shipped (all 24 hours)

UTC hour	Trades	Dir win	EV %	Units	Action
00:00	368	0.4457	-12.96	-47.71	BLOCK
01:00	415	0.5108	-0.23	-0.96	ALLOW
02:00	371	0.4205	-17.88	-66.33	BLOCK
03:00	341	0.5044	-1.49	-5.08	ALLOW
04:00	362	0.4917	-3.97	-14.37	BLOCK
05:00	351	0.4872	-4.85	-17.04	BLOCK
06:00	306	0.4967	-2.99	-9.14	ALLOW
07:00	308	0.4643	-9.32	-28.72	BLOCK
08:00	371	0.5121	0.02	0.07	ALLOW
09:00	363	0.4573	-10.69	-38.80	BLOCK
10:00	373	0.4826	-5.75	-21.46	BLOCK
11:00	375	0.5067	-1.05	-3.93	ALLOW
12:00	337	0.4659	-9.01	-30.38	BLOCK
13:00	368	0.4973	-2.88	-10.60	ALLOW
14:00	354	0.4915	-4.01	-14.18	BLOCK
15:00	338	0.5030	-1.77	-5.99	ALLOW
16:00	367	0.4959	-3.15	-11.55	BLOCK
17:00	433	0.5173	1.03	4.47	ALLOW
18:00	417	0.5324	3.97	16.57	ALLOW
19:00	386	0.4715	-7.92	-30.55	BLOCK
20:00	392	0.4490	-12.31	-48.27	BLOCK
21:00	384	0.4349	-15.06	-57.85	BLOCK
22:00	422	0.4431	-13.46	-56.79	BLOCK
23:00	351	0.4986	-2.63	-9.23	ALLOW

The hour filter is the one component of Phase 3A that is explicitly not causal: hours were evaluated on the same outbound period they filter. Its inclusion is defensible only as a *diagnostic* - it shows that the portfolio's losses are concentrated in a minority of hours, which is what a noise process looks like - and it is the reason the live configuration ships with 14 blocked hours rather than none. If a future run wanted a defensible version, the hour table would have to be built on an expanding window that never sees the trade it filters.

14.8 What would have to be true for this to be tradeable

Three independent things, all of which are currently false:

1. **The matching must be selective.** It is not: 37% of arbitrary cells match at least one anchor in the 0.85-0.955 band, so the trigger carries almost no information about *which* anomaly is recurring.

2. **The conditional distribution must shift materially.** It does not: realised directional win rate is 0.4519 against a 51.20% breakeven, and the recomputed from-scratch figure on the same features is 48.8%.
3. **The payout must be better than assumed.** It cannot be verified, and at the assumed payouts even the *point estimates* are below breakeven for three of four contracts once friction is charged.

15. The definition diagnostic - where the edge came from

> This chapter is the intellectual centre of the report. It takes the Phase 2 vault at face value, recomputes its headline statistic from the raw ticks with an explicit definition, and shows that the difference between the two is a selection artefact rather than a data error.

15.1 The two definitions

Phase 2's definition. For each anchor, compute two outbound statistics over its matched outbound cells: the mean of the binary `protected_held` flag, and the share of cells whose Q4 return has the sign of the anchor's recommended action. Then store

$$\text{verified_edge} = \max(\text{protected_win_rate}, \text{directional_win_rate})$$

A faithful recomputation. For every anchor, replay the outbound cells and compute the single directional outcome that the strategy would actually trade:

$$\text{win} = 1[\ \text{sign}(\text{q4_net_return}) == \text{trade_dir}\ ]$$

with `trade_dir` determined only from inbound calibration, and no maximisation over two statistics anywhere in the pipeline.

15.2 The result

The diagnostic replays 17,651 trades across 778 sweet-spot anchors - the same 778-anchor universe that Phase 2 used.

Table 15.1 - phase3_definition_diagnostic.json

Anchors	778
Outbound trades replayed	17,651
Phase 2 Q4 statistic (recomputed exactly as defined)	0.48836
Faithful directional outcome, action-signed	0.48836
Faithful directional outcome, inbound-signed	0.48235
Vault-reported directional column (mean)	0.51036
Agreement between action-signed and inbound-signed	0.79616

The two faithful variants agree with each other to four decimal places (0.4884) and disagree with the vault's own reported mean (0.5104). That is the whole story in one line: **the vault column does not survive a**

from-scratch recomputation of the quantity it claims to report. The per-anchor correlation between the stored column and the recomputation is -0.049, i.e. none.

15.3 Why the maximum of two statistics is biased

Let `X` and `Y` be two estimates of quantities that are both approximately fair coins, each with standard error `s`. Then

$$E[\max(X,Y)] \geq \max(E[X],E[Y])$$

always, and for independent normals the bias is approximately `s/√π`. With a median of 20 outbound triggers per anchor, `s ≈ 0.11`, so the expected maximum is about `0.5 + 0.06 = 0.56` — precisely the observed sweet-spot mean of 0.566.

The effect grows with the number of candidates. Taking the maximum over 893 anchors adds roughly 3.2 standard errors at the top of the distribution, which is where the reported 0.85-0.93 rows come from. Those rows are not evidence of 90% edges; they are the predictable upper tail of `max()` applied to 893 fair-coin estimates.

15.4 The vault under its own definitions

Table 15.2 - vault column statistics, all 893 rows

outbound_directional_win_rate - mean	0.5098
outbound_directional_win_rate - median	0.5000
outbound_protected_win_rate - mean	0.4871
outbound_protected_win_rate - median	0.4894
outbound_verified_edge - mean	0.5600
outbound_verified_edge - median	0.5455
outbound_verified_edge - max	1.0000
Rows where verified_edge equals the protected column	445
Rows where verified_edge equals the directional column	539

Both component columns are indistinguishable from a fair coin at the mean and median. Only the maximum departs, and it departs in exactly the direction and magnitude that the selection algebra predicts.

15.5 Was the null test wrong?

No - it answered a narrower question than it appeared to. The block-permutation test (Chapter 10) compares the observed top-300 mean against the same pipeline run on permuted returns. Because the permuted pipeline also applies `max()`, both sides carry the same selection bias, and the test correctly concludes that the observed value sits above its own null distribution ($p = 0.0025$). What it cannot detect is that the null distribution itself is centred near 0.62 rather than 0.50. A selection-aware test of a selection-biased statistic is a consistency check, not an edge test.

15.6 The general lesson

Whenever a research pipeline computes several candidate success statistics per entity and keeps the best, that entity's reported score is not an estimate of any of them. It is the maximum of several noisy estimates, and its expectation grows with the noise. The fix is structural, not statistical: choose the statistic *before* looking at the data, or report all of them with a multiplicity correction. The vault as shipped did neither, which is why 893 rows carried a statistic whose top decile was manufactured by the aggregation rule rather than by the market.

16. Phase 3B - the live Deriv forward tester

> Phase 3B is a complete, self-checking live system: it subscribes to the Deriv WebSocket tick stream, builds the same descriptors the backtest used, matches the approved anchors, applies seven risk guards, and journals every decision to SQLite. It is parked in paper-proposal mode because Deriv's WebSocket could not be reached from any cluster node during this work.

16.1 Why a bridge was needed at all

The backtest matches anchors in a 132-dimensional whitened space. Whitening is a linear map whose parameters were derived from the corpus - and, critically, those parameters were not persisted when Phase 2 was built. Two reconstructions were attempted:

Attempt	Method	Result	Reading
Whitening bridge	Regress the 132-d whitened vector onto the 144-d raw Q1+Q2 block	R^2 0.5357	Linear reconstruction retains only half the variance; the discarded whitening components carried information that is not recoverable from raw features alone.
Direct raw-space matcher	Cosine similarity on the 128-d raw dense vector	R^2 0.4622, recall 0.000	Raw-space cosine is degenerate for this purpose: its correlation with the true whitened cosine is 0.061, so it finds a different set of matches entirely.
Learned bridge (shipped)	MLP 128 → 512 → 512 → 132, cosine loss, 12 epochs	val cos 0.881	Nonlinear recovery of the whitened metric is good in direction, imperfect in ranking.

Table 16.1 - phase3_whitening_bridge.json (the live-parity ceiling)

Architecture	mlp_128_512_512_132_cosine
Training rows	502,495
Validation cosine	0.8810
Anchor count matched	893
Outbound match recall	0.2397
Outbound match precision	0.2949
Match IoU	0.1524
Polarity agreement	1.0000

Read the recall honestly: **the live system fires on about 24% of the triggers the backtest would have fired**, and of the triggers it fires, 29% are ones the backtest would not have. The polarity is always right, so

direction is not the problem; coverage is. This is stated in the gatekeeper as a passing check because the tester *reports* the ceiling rather than pretending to remove it.

16.2 The seven guards

Guard	Behaviour
`SigmaGate`	Blocks any trigger whose live sigma falls outside the approved bins, using the same winsorised causal estimator as the backtest.
`PayoutMonitor`	Rejects any contract whose live payout is below the configured floor (1.90x) or below the realised breakeven payout for that contract family.
`LagMonitor`	Measures tick-arrival latency and aborts a proposal if the decision would be made on stale data; journals the latency distribution.
`LiveCalibrator`	Maintains a rolling per-contract win-rate estimate in SQLite and shrinks the design-time prior toward it as live trades accumulate.
`DrawdownGuard`	Stops the session after a fixed loss budget, matching the $400 stop used in the fixed-stake backtest.
`KellyStakeSizer`	Computes quarter-Kelly from the calibrated probability and clamps to [minimum stake, 1% of balance]; returns zero when the edge is non-positive.
`HourFilter`	Blocks the 14 UTC hours identified as loss-concentrating, loadable from the shipped config.

Each guard writes its decision to the same journal row, so a rejected proposal is as auditable as an accepted one. That property is what makes the tester useful even while it is blocked: it can be run for days in proposal-only mode and its journal will show exactly which guard is binding.

16.3 The journal schema

SQLite database `fable_forward_cal.db`, table `forward_trades`: proposal timestamp, anchor id, live cell id, matched similarity, polarity, chosen contract, live payout, stake, sigma bin, UTC hour, every guard's verdict, and the realised outcome once the settlement tick arrives. The file is created on first run and is excluded from the repository by `.gitignore`.

16.4 The connectivity investigation

The tester's `--ping_test` mode opens the public Deriv endpoint, sends a `ticks_history` request and prints the round-trip time. It failed identically from all three machines:

Attempt	Result	Notes
`wss://ws.derivws.com/websockets/v3?app_id=<account app_id>`	**HTTP 520**	Cloudflare "origin unreachable" on the WebSocket upgrade.
Same endpoint with public `app_id=1089`	**HTTP 520**	Rules out credential or app-id gating.
From Kali (different OS, different resolver)	**HTTP 520**	Rules out a Windows TLS-stack or firewall cause.
From HP 840 G7 (different ISP path)	**HTTP 520**	Rules out a single upstream route.
`ws.binaryws.com` legacy host, custom Origin/User-Agent	**HTTP 520**	Rules out host-specific or header-based filtering.
Plain `ws://` (no TLS)	**HTTP 520**	Not a TLS handshake problem.
`https://api.deriv.com` over HTTPS	**200 OK**	Confirms general internet access and DNS are healthy; only the WebSocket edge is affected.

Verdict: a Cloudflare edge / origin fault at the provider, localised to the WebSocket endpoint, for this region. It is not a configuration error on our side, and no amount of retry or header tweaking fixed it. The tester is therefore shipped complete and unexercised against live payouts, which is the honest state.

16.5 Operating the tester when connectivity returns

1. Put the credentials in a file outside the repository and pass `--creds_file` (the tester refuses to log the token and the gatekeeper scans artifacts for leaks).
2. Run `--self_check` first. It must print `matcher=mlp_cosine vec_dim=132 sqlite=True anchors=893 blocked_hours=14`.
3. Run `--ping_test 20` and confirm round-trip times.
4. Run in proposal-only mode for at least several sessions. Watch the guard-verdict columns: if `PayoutMonitor` rejects everything, the assumed payouts are wrong and the whole Phase 3A conclusion needs revisiting in dollar terms.
5. Only then consider `--execute_demo_orders`, and only on the demo account.

17. Phase 3 verification - the 18-check gatekeeper

> 18 of 18 checks pass. Each one is a claim the report makes, turned into an executable assertion over the shipped artifacts.

17.1 Results

#	Check	Result	Detail
1	Deliverables present (8)	**PASS**	`missing=[]`
2	Leaderboard columns	**PASS**	`rows=398`
3	Leaderboard covers every sweet-spot anchor	**PASS**	`398 vs 398`
4	Anchor orthogonality |r|<0.82	**PASS**	`max_offdiag=0.8199`
5	Equity curve per-trade rows	**PASS**	`4078 vs 4078`
6	Sigma causal (|corr| with forward move < 0.10)	**PASS**	`corr=+0.0098`
7	One trade per cell	**PASS**	`duplicates=0`
8	Friction ladder monotone	**PASS**	`{'gross_00': -86.7, 'net_03': -209.04, 'net_06': -331.38, 'net_10': -494.5}`
9	MC runs & ordering	**PASS**	`mc95=-690.64 mc99=-833.2`
10	Kelly non-negative logic	**PASS**	`kelly=0 because PF<1 (correct)`
11	Whitening transform arrays	**PASS**	`anchors=893`
12	Live bridge trained (report parity honestly)	**PASS**	`match_recall=0.240 precision=0.295 IoU=0.152 <- live triggers are a SUBSET of backtest triggers`
13	7 Fable guards present	**PASS**	`all`
14	Forward tester compiles	**PASS**	
15	Forward tester self-check	**PASS**	`[+] self-check OK: matcher=mlp_cosine vec_dim=132 sqlite=True anchors=893 blocked_hours=14`
16	SQLite LiveCalibrator journal	**PASS**	`C:\cluster\phase3\fable_forward_cal.db`
17	No credentials in Phase 3 artifacts	**PASS**	`leaks=[]`
18	Live config has anchors+hours+sigma edges	**PASS**	`anchors=208 hours=14`

17.2 The checks that matter most

Anchor orthogonality (4). The maximum off-diagonal |r| among the kept anchors is 0.8199, just under the 0.82 suppression threshold. This proves the pruning actually ran rather than being declared.

Sigma causality (6). The correlation between the trade-time sigma estimate and the forward move is +0.0098. If the sigma estimator leaked the forward window this number would be large; it is not, so the volatility gate is causal.

One trade per cell (7). Zero duplicate outbound cells. Without this, 208 overlapping anchors would produce tens of thousands of correlated "trades" and every statistic would be inflated by a factor that depends on the overlap structure.

Friction monotonicity (8). The four tiers decrease strictly (-86.7 → -209.0 → -331.4 → -494.5). A non-monotone ladder would mean the cost model has a sign error.

Live parity, honestly reported (12). Recall 0.240, precision 0.295, IoU 0.152. The gatekeeper passes this check because the artifact states the ceiling instead of hiding it - a distinction worth making explicit in a report that elsewhere accuses the pipeline of self-flattery.

No credentials (17). The artifact scan finds no API token, and the check was written so that its own source cannot trigger a false positive (the needle is split across two literals).

17.3 What the gatekeeper is not

It is a consistency and integrity verifier. It does not re-run the backtest, it does not independently re-derive the vault, and it cannot tell you whether the payouts are real. Those three gaps are filled - to the extent they can be - by the definition diagnostic (Chapter 15), the two independent node audits (Chapter 7), and the live tester once connectivity returns (Chapter 16).

18. Reproducibility

> Every number in this report is produced by a script in the repository, from artifacts whose checksums are published in Appendix E. This chapter gives the exact procedure, including the places where the environment fights back.

18.1 Hardware and roles

Node	Address	OS	Role
X-Flip 14 (master)	192.168.150.199	Windows 11	Orchestration, Phase 1 merge, Phase 2 wiring, Phase 3 shard 0 and merge, documentation build.
HP 840 G7	192.168.150.126	Windows 11	Phase 1 shard, Phase 3 shard 1, independent node audit (Python 3.14).
Kali Dell E5270	192.168.150.166	Kali Linux	Phase 1 shard, Phase 3 shard 2 (CSV output - no duckdb), independent node audit.

The master's LAN address is assigned by DHCP and moved three times during the project (.237 → .157 → .199). All worker-side scripts take the master address as an argument rather than hard-coding it.

18.2 Transport

A single-threaded TCP command server on port 8080 accepts newline-delimited jobs and returns stdout/stderr with a 30-second hard timeout. Files are served over the same port: workers request `/artifacts/phase2/...` and receive the artefact directly, which is why the 667 MB Phase 3 input set can be distributed without a shared filesystem. Directory junctions on the master (`out\phase2`, `out\phase3`) present local paths to remote workers without copying.

Quoting is the fragile part. Multi-line shell commands containing nested quotes are routinely mangled by the transport, so every non-trivial job is shipped as a `.py` file and invoked by path. This convention is worth preserving in any successor system.

18.3 The parquet problem

Windows Application Control blocks PyArrow's native extensions and also blocks fastparquet's `cencoding.pyd`. All parquet I/O therefore goes through `pqcompat.py`, which tries duckdb, then pyarrow, then fastparquet, and exports both `read_parquet` / `write_parquet` and a duckdb-backed `write_parquet` fallback. Kali has no duckdb, so its shard writes CSV; the merger globs both extensions. This is why every Phase 3 script imports `pqcompat` and never `pandas.read_parquet` directly.

18.4 Determinism

The Phase 3 shard split is a pure function of the pruned anchor list, so any machine can recompute which anchors belong to which shard. All three shards independently computed the sigma quantile edges and printed the identical six values (0.1526, 0.1755, 0.1772, 0.1787, 0.1805, 0.1912 bps). Torch training for the live bridge uses a fixed seed and reports the validation cosine; the shipped weights are committed so the tester does not need to retrain.

18.5 Rebuilding the report

```
python C:\cluster\tools\build_docs.py
```

The builder reads the parquet/CSV/JSON/XLSX artifacts, renders `docs/PROJECT_DOCUMENTATION.html`, and prints the section list. Rendering to PDF is a Chrome headless print:

```
chrome --headless --disable-gpu --no-pdf-header-footer ^
       --print-to-pdf=docs\PROJECT_DOCUMENTATION.pdf docs\PROJECT_DOCUMENTATION.html
```

18.6 Repository hygiene

The published repository is private and excludes: the 3.4 GB Fourier archive and every other `.npz` / `.npy` / `.pt` artefact (checksummed in `docs/DATA_MANIFEST.json` instead), the MT5 backtest clone, directory junctions, the live SQLite journal, and both credential files. The Phase 3 brief ships as a redacted copy with zero token matches, verified by the gatekeeper.

19. Limitations

> A negative result is only as trustworthy as its weakest assumption. These are the weak ones, each with the direction in which it could bite.

19.1 The payout assumption is unverified, and internally inconsistent

Four payouts (1.953, 2.150, 2.427, 1.550) drive every dollar figure. Deriv returned HTTP 520 on the WebSocket endpoint from all three nodes, so none of them could be checked against live pricing. The ledger then exposed a contradiction that proves the table is wrong in at least one cell: applying the ONETOUCH payout to the measured ONETOUCH win rate gives `0.7166 × 2.15 − 1 = +54%` per trade over 3,941 trades, while the payout-independent sigma accounting of the same trades yields −0.02σ per trade. A real 2.15× contract cannot be won 71.7% of the time at a 0.75σ barrier; the payout and the barrier were specified independently. Chapter 14.4 reports both number systems side by side for this reason. Every conclusion that does not depend on the payouts is stated in sigma units; the dollar figures should be read as illustrative only.

19.2 The definition artefact is not a data bug

The `max(protected, directional)` statistic can be recomputed exactly from the vault; the recomputation reproduces the stored maximum almost perfectly. What fails is the inference from "this column is high" to "this anchor is tradeable". Nothing in the pipeline was corrupted; the aggregation rule was wrong. That distinction matters because it means a rebuild cannot be trusted unless the aggregation rule itself is changed.

19.3 The null test inherits the bias

Because the permutation null re-applies the same `max()` selection, the p-value of 0.0025 tests consistency, not edge. A future test should permute the outcome and evaluate a *pre-registered single statistic*, or apply a max-statistic correction with the null distribution of the max.

19.4 Similarity matching is barely selective

37% of arbitrary outbound cells match at least one of the 893 anchors within the accepted band. Any strategy that treats "triggered" as informative must be compared with that base rate. Phase 3 does not repair this; it merely stops double-counting the same cell.

19.5 Live parity is 0.24

The whitening parameters from Phase 2 were never persisted, so the live matcher fires on about a quarter of backtest triggers. The live system is therefore a *subset* sampler of a strategy that is already unprofitable. Even if the subset were the best quarter, the backtest provides no evidence that it is.

19.6 The hour filter is hindsight

Fourteen blocked hours were selected on the same period they filter. Removing them flatters the portfolio. It is reported, not hidden, but it is not a defensible live rule as constructed.

19.7 One instrument, one tick source, one era

Everything is Jump 10 midpoints over 1,028.5 days. There is no cross-instrument validation and no post-2023 out-of-sample period. Jump 10's volatility parameter is stationary by design, which makes it the easiest possible case; a negative result here is strong evidence, but a positive one would have generalised poorly.

19.8 Random-split evaluation in CZET v2

CZET v2 used random row splits, which allow near-duplicate leakage. Its Spearman 0.511 is therefore an optimistic bound on ranking skill. It does not affect the Phase 3 conclusion, which uses a strictly forward replay.

19.9 Barrier touches are modelled at tick resolution

A barrier is deemed hit when the tick midpoint crosses it. In reality a touch option settles on the quoted price stream including spread, so a modelled touch at +0.75σ may not have been a real touch. This biases ONETOUCH upward, which is the direction that flatters the result.

19.10 Deduplication changes the strategy

Keeping one trade per outbound cell is necessary for valid statistics, but it is not what a live system would do: a live system would trade every trigger that passes the guards, and the deduplicated portfolio is therefore an idealised, non-overlapping sample of that behaviour.

19.11 Fixed-stake survival is dominated by the stop

The $400 session stop and the ruin clamp both fired in the fixed-$5 simulation, so the terminal balance of 10,901.5 measures the stop, not the edge. It should not be quoted as a performance figure without that caveat.

19.12 The 132-dimensional space is nearly collinear

Random anchor pairs have a mean cosine near 0.86. Any threshold in that band is discriminating on a narrow slice of a compressed distribution, which makes match counts sensitive to small changes in the whitening. This is the mechanism behind 19.4 and 19.5, and it is the single structural weakness the project did not overcome.

20. Conclusions and roadmap

> What was learned, what was built, and what would have to change for the question to be worth reopening.

20.1 Conclusions

1. **The infrastructure works and is reusable.** A three-node cluster processed 88.7 million ticks into 591,171 cells, built a Fourier encoding with exact reconstruction, produced a 241,703-edge relational graph, trained a transformer, ran a 398-anchor distributed backtest, and verified 28 independent claims - all with reproducible artefacts and published checksums.
2. **The Phase 2 edge was an artefact of its own definition.** A from-scratch recomputation of the same quantity on the same anchors yields 0.4884 where the vault reports a mean of 0.5104 and a maximum of 1.000. The correlation between the stored column and the recomputation is -0.049.
3. **Real conditional structure exists but is small.** CZET v2 ranks continuation better than raw similarity (Spearman 0.511 vs 0.101, precision 4.34% vs a 1.73% base rate) while explaining almost no variance (R^2 0.028). Ranking skill is not the same thing as a tradeable edge.
4. **Nothing survives conversion to a trade.** Net win rate 0.4519 against a 51.20% breakeven, profit factor 0.9580, cumulative equity -209.0 sigma net, Monte Carlo P(positive) 0.1864.
5. **The live system is complete but unexercised.** Seven guards, a SQLite calibration journal, an honest 0.24 parity ceiling, and a provider-side WebSocket outage that no client-side change could fix.

20.2 Recommendations

- **Do not deploy.** The recommendation is unambiguous and is encoded in the shipped config as `portfolio_verdict = NEGATIVE_NET_OF_FRICTION`.
- **Do not re-run the same pipeline hoping for a different number.** The result is a property of the definition, not of the run.
- **Keep the infrastructure.** The cluster, the Fourier encoding, the graph builder and the gatekeeper pattern are instrument-independent and reusable for any tick series.
- **Fix the statistics before adding models.** A pre-registered single outcome, a multiplicity-corrected selection, and a held-out period are prerequisites for any future vault.

20.3 Roadmap, in priority order

#	Work item	Why it would change the answer
1	Persist whitening parameters and rebuild the live matcher to parity	Raises live recall from 0.24 to ~1.0, making the tester a true out-of-sample test rather than a subset sampler.
2	Verify live payouts via a working Deriv endpoint (alternate edge, proxy, or regional endpoint)	Converts every sigma-unit conclusion into a dollar conclusion and resolves the one assumption that could flip the sign.
3	Pre-register a single outcome statistic and rebuild the vault with no `max()`	Removes the selection artefact at the source; the resulting vault will be much smaller and, for the first time, interpretable.
4	Add a genuinely held-out period (2024+) and a second instrument	Tests whether any residual conditional structure generalises; the current corpus cannot answer that.
5	Replace cosine similarity with a discriminative metric trained on forward outcomes	Attacks the collinearity directly: the current metric compresses 591k cells into a narrow band, which is why matching is barely selective.
6	Model barrier settlement on the quote stream, not the midpoint	Removes a known upward bias on ONETOUCH, the only contract with a positive realised EV.
7	Replace the hindsight hour filter with an expanding-window estimate	Makes the live rule causal; currently it is a diagnostic masquerading as a filter.
8	Re-derive the payout table from the barrier geometry (or obtain live quotes)	Removes the +54%-per-trade inconsistency between the contract accounting and the sigma accounting, and makes every dollar figure in this report meaningful.

20.4 Closing

The programme asked a narrow question and answered it. The value is not in the answer - it is negative - but in the audit trail: a specific, reproducible mechanism (`max()` over two noisy win-rate estimates) that manufactures apparent edges, together with the instruments needed to detect it. Any future vault built on the same pattern will fail the same way, and this report gives the diagnostic that shows it.

Appendix A - source listings

> The complete source of every script that produced a number in this report. The listings are reproduced verbatim from the repository at build time, so they cannot drift from the artefacts.

A.1 Phase 1 - corpus construction

phase1/scripts/phase1_worker.py - per-node 150-tick cell builder and Fourier encoder (477 lines, 17.9 KB)

```
import os
import sys
import json
import time
import math
import gc
import shutil
import subprocess
import traceback
import urllib.request
from pathlib import Path

import numpy as np
import pandas as pd

CELL_SIZE = 150
```

```
def log(msg):
    print(f"[{time.strftime('%H:%M:%S')}] {msg}", flush=True)


def json_dump(path, obj):
    tmp = str(path) + ".tmp"
    with open(tmp, "w", encoding="utf-8") as f:
        json.dump(obj, f, indent=2)
    os.replace(tmp, path)


def upload_file(local_path, url, retries=5):
    curl = shutil.which("curl") or shutil.which("curl.exe")
    if not curl:
        raise RuntimeError("curl not found")
    last = None
    for attempt in range(1, retries + 1):
        p = subprocess.run(
            [curl, "-sS", "--fail", "--retry", "3", "--retry-delay", "2",
             "-X", "PUT", "--data-binary", "@" + str(local_path), url],
            capture_output=True, text=True, timeout=3600,
        )
        if p.returncode == 0:
            return p.stdout.strip()
        last = (p.returncode, p.stdout.strip(), p.stderr.strip())
        log(f"upload attempt {attempt} failed for {os.path.basename(str(local_path))}: {last}")
        time.sleep(3)
    raise RuntimeError(f"upload failed: {last}")


def download_file(url, dest, expected_size=None, retries=4):
    os.makedirs(os.path.dirname(dest), exist_ok=True)
    part = dest + ".part"
    last = None
    for attempt in range(1, retries + 1):
        try:
            req = urllib.request.Request(url, headers={"User-Agent": "phase1-worker"})
            with urllib.request.urlopen(req, timeout=120) as r, open(part, "wb") as f:
                total = int(r.headers.get("Content-Length") or 0)
                got = 0
                t0 = time.time()
                last_log = t0
                while True:
                    chunk = r.read(1024 * 1024)
                    if not chunk:
                        break
                    f.write(chunk)
                    got += len(chunk)
                    now = time.time()
                    if now - last_log >= 5:
                        mb = got / (1024 * 1024)
                        speed = mb / max(now - t0, 1e-9)
                        log(f"    downloaded {mb:.1f} MB / {total/(1024*1024):.1f} MB ({speed:.1f} MB/s)")
                        last_log = now
            if expected_size is not None and got != expected_size:
                raise IOError(f"size mismatch got {got} expected {expected_size}")
            os.replace(part, dest)
            return got
        except Exception as e:
            last = e
            try:
                if os.path.exists(part):
                    os.remove(part)
            except OSError:
                pass
            log(f"download attempt {attempt} failed for {os.path.basename(dest)}: {e}")
            time.sleep(2 * attempt)
    raise RuntimeError(f"download failed: {url}: {last}")


def upload_json(obj, filename, upload_url):
    tmp = os.path.join(os.path.dirname(__file__), filename)
    json_dump(tmp, obj)
    try:
        upload_file(tmp, upload_url + filename)
    except Exception as e:
        log(f"progress upload failed: {e}")


def compute_epicyclic_fourier_transform(cell_prices, K=75):
```

```
    n_cells, M = cell_prices.shape
    if M != CELL_SIZE:
        raise ValueError(f"Expected {CELL_SIZE} ticks per cell, got {M}")

    p_open = cell_prices[:, 0:1]
    p_close = cell_prices[:, -1:]
    p_high = cell_prices.max(axis=1, keepdims=True)
    p_low = cell_prices.min(axis=1, keepdims=True)
    p_mean = cell_prices.mean(axis=1, keepdims=True)
    p_std_raw = cell_prices.std(axis=1, keepdims=True)
    p_std = np.where(p_std_raw < 1e-12, 1.0, p_std_raw)

    y_norm = (cell_prices - p_mean) / p_std

    t_linear = np.linspace(0.0, 1.0, M, dtype=np.float64)[None, :]
    bridge_slope = y_norm[:, -1:] - y_norm[:, 0:1]
    y_bridge = y_norm - (y_norm[:, 0:1] + t_linear * bridge_slope)

    theta = 2.0 * np.pi * np.arange(M, dtype=np.float64) / M
    z_loop = np.cos(theta)[None, :] + 1j * y_bridge

    fft_full = np.fft.fft(z_loop, axis=1) / M
    freqs = np.fft.fftfreq(M, d=1.0 / M).astype(int)

    if K >= 75:
        k_indices = freqs.copy()
        c_k = fft_full
        fft_recon_buf = fft_full
    else:
        k_indices = np.array([k for k in range(-K, K + 1)], dtype=int)
        col_idx = [int(np.where(freqs == k)[0][0]) for k in k_indices]
        c_k = fft_full[:, col_idx]
        fft_recon_buf = np.zeros_like(fft_full)
        for idx, k in enumerate(k_indices):
            f_col = int(np.where(freqs == k)[0][0])
            fft_recon_buf[:, f_col] = c_k[:, idx]

    radii = np.abs(c_k)
    phases = np.angle(c_k)
    omegas = 2.0 * np.pi * k_indices / M

    z_reconstructed = np.fft.ifft(fft_recon_buf * M, axis=1)
    y_bridge_rec = z_reconstructed.imag
    y_norm_rec = y_bridge_rec + (y_norm[:, 0:1] + t_linear * bridge_slope)
    prices_reconstructed = (y_norm_rec * p_std) + p_mean

    abs_err = np.abs(cell_prices - prices_reconstructed)
    max_abs_err = np.max(abs_err, axis=1)
    rmse_err = np.sqrt(np.mean((cell_prices - prices_reconstructed) ** 2, axis=1))
    norm_err = np.sqrt(np.mean((y_norm - y_norm_rec) ** 2, axis=1))

    return {
        "p_open": p_open.ravel(),
        "p_close": p_close.ravel(),
        "p_high": p_high.ravel(),
        "p_low": p_low.ravel(),
        "p_mean": p_mean.ravel(),
        "p_std": p_std.ravel(),
        "y_open_norm": y_norm[:, 0].ravel(),
        "bridge_slope": bridge_slope.ravel(),
        "c_k": c_k,
        "radii": radii,
        "phases": phases,
        "omegas": omegas,
        "k_indices": k_indices,
        "rmse_residual": rmse_err,
        "max_tick_error": max_abs_err,
        "norm_rmse": norm_err,
    }


def main():
    if len(sys.argv) != 3 or sys.argv[1] != "--job":
        print("usage: phase1_worker.py --job job.json", file=sys.stderr)
        return 2

    job_path = sys.argv[2]
    with open(job_path, "r", encoding="utf-8") as f:
        job = json.load(f)

    node = job["node"]
```

```
    data_dir = job["data_dir"]
    files = job["files"]
    file_rows = job["file_rows"]
    carry_file = job.get("carry_file")
    carry_rows = int(job.get("carry_rows", 0))
    carry_file_rows = int(job.get("carry_file_rows", 0))
    K = int(job.get("harmonics_K", 75))
    out_dir = job["output_dir"]
    upload_url = job.get("upload_url")
    do_upload = bool(job.get("upload", False))
    global_tick_cursor = int(job["global_start_tick"])
    global_cell_id = int(job["global_start_cell"])

    os.makedirs(out_dir, exist_ok=True)
    os.makedirs(data_dir, exist_ok=True)

    download_files = job.get("download_files", [])
    data_url_base = job.get("data_url_base")
    if download_files:
        if not data_url_base:
            raise ValueError("download_files present but data_url_base missing")
        for item in download_files:
            name = item["name"]
            size = int(item["size"])
            dest = os.path.join(data_dir, name)
            if os.path.isfile(dest) and os.path.getsize(dest) == size:
                log(f"data already present: {name} ({size:,} bytes)")
                continue
            log(f"downloading {name} ({size:,} bytes) from {data_url_base}")
            download_file(data_url_base + name, dest, expected_size=size)

    run_started = time.time()
    progress = {
        "node": node,
        "status": "starting",
        "cells_done": 0,
        "ticks_done": 0,
        "files_done": 0,
        "files_total": len(files),
        "global_start_tick": global_tick_cursor,
        "global_start_cell": global_cell_id,
        "started_utc": time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime()),
    }
    if do_upload and upload_url:
        upload_json(progress, f"progress_{node}.json", upload_url)

    def progress_update(**kw):
        progress.update(kw)
        if do_upload and upload_url:
            upload_json(progress, f"progress_{node}.json", upload_url)

    carry_prices = np.empty(0, dtype=np.float64)
    carry_times = np.empty(0, dtype=np.int64)

    if carry_rows > 0:
        if not carry_file:
            raise ValueError("carry_rows > 0 but carry_file is null")
        cfp = os.path.join(data_dir, carry_file)
        log(f"reading carryover {carry_rows} ticks from {carry_file}")
        cdf = pd.read_csv(cfp, usecols=["epoch_ms", "bid", "ask"],
                          dtype={"epoch_ms": "int64", "bid": "float64", "ask": "float64"})
        if carry_file_rows and len(cdf) != carry_file_rows:
            raise ValueError(f"carry file row count {len(cdf)} != expected {carry_file_rows}")
        if len(cdf) < carry_rows:
            raise ValueError("carry file shorter than carry_rows")
        ctail = cdf.tail(carry_rows)
        carry_prices = ((ctail["bid"].to_numpy(np.float64) + ctail["ask"].to_numpy(np.float64)) / 2.0)
        carry_times = ctail["epoch_ms"].to_numpy(np.int64)
        del cdf, ctail
        log(f"carryover ready: {len(carry_prices)} ticks, first={carry_times[0]}, last={carry_times[-1]}")

    for old in os.listdir(out_dir):
        if old.startswith(f"chunk_{node}_") or old.startswith(f"summary_{node}"):
            try:
                os.remove(os.path.join(out_dir, old))
            except OSError:
                pass

    chunks = []
    k_indices_ref = None
    total_cells = 0
```

```
    total_ticks = 0
    max_tick_error_all = 0.0
    max_norm_rmse_all = 0.0
    cells_under_1e_11 = 0
    contiguity_ok = True
    rowcount_mismatches = []
    expected_tick = int(job["global_start_tick"])
    expected_cell = int(job["global_start_cell"])

    for fi, (fname, expected_rows) in enumerate(zip(files, file_rows), start=1):
        fp = os.path.join(data_dir, fname)
        t_file = time.time()
        log(f"[{fi}/{len(files)}] reading {fname} (expect {expected_rows:,} rows)")
        df = pd.read_csv(fp, usecols=["epoch_ms", "bid", "ask"],
                         dtype={"epoch_ms": "int64", "bid": "float64", "ask": "float64"})
        actual_rows = len(df)
        if actual_rows != expected_rows:
            msg = f"{fname}: actual rows {actual_rows:,} != manifest {expected_rows:,}; using actual"
            log("ROWCOUNT WARNING: " + msg)
            rowcount_mismatches.append({
                "file": fname, "actual": int(actual_rows), "manifest": int(expected_rows)
            })

        prices = ((df["bid"].to_numpy(np.float64) + df["ask"].to_numpy(np.float64)) / 2.0)
        times = df["epoch_ms"].to_numpy(np.int64)
        del df
        gc.collect()

        if len(carry_prices) > 0:
            prices = np.concatenate([carry_prices, prices])
            times = np.concatenate([carry_times, times])
            carry_prices = np.empty(0, dtype=np.float64)
            carry_times = np.empty(0, dtype=np.int64)

        n_complete = len(prices) // CELL_SIZE
        n_used = n_complete * CELL_SIZE
        if n_complete == 0:
            carry_prices = prices.copy()
            carry_times = times.copy()
            log(f"    no complete cell yet; carry={len(carry_prices)}")
            continue

        cell_prices = prices[:n_used].reshape(n_complete, CELL_SIZE)
        cell_times = times[:n_used].reshape(n_complete, CELL_SIZE)
        carry_prices = prices[n_used:].copy()
        carry_times = times[n_used:].copy()

        ft = compute_epicyclic_fourier_transform(cell_prices, K=K)
        if k_indices_ref is None:
            k_indices_ref = ft["k_indices"]

        cell_ids = np.arange(global_cell_id, global_cell_id + n_complete, dtype=np.int64)
        start_ticks = global_tick_cursor + np.arange(n_complete, dtype=np.int64) * CELL_SIZE
        end_ticks = start_ticks + (CELL_SIZE - 1)

        if int(start_ticks[0]) != expected_tick or int(cell_ids[0]) != expected_cell:
            contiguity_ok = False
            log(f"    CONTIGUITY GAP: expected tick {expected_tick}, cell {expected_cell}; "
                f"got tick {int(start_ticks[0])}, cell {int(cell_ids[0])}")
        if n_complete > 1 and not np.all(start_ticks[1:] == end_ticks[:-1] + 1):
            contiguity_ok = False
            log("    CONTIGUITY GAP inside chunk")

        chunk = pd.DataFrame({
            "cell_id": cell_ids,
            "cell_label": [f"Cell_{c}" for c in cell_ids],
            "tick_start_idx": start_ticks,
            "tick_end_idx": end_ticks,
            "time_open_ms": cell_times[:, 0],
            "time_close_ms": cell_times[:, -1],
            "p_open": ft["p_open"],
            "p_high": ft["p_high"],
            "p_low": ft["p_low"],
            "p_close": ft["p_close"],
            "p_mean": ft["p_mean"],
            "p_std": ft["p_std"],
            "y_open_norm": ft["y_open_norm"],
            "bridge_slope": ft["bridge_slope"],
            "rmse_residual": ft["rmse_residual"],
            "max_tick_error": ft["max_tick_error"],
            "norm_rmse": ft["norm_rmse"],
```

```
            "source_file": fname,
            "price_source": "mid(bid,ask)",
        })

        chunk_catalog_name = f"chunk_{node}_{fi:02d}_catalog.csv"
        chunk_npz_name = f"chunk_{node}_{fi:02d}_functions.npz"
        chunk_catalog_path = os.path.join(out_dir, chunk_catalog_name)
        chunk_npz_path = os.path.join(out_dir, chunk_npz_name)

        log(f"    writing {chunk_catalog_name} and {chunk_npz_name}")
        chunk.to_csv(chunk_catalog_path, index=False)
        np.savez_compressed(
            chunk_npz_path,
            cell_id=cell_ids,
            raw_tick_cells=cell_prices.astype(np.float64),
            c_k=ft["c_k"],
            k_indices=ft["k_indices"],
            p_mean=ft["p_mean"],
            p_std=ft["p_std"],
            y_open_norm=ft["y_open_norm"],
            bridge_slope=ft["bridge_slope"],
        )

        max_err = float(np.max(ft["max_tick_error"]))
        max_norm = float(np.max(ft["norm_rmse"]))
        under = int(np.sum(ft["max_tick_error"] < 1e-11))
        max_tick_error_all = max(max_tick_error_all, max_err)
        max_norm_rmse_all = max(max_norm_rmse_all, max_norm)
        cells_under_1e_11 += under

        chunks.append({
            "file": fname,
            "catalog": chunk_catalog_name,
            "npz": chunk_npz_name,
            "cells": int(n_complete),
            "ticks": int(n_used),
            "first_cell_id": int(cell_ids[0]),
            "last_cell_id": int(cell_ids[-1]),
            "first_tick_idx": int(start_ticks[0]),
            "last_tick_idx": int(end_ticks[-1]),
            "max_tick_error": max_err,
            "max_norm_rmse": max_norm,
            "cells_under_1e_11": under,
            "catalog_bytes": os.path.getsize(chunk_catalog_path),
            "npz_bytes": os.path.getsize(chunk_npz_path),
        })

        total_cells += n_complete
        total_ticks += n_used
        global_cell_id += n_complete
        global_tick_cursor += n_used
        expected_tick = int(end_ticks[-1]) + 1
        expected_cell = int(cell_ids[-1]) + 1

        log(f"    built {n_complete:,} cells | Cell_{cell_ids[0]}..Cell_{cell_ids[-1]} | "
            f"max_err={max_err:.3e} | carry={len(carry_prices)} | {time.time()-t_file:.1f}s")

        if do_upload and upload_url:
            for p in (chunk_catalog_path, chunk_npz_path):
                log(f"    uploading {os.path.basename(p)} ({os.path.getsize(p):,} bytes)")
                upload_file(p, upload_url + os.path.basename(p))

        progress_update(status="running", cells_done=total_cells, ticks_done=total_ticks,
                        files_done=fi, last_file=fname,
                        last_cell=int(cell_ids[-1]), max_tick_error=max_err,
                        chunks_done=len(chunks), rowcount_mismatches=rowcount_mismatches)

        del ft, chunk, cell_prices, cell_times, prices, times
        gc.collect()

    if total_cells == 0:
        raise RuntimeError("no complete cells built")
    if k_indices_ref is None:
        raise RuntimeError("no harmonic indices captured")

    lossless_ok = max_tick_error_all < 1e-9 and max_norm_rmse_all < 1e-12
    status = "ok" if (contiguity_ok and lossless_ok and not rowcount_mismatches) else (
        "rowcount_mismatch" if (contiguity_ok and lossless_ok and rowcount_mismatches)
        else "verification_failed"
    )
```

```
    summary = {
        "node": node,
        "status": status,
        "harmonics_K": K,
        "cells": int(total_cells),
        "ticks": int(total_ticks),
        "first_cell_id": int(chunks[0]["first_cell_id"]),
        "last_cell_id": int(chunks[-1]["last_cell_id"]),
        "first_tick_idx": int(chunks[0]["first_tick_idx"]),
        "last_tick_idx": int(chunks[-1]["last_tick_idx"]),
        "carry_ticks_left": int(len(carry_prices)),
        "contiguity_ok": bool(contiguity_ok),
        "lossless_ok": bool(lossless_ok),
        "max_tick_error": float(max_tick_error_all),
        "max_norm_rmse": float(max_norm_rmse_all),
        "cells_under_1e_11": int(cells_under_1e_11),
        "elapsed_sec": round(time.time() - run_started, 2),
        "files": files,
        "price_source": "mid(bid,ask)",
        "rowcount_mismatches": rowcount_mismatches,
        "chunks": chunks,
    }

    summary_path = os.path.join(out_dir, f"summary_{node}.json")
    json_dump(summary_path, summary)
    log("summary: " + json.dumps({k: summary[k] for k in (
        "status", "cells", "ticks", "first_cell_id", "last_cell_id",
        "contiguity_ok", "lossless_ok", "max_tick_error", "max_norm_rmse",
        "cells_under_1e_11", "elapsed_sec")}))
    log(f"wrote {summary_path} with {len(chunks)} chunks")

    if do_upload and upload_url:
        progress_update(status="uploading", summary=summary,
                        cells_done=total_cells, ticks_done=total_ticks,
                        files_done=len(files))
        log(f"uploading summary_{node}.json")
        upload_file(summary_path, upload_url + os.path.basename(summary_path))
        upload_json(progress, f"progress_{node}.json", upload_url)

    if not (contiguity_ok and lossless_ok):
        log("VERIFICATION FAILED")
        return 1
    if rowcount_mismatches:
        log("COMPLETE WITH ROWCOUNT WARNINGS")
        return 0
    log("PHASE 1 WORKER COMPLETE")
    return 0


if __name__ == "__main__":
    try:
        sys.exit(main())
    except Exception:
        traceback.print_exc()
        sys.exit(1)
```

The worker is the only place where the 150-tick cell boundary is defined. It reads raw ticks, validates continuity, writes the cell record and computes the 75-harmonic epicycle encoding. Every downstream phase depends on the invariant it establishes: cell `n+1` starts exactly one tick after cell `n` ends.

phase1/scripts/phase1_merge.py - node merge, seam audit, catalog assembly (300 lines, 11.6 KB)

```
import os
import sys
import json
import time
import shutil
import zipfile
import argparse
from pathlib import Path

import numpy as np
import pandas as pd
from numpy.lib.format import open_memmap, write_array_header_1_0, dtype_to_descr


def log(msg):
    print(f"[{time.strftime('%H:%M:%S')}] {msg}", flush=True)
```

```
def load_chunk_index(out_dir):
    rows = []
    for name in sorted(os.listdir(out_dir)):
        if not (name.startswith("chunk_") and name.endswith("_catalog.csv")):
            continue
        path = os.path.join(out_dir, name)
        df = pd.read_csv(path, usecols=["cell_id", "tick_start_idx", "tick_end_idx"])
        if len(df) == 0:
            continue
        npz = path[:-len("_catalog.csv")] + "_functions.npz"
        if not os.path.isfile(npz):
            raise FileNotFoundError(npz)
        rows.append({
            "catalog": path,
            "npz": npz,
            "cells": int(len(df)),
            "first_cell": int(df["cell_id"].iloc[0]),
            "last_cell": int(df["cell_id"].iloc[-1]),
            "first_tick": int(df["tick_start_idx"].iloc[0]),
            "last_tick": int(df["tick_end_idx"].iloc[-1]),
        })
    rows.sort(key=lambda r: r["first_cell"])
    return rows


def verify_chunk_chain(rows):
    problems = []
    for i, r in enumerate(rows):
        if r["last_cell"] - r["first_cell"] + 1 != r["cells"]:
            problems.append(f"cell count mismatch in {os.path.basename(r['catalog'])}")
        if r["last_tick"] - r["first_tick"] + 1 != r["cells"] * 150:
            problems.append(f"tick span mismatch in {os.path.basename(r['catalog'])}")
        if i > 0:
            p = rows[i - 1]
            if r["first_cell"] != p["last_cell"] + 1:
                problems.append(f"cell gap {p['last_cell']} -> {r['first_cell']}")
            if r["first_tick"] != p["last_tick"] + 1:
                problems.append(f"tick gap {p['last_tick']} -> {r['first_tick']}")
    return problems


def write_npz_streaming(npz_path, rows, catalog, temp_dir):
    os.makedirs(temp_dir, exist_ok=True)
    n = len(catalog)
    m = 150
    ck_path = os.path.join(temp_dir, "c_k.npy")
    raw_path = os.path.join(temp_dir, "raw_tick_cells.npy")
    rad_path = os.path.join(temp_dir, "radii.npy")
    pha_path = os.path.join(temp_dir, "phases.npy")

    ck_mm = open_memmap(ck_path, mode="w+", dtype=np.complex128, shape=(n, m))
    raw_mm = open_memmap(raw_path, mode="w+", dtype=np.float64, shape=(n, m))
    rad_mm = open_memmap(rad_path, mode="w+", dtype=np.float64, shape=(n, m))
    pha_mm = open_memmap(pha_path, mode="w+", dtype=np.float64, shape=(n, m))

    k_indices_ref = None
    p_mean = np.empty(n, dtype=np.float64)
    p_std = np.empty(n, dtype=np.float64)
    y_open = np.empty(n, dtype=np.float64)
    slope = np.empty(n, dtype=np.float64)
    cell_id = np.empty(n, dtype=np.int64)

    pos = 0
    for r in rows:
        with np.load(r["npz"], allow_pickle=False) as z:
            ids = z["cell_id"].astype(np.int64)
            if len(ids) != r["cells"]:
                raise ValueError(f"{r['npz']}: cell count {len(ids)} != {r['cells']}")
            if int(ids[0]) != r["first_cell"] or int(ids[-1]) != r["last_cell"]:
                raise ValueError(f"{r['npz']}: cell id range mismatch")
            sl = slice(pos, pos + len(ids))
            ck = z["c_k"]
            raw = z["raw_tick_cells"]
            if ck.shape != (len(ids), m) or raw.shape != (len(ids), m):
                raise ValueError(f"{r['npz']}: bad shapes ck={ck.shape} raw={raw.shape}")
            ck_mm[sl] = ck
            raw_mm[sl] = raw
            rad_mm[sl] = np.abs(ck)
            pha_mm[sl] = np.angle(ck)
```

```
            cell_id[sl] = ids
            p_mean[sl] = z["p_mean"]
            p_std[sl] = z["p_std"]
            y_open[sl] = z["y_open_norm"]
            slope[sl] = z["bridge_slope"]
            if k_indices_ref is None:
                k_indices_ref = z["k_indices"].astype(np.int64)
            pos += len(ids)
        log(f"  streamed {os.path.basename(r['npz'])} ({len(ids):,} cells)")

    if pos != n:
        raise ValueError(f"streamed {pos} cells, expected {n}")
    if k_indices_ref is None:
        raise ValueError("no k_indices found")
    if not np.array_equal(cell_id, catalog["cell_id"].to_numpy(np.int64)):
        raise ValueError("npz cell_id order does not match catalog order")

    for mm in (ck_mm, raw_mm, rad_mm, pha_mm):
        mm.flush()
    del ck_mm, raw_mm, rad_mm, pha_mm

    log(f"packing {npz_path}")
    with zipfile.ZipFile(npz_path, "w", compression=zipfile.ZIP_STORED, allowZip64=True) as zf:
        zf.write(ck_path, arcname="c_k.npy")
        zf.write(raw_path, arcname="raw_tick_cells.npy")
        zf.write(rad_path, arcname="radii.npy")
        zf.write(pha_path, arcname="phases.npy")

        for name, arr in (
            ("cell_id", cell_id),
            ("k_indices", k_indices_ref),
            ("p_mean", p_mean),
            ("p_std", p_std),
            ("y_open_norm", y_open),
            ("bridge_slope", slope),
        ):
            member = name + ".npy"
            with zf.open(member, "w", force_zip64=True) as fp:
                write_array_header_1_0(fp, {
                    "shape": arr.shape,
                    "fortran_order": False,
                    "descr": dtype_to_descr(arr.dtype),
                })
                fp.write(arr.tobytes(order="C"))

    for p in (ck_path, raw_path, rad_path, pha_path):
        try:
            os.remove(p)
        except OSError:
            pass


def write_excel(path, catalog, rows, n_rows):
    rows = list(rows)
    pieces = []
    raw_pieces = []
    got = 0
    for r in rows:
        take = min(r["cells"], n_rows - got)
        if take <= 0:
            break
        with np.load(r["npz"], allow_pickle=False) as z:
            pieces.append(z["c_k"][:take])
            raw_pieces.append(z["raw_tick_cells"][:take])
        got += take
    if got == 0:
        raise ValueError("no rows for Excel export")

    ck = np.vstack(pieces)
    raw = np.vstack(raw_pieces)
    meta = catalog.iloc[:got].copy()

    df_ticks = pd.DataFrame(raw, columns=[f"Tick_{i+1}" for i in range(raw.shape[1])])
    df_ticks.insert(0, "cell_label", meta["cell_label"].values)

    k_idx = np.load(rows[0]["npz"], allow_pickle=False)["k_indices"].astype(int)
    fn = {}
    for j, k in enumerate(k_idx):
        tag = "0" if k == 0 else (f"p{k}" if k > 0 else f"m{abs(k)}")
        fn[f"R_{tag}"] = np.abs(ck[:, j])
        fn[f"Phi_{tag}"] = np.angle(ck[:, j])
```

```
    df_fn = pd.DataFrame(fn)
    df_fn.insert(0, "cell_label", meta["cell_label"].values)

    log(f"writing Excel {path} ({got:,} rows, {df_fn.shape[1]} function columns)")
    with pd.ExcelWriter(path, engine="openpyxl") as writer:
        meta.to_excel(writer, sheet_name="Cell_Catalog", index=False)
        df_fn.to_excel(writer, sheet_name="Epicycle_Math_Functions", index=False)
        df_ticks.to_excel(writer, sheet_name="Raw_150_Tick_Cells", index=False)


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--out", default=r"C:\cluster\phase1\out")
    ap.add_argument("--final", default=r"C:\cluster\phase1\final")
    ap.add_argument("--excel-rows", type=int, default=10000)
    ap.add_argument("--skip-excel", action="store_true")
    ap.add_argument("--expected-first-cell", type=int, default=1)
    ap.add_argument("--expected-last-cell", type=int, default=591171)
    ap.add_argument("--expected-cells", type=int, default=591171)
    ap.add_argument("--skip-boundaries", action="store_true")
    args = ap.parse_args()

    os.makedirs(args.final, exist_ok=True)
    temp_dir = os.path.join(args.final, "_tmp_npy")
    started = time.time()

    rows = load_chunk_index(args.out)
    if not rows:
        raise RuntimeError(f"no chunks found in {args.out}")
    log(f"found {len(rows)} chunks, {sum(r['cells'] for r in rows):,} cells")
    problems = verify_chunk_chain(rows)
    if problems:
        for p in problems:
            log("CHAIN PROBLEM: " + p)
        raise RuntimeError(f"{len(problems)} chunk chain problems")

    catalog_parts = []
    for r in rows:
        catalog_parts.append(pd.read_csv(r["catalog"]))
    catalog = pd.concat(catalog_parts, ignore_index=True)
    catalog = catalog.sort_values("cell_id", kind="stable").reset_index(drop=True)
    del catalog_parts

    ids = catalog["cell_id"].to_numpy(np.int64)
    starts = catalog["tick_start_idx"].to_numpy(np.int64)
    ends = catalog["tick_end_idx"].to_numpy(np.int64)
    if ids[0] != args.expected_first_cell:
        raise ValueError(f"first cell {ids[0]} != {args.expected_first_cell}")
    if ids[-1] != args.expected_last_cell:
        raise ValueError(f"last cell {ids[-1]} != {args.expected_last_cell}")
    if len(catalog) != args.expected_cells:
        raise ValueError(f"cell count {len(catalog)} != {args.expected_cells}")
    if not np.all(ids[1:] == ids[:-1] + 1):
        raise ValueError("catalog cell ids are not contiguous")
    if not np.all(starts[1:] == ends[:-1] + 1):
        raise ValueError("catalog tick indices are not contiguous")
    expected_first_tick = (args.expected_first_cell - 1) * 150 + 1
    if starts[0] != expected_first_tick:
        raise ValueError(f"first tick {starts[0]} != {expected_first_tick}")

    expected_last_tick = int(ids[-1]) * 150
    tail = int(catalog["tick_end_idx"].iloc[-1])
    if tail != expected_last_tick:
        raise ValueError(f"last tick {tail} != {expected_last_tick}")
    log(f"verified catalog: Cell_{ids[0]}..Cell_{ids[-1]}, ticks {starts[0]}..{tail}")

    parquet_path = os.path.join(args.final, "jump10_phase1_cell_catalog.parquet")
    log(f"writing {parquet_path}")
    catalog.to_parquet(parquet_path, index=False, engine="fastparquet")

    npz_path = os.path.join(args.final, "jump10_phase1_fourier_functions.npz")
    write_npz_streaming(npz_path, rows, catalog, temp_dir)

    excel_path = os.path.join(args.final, "jump10_phase1_inspection.xlsx")
    if not args.skip_excel:
        write_excel(excel_path, catalog, rows, args.excel_rows)

    report = {
        "status": "ok",
        "cells": int(len(catalog)),
        "ticks": int(tail),
```

```
        "first_cell_id": int(ids[0]),
        "last_cell_id": int(ids[-1]),
        "first_tick_idx": int(starts[0]),
        "last_tick_idx": int(tail),
        "chunks": len(rows),
        "max_tick_error": float(catalog["max_tick_error"].max()),
        "max_norm_rmse": float(catalog["norm_rmse"].max()),
        "cells_under_1e_11": int((catalog["max_tick_error"] < 1e-11).sum()),
        "contiguity_ok": True,
        "node_boundaries": [],
        "outputs": {
            "catalog_parquet": parquet_path,
            "functions_npz": npz_path,
            "inspection_xlsx": excel_path if not args.skip_excel else None,
        },
        "elapsed_sec": round(time.time() - started, 2),
    }

    if not args.skip_boundaries:
        for cid in (171621, 381378):
            row = catalog.loc[catalog["cell_id"] == cid]
            nxt = catalog.loc[catalog["cell_id"] == cid + 1]
            if len(row) and len(nxt):
                report["node_boundaries"].append({
                    "left_cell": int(cid),
                    "left_last_tick": int(row["tick_end_idx"].iloc[0]),
                    "right_cell": int(cid + 1),
                    "right_first_tick": int(nxt["tick_start_idx"].iloc[0]),
                    "ok": bool(int(nxt["tick_start_idx"].iloc[0]) == int(row["tick_end_idx"].iloc[0]) + 1),
                })

    report_path = os.path.join(args.final, "phase1_verification.json")
    with open(report_path, "w", encoding="utf-8") as f:
        json.dump(report, f, indent=2)
    log("verification: " + json.dumps(report, indent=2))
    log("PHASE 1 MERGE COMPLETE")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

The merger is also the seam auditor. The two internal node boundaries (cells 171,621|171,622 and 381,378|381,379) are checked for tick continuity explicitly, because a corrupted seam would silently create a fake 150-tick "anomaly" at two points in the corpus and nothing downstream would notice.

phase1/scripts/verify_phase1.py - independent reconstruction verifier (118 lines, 4.9 KB)

```
import argparse
import json
import os
import zipfile

import numpy as np
import pandas as pd
from openpyxl import load_workbook


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--final", default=r"C:\cluster\phase1\final")
    args = ap.parse_args()

    npz_path = os.path.join(args.final, "jump10_phase1_fourier_functions.npz")
    parquet_path = os.path.join(args.final, "jump10_phase1_cell_catalog.parquet")
    xlsx_path = os.path.join(args.final, "jump10_phase1_inspection.xlsx")

    report = {"status": "ok", "checks": {}}

    with zipfile.ZipFile(npz_path) as zf:
        bad = zf.testzip()
    report["checks"]["npz_crc_ok"] = bad is None
    report["checks"]["npz_crc_bad_member"] = bad

    catalog = pd.read_parquet(parquet_path, engine="fastparquet")
    n = len(catalog)
    expected = np.arange(1, n + 1, dtype=np.int64)
    first = catalog["tick_start_idx"].to_numpy(np.int64)
```

```
last = catalog["tick_end_idx"].to_numpy(np.int64)
report["checks"]["catalog_rows"] = n
report["checks"]["catalog_columns"] = int(catalog.shape[1])
report["checks"]["cell_ids_contiguous"] = bool(np.array_equal(catalog["cell_id"].to_numpy(np.int64), expected))
report["checks"]["cell_labels_match"] = bool(
    np.array_equal(catalog["cell_label"].astype(str).to_numpy(), np.char.add("Cell_", expected.astype(str)))
)
report["checks"]["tick_ranges_contiguous"] = bool(
    np.all(first == 1 + 150 * np.arange(n, dtype=np.int64))
    and np.all(last == 150 * np.arange(1, n + 1, dtype=np.int64))
)
report["checks"]["catalog_first_tick"] = int(first[0])
report["checks"]["catalog_last_tick"] = int(last[-1])
numeric = catalog.select_dtypes(include=[np.number])
report["checks"]["catalog_all_finite"] = bool(np.isfinite(numeric.to_numpy()).all())

sample = np.array(
    sorted(set([0, 1, 13505, 171620, 171621, 210000, 381377, 381378, 400000, 591169, 591170]))
)
with np.load(npz_path, allow_pickle=False) as z:
    ids = z["cell_id"][sample]
    ck = z["c_k"][sample]
    raw = z["raw_tick_cells"][sample]
    radii = z["radii"][sample]
    phases = z["phases"][sample]
    p_mean = z["p_mean"][sample]
    p_std = z["p_std"][sample]
    y_open = z["y_open_norm"][sample]
    slope = z["bridge_slope"][sample]
    k_idx = z["k_indices"].astype(np.int64)
    report["checks"]["npz_cell_id_count"] = int(z["cell_id"].shape[0])
    report["checks"]["npz_cell_id_ok"] = bool(np.array_equal(z["cell_id"].astype(np.int64), expected))

t = np.linspace(0.0, 1.0, 150, dtype=np.float64)[None, :]
z_rec = np.fft.ifft(ck * 150.0, axis=1)
y_norm_rec = z_rec.imag + y_open[:, None] + t * slope[:, None]
prices_rec = y_norm_rec * p_std[:, None] + p_mean[:, None]
err = np.abs(prices_rec - raw)
report["checks"]["sample_rows"] = sample.tolist()
report["checks"]["sample_cell_ids"] = ids.astype(int).tolist()
report["checks"]["sample_max_abs_error"] = float(err.max())
report["checks"]["sample_max_norm_rmse"] = float(
    np.sqrt(np.mean((prices_rec - raw) ** 2 / (p_std[:, None] ** 2), axis=1)).max()
)
report["checks"]["sample_all_finite"] = bool(
    np.isfinite(ck).all()
    and np.isfinite(raw).all()
    and np.isfinite(radii).all()
    and np.isfinite(phases).all()
    and np.isfinite(prices_rec).all()
)
report["checks"]["radii_match"] = bool(np.allclose(radii, np.abs(ck), rtol=0.0, atol=0.0))
report["checks"]["phases_match"] = bool(np.allclose(phases, np.angle(ck), rtol=0.0, atol=0.0))
report["checks"]["k_indices_ok"] = bool(np.array_equal(k_idx, np.fft.fftfreq(150, d=1.0 / 150).astype(np.int64)))
report["checks"]["harmonics_count"] = int(len(k_idx))
report["checks"]["lossless_ok"] = bool(err.max() < 1e-11)

wb = load_workbook(xlsx_path, read_only=True)
for name in ("Cell_Catalog", "Epicycle_Math_Functions", "Raw_150_Tick_Cells"):
    ws = wb[name]
    report["checks"][f"xlsx_{name}"] = [int(ws.max_row), int(ws.max_column)]
wb.close()

report["outputs"] = {
    "catalog_parquet_bytes": os.path.getsize(parquet_path),
    "functions_npz_bytes": os.path.getsize(npz_path),
    "inspection_xlsx_bytes": os.path.getsize(xlsx_path),
}
report["checks"]["all_ok"] = bool(
    report["checks"]["npz_crc_ok"]
    and report["checks"]["cell_ids_contiguous"]
    and report["checks"]["cell_labels_match"]
    and report["checks"]["tick_ranges_contiguous"]
    and report["checks"]["catalog_all_finite"]
    and report["checks"]["npz_cell_id_ok"]
    and report["checks"]["sample_all_finite"]
    and report["checks"]["radii_match"]
    and report["checks"]["phases_match"]
    and report["checks"]["k_indices_ok"]
    and report["checks"]["lossless_ok"]
    and report["checks"]["xlsx_Cell_Catalog"] == [10001, int(catalog.shape[1] + 0)]
```

```
    )
    print(json.dumps(report, indent=2))


if __name__ == "__main__":
    main()
```

A.2 Phase 2 - relational graph, vault and models

phase2/pqcompat.py - parquet shim (duckdb -> pyarrow -> fastparquet) (171 lines, 5.1 KB)

```
"""
Parquet compatibility shim for this machine.

Situation (2026-10-27):
  * pyarrow's _compute.pyd is blocked by an Application Control policy.
  * fastparquet's cencoding.pyd is now blocked too (it worked for a while).
  * duckdb is NOT blocked and ships its own parquet reader/writer -> preferred.

Order of preference: duckdb -> pyarrow -> fastparquet (with the pure-python
cramjam stub so UNCOMPRESSED parquet still works where duckdb is missing).
"""
import sys
import os
import gzip
import types
import sqlite3


def _install_stub():
    if "cramjam" in sys.modules:
        return
    mod = types.ModuleType("cramjam")

    class Buffer(bytearray):
        pass

    def _unsupported(name):
        def _f(*a, **k):
            raise NotImplementedError(
                f"cramjam.{name} unavailable: the cramjam DLL is blocked by "
                f"Application Control on this host. Write parquet with compression=None."
            )
        return _f

    class _Gzip:
        @staticmethod
        def compress(data, level=6, **kw):
            return gzip.compress(bytes(data), compresslevel=int(level or 6))

        @staticmethod
        def decompress(data, output_len=None, **kw):
            return gzip.decompress(bytes(data))

        @staticmethod
        def decompress_into(data, out):
            b = gzip.decompress(bytes(data))
            mv = memoryview(out)
            mv[:len(b)] = b
            return len(b)

    class _Snappy:
        compress_raw = staticmethod(_unsupported("snappy.compress_raw"))
        decompress_raw = staticmethod(_unsupported("snappy.decompress_raw"))
        decompress_raw_into = staticmethod(_unsupported("snappy.decompress_raw_into"))
        compress = staticmethod(_unsupported("snappy.compress"))
        decompress = staticmethod(_unsupported("snappy.decompress"))

    class _Brotli:
        compress = staticmethod(_unsupported("brotli.compress"))
        decompress = staticmethod(_unsupported("brotli.decompress"))
        decompress_into = staticmethod(_unsupported("brotli.decompress_into"))

    class _Lz4:
        compress = staticmethod(_unsupported("lz4.compress"))
        decompress = staticmethod(_unsupported("lz4.decompress"))
        compress_block = staticmethod(_unsupported("lz4.compress_block"))
        decompress_block = staticmethod(_unsupported("lz4.decompress_block"))
```

```
    class _Zstd:
        compress = staticmethod(_unsupported("zstd.compress"))
        decompress = staticmethod(_unsupported("zstd.decompress"))
        decompress_into = staticmethod(_unsupported("zstd.decompress_into"))

    mod.Buffer = Buffer
    mod.gzip = _Gzip
    mod.snappy = _Snappy
    mod.brotli = _Brotli
    mod.lz4 = _Lz4
    mod.zstd = _Zstd
    sys.modules["cramjam"] = mod


def _duckdb():
    import duckdb
    return duckdb


def import_fastparquet():
    try:
        import fastparquet  # noqa: F401
        return fastparquet
    except ImportError as e:
        if "cramjam" not in str(e):
            raise
        _install_stub()
        import fastparquet  # noqa: F401
        return fastparquet


def engine():
    """'duckdb' | 'pyarrow' | 'fastparquet' | None"""
    try:
        _duckdb()
        return "duckdb"
    except Exception:
        pass
    try:
        import pyarrow  # noqa: F401
        return "pyarrow"
    except Exception:
        pass
    try:
        import_fastparquet()
        return "fastparquet"
    except Exception:
        return None


def _q(path):
    return "'" + str(path).replace("'", "''") + "'"


def write_parquet(df, path):
    path = os.path.abspath(str(path))
    if os.path.exists(path):
        os.remove(path)
    eng = engine()
    if eng == "duckdb":
        con = _duckdb().connect()
        try:
            con.register("_df", df)
            con.execute(f"COPY _df TO {_q(path)} (FORMAT PARQUET, COMPRESSION SNAPPY)")
        finally:
            con.close()
    elif eng == "pyarrow":
        df.to_parquet(path, index=False, engine="pyarrow")
    elif eng == "fastparquet":
        df.to_parquet(path, index=False, engine="fastparquet", compression=None)
    else:
        raise RuntimeError("no usable parquet engine (pyarrow/fastparquet blocked, duckdb missing)")
    return eng


def read_parquet(path):
    path = os.path.abspath(str(path))
    eng = engine()
    if eng == "duckdb":
        con = _duckdb().connect()
```

```
        try:
            return con.execute(f"SELECT * FROM read_parquet({_q(path)})").df()
        finally:
            con.close()
    if eng == "pyarrow":
        import pandas as pd
        return pd.read_parquet(path, engine="pyarrow")
    if eng == "fastparquet":
        import pandas as pd
        return pd.read_parquet(path, engine="fastparquet")
    raise RuntimeError("no usable parquet engine")


if __name__ == "__main__":
    import pandas as pd
    import numpy as np
    print("engine:", engine())
    df = pd.DataFrame({"a": np.arange(10), "b": np.linspace(0, 1, 10), "c": list("abcdefghij")})
    p = r"C:\cluster\phase2\_pq_test.parquet"
    print("write ->", write_parquet(df, p))
    back = read_parquet(p)
    print(back.head(3).to_string())
    print("roundtrip ok:", back.shape == df.shape)
```

This 143-line file is why the project runs at all on the master machine. Application Control blocks PyArrow's native modules and fastparquet's `cencoding.pyd`; duckdb is the only working engine there, and it is absent on Kali. Every other script imports this shim instead of pandas' parquet readers.

phase2/phase2_step1_zone_and_covariant_db.py - quartile cut, descriptor bank, hub selection (335 lines, 14.0 KB)

```
"""
PHASE 2 - STEP 1 (v2)
Structural zone decomposer + distributed relational covariant scanner.

v2 fixes (post phase2_mathematicalfix.md):
  * reads the repaired/whitened cache with mmap (never loads 2.6 GB into RAM)
  * top-K capped peer evaluation  -> O(N*D) GEMM + O(K*D) per row, no more
    materialising 500k-element peer sets or R_mat[peers] copies
  * growable numpy edge buffers instead of multi-million python tuples
  * parquet via pyarrow when available, else fastparquet
  * periodic checkpointing + ETA so a long shard can be resumed/monitored
"""
import os
import sys
import json
import time
import argparse

for _v in ("OMP_NUM_THREADS", "MKL_NUM_THREADS", "OPENBLAS_NUM_THREADS",
           "NUMEXPR_NUM_THREADS", "VECLIB_MAXIMUM_THREADS", "BLIS_NUM_THREADS"):
    os.environ.setdefault(_v, "8")

import numpy as np
import pandas as pd

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

FIB_TICKS = [35, 57, 75, 93, 118]


# ---------------------------------------------------------------------------
# parquet helper (pyarrow is blocked by Application Control on this host)
# ---------------------------------------------------------------------------
import pqcompat

ENGINE = pqcompat.engine()


def write_parquet(df, path):
    if ENGINE is not None:
        pqcompat.write_parquet(df, path)
    else:
        df.to_csv(path.replace(".parquet", ".csv"), index=False)
    return ENGINE


# ---------------------------------------------------------------------------
# zone extraction (kept for provenance / cold start; not used when cache exists)
```

```
# ----------------------------------------------------------------------------
def zone_whitened_fft(y_slice, k_max=12):
    L = y_slice.shape[1]
    if L < 4:
        return np.zeros((y_slice.shape[0], max(1, k_max * 2 + 3)), dtype=np.float32)
    t_lin = np.linspace(0.0, 1.0, L, dtype=np.float32)[None, :]
    slope = y_slice[:, -1:] - y_slice[:, 0:1]
    bridge = y_slice - (y_slice[:, 0:1] + t_lin * slope)
    fft_c = np.fft.rfft(bridge, axis=1)[:, 1:k_max + 1] / max(L, 1)
    k_weights = np.arange(1, fft_c.shape[1] + 1, dtype=np.float32)[None, :]
    fft_w = fft_c * k_weights
    vec = np.hstack([fft_w.real, fft_w.imag, slope.astype(np.float32)]).astype(np.float32)
    norms = np.linalg.norm(vec, axis=1, keepdims=True)
    norms = np.where(norms < 1e-9, 1.0, norms)
    return vec / norms


# ----------------------------------------------------------------------------
# distributed relational covariant scan
# ----------------------------------------------------------------------------
def _boundaries(N, weights):
    b = [0]
    s = 0.0
    for w in weights:
        s += w
        b.append(int(round(N * s)))
    b[-1] = N
    return b


class EdgeBuffer:
    """Growable (n,4) float32 edge store - no multi-million python tuples."""

    def __init__(self, cap_hint=1_000_000):
        self.blocks = []
        self.n = 0
        self.block = np.empty((cap_hint, 4), dtype=np.float32)
        self.fill = 0

    def add(self, src, dst, cov, lock):
        if self.fill >= self.block.shape[0]:
            self.blocks.append(self.block[:self.fill])
            self.block = np.empty((max(65536, self.block.shape[0]), 4), dtype=np.float32)
            self.fill = 0
        self.block[self.fill] = (src, dst, cov, lock)
        self.fill += 1
        self.n += 1

    def array(self):
        parts = self.blocks + [self.block[:self.fill]]
        if not parts:
            return np.empty((0, 4), dtype=np.float32)
        return np.vstack(parts)


def scan_relational_covariant_shard(
    cache_path,
    node_id=0,
    num_nodes=3,
    sim_threshold=0.85,
    min_covariant_peers=12,
    chunk_size=256,
    top_peers=512,
    max_edges=24,
    output_dir="C:/cluster/phase2/shards",
    checkpoint_secs=120,
    max_rows=None,
    row_start_override=None,
    row_end_override=None,
):
    os.makedirs(output_dir, exist_ok=True)
    z = np.load(cache_path, mmap_mode="r")
    keys = set(z.files)

    if "trigger_vec_dense" in keys:
        T_mat = z["trigger_vec_dense"]
        R_mat = z["resolution_vec_dense"]
        mode = "dense(whitened)"
    else:
        T_mat = z["trigger_vec"]
        R_mat = z["resolution_vec"]
```

```
    mode = "fft(whitened)"

N, D = T_mat.shape
held_all = z["protected_held"]
pol_all = z["ext_polarity"]
ret_all = z["q4_net_return"]
tick_all = z["ext_tick"]
bico_key = "phase_closure_abs_cos" if "phase_closure_abs_cos" in keys else "bicoherence"
bico_all = z[bico_key]

if row_start_override is not None and row_end_override is not None:
    row_start, row_end = int(row_start_override), int(row_end_override)
else:
    weights = [0.40, 0.30, 0.30] if num_nodes == 3 else [1.0 / num_nodes] * num_nodes
    bounds = _boundaries(N, weights)
    row_start, row_end = bounds[node_id], bounds[node_id + 1]
if max_rows:
    row_end = min(row_end, row_start + max_rows)

print(f"[*] Node {node_id}/{num_nodes}: rows {row_start:,}..{row_end:,} of {N:,} "
      f"| T={T_mat.shape} D={D} | thr={sim_threshold} min_peers={min_covariant_peers} "
      f"top_peers={top_peers} chunk={chunk_size} | mode={mode} | engine={ENGINE}", flush=True)

T_T = np.asarray(T_mat).T.copy()  # (D,N) contiguous for fast GEMM
node_rows = []   # list of tuples (compact)
edges = EdgeBuffer()

n_pass = 0
n_scanned = 0
t_start = time.time()
t_ckpt = time.time()
stats = {"max_abs_r": 0.0, "max_abs_r_cell": 0, "max_lock": -2.0, "max_lock_cell": 0,
         "max_abs_r_edge": 0.0, "edges": 0, "anomalies": 0}
n_chunks = (row_end - row_start + chunk_size - 1) // chunk_size
chunk_i = 0

def checkpoint(tag="partial"):
    if not node_rows:
        return
    cols = ["anchor_cell_id", "covariant_peer_count", "peers_evaluated",
            "pos_corr_peers", "neg_mirror_peers", "mean_0_50_similarity",
            "max_0_50_similarity", "mean_50_100_lock_corr", "frac_75_100_exact_lock",
            "protected_ext_win_rate", "directional_50_100_win_rate",
            "direction_consistency", "anomaly_z_score", "bicoherence_phase_lock",
            "target_ext_tick_median", "recommended_action"]
    dfn = pd.DataFrame.from_records(node_rows, columns=cols)
    write_parquet(dfn, os.path.join(output_dir, f"covariant_anomalies_node{node_id}_{tag}.parquet"))
    e = edges.array()
    dfe = pd.DataFrame({"src_idx": e[:, 0].astype(np.int64),
                        "dst_idx": e[:, 1].astype(np.int64),
                        "cov_0_50": e[:, 2], "lock_50_100": e[:, 3]})
    write_parquet(dfe, os.path.join(output_dir, f"covariant_edges_node{node_id}_{tag}.parquet"))
    with open(os.path.join(output_dir, f"covariant_stats_node{node_id}_{tag}.json"), "w") as f:
        json.dump({"node": node_id, "rows_done": n_scanned, "row_end": row_end,
                   "anomalies": n_pass, "source": mode, "threshold": sim_threshold,
                   **stats}, f, indent=2)

for i0 in range(row_start, row_end, chunk_size):
    i1 = min(i0 + chunk_size, row_end)
    Q = np.asarray(T_mat[i0:i1], dtype=np.float32)
    sim = Q @ T_T  # (rows_in_chunk, N) float32

    for lr in range(i1 - i0):
        g = i0 + lr
        row = sim[lr]
        lo, hi = max(0, g - 2), min(N, g + 3)
        row[lo:hi] = 0.0
        absr = np.abs(row)
        cand = np.flatnonzero(absr >= sim_threshold)
        n_cand = int(cand.size)
        if n_cand < min_covariant_peers:
            continue

        if n_cand > top_peers:
            part = cand[np.argpartition(absr[cand], -top_peers)[-top_peers:]]
            sel = part[np.argsort(absr[part])[::-1]]
        else:
            sel = cand[np.argsort(absr[cand])[::-1]]
        k = sel.size

        signed = row[sel]
```

```
            pol = np.sign(signed)
            z_res = R_mat[g]
            y_res = np.asarray(R_mat[sel], dtype=np.float32) * pol[:, None]
            cont = y_res @ np.asarray(z_res, dtype=np.float32)  # (k,)

            mean_cont = float(cont.mean())
            frac_lock = float(np.mean(cont >= 0.65))
            p_out = np.asarray(held_all[sel])
            p_dirs = np.asarray(pol_all[sel], dtype=np.int32) * pol
            dir_cons = float(abs(p_dirs.mean()))
            prot_win = float(p_out.mean())
            aligned_ret = np.asarray(ret_all[sel]) * pol
            med_ret = float(np.median(aligned_ret))
            dir_win = float(np.mean(np.sign(aligned_ret + 1e-18) == np.sign(med_ret + 1e-18)))
            edge_eff = max(prot_win, dir_win * dir_cons)
            z_sc = (edge_eff - 0.50) * np.sqrt(k) / 0.50
            mean_sim = float(absr[sel].mean())
            max_sim = float(absr[sel].max())

            stats["max_abs_r"] = max(stats["max_abs_r"], max_sim)
            if max_sim >= stats["max_abs_r"]:
                stats["max_abs_r_cell"] = int(g + 1)
            if mean_cont > stats["max_lock"]:
                stats["max_lock"] = mean_cont
                stats["max_lock_cell"] = int(g + 1)

            if edge_eff >= 0.53 or mean_cont >= 0.30:
                node_rows.append((
                    int(g + 1), n_cand, int(k),
                    int(np.sum(pol > 0)), int(np.sum(pol < 0)),
                    round(mean_sim, 4), round(max_sim, 4),
                    round(mean_cont, 4), round(frac_lock, 4),
                    round(prot_win, 4), round(dir_win, 4), round(dir_cons, 4),
                    round(float(z_sc), 3), round(float(bico_all[g]), 4),
                    int(np.median(np.asarray(tick_all[sel]))),
                    "SELL_PROTECTED_HIGH" if float(p_dirs.mean()) < 0 else "BUY_PROTECTED_LOW",
                ))
                n_pass += 1
                for j in range(min(max_edges, k)):
                    src, dst = int(g), int(sel[j])
                    if src == dst:
                        continue
                    if src > dst:
                        src, dst = dst, src
                    lock = float(cont[j])
                    edges.add(src, dst, float(signed[j]), lock)
                    if abs(float(signed[j])) > stats["max_abs_r_edge"]:
                        stats["max_abs_r_edge"] = abs(float(signed[j]))

        n_scanned += (i1 - i0)
        chunk_i += 1
        now = time.time()
        if now - t_ckpt >= checkpoint_secs:
            stats["anomalies"] = n_pass
            stats["edges"] = edges.n
            checkpoint()
            t_ckpt = now

        if chunk_i % 5 == 0 or chunk_i == n_chunks:
            el = time.time() - t_start
            rate = n_scanned / max(el, 1e-9)
            eta = (row_end - i1) / max(rate, 1e-9)
            print(f"    -> node {node_id}: {i1:,}/{row_end:,} rows "
                  f"({100.0 * (i1 - row_start) / max(1, row_end - row_start):5.1f}%) "
                  f"| {rate:,.0f} rows/s | elapsed {el/60:.1f}m | eta {eta/60:.1f}m "
                  f"| anomalies {n_pass:,} | edges {edges.n:,}", flush=True)

    stats["anomalies"] = n_pass
    stats["edges"] = edges.n
    checkpoint("final")

    # compact final artifacts (also overwrite the unsuffixed names used downstream)
    cols = ["anchor_cell_id", "covariant_peer_count", "peers_evaluated",
            "pos_corr_peers", "neg_mirror_peers", "mean_0_50_similarity",
            "max_0_50_similarity", "mean_50_100_lock_corr", "frac_75_100_exact_lock",
            "protected_ext_win_rate", "directional_50_100_win_rate",
            "direction_consistency", "anomaly_z_score", "bicoherence_phase_lock",
            "target_ext_tick_median", "recommended_action"]
    dfn = pd.DataFrame.from_records(node_rows, columns=cols)
    e = edges.array()
    dfe = pd.DataFrame({"src_idx": e[:, 0].astype(np.int64),
```

```
                        "dst_idx": e[:, 1].astype(np.int64),
                        "cov_0_50": e[:, 2], "lock_50_100": e[:, 3]})
    write_parquet(dfn, os.path.join(output_dir, f"covariant_anomalies_node{node_id}.parquet"))
    write_parquet(dfe, os.path.join(output_dir, f"covariant_edges_node{node_id}.parquet"))
    with open(os.path.join(output_dir, f"covariant_stats_node{node_id}.json"), "w") as f:
        json.dump({"node": node_id, "rows_scanned": n_scanned, "row_start": row_start,
                   "row_end": row_end, "anomalies": n_pass, "source": mode,
                   "threshold": sim_threshold, "top_peers": top_peers,
                   "minutes": round((time.time() - t_start) / 60.0, 2), **stats}, f, indent=2)
    print(f"[+] node {node_id} done: {n_pass:,} anomaly hubs, {edges.n:,} edges, "
          f"{(time.time() - t_start)/60:.1f} min -> {output_dir}", flush=True)


if __name__ == "__main__":
    p = argparse.ArgumentParser()
    p.add_argument("--npz_path", type=str, default="C:/cluster/phase2/jump10_phase2_subzones.npz")
    p.add_argument("--out_dir", type=str, default="C:/cluster/phase2")
    p.add_argument("--node_id", type=int, default=0)
    p.add_argument("--num_nodes", type=int, default=3)
    p.add_argument("--sim_threshold", type=float, default=0.85)
    p.add_argument("--min_peers", type=int, default=12)
    p.add_argument("--top_peers", type=int, default=512)
    p.add_argument("--max_edges", type=int, default=24)
    p.add_argument("--chunk_size", type=int, default=256)
    p.add_argument("--checkpoint_secs", type=int, default=120)
    p.add_argument("--max_rows", type=int, default=0)
    p.add_argument("--row_start", type=int, default=-1)
    p.add_argument("--row_end", type=int, default=-1)
    a = p.parse_args()

    if not os.path.exists(a.npz_path):
        print(f"[-] cache not found: {a.npz_path}", flush=True)
        sys.exit(1)
    scan_relational_covariant_shard(
        a.npz_path,
        node_id=a.node_id, num_nodes=a.num_nodes,
        sim_threshold=a.sim_threshold, min_covariant_peers=a.min_peers,
        chunk_size=a.chunk_size, top_peers=a.top_peers, max_edges=a.max_edges,
        output_dir=os.path.join(a.out_dir, "shards"),
        checkpoint_secs=a.checkpoint_secs,
        max_rows=(a.max_rows or None),
        row_start_override=(a.row_start if a.row_start >= 0 else None),
        row_end_override=(a.row_end if a.row_end >= 0 else None),
    )
```

This is where the 64-dimensional dense descriptor is built and where the hub rule lives: an inbound cell is a hub if its peer count at |r| ≥ 0.85 puts its anomaly z-score in the top 5,000 of the corpus.

phase2/phase2_dense_anomaly_bank.py - the six descriptor families (320 lines, 14.2 KB)

```
import numpy as np
from scipy.signal import hilbert

try:
    import torch
    import torch.nn as nn
    import torch.nn.functional as F
    TORCH_OK = True
except Exception:
    torch = None
    nn = None
    F = None
    TORCH_OK = False


FEATURE_NAMES = [
    "s_y", "int_t_dy", "int_y_dt", "levy_area", "quad_var", "cubic_var", "int_y2_dt", "int_t2_dy", "lead_lag_cov",
    "rqa_rr", "rqa_det", "rqa_lam", "rqa_diag_entropy",
    "ord_p0", "ord_p1", "ord_p2", "ord_p3", "ord_p4", "ord_p5", "perm_entropy", "forbidden_divergence",
    "arc_length", "elastic_energy", "mean_signed_curvature", "max_abs_curvature", "inflection_rate", "total_turning",
    "scat_s1_mean", "scat_s1_std", "scat_s1_2nd", "scat_s2_mean", "scat_s2_std", "scat_s2_2nd",
    "scat_s4_mean", "scat_s4_std", "scat_s4_2nd", "scat_s8_mean", "scat_s8_std", "scat_s8_2nd",
    "h0_max_gap", "h0_second_gap", "h0_persistence_entropy", "h1_loop_closure",
    "hurst_q1", "hurst_q2", "multifractal_width", "katz_dim", "higuchi_proxy",
    "hilbert_freq_mean", "hilbert_freq_std", "hilbert_amp_kurt",
    "hvg_edge_density", "hvg_degree_mean", "hvg_degree_std", "hvg_degree_max",
    "cumulant4", "trispectrum_mag", "trispectrum_phase",
    "emd_extrema_rate", "emd_envelope_asymmetry", "emd_zero_cross_rate",
```

```
    "dist_skew", "dist_kurtosis", "range_ratio",
]

N_FEATURES = len(FEATURE_NAMES)
assert N_FEATURES == 64, N_FEATURES


def _safe_div(a, b, eps=1e-9):
    return a / (b + eps)


def _entropy_from_counts(counts, axis=-1):
    total = np.sum(counts, axis=axis, keepdims=True)
    p = counts / np.maximum(total, 1e-9)
    return -np.sum(p * np.log(p + 1e-9), axis=axis, keepdims=True)


def _hvg_features(y):
    n, length = y.shape
    degree = np.zeros((n, length), dtype=np.float64)
    total_edges = np.zeros(n, dtype=np.float64)
    possible = 0
    max_lag = min(8, length - 1)
    for d in range(1, max_lag + 1):
        width = length - d + 1
        if d == 1:
            edge = np.ones((n, width), dtype=bool)
        else:
            mids = [y[:, i:i + width] for i in range(1, d)]
            middle_max = np.maximum.reduce(mids)
            edge = middle_max < np.minimum(y[:, :width], y[:, d - 1:])
        total_edges += edge.sum(axis=1)
        degree[:, :width] += edge
        degree[:, d - 1:] += edge
        possible += width
    possible = max(possible, 1)
    return np.column_stack([
        total_edges / possible,
        degree.mean(axis=1),
        degree.std(axis=1),
        degree.max(axis=1),
    ])


def _rqa_features(y):
    n, length = y.shape
    v0 = y[:, :-4]
    v1 = y[:, 2:-2]
    v2 = y[:, 4:]
    if v0.shape[1] < 4:
        rr = np.zeros((n, 1))
        det = np.zeros((n, 1))
        lam = np.zeros((n, 1))
        diag_entropy = np.zeros((n, 1))
        h1 = np.zeros((n, 1))
        return np.hstack([rr, det, lam, diag_entropy]), h1
    idx = np.linspace(0, v0.shape[1] - 1, min(16, v0.shape[1])).astype(int)
    v = np.stack([v0[:, idx], v1[:, idx], v2[:, idx]], axis=-1)
    dist_sq = np.sum((v[:, :, None, :] - v[:, None, :, :]) ** 2, axis=-1)
    eps = np.percentile(dist_sq, 25, axis=(1, 2), keepdims=True) + 1e-7
    rec = (dist_sq <= eps).astype(np.float64)
    eye = 1.0 - np.eye(rec.shape[1], dtype=np.float64)[None, :, :]
    rec_off = rec * eye
    total = rec_off.sum(axis=(1, 2), keepdims=True)
    rr = rec_off.mean(axis=(1, 2), keepdims=True)
    det_mat = rec_off[:, :-1, :-1] * rec_off[:, 1:, 1:]
    det = det_mat.sum(axis=(1, 2), keepdims=True)
    det = _safe_div(det, total)
    lam_mat = rec_off[:, :-1, :] * rec_off[:, 1:, :]
    lam = lam_mat.sum(axis=(1, 2), keepdims=True)
    lam = _safe_div(lam, total)
    diag_counts = []
    for lag in range(1, rec_off.shape[1]):
        c = np.sum(rec_off[:, :-lag, :-lag] * rec_off[:, lag:, lag:], axis=(1, 2))
        diag_counts.append(c[:, None])
    if diag_counts:
        diag_counts_arr = np.hstack(diag_counts)
    else:
        diag_counts_arr = np.zeros((n, 1))
    diag_entropy = _entropy_from_counts(diag_counts_arr, axis=1)
    h1 = np.min(dist_sq + np.eye(dist_sq.shape[1])[None, :, :] * 1e12, axis=(1, 2)).reshape(n, 1)
```

```
    return np.hstack([rr.reshape(n, 1), det.reshape(n, 1), lam.reshape(n, 1), diag_entropy.reshape(n, 1)]), h1


def _ordinal_features(y):
    w0 = y[:, :-2]
    w1 = y[:, 1:-1]
    w2 = y[:, 2:]
    c01 = (w0 < w1).astype(np.int32)
    c12 = (w1 < w2).astype(np.int32)
    c02 = (w0 < w2).astype(np.int32)
    code = c01 * 4 + c12 * 2 + c02
    valid = [0, 2, 3, 4, 5, 7]
    probs = np.stack([(code == c).mean(axis=1) for c in valid], axis=1)
    perm_entropy = -np.sum(probs * np.log(probs + 1e-9), axis=1, keepdims=True) / np.log(6.0)
    forbidden = np.sum((probs - (1.0 / 6.0)) ** 2, axis=1, keepdims=True)
    return np.hstack([probs, perm_entropy, forbidden])


def _scattering_features(y):
    parts = []
    for scale in (1, 2, 4, 8):
        if y.shape[1] <= scale:
            parts.extend([np.zeros((y.shape[0], 1))] * 3)
            continue
        d1 = np.abs(y[:, scale:] - y[:, :-scale])
        parts.append(d1.mean(axis=1, keepdims=True))
        parts.append(d1.std(axis=1, keepdims=True))
        if d1.shape[1] > 2:
            d2 = np.abs(d1[:, 2:] - d1[:, :-2])
            parts.append(d2.mean(axis=1, keepdims=True))
        else:
            parts.append(np.zeros((y.shape[0], 1)))
    return np.hstack(parts)


def compute_quartile_brute_force_features(y_zone):
    y = np.asarray(y_zone, dtype=np.float64)
    n, length = y.shape
    if length < 9:
        raise ValueError(f"zone too short: {length}")
    dy = np.diff(y, axis=1)
    ddy = np.diff(dy, axis=1)
    t = np.linspace(0.0, 1.0, length, dtype=np.float64)[None, :]
    dt = 1.0 / (length - 1)
    y0 = y - y[:, 0:1]

    s_y = y0[:, -1:]
    int_t_dy = np.sum(t[:, :-1] * dy, axis=1, keepdims=True)
    int_y_dt = np.sum(y0[:, :-1] * dt, axis=1, keepdims=True)
    levy_area = 0.5 * (int_t_dy - int_y_dt)
    quad_var = np.sum(dy ** 2, axis=1, keepdims=True)
    cubic_var = np.sum(dy ** 3, axis=1, keepdims=True)
    int_y2_dt = np.sum((y0[:, :-1] ** 2) * dt, axis=1, keepdims=True)
    int_t2_dy = np.sum((t[:, :-1] ** 2) * dy, axis=1, keepdims=True)
    lead_lag_cov = np.sum(dy[:, 1:] * dy[:, :-1], axis=1, keepdims=True)
    rough = np.hstack([s_y, int_t_dy, int_y_dt, levy_area, quad_var, cubic_var, int_y2_dt, int_t2_dy, lead_lag_cov])

    rqa, h1_loop = _rqa_features(y)
    ordinal = _ordinal_features(y)

    ds = np.sqrt(dt ** 2 + dy ** 2)
    arc_length = np.sum(ds, axis=1, keepdims=True)
    kappa = (dt * ddy) / ((dt ** 2 + dy[:, :-1] ** 2) ** 1.5 + 1e-9)
    elastic_energy = np.mean(kappa ** 2, axis=1, keepdims=True)
    mean_signed_curvature = np.mean(kappa, axis=1, keepdims=True)
    max_abs_curvature = np.max(np.abs(kappa), axis=1, keepdims=True)
    inflection_rate = np.mean((kappa[:, 1:] * kappa[:, :-1]) < 0, axis=1, keepdims=True)
    angles = np.arctan2(dy, dt)
    total_turning = np.sum(np.abs(np.diff(angles, axis=1)), axis=1, keepdims=True)
    frenet = np.hstack([arc_length, elastic_energy, mean_signed_curvature, max_abs_curvature, inflection_rate,
total_turning])

    scattering = _scattering_features(y)

    abs_dy = np.sort(np.abs(dy), axis=1)
    h0_max = abs_dy[:, -1:]
    h0_second = abs_dy[:, -2:-1]
    h0_entropy = _entropy_from_counts(abs_dy, axis=1)
    hvg = _hvg_features(y)
    topology = np.hstack([h0_max.reshape(n, 1), h0_second.reshape(n, 1), h0_entropy.reshape(n, 1), h1_loop.reshape(n, 1)])
```

```
    tau1 = np.mean(np.abs(y[:, 1:] - y[:, :-1]), axis=1, keepdims=True) + 1e-7
    tau4 = np.mean(np.abs(y[:, 4:] - y[:, :-4]), axis=1, keepdims=True) + 1e-7
    hurst_q1 = np.log2(tau4 / tau1) / 2.0
    tau1_q2 = np.sqrt(np.mean((y[:, 1:] - y[:, :-1]) ** 2, axis=1, keepdims=True)) + 1e-7
    tau4_q2 = np.sqrt(np.mean((y[:, 4:] - y[:, :-4]) ** 2, axis=1, keepdims=True)) + 1e-7
    hurst_q2 = np.log2(tau4_q2 / tau1_q2) / 2.0
    multifractal_width = hurst_q1 - hurst_q2
    max_planar_dist = np.max(np.abs(y - y[:, 0:1]), axis=1, keepdims=True) + 1e-7
    katz_dim = np.log10(length) / (np.log10(max_planar_dist / (arc_length + 1e-7)) + np.log10(length) + 1e-7)
    higuchi_proxy = np.log(arc_length + 1e-7) / (np.log(max_planar_dist + 1e-7) + 1e-7)
    multifractal = np.hstack([hurst_q1, hurst_q2, multifractal_width, katz_dim, higuchi_proxy])

    analytic = hilbert(y0, axis=1)
    inst_amp = np.abs(analytic)
    inst_phase = np.unwrap(np.angle(analytic), axis=1)
    inst_freq = np.diff(inst_phase, axis=1)
    hfreq_mean = np.mean(inst_freq, axis=1, keepdims=True)
    hfreq_std = np.std(inst_freq, axis=1, keepdims=True)
    amp_kurt = np.mean(((inst_amp - inst_amp.mean(axis=1, keepdims=True)) / (inst_amp.std(axis=1, keepdims=True) + 1e-7))
** 4, axis=1, keepdims=True)
    hilbert_feats = np.hstack([hfreq_mean, hfreq_std, amp_kurt])

    spectrum = np.fft.rfft(y, axis=1) / length
    tris = []
    for k1, k2 in ((1, 2), (1, 3), (2, 3)):
        if k1 + k2 < spectrum.shape[1]:
            val = spectrum[:, k1] * spectrum[:, k2] * np.conj(spectrum[:, k1 + k2])
            tris.append(np.abs(val)[:, None])
            tris.append(np.angle(val)[:, None])
    if tris:
        tris_arr = np.hstack(tris)
        tris_mag = np.mean(tris_arr[:, 0::2], axis=1, keepdims=True)
        tris_phase = np.mean(tris_arr[:, 1::2], axis=1, keepdims=True)
    else:
        tris_mag = np.zeros((n, 1))
        tris_phase = np.zeros((n, 1))
    centered = y - y.mean(axis=1, keepdims=True)
    std = y.std(axis=1, keepdims=True) + 1e-7
    z = centered / std
    cumulant4 = np.mean(z ** 4, axis=1, keepdims=True) - 3.0
    trispectrum = np.hstack([cumulant4.reshape(n, 1), tris_mag.reshape(n, 1), tris_phase.reshape(n, 1)])

    diff_sign = np.diff(np.signbit(y).astype(np.int8), axis=1)
    extremas = np.abs(np.diff(np.signbit(dy).astype(np.int8), axis=1)).sum(axis=1, keepdims=True)
    emd_extrema_rate = extremas / max(length - 2, 1)
    med = np.median(y, axis=1, keepdims=True)
    emd_env_asym = (y.max(axis=1, keepdims=True) - med) / (med - y.min(axis=1, keepdims=True) + 1e-7)
    emd_zero_cross = np.mean(diff_sign != 0, axis=1, keepdims=True)
    emd = np.hstack([emd_extrema_rate.reshape(n, 1), emd_env_asym.reshape(n, 1), emd_zero_cross.reshape(n, 1)])

    dist_skew = np.mean(z ** 3, axis=1, keepdims=True)
    dist_kurt = np.mean(z ** 4, axis=1, keepdims=True)
    range_ratio = (y.max(axis=1, keepdims=True) - y.min(axis=1, keepdims=True)) / std
    higher = np.hstack([dist_skew.reshape(n, 1), dist_kurt.reshape(n, 1), range_ratio.reshape(n, 1)])

    raw = np.hstack([
        rough, rqa, ordinal, frenet, scattering, topology, multifractal,
        hilbert_feats, hvg, trispectrum, emd, higher,
    ]).astype(np.float32)
    return np.nan_to_num(raw, nan=0.0, posinf=0.0, neginf=0.0)


if TORCH_OK:
    class LearnableGeometricAnomalyBank(nn.Module):
        def __init__(self, seq_len=38, num_wavelets=16, rqa_kernels=8, out_dim=128):
            super().__init__()
            self.num_wavelets = num_wavelets
            self.wavelet_freqs = nn.Parameter(torch.linspace(1.0, 18.0, num_wavelets))
            self.wavelet_scales = nn.Parameter(torch.ones(num_wavelets) * 0.15)
            self.wavelet_phases = nn.Parameter(torch.zeros(num_wavelets))
            self.delay_conv = nn.Conv1d(1, rqa_kernels, kernel_size=5, stride=1, dilation=2, padding=4)
            self.rqa_log_eps = nn.Parameter(torch.linspace(-2.0, 1.0, rqa_kernels))
            self.rqa_temp = nn.Parameter(torch.ones(rqa_kernels) * 5.0)
            self.lead_lag_bilinear = nn.Bilinear(8, 8, 32)
            self.path_proj = nn.Linear(1, 8)
            total = num_wavelets * 2 + rqa_kernels * 2 + 32 + 4
            self.fusion_net = nn.Sequential(
                nn.Linear(total, 256),
                nn.GELU(),
                nn.LayerNorm(256),
                nn.Linear(256, out_dim),
```

```
                nn.GELU(),
                nn.LayerNorm(out_dim),
            )

        def forward(self, y_ticks):
            b, length = y_ticks.shape
            t = torch.linspace(-0.5, 0.5, length, device=y_ticks.device)[None, None, :]
            freqs = self.wavelet_freqs[None, :, None]
            scales = torch.clamp(self.wavelet_scales[None, :, None].abs(), min=0.03)
            phases = self.wavelet_phases[None, :, None]
            envelope = torch.exp(-0.5 * (t / scales) ** 2)
            kernel_real = envelope * torch.cos(2.0 * np.pi * freqs * t + phases)
            kernel_imag = envelope * torch.sin(2.0 * np.pi * freqs * t + phases)
            y_exp = y_ticks[:, None, :]
            w_real = torch.mean(y_exp * kernel_real, dim=-1)
            w_imag = torch.mean(y_exp * kernel_imag, dim=-1)
            w_modulus = torch.sqrt(w_real ** 2 + w_imag ** 2 + 1e-8)
            w_phase = torch.atan2(w_imag, w_real + 1e-8)

            h_delay = self.delay_conv(y_exp)
            idx = torch.linspace(0, length - 1, min(12, length), device=y_ticks.device).long()
            h_sub = h_delay[:, :, idx]
            dist = (h_sub[:, :, :, None] - h_sub[:, :, None, :]) ** 2
            eps = torch.exp(self.rqa_log_eps)[None, :, None, None]
            temp = torch.clamp(self.rqa_temp.abs(), min=1.0)[None, :, None, None]
            soft_rec = torch.sigmoid(temp * (eps - dist))
            rqa_density = torch.mean(soft_rec, dim=(-1, -2))
            rqa_diag = torch.mean(soft_rec[:, :, :-1, :-1] * soft_rec[:, :, 1:, 1:], dim=(-1, -2))

            dy = y_ticks[:, 1:] - y_ticks[:, :-1]
            h_path = self.path_proj(dy[:, :, None])
            cum_path = torch.cumsum(h_path, dim=1) / length
            lead_vec = torch.mean(h_path[:, 1:, :], dim=1)
            lag_vec = torch.mean(cum_path[:, :-1, :], dim=1)
            rough_sig = self.lead_lag_bilinear(lead_vec, lag_vec)

            ddy = dy[:, 1:] - dy[:, :-1]
            dt = 1.0 / length
            kappa = (dt * ddy) / ((dt ** 2 + dy[:, :-1] ** 2) ** 1.5 + 1e-8)
            curv_energy = torch.mean(kappa ** 2, dim=-1, keepdim=True)
            curv_skew = torch.mean(kappa ** 3, dim=-1, keepdim=True)
            levy = 0.5 * (
                torch.sum(t[:, 0, :-1] * dy, dim=-1, keepdim=True)
                - torch.sum(y_ticks[:, :-1] * dt, dim=-1, keepdim=True)
            )
            drift = y_ticks[:, -1:] - y_ticks[:, 0:1]
            combined = torch.cat([
                w_modulus, w_phase, rqa_density, rqa_diag,
                rough_sig, curv_energy, curv_skew, levy, drift,
            ], dim=-1)
            return self.fusion_net(combined)
```

Read this carefully in conjunction with Chapter 15. The descriptor is a good description of a path and a poor discriminator between paths: the six families overlap in what they measure, which is the root of the collinearity documented in Chapter 6.

phase2/phase2_step2_inbound_outbound_ai.py - outbound evaluation and vault construction (321 lines, 14.2 KB)

```
"""Phase 2 Step 2 - CZET inbound foundation training + outbound vault verification.

Adapted to the whitened feature bank (v2):
  Q_seq(cell, quartile, 72) = [desc_Qk (17) | dense_qk whitened, padded to 55]
  trigger_vec_dense (N,132) / resolution_vec_dense (N,138) for the similarity engine

Modes:
  --mode train   train pass 1 (bidirectional masked quartile reconstruction) + pass 2
                 (covariant-pair metric learning) -> czet_phase2.pt
  --mode vault   score outbound triggers against the trained hubs -> strategy vault
  --mode both    (default)

Outputs (in --phase2_dir):
  czet_phase2.pt, czet_train_log.json, phase2_strategy_vault.parquet,
  phase2_anomaly_strategies.xlsx (if openpyxl), phase2_step2_report.json
"""
import os
import sys
import json
import time
```

```
import argparse
import numpy as np
import pandas as pd

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

try:
    import pqcompat
except Exception:
    pqcompat = None

import torch
import torch.nn as nn
import torch.nn.functional as F


def read_table(path):
    if path.endswith(".parquet"):
        try:
            if pqcompat is not None:
                return pqcompat.read_parquet(path)
            return pd.read_parquet(path)
        except Exception:
            alt = path[:-len(".parquet")] + ".csv"
            if os.path.exists(alt):
                return pd.read_csv(alt)
            raise
    return pd.read_csv(path)


class CrossZoneEpicycleTransformer(nn.Module):
    def __init__(self, zone_dim=72, d_model=256, nhead=4, num_layers=4, dim_ff=768):
        super().__init__()
        self.zone_proj = nn.Linear(zone_dim, d_model)
        self.pos_embed = nn.Parameter(torch.randn(1, 4, d_model) * 0.02)
        layer = nn.TransformerEncoderLayer(
            d_model=d_model, nhead=nhead, dim_feedforward=dim_ff,
            dropout=0.1, activation="gelu", batch_first=True, norm_first=True,
        )
        self.transformer = nn.TransformerEncoder(layer, num_layers=num_layers)
        self.zone_decoder = nn.Sequential(
            nn.Linear(d_model, d_model), nn.GELU(), nn.LayerNorm(d_model),
            nn.Linear(d_model, zone_dim),
        )
        self.covariant_proj = nn.Sequential(
            nn.Linear(d_model * 2, d_model), nn.GELU(), nn.Linear(d_model, 128),
        )
        self.anomaly_head = nn.Sequential(
            nn.Linear(d_model * 2, 128), nn.GELU(), nn.LayerNorm(128), nn.Linear(128, 3),
        )

    def encode(self, q, mask=None):
        x = self.zone_proj(q) + self.pos_embed
        if mask is not None:
            x = x.clone()
            x[:, mask, :] = 0.0
        return self.transformer(x)

    def forward(self, q):
        h_fwd = self.encode(q, [2, 3])
        pred_q34 = self.zone_decoder(h_fwd[:, 2:4, :])
        h_bwd = self.encode(q, [0, 1])
        pred_q12 = self.zone_decoder(h_bwd[:, 0:2, :])
        h_early = torch.cat([h_fwd[:, 0, :], h_fwd[:, 1, :]], dim=-1)
        z_cov = F.normalize(self.covariant_proj(h_early), dim=-1)
        anom = self.anomaly_head(h_early)
        return pred_q34, pred_q12, z_cov, anom


def train(phase2_dir, epochs1, epochs2, batch_size, d_model, layers, nhead, ff,
          max_pairs, seed=1337, max_train_rows=0, resume=False):
    torch.manual_seed(seed)
    np.random.seed(seed)
    cache = np.load(os.path.join(phase2_dir, "czet_train_cache.npz"), mmap_mode="r")
    Q = cache["Q_seq"]
    N = Q.shape[0]
    zone_dim = Q.shape[2]
    n_in = int(N * 0.85)
    if max_train_rows and max_train_rows < n_in:
        train_idx = np.sort(np.random.permutation(n_in)[:max_train_rows]).astype(np.int64)
    else:
```

```
    train_idx = np.arange(n_in, dtype=np.int64)
n_tr = len(train_idx)
print(f"[*] Q_seq={Q.shape} inbound={n_in:,} outbound={N - n_in:,} train_rows={n_tr:,} "
      f"| threads={torch.get_num_threads()}", flush=True)

model = CrossZoneEpicycleTransformer(zone_dim=zone_dim, d_model=d_model, nhead=nhead,
                                     num_layers=layers, dim_ff=ff)
if resume and os.path.exists(os.path.join(phase2_dir, "czet_phase2.pt")):
    ck = torch.load(os.path.join(phase2_dir, "czet_phase2.pt"), map_location="cpu", weights_only=False)
    model.load_state_dict(ck["state_dict"])
    print("[*] resumed weights from czet_phase2.pt", ck.get("cfg"), flush=True)
n_params = sum(p.numel() for p in model.parameters())
print(f"[*] CZET params={n_params:,}", flush=True)
opt = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=1e-4)
held = cache["protected_held"]
ret = cache["q4_net_return"]
log = {"params": int(n_params), "zone_dim": int(zone_dim), "inbound": n_in, "train_rows": int(n_tr),
       "batch_size": batch_size, "pass1": [], "pass2": []}

steps = n_tr // batch_size
model.train()
for ep in range(1, epochs1 + 1):
    perm = train_idx[np.random.permutation(n_tr)]
    tot, t0 = 0.0, time.time()
    for s in range(steps):
        idx = np.sort(perm[s * batch_size:(s + 1) * batch_size])
        bq = torch.from_numpy(np.ascontiguousarray(Q[idx]))
        b_held = torch.from_numpy(np.ascontiguousarray(held[idx].astype(np.float32)))
        b_ret = torch.from_numpy(np.ascontiguousarray(ret[idx]))
        opt.zero_grad()
        p34, p12, _, anom = model(bq)
        loss = (F.mse_loss(p34, bq[:, 2:4, :]) + F.mse_loss(p12, bq[:, 0:2, :])
                + 0.25 * (F.binary_cross_entropy_with_logits(anom[:, 0], b_held)
                          + F.smooth_l1_loss(anom[:, 2], b_ret)))
        loss.backward()
        opt.step()
        tot += loss.item()
    msg = f"    pass1 epoch {ep}/{epochs1} | loss {tot / steps:.5f} | {time.time() - t0:.0f}s"
    print(msg, flush=True)
    log["pass1"].append({"epoch": ep, "loss": round(tot / steps, 5), "secs": round(time.time() - t0, 1)})
    torch.save({"state_dict": model.state_dict(),
                "cfg": {"zone_dim": zone_dim, "d_model": d_model, "nhead": nhead,
                        "num_layers": layers, "dim_ff": ff}},
               os.path.join(phase2_dir, "czet_phase2.pt"))

edges = None
for name in ("phase2_covariant_edges.parquet", "phase2_covariant_edges.csv"):
    p = os.path.join(phase2_dir, name)
    if os.path.exists(p):
        edges = read_table(p)
        break
if edges is not None and len(edges):
    e = edges[(edges["src_idx"] < n_in) & (edges["dst_idx"] < n_in)]
    if len(e) > max_pairs > 0:
        e = e.sample(n=max_pairs, random_state=seed)
    print(f"[*] pass2 pairs={len(e):,}", flush=True)
    opt2 = torch.optim.AdamW(model.parameters(), lr=1e-4, weight_decay=1e-4)
    src = e["src_idx"].to_numpy(np.int64)
    dst = e["dst_idx"].to_numpy(np.int64)
    sign = np.sign(e["cov_0_50"].to_numpy(np.float32))
    lock = np.clip(e["lock_50_100"].to_numpy(np.float32), -1, 1)
    steps2 = len(e) // batch_size
    for ep in range(1, epochs2 + 1):
        perm = np.random.permutation(len(e))
        tot, t0 = 0.0, time.time()
        for s in range(steps2):
            j = np.sort(perm[s * batch_size:(s + 1) * batch_size])
            qz = torch.from_numpy(np.ascontiguousarray(Q[src[j]]))
            qy = torch.from_numpy(np.ascontiguousarray(Q[dst[j]]))
            b_sign = torch.from_numpy(sign[j])
            b_lock = torch.from_numpy(lock[j])
            opt2.zero_grad()
            p34z, _, zz, anomz = model(qz)
            _, _, zy, _ = model(qy)
            cos = F.cosine_similarity(zz, zy * b_sign[:, None], dim=-1)
            loss_cov = F.mse_loss(cos, b_lock)
            w = torch.sigmoid(b_lock * 3.0)[:, None, None]
            loss_lock = torch.mean(w * (p34z - qz[:, 2:4, :]) ** 2)
            loss_prob = F.binary_cross_entropy_with_logits(anomz[:, 1], (b_lock >= 0.65).float())
            loss = loss_cov + 2.0 * loss_lock + 0.5 * loss_prob
            loss.backward()
```

```
                opt2.step()
                tot += loss.item()
            msg = f"    pass2 epoch {ep}/{epochs2} | loss {tot / max(1, steps2):.5f} | {time.time() - t0:.0f}s"
            print(msg, flush=True)
            log["pass2"].append({"epoch": ep, "loss": round(tot / max(1, steps2), 5),
                                 "secs": round(time.time() - t0, 1), "pairs": int(len(e))})
    torch.save({"state_dict": model.state_dict(),
                "cfg": {"zone_dim": zone_dim, "d_model": d_model, "nhead": nhead,
                        "num_layers": layers, "dim_ff": ff}},
               os.path.join(phase2_dir, "czet_phase2.pt"))
    with open(os.path.join(phase2_dir, "czet_train_log.json"), "w") as f:
        json.dump(log, f, indent=2)
    return model, log


def vault(phase2_dir, top_hubs=5000, thr=0.85, min_hits=10):
    scan = np.load(os.path.join(phase2_dir, "jump10_phase2_scan.npz"), mmap_mode="r")
    T = scan["trigger_vec_dense"]
    R = scan["resolution_vec_dense"]
    N = T.shape[0]
    n_in = int(N * 0.85)

    db = None
    for name in ("phase2_covariant_db.parquet", "phase2_covariant_db.csv"):
        p = os.path.join(phase2_dir, name)
        if os.path.exists(p):
            db = read_table(p)
            break
    if db is None or not len(db):
        print("[-] no covariant db; skipping vault", flush=True)
        return 0

    db = db[db["anchor_cell_id"] <= n_in].sort_values("anomaly_z_score", ascending=False)
    hubs = db.head(top_hubs).reset_index(drop=True)
    hub_idx = (hubs["anchor_cell_id"].to_numpy(np.int64) - 1)
    hub_idx = hub_idx[(hub_idx >= 0) & (hub_idx < n_in)]
    Th = np.ascontiguousarray(T[hub_idx])
    Rh = np.ascontiguousarray(R[hub_idx])
    held = scan["protected_held"]
    ret = scan["q4_net_return"]
    print(f"[*] vault: {len(hub_idx):,} hubs x {N - n_in:,} outbound triggers", flush=True)

    rows = []
    CH = 20000
    for h0 in range(0, len(hub_idx), 256):
        h1 = min(len(hub_idx), h0 + 256)
        sim_hits = [[] for _ in range(h1 - h0)]
        for s in range(n_in, N, CH):
            e = min(N, s + CH)
            sims = T[s:e] @ Th[h0:h1].T
            big = np.abs(sims) >= thr
            if big.any():
                ri, ci = np.nonzero(big)
                for r, c in zip(ri, ci):
                    sim_hits[c].append((s + r, sims[r, c]))
        for ci in range(h1 - h0):
            if not sim_hits[ci]:
                continue
            idx = np.fromiter((x[0] for x in sim_hits[ci]), dtype=np.int64, count=len(sim_hits[ci]))
            if len(idx) < min_hits:
                continue
            pol = np.fromiter((np.sign(x[1]) for x in sim_hits[ci]), dtype=np.float32, count=len(idx))
            h = h0 + ci
            cont = float(np.mean((R[idx] * pol[:, None]) @ Rh[h]))
            prot = float(np.mean(held[idx]))
            dirret = ret[idx] * pol
            hrow = hubs.iloc[h]
            expected = -1.0 if str(hrow["recommended_action"]) == "SELL_PROTECTED_HIGH" else 1.0
            dirwin = float(np.mean(np.sign(dirret) == expected))
            rows.append({
                "anchor_cell_id": int(hrow["anchor_cell_id"]),
                "outbound_triggers": int(len(idx)),
                "outbound_50_100_lock_corr": round(cont, 4),
                "outbound_protected_win_rate": round(prot, 4),
                "outbound_directional_win_rate": round(dirwin, 4),
                "outbound_verified_edge": round(max(prot, dirwin), 4),
            })
        print(f"    hubs {h0}-{h1} done", flush=True)

    v = pd.DataFrame(rows)
    if not v.empty:
```

```
        v = v.merge(hubs, on="anchor_cell_id", how="left", suffixes=("", "_db"))
        v = v[v["outbound_verified_edge"] >= 0.55].copy()
        nn = v["outbound_triggers"].to_numpy(float)
        pp = v["outbound_verified_edge"].clip(0.0, 1.0).to_numpy(float)
        zc = 1.959964
        lb = (pp + zc ** 2 / (2 * nn) - zc * np.sqrt(pp * (1 - pp) / nn + zc ** 2 / (4 * nn ** 2))) / (1 + zc ** 2 / nn)
        v["outbound_win_lb95"] = np.round(lb, 4)
        v["sample_flag"] = np.where(nn >= 30, "OK", "THIN")
        v = v.sort_values(["outbound_win_lb95", "outbound_triggers"], ascending=False).reset_index(drop=True)
        v.insert(0, "strategy_id", [f"STRAT_J10_{i + 1:04d}" for i in range(len(v))])
        v["phase3_status"] = "READY_FOR_EQUITY_SIM"
    out_p = os.path.join(phase2_dir, "phase2_strategy_vault.parquet")
    if pqcompat is not None and not v.empty:
        pqcompat.write_parquet(v, out_p)
    else:
        out_p = out_p.replace(".parquet", ".csv")
        v.to_csv(out_p, index=False)
    print(f"[+] vault: {len(v):,} verified strategies -> {out_p}", flush=True)
    try:
        with pd.ExcelWriter(os.path.join(phase2_dir, "phase2_anomaly_strategies.xlsx"),
                            engine="openpyxl") as w:
            v.to_excel(w, sheet_name="Phase3_Ready_Strategies", index=False)
            db.head(10000).to_excel(w, sheet_name="Inbound_Covariant_Hubs", index=False)
        print("[+] xlsx written", flush=True)
    except Exception as ex:
        print(f"[-] xlsx skipped: {type(ex).__name__} {ex}", flush=True)
    return len(v)


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--phase2_dir", default=r"C:\cluster\phase2")
    ap.add_argument("--mode", default="both", choices=["train", "vault", "both"])
    ap.add_argument("--epochs_pass1", type=int, default=4)
    ap.add_argument("--epochs_pass2", type=int, default=3)
    ap.add_argument("--batch_size", type=int, default=512)
    ap.add_argument("--d_model", type=int, default=256)
    ap.add_argument("--layers", type=int, default=4)
    ap.add_argument("--nhead", type=int, default=4)
    ap.add_argument("--dim_ff", type=int, default=768)
    ap.add_argument("--max_pairs", type=int, default=300000)
    ap.add_argument("--threads", type=int, default=0)
    ap.add_argument("--top_hubs", type=int, default=5000)
    ap.add_argument("--max_train_rows", type=int, default=0)
    ap.add_argument("--resume", action="store_true")
    a = ap.parse_args()
    if a.threads > 0:
        torch.set_num_threads(a.threads)
    if a.mode in ("train", "both"):
        train(a.phase2_dir, a.epochs_pass1, a.epochs_pass2, a.batch_size,
              a.d_model, a.layers, a.nhead, a.dim_ff, a.max_pairs,
              max_train_rows=a.max_train_rows, resume=a.resume)
    if a.mode in ("vault", "both"):
        vault(a.phase2_dir, a.top_hubs)


if __name__ == "__main__":
    main()
```

Line for line, this is the most consequential file in the project: the `verified_edge = max(protected, directional)` assignment appears here, together with the sweet-spot filter and the 10-trigger minimum.

phase2/fix_collapse_and_whiten.py - covariance whitening that produced the 132-d space (283 lines, 11.4 KB)

```
"""
PHASE 2 - COLLINEAR-COLLAPSE REPAIR
Implements C:\\cluster\\phase2_mathematicalfix.md:

  Bug A: single-shot bicoherence identity  |A*B*conj(C)| / (|A*B|*|C|) == 1.0 exactly
  Bug B: common-mode positive cone  ->  every cell at |r| >= 0.85 with every anchor

Fixes:
  * cross-sectional standardization (per feature)
  * SVD/PCA strip of the common-mode component(s) + Mahalanobis whitening
  * L2 unit-sphere projection    =>  E[r] = 0.0, sigma_r ~ 1/sqrt(D)
  * bicoherence replaced by phase-triad closure terms cos(phi1+phi2-phi3)

Memory-safe: mmap reads, two-pass chunked standardization, disk-backed outputs,
```

```
streamed zip writing. Never holds more than one float32 block of the matrix.
"""
import os
import sys
import io
import json
import time
import zipfile
import numpy as np

RAW = r"C:\cluster\phase2\jump10_phase2_subzones_raw.npz"
OUT = r"C:\cluster\phase2\jump10_phase2_subzones.npz"
TMP = r"C:\cluster\phase2\_white_tmp"
WNPZ = r"C:\cluster\phase2\whiten_transforms.npz"
REPORT = r"C:\cluster\phase2\whiten_report.json"

WHITEN_KEYS = [
    "trigger_vec", "resolution_vec",
    "trigger_vec_dense", "resolution_vec_dense",
    "dense_q1", "dense_q2", "dense_q3", "dense_q4", "dense_h1", "dense_h2",
]
CHUNK = 65536
SUBSAMPLE = 50000
STRIP_TOP_PC = 1
EPS = 1e-5
SEED = 42


def log(msg):
    print(msg, flush=True)


def two_pass_stats(A, chunk=CHUNK):
    n, d = A.shape
    s1 = np.zeros(d, dtype=np.float64)
    s2 = np.zeros(d, dtype=np.float64)
    for i in range(0, n, chunk):
        j = min(i + chunk, n)
        b = np.asarray(A[i:j], dtype=np.float64)
        s1 += b.sum(axis=0)
        s2 += (b * b).sum(axis=0)
    mu = s1 / n
    var = np.maximum(s2 / n - mu * mu, 0.0)
    return mu.astype(np.float32), np.sqrt(var).astype(np.float32)


def fit_whitener(A, valid_cols, mu, sd):
    """Covariance on a random subsample of standardized data -> whitening W."""
    n = A.shape[0]
    rng = np.random.default_rng(SEED)
    idx = np.sort(rng.choice(n, size=min(SUBSAMPLE, n), replace=False))
    mu_sub = np.asarray(mu, dtype=np.float32)[valid_cols]
    sd_sub = np.asarray(sd, dtype=np.float32)[valid_cols]
    sub = np.empty((len(idx), int(valid_cols.sum())), dtype=np.float32)
    pos = 0
    for k in range(0, len(idx), CHUNK):
        sl = idx[k:k + CHUNK]
        block = np.asarray(A[sl], dtype=np.float32)[:, valid_cols]
        block = np.clip((block - mu_sub) / sd_sub, -6.0, 6.0)
        sub[pos:pos + block.shape[0]] = block
        pos += block.shape[0]

    cov = (sub.T @ sub) / sub.shape[0]
    cov = 0.5 * (cov + cov.T)
    U, S, _ = np.linalg.svd(cov, full_matrices=False)
    var_share = (S / max(S.sum(), 1e-12)) * 100.0
    start_pc = STRIP_TOP_PC if (S[0] / S.sum() > 0.35) else 0
    keep = np.arange(len(S)) >= start_pc
    keep &= S > 1e-3
    U_keep, S_keep = U[:, keep], S[keep]
    W = (U_keep @ np.diag(1.0 / np.sqrt(S_keep + EPS))).astype(np.float32)
    info = {
        "n_features_in": int(A.shape[1]),
        "n_constant_cols_dropped": int((~valid_cols).sum()),
        "n_features_kept": int(valid_cols.sum()),
        "n_components_out": int(W.shape[1]),
        "top5_eigenvalue_variance_share_pct": [round(float(x), 3) for x in var_share[:5]],
        "stripped_top_pc": int(start_pc),
    }
    return W, info
```

```
def transform_chunked(A, valid_cols, mu, sd, W, out_path, chunk=CHUNK):
    """y = L2normalize( clip((x-mu)/sd, +-6) @ W ) written to a disk-backed .npy"""
    n = A.shape[0]
    d_out = W.shape[1]
    mu_sub = np.asarray(mu, dtype=np.float32)[valid_cols]
    sd_sub = np.asarray(sd, dtype=np.float32)[valid_cols]
    out = np.lib.format.open_memmap(out_path, mode="w+", dtype=np.float32, shape=(n, d_out))
    for i in range(0, n, chunk):
        j = min(i + chunk, n)
        b = np.asarray(A[i:j], dtype=np.float32)[:, valid_cols]
        b = np.clip((b - mu_sub) / sd_sub, -6.0, 6.0)
        y = b @ W
        nrm = np.linalg.norm(y, axis=1, keepdims=True)
        nrm[nrm < 1e-9] = 1.0
        out[i:j] = y / nrm
    out.flush()
    del out
    return out_path


def main():
    t0 = time.time()
    if not os.path.exists(RAW):
        log(f"[-] raw npz not found: {RAW}")
        sys.exit(1)
    os.makedirs(TMP, exist_ok=True)

    log(f"[*] Opening {RAW} (mmap, no full load)")
    src = np.load(RAW, mmap_mode="r")
    keys = list(src.files)
    log(f"[*] {len(keys)} arrays: " + ", ".join(keys))
    report = {"input": RAW, "output": OUT, "arrays": {}, "whiteners": {}}

    # ---------------- pass 1: fix bicoherence BEFORE touching desc_Q1 --------
    log("\n[*] Replacing bicoherence with true phase-triad closure ...")
    q1 = np.asarray(src["desc_Q1"], dtype=np.float32)
    phi1 = np.arctan2(q1[:, 8], q1[:, 0] + 1e-9)
    phi2 = np.arctan2(q1[:, 9], q1[:, 1] + 1e-9)
    phi3 = np.arctan2(q1[:, 10], q1[:, 2] + 1e-9)
    triad = phi1 + phi2 - phi3
    bico_fixed = (0.5 * (np.abs(np.cos(triad)) + np.abs(np.sin(triad)))).astype(np.float32)
    phase_closure_cos = np.abs(np.cos(triad)).astype(np.float32)
    bico_old = np.asarray(src["bicoherence"], dtype=np.float32)
    report["bicoherence"] = {
        "old_min": float(bico_old.min()), "old_median": float(np.median(bico_old)),
        "old_max": float(bico_old.max()), "old_std": float(bico_old.std()),
        "new_min": float(bico_fixed.min()), "new_median": float(np.median(bico_fixed)),
        "new_max": float(bico_fixed.max()), "new_std": float(bico_fixed.std()),
        "phase_closure_abs_cos_median": float(np.median(phase_closure_cos)),
    }
    log(f"    old bico: min={bico_old.min():.6f} med={np.median(bico_old):.6f} "
        f"max={bico_old.max():.6f} (std={bico_old.std():.2e})")
    log(f"    new bico: min={bico_fixed.min():.6f} med={np.median(bico_fixed):.6f} "
        f"max={bico_fixed.max():.6f} (std={bico_fixed.std():.6f})")
    del bico_old, triad

    # ---------------- pass 2: whiten each descriptor matrix ------------------
    whitened_paths = {}
    for key in WHITEN_KEYS:
        if key not in src.files:
            log(f"[!] skip missing {key}")
            continue
        t1 = time.time()
        A = src[key]
        n, d = A.shape
        log(f"\n[*] Whitening `{key}` shape={A.shape} ...")
        mu, sd = two_pass_stats(A)
        sd_safe = np.where(sd < 1e-6, 1.0, sd)
        valid = sd >= 1e-6
        if valid.sum() < 2:
            log(f"[!] {key}: <2 non-constant columns, passing through")
            continue
        W, info = fit_whitener(A, valid, mu, sd_safe)
        p = os.path.join(TMP, f"{key}.npy")
        transform_chunked(A, valid, mu, sd_safe, W, p)
        whitened_paths[key] = p
        info["seconds"] = round(time.time() - t1, 2)
        report["whiteners"][key] = info
        report["arrays"][key] = {"in_shape": [n, d], "out_shape": [n, info["n_components_out"]]}
        log(f"    -> kept {info['n_features_kept']}/{d} cols, {info['n_components_out']} comps, "
```

```
        f"strip_top_pc={info['stripped_top_pc']}, top5 var%={info['top5_eigenvalue_variance_share_pct']}, "
        f"{info['seconds']}s")
    # persist W for later live/outbound scoring
    np.save(os.path.join(TMP, f"W_{key}.npy"), W)
    np.save(os.path.join(TMP, f"mu_{key}.npy"), mu)
    np.save(os.path.join(TMP, f"sd_{key}.npy"), sd_safe)
    np.save(os.path.join(TMP, f"cols_{key}.npy"), valid)
    del A, mu, sd, W

# ---------------- pass 3: stream-write the repaired npz -----------------
log(f"\n[*] Writing {OUT} ...")
big_out = {k: np.load(p, mmap_mode="r") for k, p in whitened_paths.items()}
tmp_out = OUT + ".tmp"
order = []
for k in keys:
    if k in whitened_paths:
        order.append(k)
order += [k for k in keys if k not in whitened_paths]
with zipfile.ZipFile(tmp_out, "w", compression=zipfile.ZIP_STORED, allowZip64=True) as zf:
    for k in order:
        if k in big_out:
            with zf.open(f"{k}.npy", "w", force_zip64=True) as fh:
                np.save(fh, big_out[k])
            continue
        if k == "bicoherence":
            arr = bico_fixed
        else:
            arr = np.asarray(src[k])
        with zf.open(f"{k}.npy", "w", force_zip64=True) as fh:
            np.save(fh, arr)
        if k == "bicoherence":
            with zf.open("phase_closure_abs_cos.npy", "w", force_zip64=True) as fh:
                np.save(fh, phase_closure_cos)
os.replace(tmp_out, OUT)
size = os.path.getsize(OUT)
log(f"[+] wrote {OUT} ({size/1e9:.2f} GB)")
report["output_bytes"] = size
report["elapsed_seconds"] = round(time.time() - t0, 1)

# ---------------- pass 4: verification ----------------------------------
log("\n--- VERIFYING DE-COLLAPSED COVARIANT SCORES ACROSS 591,171 CELLS ---")
new = np.load(OUT, mmap_mode="r")
T = new["trigger_vec_dense"] if "trigger_vec_dense" in new.files else new["trigger_vec"]
R = new["resolution_vec_dense"] if "resolution_vec_dense" in new.files else new["resolution_vec"]
q4_ret = new["q4_net_return"]
held = new["protected_held"]
d = T.shape[1]
log(f"[*] T shape={T.shape}  (expected sigma_r ~ {1/np.sqrt(d):.4f})")

anchors = [100, 500, 1000, 5000, 50000, 250000]
verify = []
for a in anchors:
    t1 = time.time()
    sims = T @ T[a]
    sims[max(0, a - 2): min(T.shape[0], a + 3)] = 0.0
    abs_sims = np.abs(sims)
    top5 = np.sort(abs_sims)[::-1][:5]
    rec = {
        "anchor": a,
        "top5_abs_r": [round(float(x), 4) for x in top5],
        "peers_ge_070": int((abs_sims >= 0.70).sum()),
        "peers_ge_080": int((abs_sims >= 0.80).sum()),
        "peers_ge_085": int((abs_sims >= 0.85).sum()),
        "mean_r": round(float(sims.mean()), 6),
        "p999_abs_r": round(float(np.percentile(abs_sims, 99.9)), 4),
        "seconds": round(time.time() - t1, 2),
    }
    verify.append(rec)
    log(f"Anchor {a:6d}: top5 |r|={rec['top5_abs_r']} | peers>=0.70: {rec['peers_ge_070']:5d} "
        f"| >=0.80: {rec['peers_ge_080']:4d} | >=0.85: {rec['peers_ge_085']:3d} "
        f"| mean r={rec['mean_r']:+.5f} | p99.9={rec['p999_abs_r']:.3f}")
report["anchor_verification"] = verify

# population scan: how many rows would pass min_peers / pathology check
rng = np.random.default_rng(7)
sample = np.sort(rng.choice(T.shape[0], size=300, replace=False))
counts85, counts70, means = [], [], []
for a in sample:
    s = T @ T[int(a)]
    s[max(0, int(a) - 2): min(T.shape[0], int(a) + 3)] = 0.0
    aa = np.abs(s)
```

```
        counts85.append(int((aa >= 0.85).sum()))
        counts70.append(int((aa >= 0.70).sum()))
        means.append(float(s.mean()))
    counts85 = np.array(counts85)
    counts70 = np.array(counts70)
    report["population_sample"] = {
        "rows_sampled": int(len(sample)),
        "mean_r_of_means": round(float(np.mean(means)), 6),
        "median_peers_ge_085": int(np.median(counts85)),
        "p90_peers_ge_085": int(np.percentile(counts85, 90)),
        "p99_peers_ge_085": int(np.percentile(counts85, 99)),
        "median_peers_ge_070": int(np.median(counts70)),
        "frac_rows_with_ge12_peers_085": round(float((counts85 >= 12).mean()), 4),
        "frac_rows_with_ge12_peers_070": round(float((counts70 >= 12).mean()), 4),
    }
    log("\n[*] Population sample of 300 random anchors:")
    for k, v in report["population_sample"].items():
        log(f"      {k}: {v}")
    log(f"\n[+] total elapsed {report['elapsed_seconds']}s")
    with open(REPORT, "w", encoding="utf-8") as f:
        json.dump(report, f, indent=2)
    log(f"[+] report -> {REPORT}")


if __name__ == "__main__":
    main()
```

The whitening transform built here was applied in memory and never written to disk. That single omission is why the live tester cannot reproduce the backtest's matches (Chapter 16).

phase2/phase2_czet_sweetspot.py - CZET v2 architecture, masked reconstruction, PLS blend (333 lines, 14.9 KB)

```
"""Item 06 - CZET v2 on the covariant sweet-spot manifold, LearnableGeometricAnomalyBank attached.

Universe   : hubs with 12 <= peers <= 240 and 0.85 <= mean|r| <= 0.955, plus every cell that
             is an endpoint of an edge inside that set (all cells < n_in).
Predictors : self q1/q2 (144) + peer-family mean of q1/q2 (144) + block macro descriptor (64).
Pass 1     : masked quartile reconstruction (q3/q4 from q1/q2) + protected/return heads.
Pass 2     : covariant metric learning. score = cosine + pair head(zz, zy, zz*zy, |zz-zy|,
             cos, observed cov). loss = value + BCE + weighted q3/q4 alignment + rank loss.
Linear path: 24-component PLS on the 352-d predictor block, fitted on train rows only,
             stacked with the ensembled net output by a train-fitted scalar blend.
Evaluation : R2 on held-out rows, Spearman / precision@10% on held-out edges
             (edges touching a test row; no test row is ever trained on).
Ensemble   : 3 seeds, scores and reconstructions averaged.
"""
import os
import sys
import json
import time
import numpy as np

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
import pqcompat
from phase2_step2_inbound_outbound_ai import CrossZoneEpicycleTransformer
from phase2_dense_anomaly_bank import LearnableGeometricAnomalyBank

import torch
import torch.nn as nn
import torch.nn.functional as F

BASE = r"C:\cluster\phase2"
N_IN = 502495
BLOCK = 12
N_PLS = 24
SEEDS = (11, 22, 33)


class CZETv2(nn.Module):
    def __init__(self, zone_dim=72, d_model=192, nhead=4, num_layers=3, dim_ff=512,
                 bank_out=64, seq_len=38):
        super().__init__()
        self.core = CrossZoneEpicycleTransformer(zone_dim=zone_dim, d_model=d_model,
                                                 nhead=nhead, num_layers=num_layers,
                                                 dim_ff=dim_ff)
        self.bank = LearnableGeometricAnomalyBank(seq_len=seq_len, num_wavelets=16,
                                                  rqa_kernels=8, out_dim=bank_out)
        self.fuse = nn.Sequential(
```

```
            nn.Linear(128 + bank_out * 4, d_model * 2),
            nn.GELU(), nn.LayerNorm(d_model * 2),
            nn.Linear(d_model * 2, 128),
        )
        self.pair = nn.Sequential(
            nn.Linear(4 * 128 + 2, 256), nn.GELU(), nn.LayerNorm(256),
            nn.Linear(256, 64), nn.GELU(), nn.Linear(64, 1),
        )

    def forward(self, q, yq=None):
        p34, p12, z, anom = self.core(q)
        if yq is None:
            return p34, p12, z, anom
        b = torch.cat([self.bank(yq[:, i, :]) for i in range(4)], dim=-1)
        z = F.normalize(self.fuse(torch.cat([z, b], dim=-1)), dim=-1)
        return p34, p12, z, anom

    def score_pairs(self, zs, zy, cos, cov):
        return self.pair(torch.cat([zs, zy, zs * zy, (zs - zy).abs(),
                                    cos.unsqueeze(-1), cov.unsqueeze(-1)], dim=-1)).squeeze(-1)


def build():
    db = pqcompat.read_parquet(os.path.join(BASE, "phase2_covariant_db.parquet"))
    ss = db[(db.covariant_peer_count >= 12) & (db.covariant_peer_count <= 240) &
            (db.mean_0_50_similarity >= 0.85) & (db.mean_0_50_similarity <= 0.955)]
    hub_ids = ss.anchor_cell_id.to_numpy(np.int64) - 1
    hub_ids = hub_ids[(hub_ids >= 0) & (hub_ids < N_IN)]
    edges = pqcompat.read_parquet(os.path.join(BASE, "phase2_covariant_edges.parquet"))
    hset = set(hub_ids.tolist())
    m = edges.src_idx.isin(hset) & edges.dst_idx.isin(hset)
    e = edges[m & (edges.src_idx < N_IN) & (edges.dst_idx < N_IN)].reset_index(drop=True)
    ids = np.unique(np.concatenate([hub_ids, e.src_idx.to_numpy(np.int64),
                                    e.dst_idx.to_numpy(np.int64)]))
    ids = ids[(ids >= 0) & (ids < N_IN)]
    print(f"[*] rows={len(ids):,} hubs={len(hub_ids):,} edges={len(e):,}", flush=True)

    Q = np.load(os.path.join(BASE, "czet_train_cache.npz"), mmap_mode="r")
    q = np.ascontiguousarray(Q["Q_seq"][ids])
    held = np.ascontiguousarray(Q["protected_held"][ids].astype(np.float32))
    ret = np.ascontiguousarray(Q["q4_net_return"][ids])
    z = np.load(os.path.join(BASE, "jump10_phase2_subzones.npz"), mmap_mode="r")
    yq = np.stack([np.ascontiguousarray(z[f"y_q{i}"][ids]) for i in range(1, 5)], axis=1)

    loc = {int(v): i for i, v in enumerate(ids)}
    es = np.array([loc[int(v)] for v in e.src_idx], dtype=np.int64)
    ed = np.array([loc[int(v)] for v in e.dst_idx], dtype=np.int64)
    sign = np.sign(e.cov_0_50.to_numpy(np.float32))
    cov = np.abs(e.cov_0_50.to_numpy(np.float32))
    lock = np.clip(e.lock_50_100.to_numpy(np.float32), -1, 1)

    n = len(ids)
    acc = np.zeros((n, q.shape[2] * 2))
    cnt = np.zeros(n)
    q2 = q[:, :2, :].astype(np.float64)
    for a, b in zip(es, ed):
        acc[a] += q2[b].ravel()
        acc[b] += q2[a].ravel()
        cnt[a] += 1
        cnt[b] += 1
    fam = np.divide(acc, np.maximum(cnt, 1)[:, None])
    mac = np.nan_to_num(np.load(os.path.join(BASE, "_macro_desc.npy"))[ids // BLOCK].astype(np.float64))
    Xp = np.concatenate([q2.reshape(n, -1), fam, mac], axis=1)
    print(f"[*] predictor block {Xp.shape}", flush=True)
    return ids, q, yq, held, ret, es, ed, sign, cov, lock, Xp


def fit_pls(X, Y, tr, kc=N_PLS):
    mu = X[tr].mean(0)
    sd = X[tr].std(0) + 1e-9
    Xs = (X - mu) / sd
    my = Y[tr].mean(0)
    Xr = Xs[tr].copy()
    Yr = (Y - my)[tr].copy()
    W, P, C = [], [], []
    for _ in range(kc):
        u = Yr[:, 0]
        w = Xr.T @ u
        w /= np.linalg.norm(w) + 1e-12
        t = Xr @ w
        pp = Xr.T @ t / (t @ t)
```

```
        cc = Yr.T @ t / (t @ t)
        Xr -= np.outer(t, pp)
        Yr -= np.outer(t, cc)
        W.append(w)
        P.append(pp)
        C.append(cc)
    Wm, Pm, Cm = np.column_stack(W), np.column_stack(P), np.column_stack(C)
    B = Wm @ np.linalg.pinv(Pm.T @ Wm) @ Cm.T
    return {"mu": mu, "sd": sd, "my": my, "B": B, "k": kc}


def pls_predict(pls, X):
    return ((X - pls["mu"]) / pls["sd"]) @ pls["B"] + pls["my"]


def train_one(seed, D, epochs1, epochs2, batch):
    torch.manual_seed(seed)
    np.random.seed(seed)
    tr_rows, te_rows, tr_e, te_e = D["tr_rows"], D["te_rows"], D["tr_e"], D["te_e"]
    Qt, Yt, Ht, Rt = D["Qt"], D["Yt"], D["Ht"], D["Rt"]
    model = CZETv2()
    opt = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=1e-4)
    log = {"pass1": [], "pass2": []}

    steps = max(1, len(tr_rows) // batch)
    for ep in range(1, epochs1 + 1):
        p = tr_rows[np.random.permutation(len(tr_rows))]
        tot, t0 = 0.0, time.time()
        model.train()
        for s in range(steps):
            idx = np.sort(p[s * batch:(s + 1) * batch])
            bq, by, bh, br = Qt[idx], Yt[idx], Ht[idx], Rt[idx]
            opt.zero_grad()
            p34, p12, _, anom = model(bq, by)
            loss = (F.mse_loss(p34, bq[:, 2:4, :]) + F.mse_loss(p12, bq[:, 0:2, :])
                    + 0.25 * (F.binary_cross_entropy_with_logits(anom[:, 0], bh)
                              + F.smooth_l1_loss(anom[:, 2], br)))
            loss.backward()
            opt.step()
            tot += loss.item()
        log["pass1"].append({"seed": seed, "epoch": ep, "loss": round(tot / steps, 5),
                             "secs": round(time.time() - t0, 1)})
        print(f"    [s{seed}] pass1 {ep}/{epochs1} loss {tot / steps:.5f} ({time.time() - t0:.0f}s)", flush=True)

    opt2 = torch.optim.AdamW(model.parameters(), lr=1.5e-4, weight_decay=1e-4)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt2, T_max=epochs2, eta_min=2e-5)
    S, Dd = D["S"], D["D"]
    SG, LK, CV = D["SG"], D["LK"], D["CV"]
    steps2 = max(1, len(tr_e) // batch)
    for ep in range(1, epochs2 + 1):
        p = np.random.permutation(len(tr_e))
        tot, t0 = 0.0, time.time()
        model.train()
        for s in range(steps2):
            j = torch.from_numpy(p[s * batch:(s + 1) * batch])
            sg, lk, cv = SG[j], LK[j], CV[j]
            opt2.zero_grad()
            p34z, _, zz, _ = model(Qt[S[j]], Yt[S[j]])
            _, _, zy, _ = model(Qt[Dd[j]], Yt[Dd[j]])
            cos = F.cosine_similarity(zz, zy * sg[:, None], dim=-1)
            sc = model.score_pairs(zz, zy, cos, cv)
            loss = (F.mse_loss(sc, lk) + 0.5 * F.mse_loss(cos, lk)
                    + 0.5 * F.binary_cross_entropy_with_logits(4.0 * sc, (lk >= 0.65).float())
                    + torch.mean(torch.sigmoid(lk * 3.0)[:, None, None]
                                 * (p34z - Qt[S[j]][:, 2:4, :]) ** 2))
            dl = lk[:, None] - lk[None, :]
            order = sc[:, None] - sc[None, :]
            mask = dl > 0.25
            if bool(mask.any()):
                loss = loss + 0.75 * F.softplus(-order[mask]).mean()
            loss.backward()
            opt2.step()
            tot += loss.item()
        sched.step()
        log["pass2"].append({"seed": seed, "epoch": ep, "loss": round(tot / steps2, 5),
                             "secs": round(time.time() - t0, 1)})
        print(f"    [s{seed}] pass2 {ep}/{epochs2} loss {tot / steps2:.5f} ({time.time() - t0:.0f}s)", flush=True)

    model.eval()
    with torch.no_grad():
        p34, _, _, _ = model(Qt[tr_rows], Yt[tr_rows])
```

```
        Pnet_tr = p34.numpy().reshape(len(tr_rows), -1)
        p34, _, _, _ = model(Qt[te_rows], Yt[te_rows])
        Pnet_te = p34.numpy().reshape(len(te_rows), -1)

        def scores(e_idx):
            zs_ = D["Qt"][D["es"][e_idx]]
            zy_ = D["Qt"][D["ed"][e_idx]]
            _, _, zz_, _ = model(zs_, D["Yt"][D["es"][e_idx]])
            _, _, zy2_, _ = model(zy_, D["Yt"][D["ed"][e_idx]])
            cs_ = F.cosine_similarity(zz_, zy2_ * torch.from_numpy(D["sign"][e_idx])[:, None], dim=-1)
            return model.score_pairs(zz_, zy2_, cs_, torch.from_numpy(D["cov"][e_idx])).numpy()

        s_tr = scores(tr_e)
        s_te = scores(te_e)
    return model, log, Pnet_tr, Pnet_te, s_tr, s_te


def main(epochs1=8, epochs2=30, batch=128, threads=4):
    torch.set_num_threads(threads)
    ids, q, yq, held, ret, es, ed, sign, cov, lock, Xp = build()

    n = len(ids)
    perm = np.random.default_rng(11).permutation(n)
    n_tr = int(0.8 * n)
    tr_rows = np.sort(perm[:n_tr])
    te_rows = np.sort(perm[n_tr:])
    tr_set = np.zeros(n, bool)
    tr_set[tr_rows] = True
    te_set = ~tr_set
    tr_e = np.where(tr_set[es] & tr_set[ed])[0]
    te_e = np.where(te_set[es] | te_set[ed])[0]
    print(f"[*] train rows={len(tr_rows):,} test rows={len(te_rows):,} "
          f"train edges={len(tr_e):,} test edges={len(te_e):,}", flush=True)

    Y = q[:, 2:4, :].reshape(n, -1).astype(np.float64)
    pls = fit_pls(Xp, Y, tr_rows)
    np.savez(os.path.join(BASE, "czet_phase2_pls.npz"),
             mu=pls["mu"], sd=pls["sd"], my=pls["my"], B=pls["B"], k=np.array([pls["k"]]))
    Ppls_tr = pls_predict(pls, Xp[tr_rows])
    Ppls_te = pls_predict(pls, Xp[te_rows])
    r2_pls = float(1 - ((Ppls_te - Y[te_rows]) ** 2).sum() /
                   ((Y[te_rows] - Y[te_rows].mean(0)) ** 2).sum())
    print(f"[*] PLS({pls['k']}) held-out R2 = {r2_pls:.5f}", flush=True)

    D = {"tr_rows": tr_rows, "te_rows": te_rows, "tr_e": tr_e, "te_e": te_e,
         "Qt": torch.from_numpy(q), "Yt": torch.from_numpy(yq),
         "Ht": torch.from_numpy(held), "Rt": torch.from_numpy(ret),
         "es": es, "ed": ed, "sign": sign, "cov": cov, "lock": lock,
         "S": torch.from_numpy(es[tr_e]), "D": torch.from_numpy(ed[tr_e]),
         "SG": torch.from_numpy(sign[tr_e]), "LK": torch.from_numpy(lock[tr_e]),
         "CV": torch.from_numpy(cov[tr_e])}

    logs, Ptr, Pte, Str, Ste, models = [], [], [], [], [], []
    for sd in SEEDS:
        m, lg, a, b, c, d = train_one(sd, D, epochs1, epochs2, batch)
        models.append(m)
        logs.append(lg)
        Ptr.append(a)
        Pte.append(b)
        Str.append(c)
        Ste.append(d)
        from scipy.stats import spearmanr
        print(f"    [s{sd}] seed held-out Spearman = "
              f"{float(spearmanr(d, lock[te_e]).statistic):.4f}", flush=True)

    Pnet_tr = np.mean(Ptr, axis=0)
    Pnet_te = np.mean(Pte, axis=0)
    s_te = np.mean(Ste, axis=0)
    s_tr = np.mean(Str, axis=0)
    Ytr, Yte = Y[tr_rows], Y[te_rows]

    num = float(np.sum((Pnet_tr - Ppls_tr) * (Ytr - Ppls_tr)))
    den = float(np.sum((Pnet_tr - Ppls_tr) ** 2)) + 1e-12
    lam = float(np.clip(num / den, 0.0, 1.0))
    w = np.array([lam, 1.0 - lam])
    Pst_te = w[0] * Pnet_te + w[1] * Ppls_te
    r2_net = float(1 - ((Pnet_te - Yte) ** 2).sum() / ((Yte - Yte.mean(0)) ** 2).sum())
    r2_st = float(1 - ((Pst_te - Yte) ** 2).sum() / ((Yte - Yte.mean(0)) ** 2).sum())

    lk_te = lock[te_e]
    from scipy.stats import spearmanr
```

```
    rho = float(spearmanr(s_te, lk_te).statistic)
    pear = float(np.corrcoef(s_te, lk_te)[0, 1])
    pos = lk_te >= 0.65
    prec = None
    if pos.sum() > 5:
        k10 = max(1, int(0.1 * len(s_te)))
        prec = float(pos[np.argsort(-s_te)[:k10]].mean())
    rho_cov = float(spearmanr(cov[te_e], lk_te).statistic)

    out = {
        "model": "CZETv2 + LearnableGeometricAnomalyBank + pair head + PLS path (3-seed ensemble)",
        "universe": "sweet spot: 12-240 peers, 0.85-0.955 mean|r|",
        "train_rows": int(len(tr_rows)), "test_rows": int(len(te_rows)),
        "train_edges": int(len(tr_e)), "test_edges": int(len(te_e)),
        "spearman": round(rho, 4), "pair_spearman": round(rho, 4),
        "spearman_observed_cov_baseline": round(rho_cov, 4),
        "r2": round(r2_st, 5), "oos_r2": round(r2_st, 5),
        "r2_net_only": round(r2_net, 5), "r2_pls_only": round(r2_pls, 5),
        "stack_weights": {"net": round(float(w[0]), 4), "pls": round(float(w[1]), 4)},
        "pair_score_vs_lock_pearson": round(pear, 4),
        "precision_at_10pct": None if prec is None else round(prec, 4),
        "base_rate_lock_ge_065": round(float(pos.mean()), 4),
        "params": int(sum(p.numel() for p in models[0].parameters())),
        "seeds": list(SEEDS), "epochs": {"pass1": epochs1, "pass2": epochs2},
        "protocol": "rows split 80/20 at random; test edges are those touching a test row; "
                    "PLS, stack weight and pair head are fitted on train only; R2 on held-out rows.",
    }
    print(json.dumps(out, indent=2), flush=True)
    with open(os.path.join(BASE, "czet_eval.json"), "w") as f:
        json.dump(out, f, indent=2)
    with open(os.path.join(BASE, "czet_train_log.json"), "w") as f:
        json.dump({"runs": logs, "config": {"d_model": 192, "layers": 3, "bank_out": 64,
                                            "pls_components": N_PLS, "lr_pass1": 3e-4,
                                            "lr_pass2": 1.5e-4, "batch": batch, "seeds": list(SEEDS),
                                            "universe": "sweetspot 12-240 peers, 0.85-0.955 |r|"}}, f, indent=2)
    torch.save({"state_dicts": [m.state_dict() for m in models],
                "cfg": {"zone_dim": 72, "d_model": 192, "nhead": 4, "num_layers": 3,
                        "dim_ff": 512, "bank_out": 64, "seq_len": 38},
                "stack_weights": w.tolist(), "seeds": list(SEEDS)},
               os.path.join(BASE, "czet_phase2_sweetspot.pt"))


if __name__ == "__main__":
    main()
```

phase2/phase2_czet_eval.py - held-out evaluation of the pair head (72 lines, 2.8 KB)

```
"""Out-of-sample evaluation of the trained CZET (runs on the X Flip, CPU-light)."""
import os
import sys
import json
import numpy as np
import torch

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from phase2_step2_inbound_outbound_ai import CrossZoneEpicycleTransformer, read_table

BASE = r"C:\cluster\phase2"


def main(n_pairs=8000, n_rows=4000, seed=7):
    np.random.seed(seed)
    ck = torch.load(os.path.join(BASE, "czet_phase2.pt"), map_location="cpu", weights_only=False)
    cfg = ck["cfg"]
    model = CrossZoneEpicycleTransformer(**cfg)
    model.load_state_dict(ck["state_dict"])
    model.eval()
    print("[*] loaded", cfg)

    cache = np.load(os.path.join(BASE, "czet_train_cache.npz"), mmap_mode="r")
    Q = cache["Q_seq"]
    N = Q.shape[0]
    n_in = int(N * 0.85)
    out = {"cfg": cfg, "train_rows": 150000}

    # 1) out-of-sample quartile reconstruction (last 15% never trained)
    idx = np.random.permutation(np.arange(n_in, N))[:n_rows]
    idx.sort()
    q = torch.from_numpy(np.ascontiguousarray(Q[idx]))
    with torch.no_grad():
```

```
        p34, p12, z, anom = model(q)
    for tag, pred, tgt in (("q34", p34, q[:, 2:4, :]), ("q12", p12, q[:, 0:2, :])):
        sse = float(((pred - tgt) ** 2).sum())
        sst = float(((tgt - tgt.mean(0, keepdim=True)) ** 2).sum())
        out[f"{tag}_oos_r2"] = round(1 - sse / sst, 5)
        out[f"{tag}_oos_mse"] = round(float(((pred - tgt) ** 2).mean()), 6)

    # 2) covariant metric quality on labelled pairs
    edges = read_table(os.path.join(BASE, "phase2_covariant_edges.parquet"))
    e = edges.sample(n=min(n_pairs, len(edges)), random_state=seed)
    src = e["src_idx"].to_numpy(np.int64)
    dst = e["dst_idx"].to_numpy(np.int64)
    sign = np.sign(e["cov_0_50"].to_numpy(np.float32))
    lock = e["lock_50_100"].to_numpy(np.float32)
    with torch.no_grad():
        _, _, zs, _ = model(torch.from_numpy(np.ascontiguousarray(Q[src])))
        _, _, zd, _ = model(torch.from_numpy(np.ascontiguousarray(Q[dst])))
    cos = (zs * (zd * torch.from_numpy(sign)[:, None])).sum(-1).numpy()
    lockc = np.clip(lock, -1, 1)
    out["pairs"] = int(len(e))
    out["pair_cos_vs_lock_pearson"] = round(float(np.corrcoef(cos, lockc)[0, 1]), 4)
    if len(cos) > 2:
        from scipy.stats import spearmanr
        out["pair_cos_vs_lock_spearman"] = round(float(spearmanr(cos, lockc).statistic), 4)
    pos = lockc >= 0.65
    if pos.sum() and (~pos).sum():
        order = np.argsort(-cos)
        hits = pos[order]
        tp = np.cumsum(hits)
        prec = tp / np.arange(1, len(hits) + 1)
        out["precision_at_10pct"] = round(float(prec[max(0, int(0.1 * len(hits)) - 1)]), 4)
        out["base_rate_lock_ge_065"] = round(float(pos.mean()), 4)
    print(json.dumps(out, indent=2))
    with open(os.path.join(BASE, "czet_eval.json"), "w") as f:
        json.dump(out, f, indent=2)


if __name__ == "__main__":
    main()
```

phase2/phase2_null_test.py - selection-aware block permutation test (106 lines, 4.3 KB)

```
"""Selection-aware block-permutation null test for the Phase-2 vault (item 10).

The naive per-strategy p-value is meaningless here because the 893 strategies were
*selected* on the outbound window. This script therefore re-selects the top-300 in
every permutation and compares the selected-set mean win rate, which is the only
honest test of "the vault beats a block-shuffled null".

Outputs: null_test.json, phase2_strategy_vault.parquet (null columns updated)
"""
import os
import sys
import json
import numpy as np
import pandas as pd

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
import pqcompat

BASE = r"C:\cluster\phase2"
N, N_IN = 591171, 502495
NPERM = 400
TOP = 300


def wilson(p, n, z=1.959964):
    p = np.clip(p, 0, 1)
    d = 1 + z ** 2 / n
    c = p + z ** 2 / (2 * n)
    r = z * np.sqrt(p * (1 - p) / n + z ** 2 / (4 * n ** 2))
    return (c - r) / d


def main():
    tg = np.load(os.path.join(BASE, "vault_triggers.npz"))
    idx_all, pol_all, off = tg["idx"], tg["pol"], tg["off"]
    scan = np.load(os.path.join(BASE, "jump10_phase2_scan.npz"), mmap_mode="r")
    held_o = np.asarray(scan["protected_held"])[N_IN:].astype(np.float32)
    ret_o = np.asarray(scan["q4_net_return"])[N_IN:].astype(np.float32)
```

```
    nb = len(held_o)
    n_full = nb // 150
    tail = np.arange(n_full * 150, nb, dtype=np.int64)
    base = np.arange(n_full)[:, None] * 150 + np.arange(150)[None, :]
    lis = [idx_all[off[i]:off[i + 1]] - N_IN for i in range(len(off) - 1)]
    pols = [pol_all[off[i]:off[i + 1]] for i in range(len(off) - 1)]
    ns = np.array([len(x) for x in lis], dtype=float)
    print(f"[*] {len(lis)} strategies, triggers {int(ns.sum()):,}", flush=True)

    def win_arrays(h, r):
        w = np.empty(len(lis), dtype=np.float64)
        for i, (li, po) in enumerate(zip(lis, pols)):
            w[i] = max(float(np.mean(h[li])), float(np.mean(np.sign(r[li] * po) == 1)))
        return w

    obs = win_arrays(held_o, ret_o)
    obs_lb = wilson(obs, ns)
    obs_sel = np.argsort(-obs_lb)[:TOP]
    obs_mean = float(obs[obs_sel].mean())
    obs_pool_n = int(ns[obs_sel].sum())

    rng = np.random.default_rng(20261025)
    null_mean = np.empty(NPERM)
    null_pool = np.empty(NPERM)
    for p in range(NPERM):
        pm = rng.permutation(n_full)
        pos = np.concatenate([(pm[:, None] * 150 + np.arange(150)[None, :]).ravel(), tail])
        w = win_arrays(held_o[pos], ret_o[pos])
        sel = np.argsort(-wilson(w, ns))[:TOP]
        null_mean[p] = w[sel].mean()
        null_pool[p] = np.sum(w[sel] * ns[sel]) / ns[sel].sum() if ns[sel].sum() else 0
        if (p + 1) % 100 == 0:
            print(f"    perm {p + 1}/{NPERM} | null top300 mean {null_mean[p]:.4f}", flush=True)

    p_agg = float((np.sum(null_mean >= obs_mean) + 1) / (NPERM + 1))
    obs_pool = float(np.sum(obs[obs_sel] * ns[obs_sel]) / obs_pool_n)
    p_pool = float((np.sum(null_pool >= obs_pool) + 1) / (NPERM + 1))
    per_p = np.array([float((np.sum(np.abs(null_mean - null_mean.mean()) >= abs(obs[i] - null_mean.mean())) + 1) / (NPERM +
1))
                      for i in range(len(lis))])

    out = {
        "permutations": NPERM, "block_size_ticks": 150, "top": TOP,
        "observed_top300_mean_win": round(obs_mean, 4),
        "null_top300_mean_median": round(float(np.median(null_mean)), 4),
        "null_top300_mean_p99": round(float(np.quantile(null_mean, 0.99)), 4),
        "p_selection_aware": round(p_agg, 4),
        "observed_pooled_win": round(obs_pool, 4),
        "null_pooled_median": round(float(np.median(null_pool)), 4),
        "p_pooled": round(p_pool, 4),
        "beats_null_p01": bool(p_agg < 0.01),
        "verdict": ("vault beats the block-permutation null" if p_agg < 0.01
                    else "OUTBOUND EDGE IS NOT DISTINGUISHABLE FROM A BLOCK-SHUFFLED NULL "
                         "(selection on the outbound window is the likely cause)"),
    }
    print(json.dumps(out, indent=2), flush=True)
    with open(os.path.join(BASE, "null_test.json"), "w") as f:
        json.dump(out, f, indent=2)

    v = pqcompat.read_parquet(os.path.join(BASE, "phase2_strategy_vault.parquet"))
    v["null_pval"] = np.round(per_p, 4)
    v["null_pval_selection_aware"] = round(p_agg, 4)
    v["beats_null"] = bool(p_agg < 0.01)
    v["null_verdict"] = out["verdict"]
    pqcompat.write_parquet(v, os.path.join(BASE, "phase2_strategy_vault.parquet"))


if __name__ == "__main__":
    main()
```

Note what the permutation loop permutes: the outcome. Note what it does not permute: the selection rule. This is the design decision discussed in Chapters 10 and 15.

phase2/phase2_fractal_fix.py - multi-timeframe consistency test (86 lines, 4.0 KB)

```
"""Recompute the macro/micro fractal agreement in a standardized descriptor space.

The first pass correlated raw 64-d descriptors, whose scale-dependent terms (s_y,
```

```
int_y_dt, quad_var ...) carry the absolute price level and forced every pair to ~1.
Here both vectors are z-scored per feature against the macro population first, so the
agreement measures *shape*, not price level.
"""
import os
import sys
import json
import numpy as np

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
import pqcompat
from phase2_dense_anomaly_bank import compute_quartile_brute_force_features

BASE = r"C:\cluster\phase2"
P1 = r"C:\cluster\phase1\final\jump10_phase1_fourier_functions.npz"
BLOCK = 12


def main():
    vault = pqcompat.read_parquet(os.path.join(BASE, "phase2_strategy_vault.parquet"))
    macro = np.load(os.path.join(BASE, "_macro_desc.npy"))
    print("[*] macro descriptors", macro.shape, flush=True)

    px = np.load(P1, mmap_mode="r")["raw_tick_cells"]
    hub_idx = vault.anchor_cell_id.to_numpy(np.int64) - 1
    micro_paths = np.empty((len(hub_idx), 150), dtype=np.float64)
    for i in range(0, len(hub_idx), 2000):
        micro_paths[i:i + 2000] = np.asarray(px[hub_idx[i:i + 2000]], dtype=np.float64)
    micro = compute_quartile_brute_force_features(micro_paths)
    np.save(os.path.join(BASE, "_micro_desc_hubs.npy"), micro.astype(np.float32))

    mu = macro.mean(0, keepdims=True)
    sd = macro.std(0, keepdims=True) + 1e-9
    Zm = np.clip((macro - mu) / sd, -6, 6)
    Zi = np.clip((micro - mu) / sd, -6, 6)
    blocks = hub_idx // BLOCK
    Mb = Zm[blocks]
    Zi_c = Zi - Zi.mean(1, keepdims=True)
    Mb_c = Mb - Mb.mean(1, keepdims=True)
    agree = np.sum(Zi_c * Mb_c, axis=1) / (
        np.sqrt(np.sum(Zi_c ** 2, axis=1) * np.sum(Mb_c ** 2, axis=1)) + 1e-12)

    # scale-free path shape agreement: z-scored 150-point paths
    mp = micro_paths - micro_paths.mean(1, keepdims=True)
    mp /= (mp.std(1, keepdims=True) + 1e-12)
    macro_paths = np.empty((len(set(blocks.tolist())), 150))
    pos = {b: i for i, b in enumerate(sorted(set(blocks.tolist())))}
    for i, b in enumerate(sorted(pos)):
        s = b * BLOCK
        e = min(len(px), s + BLOCK)
        blk = np.asarray(px[s:e], dtype=np.float64)
        if blk.shape[0] < BLOCK:
            blk = np.vstack([blk, np.repeat(blk[-1:], BLOCK - blk.shape[0], axis=0)])
        macro_paths[i] = blk.reshape(BLOCK, 150).mean(0)
    macro_paths -= macro_paths.mean(1, keepdims=True)
    macro_paths /= (macro_paths.std(1, keepdims=True) + 1e-12)
    mpb = macro_paths[[pos[b] for b in blocks.tolist()]]
    shape = np.sum(mp * mpb, axis=1) / 150.0

    q90 = float(np.quantile(agree, 0.90))
    vault["fractal_macro_agreement"] = np.round(agree, 4)
    vault["fractal_shape_agreement"] = np.round(shape, 4)
    vault["is_fractal_anomaly"] = agree >= q90
    pqcompat.write_parquet(vault, os.path.join(BASE, "phase2_strategy_vault.parquet"))

    rep = {
        "descriptor_agreement": {"min": round(float(agree.min()), 4), "median": round(float(np.median(agree)), 4),
                                 "p75": round(float(np.quantile(agree, .75)), 4), "max": round(float(agree.max()), 4),
                                 "std": round(float(agree.std()), 4), "p90_threshold": round(q90, 4)},
        "path_shape_agreement": {"median": round(float(np.median(shape)), 4),
                                 "p90": round(float(np.quantile(shape, .9)), 4)},
        "is_fractal_anomalies": int((agree >= q90).sum()),
        "agreement_vs_verified_edge": round(float(np.corrcoef(agree, vault.outbound_verified_edge.to_numpy(float))[0, 1]),
4),
        "fractal_anomalies_mean_edge": round(float(vault.loc[agree >= q90, "outbound_verified_edge"].mean()), 4),
        "non_fractal_mean_edge": round(float(vault.loc[agree < q90, "outbound_verified_edge"].mean()), 4),
    }
    print(json.dumps(rep, indent=2), flush=True)
    with open(os.path.join(BASE, "fractal_report.json"), "w") as f:
        json.dump(rep, f, indent=2)
```

```
if __name__ == "__main__":
    main()
```

phase2/phase2_crawled_operators.py - nine complexity operators (169 lines, 7.9 KB)

```
"""Item 03 - crawled mathematical trajectory operators.

Computes the literature-derived invariants that were NOT in the original 12-angle
brute-force bank, over the strictly causal 0-50% window of every cell, and writes
crawled_math_operators.npz + crawled_math_operators_manifest.json.

Operators
  HVG degree entropy + entropy gap vs the iid-exponential law  (Luque et al., PRE 80, 046103)
  Hilbert-Huang instantaneous frequency mean/std               (Huang et al., Proc. R. Soc. A 454, 903)
  Normalized Levy area of the (t, y) path                      (Chevyrev & Kormilitzin, arXiv:1603.03788, eq. 12)
  RQA determinism / laminarity                                 (Marwan et al., Phys. Rep. 438, 237)
  Ordinal permutation entropy                                  (Bandt & Pompe, PRL 88, 174102)
"""
import os
import sys
import json
import numpy as np

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
import pqcompat
from scipy.signal import hilbert
from phase2_dense_anomaly_bank import _rqa_features, _ordinal_features

BASE = r"C:\cluster\phase2"
P1 = r"C:\cluster\phase1\final\jump10_phase1_fourier_functions.npz"
N = 591171
WIN = 75
CHUNK = 4000

K = np.arange(2, 200)
P_TH = (1.0 / 3.0) * (2.0 / 3.0) ** (K - 2)
H_TH = float(-np.sum(P_TH * np.log2(P_TH)))


def hvg_degree(y):
    """Exact horizontal-visibility degree (all lags) via next/prev greater-or-equal."""
    n, L = y.shape
    i = np.arange(L)
    upper = np.triu(np.ones((L, L), dtype=bool), 1)[None]
    lower = np.tril(np.ones((L, L), dtype=bool), -1)[None]
    nxt = (y[:, None, :] >= y[:, :, None]) & upper
    prv = (y[:, :, None] >= y[:, None, :]) & lower
    has_n = nxt.any(2)
    has_p = prv.any(2)
    jn = np.argmax(nxt, axis=2)
    jp = L - 1 - np.argmax(prv[:, :, ::-1], axis=2)
    deg = (jn - i[None, :] - 1) + (i[None, :] - jp - 1) + has_n.astype(np.int32) + has_p.astype(np.int32)
    return np.clip(deg, 0, L - 1).astype(np.float32)


def hvg_entropy(deg):
    n, length = deg.shape
    hist = np.zeros((n, length + 1), dtype=np.float32)
    rows = np.repeat(np.arange(n), length)
    np.add.at(hist, (rows, deg.astype(np.int32).ravel()), 1.0)
    p = hist / np.maximum(hist.sum(1, keepdims=True), 1e-9)
    return -np.sum(np.where(p > 0, p * np.log2(np.where(p > 0, p, 1)), 0.0), axis=1)


def main():
    npz_out = os.path.join(BASE, "crawled_math_operators.npz")
    px = np.load(P1, mmap_mode="r")["raw_tick_cells"]
    names = ["hvg_degree_entropy", "hvg_entropy_gap", "hvg_max_degree",
             "hh_inst_freq_mean", "hh_inst_freq_std",
             "levy_area_z", "rqa_det", "rqa_lam", "perm_entropy"]
    arrs = {}
    cached = os.path.exists(npz_out) and "--force" not in sys.argv
    if cached:
        d = np.load(npz_out)
        arrs = {n: d[n] for n in names}
        print("[*] using cached crawled_math_operators.npz", flush=True)
    for n in names:
        if not cached:
            arrs[n] = np.lib.format.open_memmap(os.path.join(BASE, f"_op_{n}.npy"), mode="w+",
```

```
                                                dtype=np.float32, shape=(N,))
    t = np.linspace(0.0, 1.0, WIN)
    for s in ([] if cached else range(0, N, CHUNK)):
        e = min(N, s + CHUNK)
        p = np.asarray(px[s:e, :WIN], dtype=np.float64)
        y = p - p.mean(1, keepdims=True)
        y = y / (y.std(1, keepdims=True) + 1e-12)
        dy = np.diff(y, axis=1)
        dt = 1.0 / (WIN - 1)
        area = 0.5 * (np.sum(t[:-1] * dy, axis=1) - np.sum(y[:, :-1] * dt, axis=1))
        arrs["levy_area_z"][s:e] = area / (np.abs(dy).sum(1) + 1e-9)

        ana = hilbert(y, axis=1)
        inst = np.diff(np.unwrap(np.angle(ana), axis=1), axis=1) / (2 * np.pi * dt)
        arrs["hh_inst_freq_mean"][s:e] = inst.mean(1) / 100.0
        arrs["hh_inst_freq_std"][s:e] = inst.std(1) / 100.0

        rqa, _ = _rqa_features(y)
        arrs["rqa_det"][s:e] = rqa[:, 1]
        arrs["rqa_lam"][s:e] = rqa[:, 2]

        ord_feat = _ordinal_features(y)
        arrs["perm_entropy"][s:e] = ord_feat[:, 6]

        deg = hvg_degree(y)
        h = hvg_entropy(deg)
        arrs["hvg_degree_entropy"][s:e] = h
        arrs["hvg_entropy_gap"][s:e] = h - H_TH
        arrs["hvg_max_degree"][s:e] = deg.max(1)
        print(f"    operators {e:,}/{N:,}", flush=True)

    if not cached:
        for n in names:
            arrs[n].flush()
        np.savez(npz_out, **{n: np.asarray(arrs[n]) for n in names})

    vault = pqcompat.read_parquet(os.path.join(BASE, "phase2_strategy_vault.parquet"))
    ai = vault.anchor_cell_id.to_numpy(np.int64) - 1
    assoc = {}
    for n in names:
        v = np.asarray(arrs[n])[ai]
        assoc[n] = {
            "min": round(float(np.nanmin(v)), 5), "median": round(float(np.nanmedian(v)), 5),
            "max": round(float(np.nanmax(v)), 5),
            "corr_vs_outbound_edge": round(float(np.corrcoef(v, vault.outbound_verified_edge.to_numpy(float))[0, 1]), 4),
        }
        ptmp = os.path.join(BASE, f"_op_{n}.npy")
        if os.path.exists(ptmp):
            try:
                os.remove(ptmp)
            except Exception:
                pass

    manifest = {
        "generated": "2026-10-25",
        "purpose": "Literature-sourced trajectory invariants crawled for Phase 2 item 03 and computed on the "
                   "causal 0-50% window (ticks 1..75) of all 591,171 cells.",
        "window": {"start_tick": 1, "end_tick": 75, "length": WIN,
                   "preprocessing": "per-window mean-centering and unit-variance scaling (df=74)"},
        "operators": [
            {"name": "hvg_degree_entropy", "family": "complex networks",
             "formula": "H(d) = -sum_k p_k log2 p_k over the horizontal-visibility-graph degree distribution",
             "reference": "Luque, Lacasa, Ballesteros & Luque, Phys. Rev. E 80, 046103 (2009),
doi:10.1103/PhysRevE.80.046103",
             "note": "iid series produce P(k)=(1/3)(2/3)^(k-2) regardless of amplitude law"},
            {"name": "hvg_entropy_gap", "family": "complex networks",
             "formula": "H_empirical - H_theory with H_theory = %.6f bits for the iid exponential law" % H_TH,
             "reference": "same as hvg_degree_entropy", "note": "positive gap = excess order vs white noise"},
            {"name": "hvg_max_degree", "family": "complex networks", "formula": "max_i deg(i)",
             "reference": "same as hvg_degree_entropy"},
            {"name": "hh_inst_freq_mean", "family": "Hilbert-Huang",
             "formula": "mean of d/dt arg(Hilbert(y)) / 2*pi", "reference": "Huang et al., Proc. R. Soc. A 454, 903
(1998)"},
            {"name": "hh_inst_freq_std", "family": "Hilbert-Huang",
             "formula": "std of the instantaneous frequency", "reference": "same as hh_inst_freq_mean"},
            {"name": "levy_area_z", "family": "rough paths",
             "formula": "A = 0.5*(int t dy - int y dt), normalized by total variation",
             "reference": "Chevyrev & Kormilitzin, arXiv:1603.03788 eq. (12); Lyons (1998) rough path theory"},
            {"name": "rqa_det", "family": "recurrence quantification",
             "formula": "DET = sum(diagonal recurrence lengths >= 2) / total recurrence (16-pt delay embedding)",
             "reference": "Marwan, Romano, Thiel & Kurths, Phys. Rep. 438, 237 (2007)"},
```

```
            {"name": "rqa_lam", "family": "recurrence quantification",
             "formula": "LAM = vertical-line recurrence / total recurrence", "reference": "same as rqa_det"},
            {"name": "perm_entropy", "family": "ordinal statistics",
             "formula": "H = -sum p(pi) ln p(pi) / ln 6 over the 6 ordinal patterns of order 3",
             "reference": "Bandt & Pompe, Phys. Rev. Lett. 88, 174102 (2002)"},
        ],
        "arrays": {"path": "crawled_math_operators.npz", "rows": N, "dtype": "float32", "columns": names},
        "vault_association": assoc,
    }
    with open(os.path.join(BASE, "crawled_math_operators_manifest.json"), "w") as f:
        json.dump(manifest, f, indent=2)
    print(json.dumps({"H_theory": round(H_TH, 4), "association": assoc}, indent=2), flush=True)


if __name__ == "__main__":
    main()
```

phase2/phase2_merge_db.py - statement database assembly (154 lines, 7.0 KB)

```
"""Merge the 3 node shards into the Phase-2 Relational Covariant Database.

Runs on the X Flip (cheap: only the hub/edge tables move, never the matrices).
Produces:
  C:\\cluster\\phase2\\phase2_covariant_db.parquet
  C:\\cluster\\phase2\\phase2_covariant_edges.parquet
  C:\\cluster\\phase2\\phase2_covariant_summary.json
"""
import os
import sys
import json
import glob
import numpy as np
import pandas as pd

sys.path.insert(0, r"C:\cluster\phase2")
import pqcompat

BASE = r"C:\cluster\phase2"
SHARDS = os.path.join(BASE, "shards")
RECV = r"C:\cluster\phase1\out\phase2\shards_received"
N_TOTAL = 591171

COLS = ["anchor_cell_id", "covariant_peer_count", "peers_evaluated",
        "pos_corr_peers", "neg_mirror_peers", "mean_0_50_similarity",
        "max_0_50_similarity", "mean_50_100_lock_corr", "frac_75_100_exact_lock",
        "protected_ext_win_rate", "directional_50_100_win_rate",
        "direction_consistency", "anomaly_z_score", "bicoherence_phase_lock",
        "target_ext_tick_median", "recommended_action"]


def load_any(path):
    if path.endswith(".parquet"):
        return pqcompat.read_parquet(path)
    df = pd.read_csv(path)
    if "recommended_action" not in df.columns:
        return df
    return df


def main():
    node_files = {}
    for ext in ("parquet", "csv"):
        for pat in (os.path.join(SHARDS, f"covariant_anomalies_node?.{ext}"),
                    os.path.join(RECV, f"covariant_anomalies_node?.{ext}")):
            for f in sorted(glob.glob(pat)):
                key = os.path.basename(f).split("_node")[1].split(".")[0]
                node_files.setdefault(key, f)
    if not node_files:
        print("[-] no shard files found", flush=True)
        sys.exit(1)
    print("[*] hub shards:", node_files, flush=True)

    parts = []
    for k, f in sorted(node_files.items()):
        df = load_any(f)
        df["shard"] = int(k)
        parts.append(df)
        print(f"    node {k}: {len(df):,} anomaly hubs  <- {f}", flush=True)
    db = pd.concat(parts, ignore_index=True)
    before = len(db)
```

```
db = db.sort_values("anomaly_z_score", ascending=False)
db = db.drop_duplicates(subset="anchor_cell_id", keep="first").reset_index(drop=True)
print(f"[*] merged {before:,} -> {len(db):,} unique anomaly hubs", flush=True)

for c in COLS:
    if c not in db.columns:
        db[c] = np.nan
db = db[COLS + ["shard"]]

all_anchors = set(db["anchor_cell_id"].astype(np.int64))
missing = N_TOTAL - len(all_anchors)
out_db = os.path.join(BASE, "phase2_covariant_db.parquet")
pqcompat.write_parquet(db, out_db)
print(f"[+] {out_db}  ({os.path.getsize(out_db)/1e6:.2f} MB)", flush=True)

# ---- edges ------------------------------------------------------------
eparts = []
for k in sorted(node_files):
    for ext in ("parquet", "csv"):
        for pat in (os.path.join(SHARDS, f"covariant_edges_node{k}.{ext}"),
                    os.path.join(RECV, f"covariant_edges_node{k}.{ext}")):
            for f in glob.glob(pat):
                e = load_any(f)
                if len(e):
                    eparts.append(e)
if eparts:
    edges = pd.concat(eparts, ignore_index=True)
    edges["abs_cov"] = edges["cov_0_50"].abs()
    edges = edges.sort_values("abs_cov", ascending=False)
    s = edges["src_idx"].to_numpy(np.int64)
    d = edges["dst_idx"].to_numpy(np.int64)
    edges["pair"] = np.minimum(s, d) * np.int64(N_TOTAL + 1) + np.maximum(s, d)
    edges = edges.drop_duplicates(subset="pair", keep="first").drop(columns=["pair", "abs_cov"])
    out_e = os.path.join(BASE, "phase2_covariant_edges.parquet")
    pqcompat.write_parquet(edges, out_e)
    n_edges = len(edges)
    print(f"[+] {out_e}  ({n_edges:,} unique undirected edges, {os.path.getsize(out_e)/1e6:.2f} MB)", flush=True)
else:
    n_edges = 0
    edges = pd.DataFrame(columns=["src_idx", "dst_idx", "cov_0_50", "lock_50_100"])

# ---- summary ----------------------------------------------------------
top = db.head(25)
summary = {
    "cells_total": N_TOTAL,
    "anomaly_hubs": int(len(db)),
    "hub_rate_pct": round(100.0 * len(db) / N_TOTAL, 3),
    "cells_not_anomalous": int(missing),
    "unique_undirected_edges": int(n_edges),
    "shards": {k: int(len(p)) for k, p in zip(sorted(node_files), parts)},
    "covariant_similarity": {
        "max_single_peer_abs_r": round(float(db["max_0_50_similarity"].max()), 4),
        "max_single_peer_abs_r_anchor": int(db.loc[db["max_0_50_similarity"].idxmax(), "anchor_cell_id"]),
        "median_mean_abs_r": round(float(db["mean_0_50_similarity"].median()), 4),
        "p99_mean_abs_r": round(float(np.percentile(db["mean_0_50_similarity"], 99)), 4),
    },
    "continuation_lock": {
        "max_mean_lock_corr": round(float(db["mean_50_100_lock_corr"].max()), 4),
        "max_mean_lock_corr_anchor": int(db.loc[db["mean_50_100_lock_corr"].idxmax(), "anchor_cell_id"]),
        "max_frac_exact_lock": round(float(db["frac_75_100_exact_lock"].max()), 4),
        "median_mean_lock_corr": round(float(db["mean_50_100_lock_corr"].median()), 4),
    },
    "edge_quality": {
        "max_protected_ext_win_rate": round(float(db["protected_ext_win_rate"].max()), 4),
        "median_protected_ext_win_rate": round(float(db["protected_ext_win_rate"].median()), 4),
        "max_directional_win_rate": round(float(db["directional_50_100_win_rate"].max()), 4),
        "median_directional_win_rate": round(float(db["directional_50_100_win_rate"].median()), 4),
        "max_anomaly_z_score": round(float(db["anomaly_z_score"].max()), 3),
        "hubs_z_ge_3": int((db["anomaly_z_score"] >= 3).sum()),
        "hubs_protected_win_ge_60pct": int((db["protected_ext_win_rate"] >= 0.60).sum()),
        "hubs_directional_win_ge_60pct": int((db["directional_50_100_win_rate"] >= 0.60).sum()),
    },
    "peer_counts": {
        "median": int(db["covariant_peer_count"].median()),
        "p90": int(np.percentile(db["covariant_peer_count"], 90)),
        "max": int(db["covariant_peer_count"].max()),
        "median_evaluated": int(db["peers_evaluated"].median()),
    },
    "actions": db["recommended_action"].value_counts().to_dict(),
    "mean_ext_tick": round(float(db["target_ext_tick_median"].mean()), 1),
    "top_10_hubs": top.head(10)[["anchor_cell_id", "covariant_peer_count",
```

```
                                    "mean_0_50_similarity", "max_0_50_similarity",
                                    "mean_50_100_lock_corr", "protected_ext_win_rate",
                                    "directional_50_100_win_rate", "anomaly_z_score",
                                    "recommended_action"]].to_dict(orient="records"),
    }
    with open(os.path.join(BASE, "phase2_covariant_summary.json"), "w") as f:
        json.dump(summary, f, indent=2)
    print(json.dumps(summary, indent=2), flush=True)


if __name__ == "__main__":
    main()
```

phase2/phase2_report.py - eleven-sheet workbook writer (179 lines, 9.0 KB)

```
"""Phase 2 anomaly-efficiency report (X Flip, light).

Inputs : phase2_covariant_db.parquet, phase2_covariant_edges.parquet,
         phase2_covariant_summary.json, phase2_strategy_vault.(parquet|csv)
Outputs: phase2_anomaly_efficiency_report.json
         phase2_anomaly_efficiency_report.xlsx
"""
import os
import sys
import json
import numpy as np
import pandas as pd

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
import pqcompat

BASE = r"C:\cluster\phase2"


def read_table(p):
    return pqcompat.read_parquet(p) if p.endswith(".parquet") else pd.read_csv(p)


def main():
    db = read_table(os.path.join(BASE, "phase2_covariant_db.parquet"))
    edges = read_table(os.path.join(BASE, "phase2_covariant_edges.parquet"))
    with open(os.path.join(BASE, "phase2_covariant_summary.json")) as f:
        summ = json.load(f)
    vault = None
    for n in ("phase2_strategy_vault.parquet", "phase2_strategy_vault.csv"):
        p = os.path.join(BASE, n)
        if os.path.exists(p):
            vault = read_table(p)
            break

    n = db["covariant_peer_count"].clip(lower=1).to_numpy(float)
    for col, label in (("protected_ext_win_rate", "prot"), ("directional_50_100_win_rate", "dir")):
        p = db[col].to_numpy(float)
        db[f"{label}_edge"] = p - 0.5
        db[f"{label}_z"] = (p - 0.5) / np.sqrt(0.25 / n)
        db[f"{label}_p"] = 0.5 * np.vectorize(lambda z: __import__("math").erfc(abs(z) / np.sqrt(2)))(db[f"{label}_z"])

    db["best_win"] = db[["protected_ext_win_rate", "directional_50_100_win_rate"]].max(axis=1)
    db["best_edge"] = db["best_win"] - 0.5
    db["best_z"] = (db["best_win"] - 0.5) / np.sqrt(0.25 / n)
    db["efficiency_score"] = db["best_z"] * np.sqrt(n) / 100.0

    q = [0, .01, .05, .25, .5, .75, .95, .99, 1.0]
    dist = {}
    for col in ("covariant_peer_count", "mean_0_50_similarity", "max_0_50_similarity",
                "mean_50_100_lock_corr", "frac_75_100_exact_lock", "protected_ext_win_rate",
                "directional_50_100_win_rate", "anomaly_z_score", "best_z", "efficiency_score"):
        s = db[col].astype(float)
        dist[col] = {f"p{int(x*100)}": round(float(s.quantile(x)), 4) for x in q}

    def bucket_quality(col, bins, labels):
        out = []
        for lo, hi, lab in zip(bins[:-1], bins[1:], labels):
            m = (db[col] >= lo) & (db[col] < hi)
            if m.sum() == 0:
                continue
            out.append({
                "bucket": lab, "hubs": int(m.sum()),
                "median_best_win": round(float(db.loc[m, "best_win"].median()), 4),
                "median_lock_corr": round(float(db.loc[m, "mean_50_100_lock_corr"].median()), 4),
```

```
                "max_best_win": round(float(db.loc[m, "best_win"].max()), 4),
                "hubs_win_ge_60pct": int((db.loc[m, "best_win"] >= 0.60).sum()),
            })
        return out

    by_peers = bucket_quality("covariant_peer_count", [12, 25, 50, 100, 250, 1000, 1e9],
                              ["12-24", "25-49", "50-99", "100-249", "250-999", "1000+"])
    by_r = bucket_quality("mean_0_50_similarity", [0.85, 0.88, 0.90, 0.92, 0.94, 0.96, 1.01],
                          ["0.85-0.88", "0.88-0.90", "0.90-0.92", "0.92-0.94", "0.94-0.96", "0.96+"])

    report = {
        "generated_from": "phase2_covariant_db.parquet",
        "coverage": {"cells": summ["cells_total"], "hubs": len(db),
                     "hub_rate_pct": summ["hub_rate_pct"],
                     "edges": int(len(edges)), "vault_strategies": 0 if vault is None else int(len(vault))},
        "answers": {
            "highest_covariant_similarity": summ["covariant_similarity"],
            "strongest_continuation_lock": summ["continuation_lock"],
            "best_verified_edge": {
                "max_protected_win": round(float(db["protected_ext_win_rate"].max()), 4),
                "max_directional_win": round(float(db["directional_50_100_win_rate"].max()), 4),
                "best_combined_win": round(float(db["best_win"].max()), 4),
                "anchor_cell_id": int(db.loc[db["best_win"].idxmax(), "anchor_cell_id"]),
            },
            "statistically_significant_hubs_z5": int((db["best_z"] >= 5).sum()),
            "hubs_win_ge_60pct": int((db["best_win"] >= 0.60).sum()),
            "hubs_win_ge_70pct": int((db["best_win"] >= 0.70).sum()),
        },
        "distributions": dist,
        "efficiency_by_peer_count": by_peers,
        "efficiency_by_similarity": by_r,
        "top_25_by_efficiency": db.sort_values("best_z", ascending=False).head(25)[
            ["anchor_cell_id", "covariant_peer_count", "mean_0_50_similarity", "max_0_50_similarity",
             "mean_50_100_lock_corr", "frac_75_100_exact_lock", "protected_ext_win_rate",
             "directional_50_100_win_rate", "best_win", "best_z", "recommended_action"]
        ].round(4).to_dict(orient="records"),
        "edge_table": {
            "median_cov": round(float(edges["cov_0_50"].abs().median()), 4) if len(edges) else None,
            "p90_cov": round(float(edges["cov_0_50"].abs().quantile(0.9)), 4) if len(edges) else None,
            "max_cov": round(float(edges["cov_0_50"].abs().max()), 4) if len(edges) else None,
            "median_abs_lock": round(float(edges["lock_50_100"].abs().median()), 4) if len(edges) else None,
            "edges_lock_ge_065": int((edges["lock_50_100"].abs() >= 0.65).sum()) if len(edges) else 0,
        },
        "verdict": None,
    }
    best = report["answers"]["best_verified_edge"]["best_combined_win"]
    report["verdict"] = (
        "Phase 2 produces tradeable structure: "
        f"{report['answers']['hubs_win_ge_60pct']:,} of {len(db):,} hubs exceed 60% out-of-sample win rates "
        f"(best {best:.1%}, {report['answers']['statistically_significant_hubs_z5']:,} hubs at z>=5)."
        if best >= 0.60 else
        f"No hub clears a robust 60% out-of-sample edge (best {best:.1%}); Phase 2 shows covariant structure but no proven
directional edge."
    )

    for name in ("czet_eval.json", "czet_train_log.json", "null_test.json",
                 "fractal_report.json", "phase2_enrichment_report.json",
                 "crawled_math_operators_manifest.json"):
        pth = os.path.join(BASE, name)
        if os.path.exists(pth):
            with open(pth) as f:
                report[name.replace(".json", "")] = json.load(f)

    cop = os.path.join(BASE, "crawled_math_operators.npz")
    if os.path.exists(cop) and vault is not None and len(vault):
        zc = np.load(cop)
        ai = vault["anchor_cell_id"].to_numpy(np.int64) - 1
        rows = []
        for n in zc.files:
            v = zc[n]
            vh = v[ai]
            rows.append({
                "operator": n,
                "all_median": round(float(np.nanmedian(v)), 5),
                "all_p05": round(float(np.nanpercentile(v, 5)), 5),
                "all_p95": round(float(np.nanpercentile(v, 95)), 5),
                "vault_median": round(float(np.nanmedian(vh)), 5),
                "corr_vs_outbound_edge": round(float(np.corrcoef(vh, vault["outbound_verified_edge"])[0, 1]), 4),
            })
        report["crawled_operators"] = rows
```

```
    with open(os.path.join(BASE, "phase2_anomaly_efficiency_report.json"), "w") as f:
        json.dump(report, f, indent=2)
    print(json.dumps({k: report[k] for k in ("coverage", "answers", "edge_table", "verdict")}, indent=2))

    try:
        with pd.ExcelWriter(os.path.join(BASE, "phase2_anomaly_efficiency_report.xlsx"),
                            engine="openpyxl") as w:
            pd.DataFrame([report["coverage"]]).to_excel(w, sheet_name="Coverage", index=False)
            pd.DataFrame(report["answers"]["best_verified_edge"], index=["value"]).T.to_excel(w, sheet_name="Answers")
            pd.DataFrame(dist).T.to_excel(w, sheet_name="Distributions")
            pd.DataFrame(by_peers).to_excel(w, sheet_name="Efficiency_by_Peers", index=False)
            pd.DataFrame(by_r).to_excel(w, sheet_name="Efficiency_by_Sim", index=False)
            db.sort_values("best_z", ascending=False).head(10000).to_excel(w, sheet_name="Top_Hubs", index=False)
            if vault is not None and len(vault):
                vault.to_excel(w, sheet_name="Strategy_Vault", index=False)
            if "crawled_operators" in report:
                pd.DataFrame(report["crawled_operators"]).to_excel(w, sheet_name="Crawled_Operators", index=False)
            cz = report.get("czet_eval", {})
            if cz:
                pd.DataFrame([cz]).T.to_excel(w, sheet_name="CZET_Eval")
            nt = report.get("null_test", {})
            if nt:
                pd.DataFrame([{k: (str(v) if isinstance(v, (dict, list)) else v) for k, v in nt.items()}]).T.to_excel(w,
sheet_name="Null_Test")
            fr = report.get("fractal_report")
            if isinstance(fr, dict):
                flat = {k: (str(v) if isinstance(v, (dict, list)) else v) for k, v in fr.items()}
                pd.DataFrame([flat]).T.to_excel(w, sheet_name="Fractal_vs_Macro")
        print("[+] xlsx written")
    except Exception as e:
        print(f"[-] xlsx skipped: {type(e).__name__} {e}")


if __name__ == "__main__":
    main()
```

phase2/phase2_gatekeeper_verifier.py - the ten Phase 2 checks (129 lines, 6.2 KB)

```
import os
import sys
import json
import numpy as np
import pandas as pd

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
import pqcompat

_orig_read_parquet = pd.read_parquet


def _read_parquet(p, *a, **k):
    try:
        return _orig_read_parquet(p, *a, **k)
    except Exception:
        return pqcompat.read_parquet(p)


pd.read_parquet = _read_parquet

PHASE2_DIR = "C:/cluster/phase2"

def run_phase2_gatekeeper_audit():
    print("=" * 78)
    print("         PHASE 2 MASTER GATEKEEPER VERIFICATION CHECKLIST")
    print("=" * 78)
    results = {}

    def check(item_id, name, passed, detail):
        status = "PASS" if passed else "MISSING / INCOMPLETE"
        icon = "[+]" if passed else "[!]"
        print(f"{icon} {item_id}. {name:<42} : {status} ({detail})")
        results[item_id] = {"name": name, "passed": bool(passed), "detail": detail}

    # 1. Subzones File & Contiguity
    npz_path = os.path.join(PHASE2_DIR, "jump10_phase2_subzones.npz")
    if os.path.exists(npz_path):
        z = np.load(npz_path, mmap_mode="r")
        keys = list(z.keys())
        n_rows = z["desc_Q1"].shape[0] if "desc_Q1" in keys else 0
        check("01", "591,171 Contiguous Cell Coverage", n_rows == 591171, f"Rows={n_rows:,}, Keys={len(keys)}")
```

```
    # 2. Structural Quartiles (Q1..Q4)
    has_q = all(k in keys for k in ["desc_Q1", "desc_Q2", "desc_Q3", "desc_Q4"])
    check("02", "0-25-50-75-100% & Fib Zone Decomposition", has_q, f"Q1..Q4 present={has_q}")

    # 3. 12-Angle Brute-Force Feature Density
    t_key = "trigger_vec_dense" if "trigger_vec_dense" in keys else ("trigger_vec" if "trigger_vec" in keys else None)
    t_dim = z[t_key].shape[1] if t_key else 0
    manifest_exists = os.path.exists(os.path.join(PHASE2_DIR, "crawled_math_operators_manifest.json"))
    check("03", "12-Angle Brute-Force + Webcrawl Features", t_dim >= 50 and manifest_exists,
          f"vec={t_key} (dim={t_dim}), webcrawl_manifest={manifest_exists}")

    # 4. Whitening & Bicoherence Fix Check
    bico = np.array(z["bicoherence"]) if "bicoherence" in keys else np.ones(10)
    bico_std = float(np.std(bico))
    T_sample = np.array(z[t_key][:2000])
    rand_sims = T_sample[:1000] @ T_sample[1000:2000].T
    mean_abs_r = float(np.mean(np.abs(rand_sims)))
    check("04", "Zero Collinear Collapse & Fixed Bicoherence", mean_abs_r < 0.25 and bico_std > 0.01,
          f"random |r|={mean_abs_r:.4f}, bico_std={bico_std:.4f}")
else:
    for i in ["01", "02", "03", "04"]:
        check(i, "Subzone Tensor Checks", False, "jump10_phase2_subzones.npz missing")

# 5. Relational Covariant DB & Positive/Negative Mirror Pairs
cov_db_path = os.path.join(PHASE2_DIR, "phase2_covariant_db.parquet")
edges_path = os.path.join(PHASE2_DIR, "phase2_covariant_edges.parquet")
if os.path.exists(cov_db_path) and os.path.exists(edges_path):
    df_cov = pd.read_parquet(cov_db_path)
    df_edg = pd.read_parquet(edges_path)
    has_mirrors = ("pos_corr_peers" in df_cov.columns) and ("neg_mirror_peers" in df_cov.columns)
    check("05", "Relational Covariant DB (+/-0.85 Mirrors)", len(df_cov) > 1000 and has_mirrors,
          f"Hubs={len(df_cov):,}, Edges={len(df_edg):,}, Pos/Neg Mirrors={has_mirrors}")
else:
    check("05", "Relational Covariant DB (+/-0.85 Mirrors)", False, "Parquet files missing")

# 6. Inbound Pass 1 & Pass 2 Neural Convergence (CZET + Learnable Kernels)
czet_eval_path = os.path.join(PHASE2_DIR, "czet_eval.json")
if os.path.exists(czet_eval_path):
    with open(czet_eval_path, "r") as f:
        ev = json.load(f)
    spearman = float(ev.get("spearman", ev.get("pair_spearman", 0.15)))
    r2_val = float(ev.get("r2", ev.get("oos_r2", 0.0)))
    check("06", "CZET Inbound Pass 1&2 Continuation Ranking", spearman >= 0.28 and r2_val > 0.02,
          f"Spearman={spearman:.3f}, R2={r2_val:.4f} (Needs LearnableKernel/SweetSpot retraining if low)")
else:
    check("06", "CZET Inbound Pass 1&2 Continuation Ranking", False, "czet_eval.json missing")

# 7, 8, 9, 10. Strategy Vault, Fractal Multi-Timeframe, Indicator Coupling & Null Test
vault_path = os.path.join(PHASE2_DIR, "phase2_strategy_vault.parquet")
if os.path.exists(vault_path):
    df_v = pd.read_parquet(vault_path)
    cols = set(df_v.columns)
    check("07", "Outbound Streaming Strategy Vault (>=55%)", len(df_v) >= 100, f"Vault Strategies={len(df_v):,}")

    has_fractal = any("fractal" in c.lower() or "macro" in c.lower() for c in cols)
    check("08", "Multi-Timeframe Fractal Check (150t vs 1800t)", has_fractal,
          f"Fractal column present={has_fractal}")

    has_ind = any(k in c.lower() for c in cols for k in ["rsi", "vwap", "indicator", "divergence"])
    check("09", "Coupled Indicator Functions (RSI/VWAP)", has_ind,
          f"Indicator confluence columns present={has_ind}")

    has_null = any("null" in c.lower() or "pval" in c.lower() or "wilson" in c.lower() for c in cols)
    check("10", "Statistical Null / Wilson Lower Bound Audit", has_null,
          f"Statistical bound present={has_null}")
else:
    for i, lbl in [("07", "Outbound Strategy Vault"), ("08", "Multi-TF Fractal Check"),
                   ("09", "Coupled Indicator Functions"), ("10", "Statistical Null Audit")]:
        check(i, lbl, False, "phase2_strategy_vault.parquet missing")

all_passed = all(v["passed"] for v in results.values())
print("=" * 78)
if all_passed:
    print("[VERDICT] PHASE 2 IS 100% COMPLETE & VERIFIED. CLEAR TO LAUNCH PHASE 3!")
else:
    missing = [k for k, v in results.items() if not v["passed"]]
    print(f"[VERDICT] PHASE 2 HAS {len(missing)} UNFINISHED ITEM(S): {missing}")
    print("          Complete the missing items above before moving to Phase 3.")
print("=" * 78)
```

```
    with open(os.path.join(PHASE2_DIR, "phase2_gatekeeper_status.json"), "w") as f:
        json.dump({"all_passed": all_passed, "checks": results}, f, indent=2)

if __name__ == "__main__":
    run_phase2_gatekeeper_audit()
```

phase2/phase2_vault_enrich.py - indicator confluence columns (276 lines, 12.3 KB)

```
"""Phase 2 vault enrichment: items 08 (macro fractal), 09 (RSI/VWAP), 10 (null test).

Inputs : jump10_phase2_scan.npz, jump10_phase1_fourier_functions.npz (raw_tick_cells),
         phase2_covariant_db.parquet, phase2_strategy_vault.parquet
Outputs: vault_triggers.npz, indicator_features.npz, phase2_strategy_vault.parquet (enriched),
         phase2_anomaly_strategies.xlsx, phase2_enrichment_report.json
"""
import os
import sys
import json
import numpy as np
import pandas as pd

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
import pqcompat
from phase2_dense_anomaly_bank import compute_quartile_brute_force_features

BASE = r"C:\cluster\phase2"
P1 = r"C:\cluster\phase1\final\jump10_phase1_fourier_functions.npz"
N = 591171
N_IN = 502495
BLOCK = 12
CHUNK = 40000


def read_table(p):
    return pqcompat.read_parquet(p) if p.endswith(".parquet") else pd.read_csv(p)


def recompute_triggers(thr=0.85, min_hits=10, top_hubs=5000, batch=256):
    scan = np.load(os.path.join(BASE, "jump10_phase2_scan.npz"), mmap_mode="r")
    T = scan["trigger_vec_dense"]
    db = read_table(os.path.join(BASE, "phase2_covariant_db.parquet"))
    db = db[db.anchor_cell_id <= N_IN].sort_values("anomaly_z_score", ascending=False).reset_index(drop=True)
    hubs = db.head(top_hubs)
    vault = read_table(os.path.join(BASE, "phase2_strategy_vault.parquet"))
    pos = {int(a): i for i, a in enumerate(hubs.anchor_cell_id.to_numpy())}
    want = [pos[int(a)] for a in vault.anchor_cell_id]
    if any(int(a) not in pos for a in vault.anchor_cell_id):
        raise RuntimeError("vault anchor not in top_hubs set")
    Th = np.ascontiguousarray(T[[int(a) - 1 for a in hubs.anchor_cell_id.to_numpy()]])

    out_idx, out_pol, out_off = [], [], [0]
    for b0 in range(0, len(want), batch):
        sel = want[b0:b0 + batch]
        hits = [[] for _ in sel]
        for s in range(N_IN, N, 20000):
            e = min(N, s + 20000)
            sims = T[s:e] @ Th[sel].T
            big = np.abs(sims) >= thr
            if big.any():
                ri, ci = np.nonzero(big)
                for r, c in zip(ri, ci):
                    hits[c].append((s + r, sims[r, c]))
        for lst in hits:
            idx = np.fromiter((x[0] for x in lst), dtype=np.int64, count=len(lst))
            pol = np.fromiter((np.sign(x[1]) for x in lst), dtype=np.float32, count=len(lst))
            out_idx.append(idx)
            out_pol.append(pol)
            out_off.append(out_off[-1] + len(idx))
        print(f"    triggers {b0 + len(sel)}/{len(want)}", flush=True)
    np.savez(os.path.join(BASE, "vault_triggers.npz"),
             idx=np.concatenate(out_idx) if out_idx else np.zeros(0, np.int64),
             pol=np.concatenate(out_pol) if out_pol else np.zeros(0, np.float32),
             off=np.asarray(out_off, np.int64),
             hubs=vault.anchor_cell_id.to_numpy(np.int64))
    return vault, out_idx, out_pol


def indicator_block():
    cache = os.path.join(BASE, "indicator_features.npz")
    if os.path.exists(cache):
```

```
        d = np.load(cache)
        return {k: d[k] for k in d.files}
    z1 = np.load(P1, mmap_mode="r")
    px = z1["raw_tick_cells"]
    out = {}
    names = ["rsi_end1", "rsi_end2", "rsi_max1", "rsi_max2", "rsi_min1", "rsi_min2",
             "px_high1", "px_high2", "px_low1", "px_low2", "vwap_z1", "vwap_z2", "vwap_stretch1"]
    for n in names:
        out[n] = np.lib.format.open_memmap(os.path.join(BASE, f"_ind_{n}.npy"), mode="w+",
                                           dtype=np.float32, shape=(N,))
    W = 14
    for s in range(0, N, CHUNK):
        e = min(N, s + CHUNK)
        p = np.asarray(px[s:e], dtype=np.float64)
        d = np.diff(p, axis=1)
        up = np.clip(d, 0.0, None)
        dn = np.clip(-d, 0.0, None)
        cu = np.cumsum(up, axis=1)
        cd = np.cumsum(dn, axis=1)
        cs_u = np.concatenate([np.zeros((e - s, 1)), cu], axis=1)
        cs_d = np.concatenate([np.zeros((e - s, 1)), cd], axis=1)
        su = cs_u[:, W:] - cs_u[:, :-W]
        sd = cs_d[:, W:] - cs_d[:, :-W]
        rsi = 100.0 * su / (su + sd + 1e-12)
        full = np.full((e - s, p.shape[1]), 50.0)
        full[:, W:] = rsi
        full[:, 1:W] = rsi[:, :1]
        h1, h2 = full[:, :75], full[:, 75:]
        out["rsi_end1"][s:e] = full[:, 74]
        out["rsi_end2"][s:e] = full[:, -1]
        out["rsi_max1"][s:e] = h1.max(1)
        out["rsi_max2"][s:e] = h2.max(1)
        out["rsi_min1"][s:e] = h1.min(1)
        out["rsi_min2"][s:e] = h2.min(1)
        out["px_high1"][s:e] = p[:, :75].max(1)
        out["px_high2"][s:e] = p[:, 75:].max(1)
        out["px_low1"][s:e] = p[:, :75].min(1)
        out["px_low2"][s:e] = p[:, 75:].min(1)
        csum = np.cumsum(p, axis=1)
        cs2 = np.cumsum(p * p, axis=1)
        cs_pad = np.concatenate([np.zeros((e - s, 1)), csum], axis=1)
        cs2_pad = np.concatenate([np.zeros((e - s, 1)), cs2], axis=1)
        VW = 55
        m = (cs_pad[:, VW:] - cs_pad[:, :-VW]) / VW
        v = (cs2_pad[:, VW:] - cs2_pad[:, :-VW]) / VW - m * m
        sdv = np.sqrt(np.maximum(v, 1e-12))
        dev = p[:, VW - 1:] - m
        zsc = dev / sdv
        ncols = zsc.shape[1]
        fullz = np.zeros((e - s, p.shape[1]))
        fullz[:, VW - 1:] = zsc
        fullz[:, :VW - 1] = zsc[:, :1]
        out["vwap_z1"][s:e] = fullz[:, 74]
        out["vwap_z2"][s:e] = fullz[:, -1]
        out["vwap_stretch1"][s:e] = np.abs(fullz[:, :75]).max(1)
        print(f"    indicators {e:,}/{N:,} ({ncols} cols)", flush=True)
    for n in names:
        out[n].flush()
    np.savez(os.path.join(BASE, "indicator_features.npz"), **{n: np.asarray(out[n]) for n in names})
    return {n: np.asarray(out[n]) for n in names}


def macro_fractal(vault_anchor):
    z1 = np.load(P1, mmap_mode="r")
    px = z1["raw_tick_cells"]
    n_blocks = int(np.ceil(N / BLOCK))
    cached = os.path.join(BASE, "_macro_desc.npy")
    if os.path.exists(cached):
        macro_desc = np.load(cached)
    else:
      macro_desc = np.lib.format.open_memmap(cached, mode="w+", dtype=np.float32, shape=(n_blocks, 64))
      for b0 in range(0, n_blocks, 4000):
        b1 = min(n_blocks, b0 + 4000)
        rows = np.empty((b1 - b0, 150), dtype=np.float64)
        for i, bi in enumerate(range(b0, b1)):
            s = bi * BLOCK
            e = min(N, s + BLOCK)
            blk = np.asarray(px[s:e], dtype=np.float64)
            if blk.shape[0] < BLOCK:
                blk = np.vstack([blk, np.repeat(blk[-1:], BLOCK - blk.shape[0], axis=0)])
            rows[i] = blk.reshape(BLOCK, 150).mean(axis=0)
```

```
        macro_desc[b0:b1] = compute_quartile_brute_force_features(rows)
        print(f"    macro desc {b1:,}/{n_blocks:,}", flush=True)
    macro_desc = np.asarray(macro_desc)

    hub_idx = vault_anchor.astype(np.int64) - 1
    micro = np.empty((len(hub_idx), 150), dtype=np.float64)
    for i in range(0, len(hub_idx), 2000):
        sl = hub_idx[i:i + 2000]
        micro[i:i + 2000] = np.asarray(px[sl], dtype=np.float64)
    micro_desc = compute_quartile_brute_force_features(micro)
    blocks = hub_idx // BLOCK

    def cos64(a, b):
        an = (a - a.mean(1, keepdims=True)) / (a.std(1, keepdims=True) + 1e-9)
        bn = (b - b.mean(1, keepdims=True)) / (b.std(1, keepdims=True) + 1e-9)
        return np.sum(an * bn, axis=1) / a.shape[1]

    agree = cos64(micro_desc, macro_desc[blocks])
    return agree, micro_desc, macro_desc


def main():
    vault, trig_idx, trig_pol = recompute_triggers()
    print(f"[*] vault rows={len(vault)} triggers={len(trig_idx)}", flush=True)
    scan = np.load(os.path.join(BASE, "jump10_phase2_scan.npz"), mmap_mode="r")
    held = np.asarray(scan["protected_held"])
    ret = np.asarray(scan["q4_net_return"])

    print("[*] item 08 macro fractal ...", flush=True)
    agree, _, _ = macro_fractal(vault.anchor_cell_id.to_numpy(np.int64))
    is_frac = agree >= float(np.quantile(agree, 0.90))
    print(f"    agreement med={np.median(agree):.4f} p90={np.quantile(agree,0.9):.4f} "
          f"frac_anomalies={int(is_frac.sum())}", flush=True)

    print("[*] item 09 indicator block ...", flush=True)
    ind = indicator_block()

    print("[*] item 10 block-permutation null ...", flush=True)
    rng = np.random.default_rng(20261025)
    nb = N - N_IN
    n_perm = 200
    n_full = nb // 150
    idx_base = (np.arange(n_full)[:, None] * 150 + np.arange(150)[None, :]).ravel()
    tail = np.arange(n_full * 150, nb, dtype=np.int64)
    perms = np.empty((n_perm, len(trig_idx)), dtype=np.float32)
    perm_held = np.empty((n_perm, len(trig_idx)), dtype=np.float32)
    held_o = held[N_IN:]
    ret_o = ret[N_IN:]
    li_all = [idx - N_IN for idx in trig_idx]
    for p in range(n_perm):
        pm = rng.permutation(n_full)
        newpos = np.concatenate([(pm[:, None] * 150 + np.arange(150)[None, :]).ravel(), tail])
        ph = held_o[newpos]
        pr = ret_o[newpos]
        for i, li in enumerate(li_all):
            perms[p, i] = float(np.mean(np.sign(pr[li] * trig_pol[i]) == 1))
            perm_held[p, i] = float(np.mean(ph[li]))
        if (p + 1) % 50 == 0:
            print(f"    perm {p + 1}/{n_perm}", flush=True)

    obs_dir, obs_prot, pvals = [], [], []
    for i, idx in enumerate(trig_idx):
        li = idx - N_IN
        od = float(np.mean(np.sign(ret[li] * trig_pol[i]) == 1))
        op = float(np.mean(held[li]))
        obs_dir.append(od)
        obs_prot.append(op)
        null = np.maximum(perms[:, i], perm_held[:, i])
        pvals.append(float((np.sum(null >= max(od, op)) + 1) / (n_perm + 1)))

    vault = vault.copy()
    vault["fractal_macro_agreement"] = np.round(agree, 4)
    vault["is_fractal_anomaly"] = is_frac
    vault["rsi_divergence_confluence"] = np.round(
        np.where((ind["px_high2"][vault.anchor_cell_id - 1] > ind["px_high1"][vault.anchor_cell_id - 1])
                 & (ind["rsi_max2"][vault.anchor_cell_id - 1] < ind["rsi_max1"][vault.anchor_cell_id - 1]), 1.0,
                 np.where((ind["px_low2"][vault.anchor_cell_id - 1] < ind["px_low1"][vault.anchor_cell_id - 1])
                          & (ind["rsi_min2"][vault.anchor_cell_id - 1] > ind["rsi_min1"][vault.anchor_cell_id - 1]), -1.0,
0.0)), 2)
    vault["vwap_band_anomaly_score"] = np.round(ind["vwap_stretch1"][vault.anchor_cell_id - 1], 3)
    vault["rsi_end1"] = np.round(ind["rsi_end1"][vault.anchor_cell_id - 1], 2)
```

```
    vault["vwap_z1"] = np.round(ind["vwap_z1"][vault.anchor_cell_id - 1], 3)
    vault["null_pval"] = np.round(pvals, 4)
    vault["null_p99"] = np.round(np.quantile(np.maximum(perms, perm_held), 0.99, axis=0), 4)
    vault["beats_null"] = vault["null_pval"] < 0.01
    vault["outbound_protected_win_rate"] = np.round(obs_prot, 4)
    vault["outbound_directional_win_rate"] = np.round(obs_dir, 4)
    vault["outbound_verified_edge"] = np.round(np.maximum(obs_prot, obs_dir), 4)

    pqcompat.write_parquet(vault, os.path.join(BASE, "phase2_strategy_vault.parquet"))
    try:
        with pd.ExcelWriter(os.path.join(BASE, "phase2_anomaly_strategies.xlsx"), engine="openpyxl") as w:
            vault.to_excel(w, sheet_name="Phase3_Ready_Strategies", index=False)
            vault[vault["beats_null"]].to_excel(w, sheet_name="Null_Verified", index=False)
    except Exception as ex:
        print(f"[-] xlsx: {type(ex).__name__} {ex}", flush=True)

    top300 = vault.nlargest(min(300, len(vault)), "outbound_win_lb95")
    rep = {
        "vault_rows": int(len(vault)),
        "fractal": {"median_agreement": round(float(np.median(agree)), 4),
                    "p90_agreement": round(float(np.quantile(agree, 0.9)), 4),
                    "is_fractal_anomalies": int(is_frac.sum())},
        "indicators": {
            "rsi_divergence_bearish": int((vault["rsi_divergence_confluence"] == 1).sum()),
            "rsi_divergence_bullish": int((vault["rsi_divergence_confluence"] == -1).sum()),
            "vwap_stretch_median": round(float(vault["vwap_band_anomaly_score"].median()), 3),
        },
        "null_test": {
            "permutations": n_perm, "block_size_ticks": 150,
            "top300_observed_mean_win": round(float(top300["outbound_verified_edge"].mean()), 4),
            "top300_null_p99_mean": round(float(top300["null_p99"].mean()), 4),
            "top300_beats_null": int(top300["beats_null"].sum()),
            "vault_beats_null": int(vault["beats_null"].sum()),
            "median_pval": round(float(np.median(pvals)), 4),
        },
    }
    with open(os.path.join(BASE, "phase2_enrichment_report.json"), "w") as f:
        json.dump(rep, f, indent=2)
    print(json.dumps(rep, indent=2), flush=True)


if __name__ == "__main__":
    main()
```

phase2/node_export_840g7.py - HP 840 G7 independent audit (57 lines, 2.5 KB)

```
"""Runs on 840g7 - independent re-verification of the Phase 2 relational DB / vault
and regeneration of the strategy workbook (offloads openpyxl work from XFlip)."""
import os
import sys
import json
import numpy as np

BASE = r"C:\cluster\phase2"
sys.path.insert(0, BASE)
import pqcompat
import pandas as pd

db = pqcompat.read_parquet(os.path.join(BASE, "phase2_covariant_db.parquet"))
ed = pqcompat.read_parquet(os.path.join(BASE, "phase2_covariant_edges.parquet"))
v = pqcompat.read_parquet(os.path.join(BASE, "phase2_strategy_vault.parquet"))

n_und = len({(min(a, b), max(a, b)) for a, b in zip(ed.src_idx, ed.dst_idx)})
sym = len({(a, b) for a, b in zip(ed.src_idx, ed.dst_idx)} & {(b, a) for a, b in zip(ed.src_idx, ed.dst_idx)})
loops = int((ed.src_idx == ed.dst_idx).sum())
hubs = int(db.anchor_cell_id.nunique())
dup_cols = int(ed.duplicated().sum())
res = {
    "node": "840g7",
    "db_rows": int(len(db)),
    "db_unique_anchors": hubs,
    "edge_rows": int(len(ed)),
    "edge_unique_undirected": int(n_und),
    "edge_rows_that_are_mirrored": int(sym),
    "edge_self_loops": loops,
    "edge_exact_duplicate_rows": dup_cols,
    "vault_rows": int(len(v)),
    "vault_ok_rows": int((v.sample_flag == "OK").sum()),
    "vault_thin_rows": int((v.sample_flag == "THIN").sum()),
    "vault_min_edge": round(float(v.outbound_verified_edge.min()), 4),
```

```
    "vault_max_edge": round(float(v.outbound_verified_edge.max()), 4),
    "vault_rows_above_55pct": int((v.outbound_verified_edge >= 0.55).sum()),
    "beats_null_rows": int(v.beats_null.astype(bool).sum()),
    "czet_spearman": None,
}
ce = os.path.join(BASE, "czet_eval.json")
if os.path.exists(ce):
    res["czet_spearman"] = json.load(open(ce))["spearman"]

pad = lambda s, n=44: s + " " * max(0, n - len(s))
for k, val in res.items():
    print(pad(k) + ": " + str(val))

with pd.ExcelWriter(os.path.join(BASE, "phase2_anomaly_strategies.xlsx"), engine="openpyxl") as w:
    v.to_excel(w, sheet_name="Phase3_Ready_Strategies", index=False)
    v[v.beats_null.astype(bool)].to_excel(w, sheet_name="Null_Verified", index=False)
    v[(v.sample_flag == "OK") & (v.outbound_verified_edge >= 0.55)].sort_values(
        "outbound_win_lb95", ascending=False).to_excel(w, sheet_name="Ranked_by_Wilson_LB", index=False)
    db.sort_values("anomaly_z_score", ascending=False).head(2000).to_excel(w, sheet_name="Inbound_Covariant_Hubs",
index=False)
print("[+] strategies workbook written")

with open(os.path.join(BASE, "node_verify_840g7.json"), "w") as f:
    json.dump(res, f, indent=2)
```

phase2/node_verify_kali.py - Kali independent audit (46 lines, 1.9 KB)

```
"""Runs on Kali - independent edge-graph audit from the CSV (no parquet engine needed)."""
import os
import json
import numpy as np
import pandas as pd

BASE = os.path.expanduser("~/cluster/phase2")
ed = pd.read_csv(os.path.join(BASE, "phase2_covariant_edges.csv"))
db = pd.read_csv(os.path.join(BASE, "phase2_covariant_db.csv"))

es = ed.src_idx.to_numpy(np.int64)
ds = ed.dst_idx.to_numpy(np.int64)
lo = np.minimum(es, ds)
hi = np.maximum(es, ds)
pairs = np.stack([lo, hi], 1)
uniq = np.unique(pairs, axis=0)

deg = np.bincount(np.concatenate([lo, hi]))
res = {
    "node": "kali",
    "edge_rows": int(len(ed)),
    "unique_undirected_edges": int(len(uniq)),
    "self_loops": int((es == ds).sum()),
    "duplicate_pairs": int(len(ed) - len(uniq)),
    "nodes_touched": int((deg > 0).sum()),
    "max_degree": int(deg.max()),
    "median_degree_nonzero": float(np.median(deg[deg > 0])),
    "cov_min": round(float(ed.cov_0_50.abs().min()), 4),
    "cov_median": round(float(ed.cov_0_50.abs().median()), 4),
    "cov_max": round(float(ed.cov_0_50.abs().max()), 4),
    "lock_median_abs": round(float(ed.lock_50_100.abs().median()), 4),
    "lock_ge_065": int((ed.lock_50_100.abs() >= 0.65).sum()),
    "db_rows": int(len(db)),
    "db_hubs": int(db.anchor_cell_id.nunique()),
    "hubs_appearing_in_edges": int(np.isin(db.anchor_cell_id.to_numpy(np.int64) - 1, np.unique(np.concatenate([lo,
hi])))).sum()),
    "peers_mean": round(float(db.covariant_peer_count.mean()), 3),
    "peers_median": float(db.covariant_peer_count.median()),
    "mean_sim_median": round(float(db.mean_0_50_similarity.median()), 4),
    "cells_above_085_mean_sim": int((db.mean_0_50_similarity >= 0.85).sum()),
}
pad = lambda s, n=40: s + " " * max(0, n - len(s))
for k, val in res.items():
    print(pad(k) + ": " + str(val))
with open(os.path.join(BASE, "node_verify_kali.json"), "w") as f:
    json.dump(res, f, indent=2)
print("[+] kali audit written")
```

A.3 Phase 3 - replay engine, tester, verifier

phase3/phase3_master_engine.py - shard/merge engine, contract auction, friction ladder, Monte Carlo (731 lines, 33.3 KB)

```
"""PHASE 3 STAGE 3A - distributed tick-exact backtester, Fable sigma survival audit,
Deriv contract EV auction, deduplicated portfolio equity + Monte Carlo.

Usage
-----
worker : python -u phase3_master_engine.py --node_id K --num_nodes 3
merge  : python -u phase3_master_engine.py --merge

Design notes
------------
* Strict causality: direction, dip/TP/SL quantiles, sigma bins and the per-bin
  Wilson lower bounds used by the EV gate are all calibrated on INBOUND cells
  (0..502,494). Outbound cells (502,495..591,171) are only ever scored.
* Similarity band: 0.85 <= |r| <= 0.955 (the Phase 2 sweet spot).
* Orthogonal pruning: greedy on outbound_win_lb95, suppressing any anchor whose
  |r| with an already kept anchor is >= 0.82.
* Parquet I/O goes through pqcompat (duckdb first) with a CSV fallback so the
  engine runs on Windows (AppControl blocks fastparquet) and on Kali alike.
"""
import os
import sys
import json
import time
import argparse
import numpy as np
import pandas as pd

BASE3 = os.path.dirname(os.path.abspath(__file__))
for cand in (BASE3, r"C:\cluster\phase2"):
    if os.path.isdir(cand) and cand not in sys.path:
        sys.path.insert(0, cand)

N_IN = 502495
SIM_MIN, SIM_MAX = 0.85, 0.955
PEER_MIN, PEER_MAX = 12, 400
ORTHO_CUT = 0.82
FRICTION_TIERS = (("gross_00", 0.0), ("net_03", 0.03), ("net_06", 0.06), ("net_10", 0.10))
PAYOUT = {"RISE_FALL": 1.953, "ONETOUCH_075": 2.150, "BARRIER_035": 2.427, "NOTOUCH_10": 1.55}

_cfg = {"inputs": None, "catalog_times": None, "phase2_dir": r"C:\cluster\phase2"}


# ------------------------------------------------------------------------- io
def _pq():
    try:
        import pqcompat
        return pqcompat
    except Exception:
        return None


def load_table(path):
    p = _pq()
    if path.endswith(".parquet"):
        if p is not None:
            try:
                return p.read_parquet(path)
            except Exception:
                pass
        csv = path[:-8] + ".csv"
        if os.path.exists(csv):
            return pd.read_csv(csv)
        raise RuntimeError(f"cannot read {path}")
    return pd.read_csv(path)


def save_table(df, path):
    p = _pq()
    if p is not None:
        try:
            p.write_parquet(df, path)
            return
        except Exception:
            pass
    df.to_csv(path[:-8] + ".csv", index=False)
```

```
def wilson_lb(wins, n, z=1.96):
    n = int(n)
    if n <= 0:
        return 0.0
    p = wins / n
    den = 1.0 + z * z / n
    ctr = p + z * z / (2.0 * n)
    half = z * np.sqrt((p * (1 - p) + z * z / (4.0 * n)) / n)
    return float(max(0.0, (ctr - half) / den))


# ---------------------------------------------------------------------- sigma (Fable)
def winsorized_sigma(raw, window_cells=12, chunk=20000, sample_step=25):
    """1,800-tick (12-cell) rolling winsorized sigma up to each cell's tick-74 mark."""
    N = raw.shape[0]
    sample = np.asarray(raw[::sample_step], dtype=np.float64)
    absd = np.abs(np.diff(sample, axis=1))
    global_med = float(np.median(absd))
    cap = max(global_med * 6.0, 1e-9)
    cell_var = np.empty(N, dtype=np.float64)
    for s in range(0, N, chunk):
        e = min(N, s + chunk)
        d = np.diff(np.asarray(raw[s:e], dtype=np.float64), axis=1)
        sq = np.where(np.abs(d) <= cap, d * d, np.nan)
        with np.errstate(invalid="ignore"):
            v = np.nanmean(sq, axis=1)
        cell_var[s:e] = np.where(np.isfinite(v), v, global_med ** 2)
    csum = np.concatenate([[0.0], np.cumsum(cell_var)])
    idx = np.arange(N)
    lo = np.maximum(0, idx - (window_cells - 1))
    cnt = idx + 1 - lo
    roll = (csum[idx + 1] - csum[lo]) / cnt
    return np.sqrt(np.maximum(roll, 1e-12)).astype(np.float32), global_med, cap


def sigma_bins(rel_bps, n_in=N_IN):
    qs = np.quantile(rel_bps[:n_in], [0.0, 0.2, 0.4, 0.6, 0.8, 1.0])
    bid = np.clip(np.digitize(rel_bps, qs[1:-1]), 0, 4)
    return qs, bid


# ---------------------------------------------------------------------- helpers
def orthogonal_pruning(T_anchors, wlb):
    order = np.argsort(-wlb)
    A = T_anchors[order].astype(np.float32)
    A /= np.maximum(np.linalg.norm(A, axis=1, keepdims=True), 1e-9)
    kept, kept_pos = [], []
    for i in range(len(order)):
        if kept:
            s = np.abs(A[kept] @ A[i])
            if np.any(s >= ORTHO_CUT):
                continue
        kept.append(i)
        kept_pos.append(order[i])
    return np.array(sorted(kept_pos), dtype=np.int64), len(kept), len(order)


def first_cross(v, level, ge=True):
    idx = np.nonzero(v >= level)[0] if ge else np.nonzero(v <= level)[0]
    return int(idx[0]) if len(idx) else 10 ** 9


def vectorized_outcomes(paths, tp=0.75, barrier=0.35):
    close = paths[:, -1]
    maxv = paths.max(axis=1)
    return {
        "close": close,
        "win_dir": (close > 0).astype(np.int8),
        "win_touch": (maxv >= tp).astype(np.int8),
        "win_bar": (close >= barrier).astype(np.int8),
    }


def run_node(node_id, num_nodes, out_dir, top_elite=0):
    t0 = time.time()
    inputs = os.environ.get("P3_INPUTS", os.path.join(BASE3, "phase3_inputs.npz"))
    vault_path = os.environ.get("P3_VAULT", os.path.join(_cfg["phase2_dir"], "phase2_strategy_vault.parquet"))
    times_path = os.environ.get("P3_TIMES", os.path.join(BASE3, "phase3_catalog_times.npy"))
    os.makedirs(os.path.join(out_dir, "shards"), exist_ok=True)
```

```
    z = np.load(inputs)
    raw = z["raw"]
    T_all = z["trigger_vec"]
    N = raw.shape[0]
    print(f"[*] node {node_id}/{num_nodes}: inputs {raw.shape} {T_all.shape}", flush=True)

    df_vault = load_table(vault_path)
    df_vault = df_vault[(df_vault.covariant_peer_count >= PEER_MIN) &
                        (df_vault.covariant_peer_count <= PEER_MAX) &
                        (df_vault.mean_0_50_similarity >= SIM_MIN) &
                        (df_vault.mean_0_50_similarity <= SIM_MAX)].copy()
    df_vault = df_vault.reset_index(drop=True)
    print(f"[*] vault rows inside sweet spot: {len(df_vault):,}", flush=True)

    anchor0 = df_vault.anchor_cell_id.to_numpy(np.int64) - 1
    T_anch = T_all[anchor0]
    wlb = df_vault.outbound_win_lb95.to_numpy(np.float64)
    keep_idx, n_kept, n_before = orthogonal_pruning(T_anch, wlb)
    print(f"[*] orthogonal pruning: {n_before} -> {n_kept} anchors (|r_anchor| >= {ORTHO_CUT} suppressed)", flush=True)
    if top_elite:
        order = np.argsort(-(df_vault.outbound_win_lb95.to_numpy()[keep_idx]))
        keep_idx = keep_idx[order[:top_elite]]
        n_kept = len(keep_idx)
        print(f"[*] elite cap -> {n_kept}", flush=True)

    df_kept = df_vault.iloc[keep_idx].reset_index(drop=True)
    anchor0 = anchor0[keep_idx]
    T_anch = T_all[anchor0]

    bounds = [int(round(n_kept * i / num_nodes)) for i in range(num_nodes + 1)]
    s0, s1 = bounds[node_id], bounds[node_id + 1]
    df_shard = df_kept.iloc[s0:s1].reset_index(drop=True)
    T_shard = T_anch[s0:s1]
    print(f"[*] shard strategies {s0}..{s1 - 1} ({len(df_shard)})", flush=True)
    if len(df_shard) == 0:
        return

    print("[*] computing Fable winsorized sigma ...", flush=True)
    sig_w, med_abs, cap = winsorized_sigma(raw)
    p74 = np.asarray(raw[:, 74], dtype=np.float64)
    rel_bps = (sig_w / np.maximum(np.abs(p74), 1e-9)) * 1e4
    qs, bid = sigma_bins(rel_bps)
    print(f"    sigma bins (bps): {np.round(qs, 3).tolist()}  med|d|={med_abs:.6g} cap={cap:.6g}", flush=True)

    try:
        times_ms = np.load(times_path)
    except Exception:
        times_ms = np.zeros(N, dtype=np.int64)
    hours = ((times_ms // 3600000) % 24).astype(np.int8) if times_ms.any() else np.zeros(N, np.int8)

    raw_out = np.asarray(raw[N_IN:], dtype=np.float64)
    std_out = raw_out[:, :75].std(axis=1)
    std_out = np.where(std_out < 1e-7, 1.0, std_out)
    paths_out = (raw_out[:, 75:] - raw_out[:, 74:75]) / std_out[:, None]
    T_out = T_all[N_IN:]
    sig_out = bid[N_IN:]
    hr_out = hours[N_IN:]
    n_out = raw_out.shape[0]
    print(f"[*] outbound cells {n_out:,}", flush=True)

    cal_step = 5
    T_cal = T_all[:N_IN:cal_step]
    raw_cal = np.asarray(raw[:N_IN:cal_step], dtype=np.float64)
    std_cal = raw_cal[:, :75].std(axis=1)
    std_cal = np.where(std_cal < 1e-7, 1.0, std_cal)
    paths_cal = (raw_cal[:, 75:] - raw_cal[:, 74:75]) / std_cal[:, None]
    sig_cal = bid[:N_IN:cal_step]
    print(f"[*] calibration cells {paths_cal.shape[0]:,}", flush=True)

    sim_out = (T_out @ T_shard.T).astype(np.float32)
    sim_cal = (T_cal @ T_shard.T).astype(np.float32)
    print(f"[*] similarity matrices {sim_out.shape} {sim_cal.shape} "
          f"({time.time() - t0:.0f}s)", flush=True)

    strat_rows, ledger = [], []
    bin_tot = {b: dict(n=0, dir=0, touch=0, bar=0, mult=0) for b in range(5)}
    hour_tot = {h: dict(n=0, win=0, units=0.0) for h in range(24)}

    for j in range(len(df_shard)):
        row = df_shard.iloc[j]
```

```
        sid = str(row.get("strategy_id", f"S{s0 + j}"))
        anc = int(row.anchor_cell_id)
        cal = sim_cal[:, j]
        chits = np.nonzero((np.abs(cal) >= SIM_MIN) & (np.abs(cal) <= SIM_MAX))[0]

        if len(chits) >= 20:
            cpol = np.sign(cal[chits])[:, None]
            aligned = paths_cal[chits] * cpol
            med_close = float(np.median(aligned[:, -1]))
            dir_sign = 1.0 if med_close >= 0 else -1.0
            sgn = aligned * dir_sign
            dip = sgn[:, :45].min(axis=1)
            limit_dip = float(np.clip(np.percentile(dip, 40), -1.20, -0.25))
            tp_cal = float(np.clip(np.percentile(sgn.max(axis=1), 65), 0.60, 2.50))
            sl_cal = float(np.clip(np.percentile(sgn.min(axis=1), 15), -2.50, -0.75))
            mode1_in = float(np.mean(sgn[:, -1]))
            filled = dip <= limit_dip
            mode3_in = float(np.mean(sgn[filled, -1] - limit_dip)) if filled.any() else -9.0

            occ = vectorized_outcomes(sgn)
            occ["win_notouch"] = (sgn.min(axis=1) > -1.0).astype(np.int8)
            wdir_all = float(occ["win_dir"].mean())
            wtouch_all = float(occ["win_touch"].mean())
            wbar_all = float(occ["win_bar"].mean())

            wlb_by_bin = {}
            for b in range(5):
                m = sig_cal[chits] == b
                nb = int(m.sum())
                if nb >= 12:
                    wlb_by_bin[b] = (
                        wilson_lb(int(occ["win_dir"][m].sum()), nb),
                        wilson_lb(int(occ["win_touch"][m].sum()), nb),
                        wilson_lb(int(occ["win_bar"][m].sum()), nb),
                        wilson_lb(int(occ["win_notouch"][m].sum()), nb),
                    )
            occ["win_notouch"] = (sgn.min(axis=1) > -1.0).astype(np.int8)
            fb = (wilson_lb(int(occ["win_dir"].sum()), len(chits)),
                  wilson_lb(int(occ["win_touch"].sum()), len(chits)),
                  wilson_lb(int(occ["win_bar"].sum()), len(chits)),
                  wilson_lb(int(occ["win_notouch"].sum()), len(chits)))
            if not wlb_by_bin:
                wlb_by_bin = {2: fb}
        else:
            dir_sign = -1.0 if "SELL" in str(row.get("recommended_action", "BUY")).upper() else 1.0
            limit_dip, tp_cal, sl_cal = -0.50, 1.25, -1.25
            mode1_in, mode3_in = -9.0, -9.0
            wdir_all = wtouch_all = wbar_all = 0.0
            fb = (0.0, 0.0, 0.0, 0.0)
            wlb_by_bin = {2: fb}

        sims = sim_out[:, j]
        hits = np.nonzero((np.abs(sims) >= SIM_MIN) & (np.abs(sims) <= SIM_MAX))[0]
        if len(hits) == 0:
            continue

        pol = np.sign(sims[hits])
        curve = paths_out[hits] * pol[:, None] * dir_sign
        n_h = len(hits)

        close = curve[:, -1]
        maxv = curve.max(axis=1)
        minv = curve.min(axis=1)
        w_dir = close > 0
        w_touch = maxv >= 0.75
        w_bar = close >= 0.35
        w_notouch = minv > -1.0

        tp_first = np.array([first_cross(c, tp_cal) for c in curve])
        sl_first = np.array([first_cross(c, sl_cal, ge=False) for c in curve])
        mult_win = tp_first < sl_first
        mult_gross = np.where(mult_win, tp_cal, sl_cal)

        e2 = np.where(mult_win, tp_cal, np.where(sl_first < 75, sl_cal, close))

        fill = curve[:, :45].min(axis=1) <= limit_dip
        e1_gross = np.zeros(n_h)
        for i in np.nonzero(fill)[0]:
            post = curve[i, :] - limit_dip
            ts = first_cross(post, sl_cal - limit_dip, ge=False)
            tpp = first_cross(post, tp_cal - limit_dip)
```

```
            e1_gross[i] = (tp_cal - limit_dip) if tpp < ts else (
                (sl_cal - limit_dip) if ts < len(post) else post[-1])

        bins_h = sig_out[hits]
        hrs_h = hr_out[hits]

        fallback = fb
        cons_ev = {}
        for cname, widx in (("RISE_FALL", 0), ("ONETOUCH_075", 1), ("BARRIER_035", 2), ("NOTOUCH_10", 3)):
            cons_ev[cname] = float(fallback[widx] * PAYOUT[cname] - 1.0)
        ev_mult = fallback[0] * (1.0 + (tp_cal - 0.03) / abs(sl_cal + 0.03)) - 1.0
        safe_bins = []
        for b in range(5):
            m = bins_h == b
            if m.sum() >= 8 and b in wlb_by_bin:
                if wlb_by_bin[b][0] * PAYOUT["RISE_FALL"] - 1.0 > 0.02:
                    safe_bins.append(b)
        cons_ev["MULTIPLIER_TP_SL"] = float(ev_mult)
        best_contract = max(cons_ev, key=cons_ev.get)

        realized = {
            "RISE_FALL": float(w_dir.mean()),
            "ONETOUCH_075": float(w_touch.mean()),
            "BARRIER_035": float(w_bar.mean()),
            "NOTOUCH_10": float(w_notouch.mean()),
            "MULTIPLIER_TP_SL": float(mult_win.mean()),
        }
        modes_net = {
            "DIRECTIONAL_CLOSE_149": close - 0.03,
            "EXCURSION_TOUCH_SNIPER": e2 - 0.03,
            "PROTECTED_EXTREMUM_LIMIT_FADE": np.where(fill, e1_gross - 0.03, np.nan),
        }
        in_modes = {"DIRECTIONAL_CLOSE_149": mode1_in, "EXCURSION_TOUCH_SNIPER": mode1_in,
                    "PROTECTED_EXTREMUM_LIMIT_FADE": mode3_in}
        best_mode = max(in_modes, key=in_modes.get)
        best_arr = modes_net[best_mode]
        best_arr = best_arr[np.isfinite(best_arr)]
        n_eff = int(len(best_arr))
        wins = int((best_arr > 0).sum())
        wr = wins / n_eff if n_eff else 0.0
        gp = float(best_arr[best_arr > 0].sum())
        gl = float(np.abs(best_arr[best_arr < 0].sum())) + 1e-9

        for i in range(n_h):
            b = int(bins_h[i])
            hr = int(hrs_h[i])
            bin_tot[b]["n"] += 1
            bin_tot[b]["dir"] += int(w_dir[i])
            bin_tot[b]["touch"] += int(w_touch[i])
            bin_tot[b]["bar"] += int(w_bar[i])
            bin_tot[b]["mult"] += int(mult_win[i])
            if 0 <= hr < 24:
                hour_tot[hr]["n"] += 1
                hour_tot[hr]["win"] += int(w_dir[i])
                hour_tot[hr]["units"] += (PAYOUT["RISE_FALL"] - 1.0) if w_dir[i] else -1.0
            ledger.append((sid, anc, int(N_IN + hits[i] + 1), float(abs(sims[hits[i]])),
                           int(pol[i]), int(dir_sign), b, hr, best_mode, best_contract,
                           float(close[i]), float(e2[i]) if np.isfinite(e2[i]) else float(close[i]),
                           float(e1_gross[i]), int(bool(fill[i])),
                           float(sig_out[hits[i]]), float(std_out[hits[i]]),
                           float(cons_ev[best_contract]),
                           float(maxv[i]), float(minv[i]), int(bool(w_notouch[i]))))

        strat_rows.append({
            "strategy_id": sid, "anchor_cell_id": anc,
            "vault_win_lb95": float(row.outbound_win_lb95),
            "covariant_peer_count": int(row.covariant_peer_count),
            "mean_0_50_similarity": float(row.mean_0_50_similarity),
            "recommended_action": str(row.recommended_action),
            "in_cal_peers": int(len(chits)),
            "out_triggers": int(n_h),
            "dir_sign": int(dir_sign),
            "limit_dip": round(limit_dip, 4), "tp_sigma": round(tp_cal, 4), "sl_sigma": round(sl_cal, 4),
            "mode_dir_win": round(float(w_dir.mean()), 4),
            "mode_touch_win": round(float(w_touch.mean()), 4),
            "mode_barrier_win": round(float(w_bar.mean()), 4),
            "mode_mult_win": round(float(mult_win.mean()), 4),
            "inbound_wlb_dir": round(float(fallback[0]), 4),
            "safe_sigma_bins": ",".join(str(b) for b in safe_bins) if safe_bins else "NONE",
            "cons_ev_rise_fall": round(cons_ev["RISE_FALL"], 4),
            "cons_ev_onetouch": round(cons_ev["ONETOUCH_075"], 4),
```

```
            "cons_ev_barrier": round(cons_ev["BARRIER_035"], 4),
            "cons_ev_multiplier": round(cons_ev["MULTIPLIER_TP_SL"], 4),
            "cons_ev_notouch": round(cons_ev["NOTOUCH_10"], 4),
            "mode_notouch_win": round(float(w_notouch.mean()), 4),
            "safest_contract": "RISE_FALL" if cons_ev["RISE_FALL"] > 0.02 else "SKIP",
            "highest_yield_contract": best_contract,
            "best_cons_ev": round(cons_ev[best_contract], 4),
            "realised_ev_pct": round((realized[best_contract] * PAYOUT.get(best_contract, 1.953) - 1.0) * 100, 2),
            "best_mode": best_mode,
            "net_mean_sigma": round(float(best_arr.mean()), 4) if n_eff else None,
            "net_total_sigma": round(float(best_arr.sum()), 3) if n_eff else None,
            "win_rate": round(wr, 4),
            "wilson_lb": round(wilson_lb(wins, n_eff), 4),
            "profit_factor": round(gp / gl, 3),
            "t_stat": round(float(best_arr.mean() / (best_arr.std() + 1e-9) * np.sqrt(n_eff)), 3) if n_eff else None,
            "verified_profitable": bool(wr >= 0.55 and gp / gl >= 1.15 and n_eff >= 15),
        })

    df_strat = pd.DataFrame(strat_rows)
    cols = ["strategy_id", "anchor_cell_id", "outbound_cell_id", "match_similarity",
            "match_polarity", "trade_dir", "sigma_bin", "utc_hour", "best_mode",
            "best_contract", "gross_dir_149", "gross_exc_sniper", "gross_prot_fade",
            "fade_filled", "sigma_bps", "cell_std_points", "cons_ev",
            "max_excursion", "min_excursion", "notouch_win"]
    df_led = pd.DataFrame(ledger, columns=cols)
    save_table(df_strat, os.path.join(out_dir, "shards", f"p3_strategies_node{node_id}.parquet"))
    save_table(df_led, os.path.join(out_dir, "shards", f"p3_ledger_node{node_id}.parquet"))

    diag = {
        "node": node_id, "strategies_audited": int(len(df_strat)),
        "trades": int(len(df_led)), "sigma_quantiles_bps": [round(float(x), 4) for x in qs],
        "global_median_abs_diff": med_abs, "winsor_cap": cap,
        "orthogonal_before": int(n_before), "orthogonal_after": int(n_kept),
        "sigma_bins": {str(b): bin_tot[b] for b in range(5)},
        "utc_hours": {str(h): hour_tot[h] for h in range(24)},
        "secs": round(time.time() - t0, 1),
    }
    with open(os.path.join(out_dir, "shards", f"p3_diag_node{node_id}.json"), "w") as f:
        json.dump(diag, f, indent=2)
    print(f"[+] node {node_id} done: {len(df_strat)} strategies, {len(df_led):,} trades "
          f"({time.time() - t0:.0f}s)", flush=True)


# ---------------------------------------------------------------------- merge
def max_dd(eq):
    peak = np.maximum.accumulate(eq)
    return float(np.min(eq - peak))


def perf(pnl):
    pnl = np.asarray(pnl, dtype=np.float64)
    if len(pnl) == 0:
        return dict(n=0, pf=0.0, sharpe=0.0, sortino=0.0, calmar=0.0, maxdd=0.0, win=0.0)
    eq = np.cumsum(pnl)
    gp = pnl[pnl > 0].sum()
    gl = np.abs(pnl[pnl < 0].sum()) + 1e-9
    sd = pnl.std()
    dd = -max_dd(eq) + 1e-9
    down = pnl[pnl < 0].std() if (pnl < 0).any() else sd
    return dict(n=int(len(pnl)), pf=float(gp / gl), sharpe=float(pnl.mean() / (sd + 1e-9) * np.sqrt(len(pnl))),
                sortino=float(pnl.mean() / (down + 1e-9) * np.sqrt(len(pnl))),
                calmar=float(eq[-1] / dd), maxdd=float(-dd), win=float((pnl > 0).mean()))


def merge(out_dir):
    import glob
    import re

    def shard_files(kind):
        found = {}
        for pat in (f"{kind}_node*.parquet", f"{kind}_node*.csv"):
            for f in glob.glob(os.path.join(out_dir, "shards", pat)):
                m = re.search(r"node(\d+)", os.path.basename(f))
                if m:
                    found.setdefault(int(m.group(1)), f)
        return [found[k] for k in sorted(found)]

    s_files = shard_files("p3_strategies")
    l_files = shard_files("p3_ledger")
    df_s = pd.concat([load_table(f) for f in s_files], ignore_index=True)
    df_l = pd.concat([load_table(f) for f in l_files], ignore_index=True)
```

```
    print(f"[*] merged {len(df_s)} strategies / {len(df_l):,} trades from {len(s_files)} shards", flush=True)

    df_s = df_s.sort_values(["verified_profitable", "wilson_lb", "profit_factor"],
                            ascending=[False, False, False]).reset_index(drop=True)

    approved = df_s[df_s.best_cons_ev > 0.02].copy()
    keep_ids = set(approved.strategy_id) if len(approved) else set(df_s.strategy_id)
    port = df_l[df_l.strategy_id.isin(keep_ids)].copy()
    mode_map = dict(zip(df_s.strategy_id, df_s.best_mode))
    eng_map = dict(zip(df_s.strategy_id, df_s.highest_yield_contract))
    port["chosen_mode"] = port.strategy_id.map(mode_map)
    port["chosen_contract"] = port.strategy_id.map(eng_map)
    port["gross_sigma"] = np.where(port.chosen_mode == "DIRECTIONAL_CLOSE_149", port.gross_dir_149,
                                   np.where(port.chosen_mode == "EXCURSION_TOUCH_SNIPER",
                                            port.gross_exc_sniper, port.gross_prot_fade))
    port = port[(port.chosen_mode != "PROTECTED_EXTREMUM_LIMIT_FADE") | (port.fade_filled == 1)].copy()
    port["payout"] = port.chosen_contract.map(PAYOUT).fillna(1.953)
    port["unit_win"] = np.where(port.chosen_contract == "RISE_FALL", (port.gross_dir_149 > 0).astype(int),
                                np.where(port.chosen_contract == "ONETOUCH_075", (port.max_excursion >= 0.75).astype(int),
                                         np.where(port.chosen_contract == "BARRIER_035", (port.gross_dir_149 >=
0.35).astype(int),
                                                  np.where(port.chosen_contract == "NOTOUCH_10", port.notouch_win,
                                                           (port.gross_exc_sniper > 0).astype(int)))))
    port["unit_pnl"] = np.where(port.unit_win == 1, port.payout - 1.0, -1.0)

    port = port.sort_values(["outbound_cell_id", "cons_ev"], ascending=[True, False])
    port = port.drop_duplicates(subset=["outbound_cell_id"], keep="first").reset_index(drop=True)
    print(f"[*] portfolio: {len(port):,} non-overlapping trades from {port.strategy_id.nunique()} strategies", flush=True)

    for name, cost in FRICTION_TIERS:
        port[f"pnl_{name}"] = port.gross_sigma - cost
        port[f"eq_{name}"] = port[f"pnl_{name}"].cumsum()
        port[f"dd_{name}"] = port[f"eq_{name}"] - port[f"eq_{name}"].cummax()

    bal_k, bal_f, k_curve, f_curve = 50.0, 50.0, [], []
    ruined = False
    for upnl in port.unit_pnl.to_numpy():
        if bal_k > 0.5:
            stake_k = float(np.clip(0.25 * max(0.002, upnl * 0.02) * bal_k, 0.50, 0.01 * bal_k))
            bal_k = max(0.0, bal_k + stake_k * upnl)
        if not ruined and bal_f >= 5.0:
            bal_f += 5.0 * upnl
            if bal_f <= 0.0:
                bal_f = 0.0
                ruined = True
        k_curve.append(round(bal_k, 2))
        f_curve.append(round(bal_f, 2))
    port["balance_account_ruined_fixed5"] = ruined
    port["balance_kelly_from_50"] = k_curve
    port["balance_fixed5_from_50"] = f_curve

    pnl_net = port.pnl_net_03.to_numpy()
    rng = np.random.default_rng(42)
    n = len(pnl_net)
    dds, ends, pfs = [], [], []
    for _ in range(5000):
        s = rng.choice(pnl_net, size=n, replace=True)
        eq = np.cumsum(s)
        dds.append(max_dd(eq))
        ends.append(float(eq[-1]))
        pfs.append(float(s[s > 0].sum() / (np.abs(s[s < 0].sum()) + 1e-9)))
    perf_net = perf(pnl_net)

    wp = float((pnl_net > 0).mean())
    aw = float(pnl_net[pnl_net > 0].mean()) if (pnl_net > 0).any() else 1.0
    al = float(abs(pnl_net[pnl_net < 0].mean())) if (pnl_net < 0).any() else 1.0
    b = aw / max(al, 1e-9)
    kelly = max(0.0, wp - (1 - wp) / b)

    contract_rows = []
    for c in ("RISE_FALL", "ONETOUCH_075", "BARRIER_035", "NOTOUCH_10", "MULTIPLIER_TP_SL"):
        sub = df_s[df_s.highest_yield_contract == c]
        contract_rows.append({
            "contract": c, "payout": PAYOUT.get(c, 1.953),
            "breakeven_pct": round(100.0 / PAYOUT.get(c, 1.953), 2),
            "strategies_chosen": int(len(sub)),
            "mean_cons_ev_pct": round(float(sub.best_cons_ev.mean() * 100), 3) if len(sub) else 0.0,
            "mean_realised_ev_pct": round(float(sub.realised_ev_pct.mean()), 3) if len(sub) else 0.0,
            "mean_win_rate": round(float(sub.win_rate.mean()), 4) if len(sub) else 0.0,
        })
```

```
diag_files = sorted(glob.glob(os.path.join(out_dir, "shards", "p3_diag_node*.json")))
sig_map, hour_map, quantiles = [], [], []
agg_b = {b: dict(n=0, dir=0, touch=0, bar=0, mult=0) for b in range(5)}
agg_h = {h: dict(n=0, win=0, units=0.0) for h in range(24)}
for f in diag_files:
    d = json.load(open(f))
    quantiles = d["sigma_quantiles_bps"]
    for b in range(5):
        for k in agg_b[b]:
            agg_b[b][k] += d["sigma_bins"][str(b)][k]
    for h in range(24):
        agg_h[h]["n"] += d["utc_hours"][str(h)]["n"]
        agg_h[h]["win"] += d["utc_hours"][str(h)]["win"]
        agg_h[h]["units"] += d["utc_hours"][str(h)]["units"]
for b in range(5):
    st = agg_b[b]
    nb = max(1, st["n"])
    rows = {
        "sigma_bin": b,
        "range_bps": f"{quantiles[b]:.3f} - {quantiles[b + 1]:.3f}",
        "triggers": st["n"],
        "rise_fall_win": round(st["dir"] / nb, 4),
        "rise_fall_ev_pct": round((st["dir"] / nb * PAYOUT["RISE_FALL"] - 1) * 100, 2),
        "onetouch_win": round(st["touch"] / nb, 4),
        "onetouch_ev_pct": round((st["touch"] / nb * PAYOUT["ONETOUCH_075"] - 1) * 100, 2),
        "barrier_win": round(st["bar"] / nb, 4),
        "barrier_ev_pct": round((st["bar"] / nb * PAYOUT["BARRIER_035"] - 1) * 100, 2),
    }
    rows["gate"] = "ALLOW" if rows["rise_fall_ev_pct"] > 1.0 else "BLOCK"
    sig_map.append(rows)
for h in range(24):
    st = agg_h[h]
    if st["n"]:
        ev = (st["win"] / st["n"]) * PAYOUT["RISE_FALL"] - 1
        hour_map.append({"utc_hour": f"{h:02d}:00", "trades": st["n"],
                         "dir_win": round(st["win"] / st["n"], 4),
                         "ev_pct": round(ev * 100, 2),
                         "units": round(st["units"], 2),
                         "action": "BLOCK" if ev < -0.03 else "ALLOW"})
blocked = [int(r["utc_hour"][:2]) for r in hour_map if r["action"] == "BLOCK"]

summary = {
    "vault_rows_in_sweet_spot": int(len(df_s)),
    "approved_ev_gt_2pct": int(len(approved)),
    "verified_profitable": int(df_s.verified_profitable.sum()),
    "portfolio_trades": int(len(port)),
    "portfolio_strategies": int(port.strategy_id.nunique()),
    "portfolio_win_rate_net03": round(float(perf_net["win"]), 4),
    "portfolio_wilson_lb_net03": round(wilson_lb(int((pnl_net > 0).sum()), len(pnl_net)), 4),
    "portfolio_profit_factor_net03": round(float(perf_net["pf"]), 3),
    "sharpe_net03": round(float(perf_net["sharpe"]), 3),
    "sortino_net03": round(float(perf_net["sortino"]), 3),
    "calmar_net03": round(float(perf_net["calmar"]), 3),
    "max_dd_net03_sigma": round(float(perf_net["maxdd"]), 2),
    "cum_equity_sigma": {k: round(float(port[f"eq_{k}"].iloc[-1]), 2) for k, _ in FRICTION_TIERS},
    "mc95_worst_dd_sigma": round(float(np.percentile(dds, 5)), 2),
    "mc99_worst_dd_sigma": round(float(np.percentile(dds, 1)), 2),
    "mc_prob_positive": round(float(np.mean(np.array(ends) > 0)), 4),
    "mc_pf_median": round(float(np.median(pfs)), 3),
    "kelly_fraction": round(kelly, 4),
    "quarter_kelly_risk_pct": round(kelly * 25.0, 2),
    "balance_fixed5_from_50": round(float(f_curve[-1]) if f_curve else 50.0, 2),
    "balance_quarter_kelly_from_50": round(float(k_curve[-1]) if k_curve else 50.0, 2),
    "blocked_utc_hours": blocked,
    "sigma_quantile_edges_bps": quantiles,
}

save_table(df_s, os.path.join(out_dir, "phase3_strategy_ev_leaderboard.parquet"))
save_table(port, os.path.join(out_dir, "phase3_deduplicated_equity_curve.parquet"))

xlsx = os.path.join(out_dir, "phase3_master_backtest_report.xlsx")
with pd.ExcelWriter(xlsx, engine="openpyxl") as w:
    pd.DataFrame([{k: (str(v) if isinstance(v, (dict, list)) else v) for k, v in summary.items()}]).T.to_excel(
        w, sheet_name="Executive_Summary", header=False)
    pd.DataFrame(contract_rows).to_excel(w, sheet_name="Contract_Comparison", index=False)
    pd.DataFrame(sig_map).to_excel(w, sheet_name="Fable_SigmaGate_Map", index=False)
    pd.DataFrame(hour_map).to_excel(w, sheet_name="Fable_HourFilter_Map", index=False)
    df_s[df_s.best_cons_ev > 0.02].to_excel(w, sheet_name="Top_Approved_Strategies", index=False)
    port.head(50000).to_excel(w, sheet_name="Chronological_Equity_Curve", index=False)

live_anchors = [int(x) for x in approved.anchor_cell_id.tolist()]
```

```
    realized_win = {
        "RISE_FALL": float((port.unit_win[port.chosen_contract == "RISE_FALL"]).mean())
        if (port.chosen_contract == "RISE_FALL").any() else None,
        "ONETOUCH_075": float((port.unit_win[port.chosen_contract == "ONETOUCH_075"]).mean())
        if (port.chosen_contract == "ONETOUCH_075").any() else None,
        "BARRIER_035": float((port.unit_win[port.chosen_contract == "BARRIER_035"]).mean())
        if (port.chosen_contract == "BARRIER_035").any() else None,
        "NOTOUCH_10": float((port.unit_win[port.chosen_contract == "NOTOUCH_10"]).mean())
        if (port.chosen_contract == "NOTOUCH_10").any() else None,
    }
    contract_floor = {c: (round(1.02 / w, 3) if w and w > 0 else None)
                      for c, w in realized_win.items()}
    cfg = {
        "approved_strategies": int(len(approved)),
        "anchor_cell_ids": live_anchors,
        "approved_anchor_cell_ids": live_anchors,
        "payouts": PAYOUT,
        "sigma_quantile_edges_bps": quantiles,
        "blocked_utc_hours": blocked,
        "ev_gate": 0.02,
        "sim_band": [SIM_MIN, SIM_MAX],
        "safe_bins_extreme": [3, 4],
        "payout_floor_r": 1.90,
        "contract_breakeven_payout": contract_floor,
        "realized_win_rate_by_contract": {k: (round(v, 4) if v is not None else None)
                                          for k, v in realized_win.items()},
        "portfolio_verdict": ("NEGATIVE_NET_OF_FRICTION" if
                              summary["portfolio_profit_factor_net03"] < 1.0 else "POSITIVE"),
        "exit_recommendation": (
            "DO NOT TRADE LIVE: signal is payout-agnostic negative (PF<1, cum equity negative at "
            "every friction tier). Contract PnL is positive ONLY under the brief's assumed payouts."),
        "payout_contingency_note": (
            "ONETOUCH 75s needs a live payout above ~1.42x to break even (assumed 2.15x); NOTOUCH "
            "needs ~1.23x (assumed 1.55x). Real Deriv payout could not be verified: the public "
            "WebSocket ws.derivws.com/websockets/v3 returns HTTP 520 from every cluster node."),
    }
    with open(os.path.join(out_dir, "phase3_live_forward_config.json"), "w") as f:
        json.dump(cfg, f, indent=2)
    with open(os.path.join(out_dir, "phase3_executive_summary.json"), "w") as f:
        json.dump(summary, f, indent=2)

    build_whitening_bridge(out_dir, approved)

    print("\n" + "=" * 74)
    print("        PHASE 3A MASTER BACKTEST & EQUITY SIMULATION SUMMARY")
    print("=" * 74)
    print(json.dumps(summary, indent=2))
    print(f"[+] {xlsx}")


def build_whitening_bridge(out_dir, approved):
    """Fits the linear bridge live-window features -> 132-d trigger_vec_dense space."""
    try:
        inputs = os.environ.get("P3_INPUTS", os.path.join(BASE3, "phase3_inputs.npz"))
        z = np.load(inputs)
        T = z["trigger_vec"]
        Q = z["Q_seq"] if "Q_seq" in z.files else None
        if Q is None:
            print("[!] Q_seq missing - whitening bridge skipped")
            return
        idx = np.concatenate([np.arange(0, N_IN, 37), np.asarray(approved.anchor_cell_id.to_numpy(np.int64) - 1)])
        X = np.asarray(Q[idx], dtype=np.float64).reshape(len(idx), -1)
        Y = np.asarray(T[idx], dtype=np.float64)
        mu, sd = X.mean(0), X.std(0) + 1e-9
        Xs = (X - mu) / sd
        A = Xs.T @ Xs + 1e3 * np.eye(Xs.shape[1])
        W = np.linalg.solve(A, Xs.T @ Y)
        pred = Xs @ W
        r2 = float(1 - ((pred - Y) ** 2).sum() / ((Y - Y.mean(0)) ** 2).sum())
        np.savez(os.path.join(out_dir, "phase3_whitening_transform.npz"),
                 mu=mu, sd=sd, W=W, bridge_r2=np.array([r2]),
                 anchor_vecs=np.asarray(T[np.asarray(approved.anchor_cell_id.to_numpy(np.int64) - 1)]),
                 anchor_ids=approved.anchor_cell_id.to_numpy(np.int64))
        print(f"[+] whitening bridge saved (linear R2 = {r2:.4f})")
    except Exception as e:
        print(f"[!] whitening bridge failed: {type(e).__name__} {e}")


if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--node_id", type=int, default=0)
```

```
    ap.add_argument("--num_nodes", type=int, default=3)
    ap.add_argument("--out_dir", default=os.path.join(BASE3))
    ap.add_argument("--merge", action="store_true")
    ap.add_argument("--top_elite", type=int, default=0)
    a = ap.parse_args()
    if a.merge:
        merge(a.out_dir)
    else:
        run_node(a.node_id, a.num_nodes, a.out_dir, a.top_elite)
```

The engine is 651 lines and runs in two modes from one file. The contract auction, the deduplication pass, the friction ladder and the Monte Carlo all live in the merge path, which is why a shard can be re-run without invalidating the portfolio statistics.

phase3/phase3_definition_diagnostic.py - the recomputation that removed the edge (92 lines, 4.3 KB)

```
"""Diagnostic: does the Phase 2 outbound edge survive as a tick-75->149 directional trade?

Compares three definitions on the SAME outbound matches:
  A) Phase 2 statistic      : mean( sign(q4_net_return * pol) == expected_action )
  B) tick-149 close sign    : mean( sign(path_149 * pol * action_sign) > 0 )
  C) inbound-calibrated dir : mean( sign(path_149 * pol * dir_sign_inbound) > 0 )
"""
import os
import sys
import json
import numpy as np
import pandas as pd

sys.path.insert(0, r"C:\cluster\phase2")
import pqcompat

P2 = r"C:\cluster\phase2"
N_IN = 502495
SIM_MIN, SIM_MAX = 0.85, 0.955

z = np.load(os.path.join(P2, "jump10_phase2_subzones.npz"), mmap_mode="r")
T = np.asarray(z["trigger_vec_dense"], dtype=np.float32)
scan = np.load(os.path.join(P2, "jump10_phase2_scan.npz"), mmap_mode="r")
qret = np.asarray(scan["q4_net_return"], dtype=np.float32)
raw = np.load(os.path.join(r"C:\cluster\phase3", "phase3_inputs.npz"))["raw"]

vault = pd.read_csv(os.path.join(r"C:\cluster\phase3", "phase3_vault.csv"))
vault = vault[(vault.covariant_peer_count >= 12) & (vault.covariant_peer_count <= 400) &
              (vault.mean_0_50_similarity >= 0.85) & (vault.mean_0_50_similarity <= 0.955)]
anchors = (vault.anchor_cell_id.to_numpy(np.int64) - 1)
anchors = anchors[anchors < N_IN]

raw_out = np.asarray(raw[N_IN:], dtype=np.float64)
std_out = raw_out[:, :75].std(axis=1)
std_out = np.where(std_out < 1e-7, 1.0, std_out)
paths = (raw_out[:, 75:] - raw_out[:, 74:75]) / std_out[:, None]
ret_out = qret[N_IN:]
T_out = T[N_IN:]
T_anch = T[anchors]

raw_cal = np.asarray(raw[:N_IN:5], dtype=np.float64)
std_cal = raw_cal[:, :75].std(axis=1)
std_cal = np.where(std_cal < 1e-7, 1.0, std_cal)
paths_cal = (raw_cal[:, 75:] - raw_cal[:, 74:75]) / std_cal[:, None]
T_cal = T[:N_IN:5]

rows = []
for j, a in enumerate(anchors):
    act = str(vault.recommended_action.iloc[j])
    expected = -1.0 if "SELL" in act.upper() else 1.0
    sims = T_out @ T_anch[j]
    hits = np.nonzero((np.abs(sims) >= SIM_MIN) & (np.abs(sims) <= SIM_MAX))[0]
    if len(hits) < 8:
        continue
    pol = np.sign(sims[hits])
    ca = T_cal @ T_anch[j]
    ch = np.nonzero((np.abs(ca) >= SIM_MIN) & (np.abs(ca) <= SIM_MAX))[0]
    if len(ch) >= 40:
        med = float(np.median(paths_cal[ch] * np.sign(ca[ch])[:, None]))
        dsign = 1.0 if med >= 0 else -1.0
    else:
        dsign = expected
```

```
    curve = paths[hits] * pol[:, None] * expected
    curve_in = paths[hits] * pol[:, None] * dsign
    rows.append({
        "anchor_cell_id": int(a + 1),
        "n": int(len(hits)),
        "win_phase2_q4stat": float(np.mean(np.sign(ret_out[hits] * pol) == expected)),
        "win_close149_action": float(np.mean(curve[:, -1] > 0)),
        "win_close149_inbound": float(np.mean(curve_in[:, -1] > 0)),
        "agree_action_vs_inbound": float(np.mean(np.sign(curve[:, -1]) == np.sign(curve_in[:, -1]))),
        "vault_reported_dirwin": float(vault.outbound_directional_win_rate.iloc[j]),
        "mean_abs_sim": float(np.abs(sims[hits]).mean()),
    })

df = pd.DataFrame(rows)
w = df.n.to_numpy(float)
print(f"anchors with >=8 matches: {len(df)}  trades: {int(df.n.sum()):,}")
for c in ("win_phase2_q4stat", "win_close149_action", "win_close149_inbound", "vault_reported_dirwin"):
    print(f"  {c:26s} trades-weighted = {np.average(df[c], weights=w):.4f}   median={df[c].median():.4f}")
print(f"  action vs inbound directional agreement = {np.average(df.agree_action_vs_inbound, weights=w):.4f}")
print(f"  corr(phase2 stat, close149_action)  = {df.win_phase2_q4stat.corr(df.win_close149_action):.3f}")
print(f"  corr(phase2 stat, vault reported)   = {df.win_phase2_q4stat.corr(df.vault_reported_dirwin):.3f}")
df.to_csv(r"C:\cluster\phase3\phase3_definition_diagnostic.csv", index=False)
json.dump({"anchors": len(df), "trades": int(df.n.sum()),
           "w_phase2": float(np.average(df.win_phase2_q4stat, weights=w)),
           "w_close_action": float(np.average(df.win_close149_action, weights=w)),
           "w_close_inbound": float(np.average(df.win_close149_inbound, weights=w)),
           "vault_reported": float(np.average(df.vault_reported_dirwin, weights=w)),
           "agree": float(np.average(df.agree_action_vs_inbound, weights=w))},
          open(r"C:\cluster\phase3\phase3_definition_diagnostic.json", "w"), indent=2)
print("[+] saved diagnostic")
```

phase3/phase3_gatekeeper_verifier.py - the eighteen Phase 3 checks (145 lines, 6.8 KB)

```
"""PHASE 3 GATEKEEPER - final verification of Stage 3A deliverables and Stage 3B readiness."""
import os
import sys
import json
import sqlite3
import subprocess
import numpy as np
import pandas as pd

sys.path.insert(0, r"C:\cluster\phase2")
import pqcompat

P3 = r"C:\cluster\phase3"
P2 = r"C:\cluster\phase2"
results = []


def check(name, ok, detail=""):
    results.append((name, bool(ok), detail))
    print(f"[{'PASS' if ok else 'FAIL'}] {name}" + (f"  ::  {detail}" if detail else ""))


# 1 - deliverables exist
files = ["phase3_strategy_ev_leaderboard.parquet", "phase3_deduplicated_equity_curve.parquet",
         "phase3_master_backtest_report.xlsx", "phase3_live_forward_config.json",
         "phase3_whitening_transform.npz", "phase3_deriv_live_forward_tester.py",
         "phase3_gatekeeper_verifier.py", "phase3_executive_summary.json"]
missing = [f for f in files if not os.path.exists(os.path.join(P3, f))]
check("Deliverables present (8)", not missing, f"missing={missing}")

summ = json.load(open(os.path.join(P3, "phase3_executive_summary.json")))
cfg = json.load(open(os.path.join(P3, "phase3_live_forward_config.json")))

# 2 - leaderboard schema + anchor count
lb = pqcompat.read_parquet(os.path.join(P3, "phase3_strategy_ev_leaderboard.parquet"))
need = {"anchor_cell_id", "cons_ev_rise_fall", "cons_ev_onetouch", "cons_ev_barrier",
        "cons_ev_notouch", "cons_ev_multiplier", "inbound_wlb_dir", "mode_dir_win",
        "mode_touch_win", "mode_barrier_win", "mode_notouch_win", "safe_sigma_bins",
        "verified_profitable"}
check("Leaderboard columns", need <= set(lb.columns), f"rows={len(lb)}")
check("Leaderboard covers every sweet-spot anchor", len(lb) == summ["vault_rows_in_sweet_spot"],
      f"{len(lb)} vs {summ['vault_rows_in_sweet_spot']}")

# 3 - orthogonal pruning really holds (|r_anchor| >= 0.82 suppressed)
T = np.load(os.path.join(P3, "phase3_inputs.npz"), mmap_mode="r")["trigger_vec"]
anc = lb.anchor_cell_id.to_numpy(np.int64) - 1
A = np.asarray(T[anc], dtype=np.float64)
```

```
An = A / np.linalg.norm(A, axis=1, keepdims=True)
S = np.abs(An @ An.T)
np.fill_diagonal(S, 0.0)
check("Anchor orthogonality |r|<0.82", float(S.max()) < 0.82, f"max_offdiag={S.max():.4f}")

# 4 - kindex leakage: sigma must be causal, i.e. uncorrelated with the forward move
eq = pqcompat.read_parquet(os.path.join(P3, "phase3_deduplicated_equity_curve.parquet"))
check("Equity curve per-trade rows", len(eq) == summ["portfolio_trades"],
      f"{len(eq)} vs {summ['portfolio_trades']}")
if "outbound_cell_id" in eq.columns and "sigma_bps" in eq.columns:
    r = np.corrcoef(eq.sigma_bps.to_numpy(float), eq.gross_dir_149.to_numpy(float))[0, 1]
    check("Sigma causal (|corr| with forward move < 0.10)", abs(r) < 0.10, f"corr={r:+.4f}")
else:
    check("Sigma causal (columns present)", False, "sigma_bps/gross_dir_149 missing")

# 5 - one trade per cell (dedup)
if "outbound_cell_id" in eq.columns:
    dup = int(eq.outbound_cell_id.duplicated().sum())
    check("One trade per cell", dup == 0, f"duplicates={dup}")

# 6 - friction ladder monotonicity
c = summ["cum_equity_sigma"]
check("Friction ladder monotone", c["gross_00"] >= c["net_03"] >= c["net_06"] >= c["net_10"],
      f"{c}")

# 7 - Monte Carlo sanity
check("MC runs & ordering", summ["mc95_worst_dd_sigma"] >= summ["mc99_worst_dd_sigma"],
      f"mc95={summ['mc95_worst_dd_sigma']} mc99={summ['mc99_worst_dd_sigma']}")
check("Kelly non-negative logic", summ["kelly_fraction"] == 0.0 and
      summ["portfolio_profit_factor_net03"] < 1.0,
      "kelly=0 because PF<1 (correct)")

# 8 - whitening bridge / live parity artifacts
w = np.load(os.path.join(P3, "phase3_whitening_transform.npz"))
bridge = json.load(open(os.path.join(P3, "phase3_whitening_bridge.json")))
check("Whitening transform arrays", all(k in w.files for k in
      ("q_mu", "q_sd", "anchor_vecs", "anchor_ids", "anchor_actions")),
      f"anchors={len(w['anchor_ids'])}")
check("Live bridge trained (report parity honestly)", os.path.exists(
    os.path.join(P3, "phase3_live_bridge.pt")),
    f"match_recall={bridge['match_recall']:.3f} precision={bridge['match_precision']:.3f} "
    f"IoU={bridge['match_iou']:.3f}  <- live triggers are a SUBSET of backtest triggers")

# 9 - forward tester: compiles, self-checks, 7 guards, SQLite
src = open(os.path.join(P3, "phase3_deriv_live_forward_tester.py"), encoding="utf-8").read()
guards = ["SIGMA_GATE", "PAYOUT_MONITOR", "LAG_GUARD", "LIVE_CALIBRATOR", "DRAWDOWN_HALT",
          "KellyStakeSizer", "HOUR_FILTER"]
check("7 Fable guards present", all(g in src for g in guards),
      ",".join(g for g in guards if g not in src) or "all")
rc = subprocess.run([sys.executable, "-c",
                     "import py_compile;py_compile.compile(r'%s',doraise=True)" %
                     os.path.join(P3, "phase3_deriv_live_forward_tester.py")],
                    capture_output=True, text=True)
check("Forward tester compiles", rc.returncode == 0, rc.stderr.strip()[:120])
rc = subprocess.run([sys.executable, os.path.join(P3, "phase3_deriv_live_forward_tester.py"),
                     "--self_check"], capture_output=True, text=True)
check("Forward tester self-check", rc.returncode == 0 and "self-check OK" in rc.stdout,
      (rc.stdout or rc.stderr).strip().splitlines()[-1][:120] if (rc.stdout or rc.stderr) else "")
db = os.path.join(P3, "fable_forward_cal.db")
ok_db = False
if os.path.exists(db):
    con = sqlite3.connect(db)
    tabs = {r[0] for r in con.execute("SELECT name FROM sqlite_master WHERE type='table'")}
    con.close()
    ok_db = "forward_trades" in tabs
check("SQLite LiveCalibrator journal", ok_db, db)

# 10 - credential hygiene: no token material in any Phase 3 artifact
hits = []
for fn in os.listdir(P3):
    p = os.path.join(P3, fn)
    if os.path.isfile(p) and os.path.getsize(p) < 8_000_000 and fn.endswith(
            (".py", ".json", ".txt", ".log", ".csv", ".md")):
        try:
            t = open(p, encoding="utf-8", errors="ignore").read()
        except OSError:
            continue
        if ("pat_" + "a9c5f3d4") in t or ("34NbJFTA3U4VL" + "tchImAaP") in t:
            hits.append(fn)
check("No credentials in Phase 3 artifacts", not hits, f"leaks={hits}")
```

```
# 11 - live config contents
check("Live config has anchors+hours+sigma edges",
      len(cfg.get("approved_anchor_cell_ids", [])) > 0 and
      len(cfg.get("blocked_utc_hours", [])) > 0 and
      len(cfg.get("sigma_quantile_edges_bps", [])) == 6,
      f"anchors={len(cfg.get('approved_anchor_cell_ids', []))} "
      f"hours={len(cfg.get('blocked_utc_hours', []))}")

print("\n" + "=" * 72)
npass = sum(1 for _, ok, _ in results if ok)
print(f"PHASE 3 GATEKEEPER: {npass}/{len(results)} PASS")
for n, ok, d in results:
    if not ok:
        print(f"  !! {n}  ::  {d}")
print("=" * 72)
json.dump([{"check": n, "pass": ok, "detail": d} for n, ok, d in results],
          open(os.path.join(P3, "phase3_gatekeeper_results.json"), "w"), indent=2)
```

phase3/build_whitening_bridge.py - linear whitening reconstruction (46 lines, 1.8 KB)

```
"""Build phase3_whitening_transform.npz: live 75-tick -> 132-d trigger vector bridge.

Fits a ridge-regularised linear map on the INBOUND half only (no lookahead) from the
Phase 2 dense quartile features (Q_seq, 4x72 = 288-d) to the Phase 2 trigger vector
(132-d cosine-state descriptor) that the Phase 3 anchor matching uses.
"""
import os
import json
import numpy as np

P2 = r"C:\cluster\phase2"
P3 = r"C:\cluster\phase3"
N_IN = 502495
RIDGE = 1e-2

z = np.load(os.path.join(P2, "czet_train_cache.npz"), mmap_mode="r")
Q = np.asarray(z["Q_seq"][:, :2, :], dtype=np.float64).reshape(len(z["Q_seq"]), -1)
T = np.load(os.path.join(P3, "phase3_inputs.npz"), mmap_mode="r")["trigger_vec"]
T = np.asarray(T, dtype=np.float64)

Q_in, T_in = Q[:N_IN], T[:N_IN]
q_mu = Q_in.mean(axis=0)
q_sd = Q_in.std(axis=0)
q_sd = np.where(q_sd < 1e-9, 1.0, q_sd)
X = (Q_in - q_mu) / q_sd

G = X.T @ X / len(X)
G.flat[:: G.shape[0] + 1] += RIDGE
W = np.linalg.solve(G, X.T @ T_in / len(X)).astype(np.float32)

pred = X @ W
r2 = 1.0 - float(((T_in - pred) ** 2).sum() / ((T_in - T_in.mean(0)) ** 2).sum())

vault = __import__("pandas").read_csv(os.path.join(P3, "phase3_vault.csv"))
anchors = vault.anchor_cell_id.to_numpy(np.int64) - 1

np.savez_compressed(
    os.path.join(P3, "phase3_whitening_transform.npz"),
    q_mu=q_mu.astype(np.float32), q_sd=q_sd.astype(np.float32), W=W,
    anchor_vecs=T[anchors].astype(np.float32), anchor_ids=vault.anchor_cell_id.to_numpy(np.int64),
    anchor_actions=vault.recommended_action.to_numpy().astype("U32"),
    bridge_r2=np.float32(r2), n_fit=np.int64(N_IN),
)
print(f"[+] bridge R2 (inbound, Q1+Q2 144->132) = {r2:.4f}   params={W.size:,}   anchors={len(anchors)}")
json.dump({"bridge_r2": r2, "n_fit": N_IN, "anchors": int(len(anchors))},
          open(os.path.join(P3, "phase3_whitening_bridge.json"), "w"), indent=2)
```

phase3/build_live_bridge.py - raw-space cosine matcher (rejected: recall 0.000) (93 lines, 3.7 KB)

```
"""Live-parity bridge: raw 75-tick window -> Phase 3 anchor space (132-d).

Live path (reproducible from ticks alone):
    y = last 75 ticks of the cell (0-50% window)
    F = [dense(y[:38]) | dense(y[38:])]          (Phase 2 dense extractor, raw 64-d each)
    F = (F - mu) / sd
    T_hat = F @ W   (ridge, or MLP)              -> 132-d, cosine vs anchor_vecs

Reports R2, cosine error, and trigger-match recall/IoU vs the true Phase 2 space.
```

```
"""
import os
import sys
import time
import json
import numpy as np

sys.path.insert(0, r"C:\cluster\phase2")
from phase2_dense_anomaly_bank import compute_quartile_brute_force_features

P2 = r"C:\cluster\phase2"
P3 = r"C:\cluster\phase3"
N_IN = 502495
CHUNK = 50000

z = np.load(os.path.join(P2, "jump10_phase2_subzones.npz"), mmap_mode="r")
T = np.load(os.path.join(P3, "phase3_inputs.npz"), mmap_mode="r")["trigger_vec"]
N = len(T)
t0 = time.time()

cache = os.path.join(P3, "_live_feat.npy")
if not os.path.exists(cache):
    q1 = np.asarray(z["y_q1"], dtype=np.float64)
    q2 = np.asarray(z["y_q2"], dtype=np.float64)
    F = np.lib.format.open_memmap(cache, mode="w+", dtype=np.float32, shape=(N, 128))
    for s in range(0, N, CHUNK):
        e = min(N, s + CHUNK)
        a = compute_quartile_brute_force_features(q1[s:e])
        b = compute_quartile_brute_force_features(q2[s:e])
        F[s:e] = np.hstack([a, b]).astype(np.float32)
        if (s // CHUNK) % 4 == 0:
            print(f"    {e:,}/{N:,}  ({time.time()-t0:.0f}s)", flush=True)
    F.flush()
    del F
print(f"[*] live feature matrix ready ({time.time()-t0:.0f}s)", flush=True)

F = np.load(cache, mmap_mode="r")
F_in = np.asarray(F[:N_IN], dtype=np.float64)
mu = F_in.mean(0)
sd = F_in.std(0)
sd = np.where(sd < 1e-9, 1.0, sd)
X = (F_in - mu) / sd
T_in = np.asarray(T[:N_IN], dtype=np.float64)

G = X.T @ X / len(X)
G.flat[:: G.shape[0] + 1] += 1e-3
W = np.linalg.solve(G, X.T @ T_in / len(X))
pred = X @ W
r2 = 1.0 - float(((T_in - pred) ** 2).sum() / ((T_in - T_in.mean(0)) ** 2).sum())
print(f"[*] linear bridge R2 = {r2:.4f}", flush=True)
del X, T_in, pred

vault = __import__("pandas").read_csv(os.path.join(P3, "phase3_vault.csv"))
anchors = vault.anchor_cell_id.to_numpy(np.int64) - 1
A = np.asarray(T[anchors], dtype=np.float64)
An = A / np.linalg.norm(A, axis=1, keepdims=True)

idx = np.arange(N_IN, N, 2)
Tt = np.asarray(T[idx], dtype=np.float64)
Ft = (np.asarray(F[idx], dtype=np.float64) - mu) / sd
Tc = Ft @ W
st = (Tt / np.linalg.norm(Tt, axis=1, keepdims=True)) @ An.T
sb = (Tc / np.linalg.norm(Tc, axis=1, keepdims=True)) @ An.T
bt = (np.abs(st) >= 0.85) & (np.abs(st) <= 0.955)
bb = (np.abs(sb) >= 0.85) & (np.abs(sb) <= 0.955)
recall = float((bt & bb).sum() / max(1, bt.sum()))
iou = float((bt & bb).sum() / max(1, (bt | bb).sum()))
pol = float((np.sign(st[bt & bb]) == np.sign(sb[bt & bb])).mean()) if (bt & bb).any() else 0.0
print(f"[*] outbound match fidelity: recall={recall:.3f} IoU={iou:.3f} polarity={pol:.3f} "
      f"true_hits={int(bt.sum()):,} bridge_hits={int(bb.sum()):,}", flush=True)

np.savez_compressed(
    os.path.join(P3, "phase3_whitening_transform.npz"),
    q_mu=mu.astype(np.float32), q_sd=sd.astype(np.float32), W=W.astype(np.float32),
    anchor_vecs=A.astype(np.float32), anchor_ids=vault.anchor_cell_id.to_numpy(np.int64),
    anchor_actions=vault.recommended_action.to_numpy().astype("U32"),
    bridge_r2=np.float32(r2), n_fit=np.int64(N_IN),
    match_recall=np.float32(recall), match_iou=np.float32(iou),
)
json.dump({"bridge_r2": r2, "match_recall": recall, "match_iou": iou,
           "polarity_agreement": pol, "n_fit": N_IN, "anchors": int(len(anchors)),
```

```
           "feature": "dense_q1_raw64+dense_q2_raw64 -> 128 -> 132"},
          open(os.path.join(P3, "phase3_whitening_bridge.json"), "w"), indent=2)
print("[+] phase3_whitening_transform.npz written")
```

phase3/train_live_bridge.py - MLP 128-512-512-132 cosine bridge (shipped) (131 lines, 4.8 KB)

```
"""Train a nonlinear live-parity bridge (75-tick dense features -> Phase 3 132-d anchor space).

Cosine-loss MLP. Trains on INBOUND only; validates on the last 10% of inbound, then measures
outbound trigger-match recall / IoU against the true Phase 2 similarity.
"""
import os
import sys
import json
import time
import numpy as np
import torch
import torch.nn as nn

sys.path.insert(0, r"C:\cluster\phase3")
sys.path.insert(0, r"C:\cluster\phase2")

P3 = r"C:\cluster\phase3"
P2 = r"C:\cluster\phase2"
N_IN = 502495
torch.manual_seed(0)
torch.set_num_threads(8)

F = np.load(os.path.join(P3, "_live_feat.npy"), mmap_mode="r")
T = np.load(os.path.join(P3, "phase3_inputs.npz"), mmap_mode="r")["trigger_vec"]
N = len(T)

Fi = np.asarray(F[:N_IN], dtype=np.float32)
mu = Fi.mean(0)
sd = Fi.std(0)
sd = np.where(sd < 1e-9, 1.0, sd)
Fi = (Fi - mu) / sd
Ti = np.asarray(T[:N_IN], dtype=np.float32)
Ti = Ti / np.maximum(np.linalg.norm(Ti, axis=1, keepdims=True), 1e-9)

n_val = 50000
Xtr, Ytr = torch.from_numpy(Fi[:-n_val]), torch.from_numpy(Ti[:-n_val])
Xva, Yva = torch.from_numpy(Fi[-n_val:]), torch.from_numpy(Ti[-n_val:])
del Fi, Ti


class Net(nn.Module):
    def __init__(self, d_in=128, d_out=132, h=512):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d_in, h), nn.GELU(), nn.LayerNorm(h),
            nn.Linear(h, h), nn.GELU(), nn.LayerNorm(h),
            nn.Linear(h, d_out),
        )

    def forward(self, x):
        return self.net(x)


net = Net().to("cpu")
opt = torch.optim.AdamW(net.parameters(), lr=1.5e-3, weight_decay=1e-4)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=12)
BS = 4096
n = len(Xtr)
best = 9.9
t0 = time.time()

for ep in range(12):
    perm = torch.randperm(n)
    tot = 0.0
    for s in range(0, n, BS):
        b = perm[s:s + BS]
        xb, yb = Xtr[b], Ytr[b]
        pred = net(xb)
        cos = (pred * yb).sum(1) / (pred.norm(dim=1) * yb.norm(dim=1) + 1e-9)
        loss = (1.0 - cos).mean()
        opt.zero_grad()
        loss.backward()
        opt.step()
        tot += float(loss) * len(b)
```

```
    sched.step()
    with torch.no_grad():
        pv = net(Xva)
        cv = ((pv * Yva).sum(1) / (pv.norm(dim=1) * Yva.norm(dim=1) + 1e-9))
        vloss = float((1.0 - cv).mean())
        acc = float((cv > 0.995).float().mean())
    print(f"  epoch {ep+1:2d}  train_cos={1-tot/n:.5f}  val_cos={float(cv.mean()):.5f}  "
          f"val>0.995={acc:.4f}  ({time.time()-t0:.0f}s)", flush=True)
    if vloss < best:
        best = vloss
        torch.save(net.state_dict(), os.path.join(P3, "phase3_live_bridge.pt"))

net.load_state_dict(torch.load(os.path.join(P3, "phase3_live_bridge.pt")))
net.eval()

import pandas as pd
vault = pd.read_csv(os.path.join(P3, "phase3_vault.csv"))
anc = vault.anchor_cell_id.to_numpy(np.int64) - 1
A = np.asarray(T[anc], dtype=np.float64)
An = A / np.linalg.norm(A, axis=1, keepdims=True)

idx = np.arange(N_IN, N, 2)
Tt = np.asarray(T[idx], dtype=np.float64)
Tt_n = Tt / np.linalg.norm(Tt, axis=1, keepdims=True)
st = Tt_n @ An.T

Fc = (np.asarray(F[idx], dtype=np.float32) - mu) / sd
with torch.no_grad():
    Tc = net(torch.from_numpy(Fc)).numpy().astype(np.float64)
Tc_n = Tc / np.maximum(np.linalg.norm(Tc, axis=1, keepdims=True), 1e-9)
sb = Tc_n @ An.T

bt = (np.abs(st) >= 0.85) & (np.abs(st) <= 0.955)
bb = (np.abs(sb) >= 0.85) & (np.abs(sb) <= 0.955)
recall = float((bt & bb).sum() / max(1, bt.sum()))
prec = float((bt & bb).sum() / max(1, bb.sum()))
iou = float((bt & bb).sum() / max(1, (bt | bb).sum()))
pol = float((np.sign(st[bt & bb]) == np.sign(sb[bt & bb])).mean()) if (bt & bb).any() else 0.0
print(f"[*] outbound parity: recall={recall:.3f} precision={prec:.3f} IoU={iou:.3f} "
      f"polarity={pol:.3f} true_hits={int(bt.sum()):,} bridge_hits={int(bb.sum()):,}", flush=True)

torch.save({"state_dict": net.state_dict(), "d_in": 128, "d_out": 132, "h": 512,
            "feat": "dense_q1(64)+dense_q2(64) raw -> standardize"},
           os.path.join(P3, "phase3_live_bridge.pt"))
np.savez_compressed(
    os.path.join(P3, "phase3_whitening_transform.npz"),
    q_mu=mu.astype(np.float32), q_sd=sd.astype(np.float32),
    anchor_vecs=A.astype(np.float32), anchor_ids=vault.anchor_cell_id.to_numpy(np.int64),
    anchor_actions=vault.recommended_action.to_numpy().astype("U32"),
    bridge_r2=np.float32(np.nan), n_fit=np.int64(N_IN),
    match_recall=np.float32(recall), match_iou=np.float32(iou), match_precision=np.float32(prec),
)
json.dump({"bridge": "mlp_128_512_512_132_cosine", "match_recall": recall,
           "match_precision": prec, "match_iou": iou, "polarity_agreement": pol,
           "val_cos": float(cv.mean()), "n_fit": N_IN, "anchors": int(len(anc))},
          open(os.path.join(P3, "phase3_whitening_bridge.json"), "w"), indent=2)
print("[+] phase3_whitening_transform.npz + phase3_live_bridge.pt written")
```

phase3/phase3_deriv_live_forward_tester.py - seven guards, SQLite journal, upgrade probe (426 lines, 18.5 KB)

```
"""PHASE 3B - Deriv live WebSocket forward tester with the 7 Fable adaptive guards.

Default mode is paper_proposal: subscribes to live ticks, evaluates the Phase 3 anchor
matches on every completed 75-tick window, queries REAL payouts through the Deriv
`proposal` endpoint, settles 75 ticks later from the live stream, and journals every
decision to SQLite. Live orders are only placed with --execute_demo_orders.

Credentials: never hardcoded.
  precedence: --api_token/--app_id  >  $env:DERIV_API_TOKEN/$env:DERIV_APP_ID  >  --creds_file
  --creds_file accepts any text file containing "API TOKEN:<tok>" and "APP ID:<id>" lines
  (e.g. the Phase 3 brief). The token is masked in every log line.

Usage
  python -u phase3_deriv_live_forward_tester.py --ping_test 20 --creds_file "...brief.md"
  python -u phase3_deriv_live_forward_tester.py --symbol JD10 --max_minutes 60
"""
from __future__ import annotations

import argparse
import asyncio
```

```
import json
import os
import re
import sqlite3
import sys
import time
from collections import deque
from datetime import datetime, timezone

import numpy as np

sys.path.insert(0, r"C:\cluster\phase2")
sys.path.insert(0, r"C:\cluster\phase3")
from phase2_dense_anomaly_bank import compute_quartile_brute_force_features  # noqa: E402

P3 = r"C:\cluster\phase3"
DB_PATH = os.path.join(P3, "fable_forward_cal.db")
SIM_MIN, SIM_MAX = 0.85, 0.955
SIGMA_LOOKBACK = 1800
SETTLE_TICKS = 75


def mask(tok: str) -> str:
    if not tok:
        return "<unset>"
    return tok[:8] + "..." + tok[-4:] if len(tok) > 14 else "***"


def load_creds(args) -> tuple[str, str]:
    app_id = args.app_id or os.environ.get("DERIV_APP_ID", "")
    token = args.api_token or os.environ.get("DERIV_API_TOKEN", "")
    if (not app_id or not token) and args.creds_file:
        try:
            text = open(args.creds_file, "r", encoding="utf-8", errors="ignore").read()
            if not app_id:
                m = re.search(r"APP\s*ID\s*[:\-]\s*([A-Za-z0-9]+)", text)
                app_id = m.group(1) if m else ""
            if not token:
                m = re.search(r"API\s*TOKEN\s*[:\-]\s*(pat_[A-Za-z0-9]+|[A-Za-z0-9]{32,})", text)
                token = m.group(1) if m else ""
        except OSError as e:
            print(f"[!] creds_file unreadable: {e}")
    return app_id, token


def wilson_lb(p: float, n: int, z: float = 1.96) -> float:
    if n <= 0:
        return 0.0
    d = 1 + z * z / n
    c = p + z * z / (2 * n)
    return (c - z * np.sqrt(p * (1 - p) / n + z * z / (4 * n * n))) / d


class FableAdaptiveGuards:
    """All 7 adaptive circuit breakers (SigmaGate, PayoutMonitor, LagMonitor,
    LiveCalibrator, DrawdownGuard, KellyStakeSizer, HourFilter)."""

    def __init__(self, cfg_path: str, db_path: str = DB_PATH):
        self.cfg = json.load(open(cfg_path, "r"))
        self.sigma_edges = np.array(self.cfg.get("sigma_quantile_edges_bps", [0.15, 0.20]))
        self.blocked_hours = set(int(h) for h in self.cfg.get("blocked_utc_hours", []))
        self.payout_floor = float(self.cfg.get("payout_floor_r", 1.90))
        self.session_stop = -400.0
        self.daily_stop = -800.0
        self.max_rtt_ms = 250.0
        self.consec_loss_limit = 8
        self.cooldown_s = 300.0

        self.rtts = deque(maxlen=40)
        self.session_pnl = 0.0
        self.daily_pnl = 0.0
        self.day = datetime.now(timezone.utc).date()
        self.consec_losses = 0
        self.pause_until = 0.0
        self.trades = []
        self.live_hits, self.live_exp = 0.0, 0.0
        self.db_path = db_path
        self._init_sqlite()

    def _init_sqlite(self):
        os.makedirs(os.path.dirname(self.db_path), exist_ok=True)
```

```
        con = sqlite3.connect(self.db_path)
        con.execute("""CREATE TABLE IF NOT EXISTS forward_trades(
            ts_utc TEXT, symbol TEXT, anchor_cell_id INTEGER, contract TEXT,
            sigma_bps REAL, sim REAL, live_payout REAL, rtt_ms REAL,
            stake REAL, won INTEGER, pnl REAL, settle_spot REAL, note TEXT)""")
        con.execute("CREATE INDEX IF NOT EXISTS ix_ts ON forward_trades(ts_utc)")
        con.commit()
        con.close()
        self.db_ok = True

    def note_payout(self, r: float) -> bool:
        return r >= self.payout_floor

    def check_all(self, sigma_bps: float, rtt_ms: float, live_payout: float) -> tuple[bool, str]:
        now = time.time()
        today = datetime.now(timezone.utc).date()
        if today != self.day:
            self.day, self.daily_pnl = today, 0.0
        if self.session_pnl <= self.session_stop:
            return False, f"DRAWDOWN_HALT session={self.session_pnl:.2f}"
        if self.daily_pnl <= self.daily_stop:
            return False, f"DAILY_HALT daily={self.daily_pnl:.2f}"
        if now < self.pause_until:
            return False, f"CONSEC_LOSS_COOLDOWN {int(self.pause_until - now)}s"
        hr = datetime.now(timezone.utc).hour
        if hr in self.blocked_hours:
            return False, f"HOUR_FILTER {hr:02d}Z"
        self.rtts.append(rtt_ms)
        if len(self.rtts) >= 5 and float(np.median(self.rtts)) > self.max_rtt_ms:
            return False, f"LAG_GUARD median RTT={np.median(self.rtts):.0f}ms"
        if not (self.sigma_edges[0] <= sigma_bps <= self.sigma_edges[-1]):
            return False, f"SIGMA_GATE {sigma_bps:.3f}bps"
        if not self.note_payout(live_payout):
            return False, f"PAYOUT_MONITOR {live_payout:.3f}x < {self.payout_floor:.2f}x"
        if self.live_exp >= 25 and self.live_hits / self.live_exp < 0.40:
            return False, f"LIVE_CALIBRATOR hit={self.live_hits/self.live_exp:.2f} (degraded)"
        return True, "ALL_7_GUARDS_GREEN"

    def stake(self, balance: float, p: float, r: float) -> float:
        if r <= 1.0 or p <= 0:
            return 0.0
        f_star = (p * r - 1.0) / (r - 1.0)
        return float(np.clip(0.25 * f_star * balance, 0.35, 0.01 * balance))

    def record(self, row: dict):
        self.trades.append(row)
        self.session_pnl += row["pnl"]
        self.daily_pnl += row["pnl"]
        self.live_hits += row["won"]
        self.live_exp += 1
        if row["won"]:
            self.consec_losses = 0
        else:
            self.consec_losses += 1
            if self.consec_losses >= self.consec_loss_limit:
                self.pause_until = time.time() + self.cooldown_s
                self.consec_losses = 0
        con = sqlite3.connect(self.db_path)
        con.execute("INSERT INTO forward_trades VALUES(?,?,?,?,?,?,?,?,?,?,?,?,?)",
                    (row["ts_utc"], row["symbol"], row["anchor_cell_id"], row["contract"],
                     row["sigma_bps"], row["sim"], row["live_payout"], row["rtt_ms"],
                     row["stake"], row["won"], row["pnl"], row["settle_spot"], row["note"]))
        con.commit()
        con.close()


class LiveMatcher:
    def __init__(self):
        w = np.load(os.path.join(P3, "phase3_whitening_transform.npz"))
        self.mu, self.sd = w["q_mu"], w["q_sd"]
        self.A = w["anchor_vecs"].astype(np.float64)
        self.An = self.A / np.linalg.norm(self.A, axis=1, keepdims=True)
        self.ids = w["anchor_ids"]
        self.actions = w["anchor_actions"]
        self.mode = "linear_ridge"
        self.W = w["W"] if "W" in w.files else None
        pt = os.path.join(P3, "phase3_live_bridge.pt")
        self.net = None
        if os.path.exists(pt):
            import torch
            ck = torch.load(pt, map_location="cpu")
```

```
            sd = ck["state_dict"] if "state_dict" in ck else ck
            sd = {k[4:] if k.startswith("net.") else k: v for k, v in sd.items()}
            self.net = torch.nn.Sequential(
                torch.nn.Linear(128, 512), torch.nn.GELU(), torch.nn.LayerNorm(512),
                torch.nn.Linear(512, 512), torch.nn.GELU(), torch.nn.LayerNorm(512),
                torch.nn.Linear(512, 132))
            self.net.load_state_dict(sd)
            self.net.eval()
            self.mode = "mlp_cosine"
        meta = os.path.join(P3, "phase3_whitening_bridge.json")
        self.parity = json.load(open(meta)) if os.path.exists(meta) else {}

    def vector(self, y75: np.ndarray) -> np.ndarray:
        f = np.hstack([compute_quartile_brute_force_features(np.asarray(y75[:38])[None, :]),
                       compute_quartile_brute_force_features(np.asarray(y75[38:75])[None, :])])
        f = (f - self.mu) / self.sd
        if self.net is not None:
            import torch
            with torch.no_grad():
                v = self.net(torch.from_numpy(f.astype(np.float32))).numpy()[0].astype(np.float64)
            return v / max(np.linalg.norm(v), 1e-9)
        return (f @ self.W)[0]

    def match(self, v: np.ndarray):
        s = self.An @ (v / max(np.linalg.norm(v), 1e-9))
        a = np.abs(s)
        k = np.nonzero((a >= SIM_MIN) & (a <= SIM_MAX))[0]
        if not len(k):
            return None
        j = k[np.argmax(a[k])]
        return int(self.ids[j]), float(a[j]), float(np.sign(s[j])), str(self.actions[j])


def winsorized_sigma_bps(prices: np.ndarray) -> float:
    d = np.abs(np.diff(prices))
    if len(d) < 30:
        return float("nan")
    med = float(np.median(d))
    cap = 6.98438 * max(med, 1e-9)
    dw = np.clip(d, 0.0, cap)
    return float(np.std(dw) / max(abs(prices[-1]), 1e-12) * 1e4)


async def ws_request(ws, payload: dict, timeout=10.0) -> dict:
    await ws.send(json.dumps(payload))
    end = time.time() + timeout
    while time.time() < end:
        msg = json.loads(await asyncio.wait_for(ws.recv(), timeout=max(0.1, end - time.time())))
        if "error" in msg:
            return {"error": msg["error"]}
        for k in ("proposal", "ping", "authorize", "tick", "buy"):
            if k in msg:
                return msg
    return {"error": {"message": "timeout"}}


def proposal_payload(contract: str, symbol: str, stake: float, spot: float | None) -> dict | None:
    if contract in ("CALL", "PUT"):
        return {"proposal": 1, "amount": stake, "basis": "stake",
                "contract_type": contract, "currency": "USD", "duration": 75,
                "duration_unit": "s", "symbol": symbol}
    if contract == "ONETOUCH":
        return {"proposal": 1, "amount": stake, "basis": "stake",
                "contract_type": "ONETOUCH", "currency": "USD", "duration": 1,
                "duration_unit": "d", "symbol": symbol,
                "barrier": f"+{max(spot * 0.00015, 0.5):.2f}" if spot else "+1.00"}
    return None


async def ping_test(app_id: str, token: str, symbol: str, n: int = 20):
    import websockets
    url = f"wss://ws.derivws.com/websockets/v3?app_id={app_id}"
    print(f"[*] PING TEST  url={url}  token={mask(token)}  symbol={symbol}")
    rtts = []
    async with websockets.connect(url, ping_interval=20, max_size=2**22) as ws:
        if token:
            r = await ws_request(ws, {"authorize": token})
            if "error" in r:
                print(f"[!] authorize failed: {r['error'].get('message')}")
            else:
                print(f"[+] authorized: loginid={r['authorize'].get('loginid')} "
```

```
                    f"balance={r['authorize'].get('balance')} {r['authorize'].get('currency')}")
        for i in range(n):
            t0 = time.perf_counter()
            r = await ws_request(ws, {"ping": 1})
            dt = (time.perf_counter() - t0) * 1000.0
            if "error" in r:
                print(f"  ping {i+1}: {r['error']}")
                continue
            rtts.append(dt)
            print(f"  ping {i+1:2d}: {dt:7.1f} ms")
            await asyncio.sleep(0.25)
        r = await ws_request(ws, {"ticks": symbol, "subscribe": 1})
        spot = None
        if "tick" in r:
            spot = float(r["tick"]["quote"])
            print(f"[+] {symbol} spot={spot}")
            await ws_request(ws, {"forget": r["tick"].get("id", "")}, timeout=2)
        for c in ("CALL", "PUT", "ONETOUCH"):
            p = proposal_payload(c, symbol, 1.0, spot or 1000.0)
            if not p:
                continue
            t0 = time.perf_counter()
            r = await ws_request(ws, p)
            dt = (time.perf_counter() - t0) * 1000.0
            if "error" in r:
                print(f"  proposal {c:9s}: ERROR {r['error'].get('message')}")
            else:
                pr = r["proposal"]
                payout = float(pr.get("payout", 0.0))
                print(f"  proposal {c:9s}: payout={payout:.3f} -> R={payout/1.0:.3f}x "
                      f"(dur={pr.get('display_value')}) {dt:.0f}ms")
    if rtts:
        med = float(np.median(rtts))
        verdict = "PASS" if med < 250 else "FAIL"
        print(f"[{verdict}] median RTT={med:.1f}ms  p95={np.percentile(rtts,95):.1f}ms  n={len(rtts)}")
        return med < 250
    return False


async def run_stream(app_id, token, symbol, cfg_path, minutes, execute_orders):
    import websockets
    guards = FableAdaptiveGuards(cfg_path)
    matcher = LiveMatcher()
    print(f"[*] matcher={matcher.mode}  parity={ {k: round(v,3) for k,v in matcher.parity.items() if isinstance(v,
(int,float))} }")
    url = f"wss://ws.derivws.com/websockets/v3?app_id={app_id}"
    ticks = deque(maxlen=SIGMA_LOOKBACK + SETTLE_TICKS + 5)
    pending = []
    balance = 50.0
    stop_at = time.time() + minutes * 60
    n_eval = n_trig = n_settled = 0

    async with websockets.connect(url, ping_interval=20, max_size=2**22) as ws:
        if token:
            await ws_request(ws, {"authorize": token})
        r = await ws_request(ws, {"ticks": symbol, "subscribe": 1})
        if "error" in r:
            print(f"[!] subscribe failed: {r['error']}")
            return
        print(f"[+] subscribed {symbol}; buffer warmup {SIGMA_LOOKBACK} ticks")
        while time.time() < stop_at:
            msg = json.loads(await asyncio.wait_for(ws.recv(), timeout=60))
            if "tick" not in msg:
                continue
            q = float(msg["tick"]["quote"])
            ticks.append(q)
            for p in list(pending):
                p["age"] += 1
                if p["age"] >= SETTLE_TICKS:
                    pending.remove(p)
                    if p["contract"] in ("CALL", "PUT"):
                        won = (q > p["spot"]) if p["contract"] == "CALL" else (q < p["spot"])
                    else:
                        won = abs(q - p["spot"]) >= p["barrier_pts"]
                    pnl = p["stake"] * (p["live_payout"] - 1.0) if won else -p["stake"]
                    balance += pnl
                    guards.record({"ts_utc": p["ts"], "symbol": symbol,
                                   "anchor_cell_id": p["anchor"], "contract": p["contract"],
                                   "sigma_bps": p["sigma"], "sim": p["sim"],
                                   "live_payout": p["live_payout"], "rtt_ms": p["rtt"],
                                   "stake": p["stake"], "won": int(won), "pnl": pnl,
```

```
                                    "settle_spot": q, "note": p["note"]})
                    n_settled += 1
                    print(f"  [SETTLE] {p['contract']:9s} anchor={p['anchor']:6d} "
                          f"{'WIN ' if won else 'LOSS'} pnl={pnl:+.2f} bal={balance:.2f}")
            if len(ticks) < 76:
                continue
            n_eval += 1
            if n_eval % 8:
                continue
            window = np.fromiter(list(ticks)[-75:], dtype=np.float64, count=75)
            sigma = winsorized_sigma_bps(np.fromiter(list(ticks)[-SIGMA_LOOKBACK:],
                                                     dtype=np.float64,
                                                     count=min(len(ticks), SIGMA_LOOKBACK)))
            if not np.isfinite(sigma):
                continue
            v = matcher.vector(window)
            m = matcher.match(v)
            if m is None:
                continue
            anchor, sim, polarity, action = m
            want = "CALL" if action.upper().startswith("BUY") else "PUT"
            t0 = time.perf_counter()
            pr = await ws_request(ws, proposal_payload(want, symbol, 1.0, q))
            rtt = (time.perf_counter() - t0) * 1000.0
            if "error" in pr:
                continue
            live_R = float(pr["proposal"].get("payout", 0.0)) / 1.0
            ok, reason = guards.check_all(sigma, rtt, live_R)
            if not ok:
                print(f"  [BLOCK] anchor={anchor} sim={sim:.3f} R={live_R:.3f} -> {reason}")
                continue
            n_trig += 1
            stake = guards.stake(balance, guards.live_hits / max(1, guards.live_exp) if guards.live_exp else 0.5, live_R)
            pending.append({"age": 0, "spot": q, "stake": stake if execute_orders else 0.0,
                            "anchor": anchor, "contract": want, "sigma": sigma, "sim": sim,
                            "live_payout": live_R, "rtt": rtt, "note": action,
                            "barrier_pts": q * 0.00015, "ts": datetime.now(timezone.utc).isoformat()})
            print(f"  [TRIGGER] {want} anchor={anchor} sim={sim:.3f} R={live_R:.3f}x "
                  f"sigma={sigma:.3f}bps rtt={rtt:.0f}ms stake={stake:.2f}")

    p = guards.session_pnl
    print(f"[+] session done: evals={n_eval} triggers={n_trig} settled={n_settled} "
          f"pnl={p:+.2f} balance={balance:.2f} db={DB_PATH}")


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--app_id", default=None)
    ap.add_argument("--api_token", default=None)
    ap.add_argument("--creds_file", default=None)
    ap.add_argument("--symbol", default="JD10")
    ap.add_argument("--ping_test", type=int, default=0)
    ap.add_argument("--minutes", type=float, default=30.0)
    ap.add_argument("--execute_demo_orders", action="store_true")
    ap.add_argument("--self_check", action="store_true")
    a = ap.parse_args()

    if a.self_check:
        g = FableAdaptiveGuards(os.path.join(P3, "phase3_live_forward_config.json"))
        mt = LiveMatcher()
        y = np.cumsum(np.random.default_rng(0).normal(0, 1, 75))
        v = mt.vector(y)
        print(f"[+] self-check OK: matcher={mt.mode} vec_dim={v.shape[0]} "
              f"sqlite={g.db_ok} anchors={len(mt.ids)} blocked_hours={len(g.blocked_hours)}")
        return

    app_id, token = load_creds(a)
    if not app_id or not token:
        print("[!] no credentials: pass --app_id/--api_token, set DERIV_APP_ID/DERIV_API_TOKEN, "
              "or --creds_file <file containing 'API TOKEN:' and 'APP ID:'>")
        return 2
    print(f"[*] app_id={app_id} token={mask(token)}")
    if a.ping_test:
        ok = asyncio.run(ping_test(app_id, token, a.symbol, a.ping_test))
        return 0 if ok else 1
    asyncio.run(run_stream(app_id, token, a.symbol,
                           os.path.join(P3, "phase3_live_forward_config.json"),
                           a.minutes, a.execute_demo_orders))
    return 0
```

```
if __name__ == "__main__":
    sys.exit(main())
```

Read the guard classes in order: each one is a single predicate over a proposal, and each one journals its verdict. The tester's value in its current blocked state is that a week of proposal-only operation will show which guard binds first on live data.

Appendix B - data dictionary

> Every artifact the report reads, with its shape and the meaning of each column.

B.1 Cell catalog - `jump10_phase1_cell_catalog.parquet`

591,171 rows, one per cell, written by the Phase 1 merger. It is the spine of the entire project: every other artifact is keyed by `cell_index`.

Column	Column	Column
`cell_id`	`p_high`	`rmse_residual`
`cell_label`	`p_low`	`max_tick_error`
`tick_start_idx`	`p_close`	`norm_rmse`
`tick_end_idx`	`p_mean`	`source_file`
`time_open_ms`	`p_std`	`price_source`
`time_close_ms`	`y_open_norm`	
`p_open`	`bridge_slope`	

B.2 Strategy ev leaderboard - `phase3_strategy_ev_leaderboard.parquet`

398 rows, one per pruned anchor. This is the object the Phase 3 auction operates on; the equity curve is one row per trade derived from it.

Column	Column	Column
`strategy_id`	`mode_dir_win`	`safest_contract`
`anchor_cell_id`	`mode_touch_win`	`highest_yield_contract`
`vault_win_lb95`	`mode_barrier_win`	`best_cons_ev`
`covariant_peer_count`	`mode_mult_win`	`realised_ev_pct`
`mean_0_50_similarity`	`inbound_wlb_dir`	`best_mode`
`recommended_action`	`safe_sigma_bins`	`net_mean_sigma`
`in_cal_peers`	`cons_ev_rise_fall`	`net_total_sigma`
`out_triggers`	`cons_ev_onetouch`	`win_rate`
`dir_sign`	`cons_ev_barrier`	`wilson_lb`
`limit_dip`	`cons_ev_multiplier`	`profit_factor`
`tp_sigma`	`cons_ev_notouch`	`t_stat`
`sl_sigma`	`mode_notouch_win`	`verified_profitable`

B.3 Deduplicated equity curve - `phase3_deduplicated_equity_curve.parquet`

4,078 rows, one per trade, one trade per outbound cell. The `gross_*` columns are sigma-unit outcomes before friction; the `pnl_*` / `eq_*` / `dd_*` triples are the cumulative ladders at 0.00, 0.03, 0.06 and 0.10 sigma friction.

Column	Column	Column
`strategy_id`	`sigma_bps`	`dd_gross_00`
`anchor_cell_id`	`cell_std_points`	`pnl_net_03`
`outbound_cell_id`	`cons_ev`	`eq_net_03`
`match_similarity`	`max_excursion`	`dd_net_03`
`match_polarity`	`min_excursion`	`pnl_net_06`
`trade_dir`	`notouch_win`	`eq_net_06`
`sigma_bin`	`chosen_mode`	`dd_net_06`
`utc_hour`	`chosen_contract`	`pnl_net_10`
`best_mode`	`gross_sigma`	`eq_net_10`
`best_contract`	`payout`	`dd_net_10`
`gross_dir_149`	`unit_win`	`balance_account_ruined_fixed5`
`gross_exc_sniper`	`unit_pnl`	`balance_kelly_from_50`
`gross_prot_fade`	`pnl_gross_00`	`balance_fixed5_from_50`
`fade_filled`	`eq_gross_00`	

B.4 Vault - `phase2_strategy_vault.parquet` (893 rows)

Vault columns

hub_cell_id	Cell index of the anchor (inbound, before the 50% cut).
best_peer_id / partner	Highest-similarity outbound partner.
inbound_peer_count	How many inbound cells match this anchor at |r| >= 0.85.
mean_0_50_similarity	Mean observable-half correlation across peers.
recommended_action	BUY_PROTECTED_LOW or SELL_PROTECTED_HIGH, from the inbound distribution.
out_trigger_count	Number of outbound cells that matched. Minimum 10.
outbound_50_100_lock_corr	Mean forward-half correlation between anchor and peers.
outbound_protected_win_rate	Mean of the binary protected_held flag over outbound peers.
outbound_directional_win_rate	Share of outbound peers whose Q4 return matched the action sign.
outbound_verified_edge	max(protected_win_rate, directional_win_rate) - the artefact.

B.5 Other artifacts

File	Contents
`phase3_inputs.npz`	`raw` (591171, 150) float32 midpoints; `trigger_vec` (591171, 132) float32 whitened descriptors. 667 MB.
`czet_train_cache.npz`	`Q_seq` (591171, 4, 72) quartile token sequences for CZET. 658 MB.
`jump10_phase2_subzones.npz`	Computed descriptors: `trigger_vec` (62), `dense_q1..h2` (64 raw each), `desc_Q1..Q4` (17 each), `y_half1/2` (75 each), `y_q1..q4` (38 each), `protected_held`, `q4_net_return`.
`jump10_phase1_fourier_functions.npz`	75-harmonic complex coefficients per cell. 3.4 GB. Not shipped; checksummed.
`phase3_live_bridge.pt`	MLP weights for the live matcher (128-512-512-132).
`phase3_whitening_transform.npz`	Mean/scale arrays used by the bridge.
`phase3_live_forward_config.json`	208 approved anchor ids, 14 blocked hours, sigma edges, payout floors, verdict.
`fable_forward_cal.db`	SQLite journal of live proposals (not shipped).

Appendix C - metric definitions

> Every statistic in this report, defined exactly as the code computes it, so that a reader can recompute any of them from the parquet.

C.1 Win rates

$$\text{win_rate} = (1/N) \sum 1[\text{unit_win_i}]$$

For a directional contract `unit_win = sign(gross_dir_149) == trade_dir`. For ONETOUCH `unit_win = max_excursion ≥ 0.75`. For NOTOUCH `unit_win = min_excursion > -1.0`. For BARRIER `unit_win = gross_dir_149 ≥ 0.35`.

C.2 Wilson lower bound

$$LB = (\hat{p} + z^2/2n - z\sqrt{(\hat{p}(1-\hat{p})/n + z^2/4n^2)}) \,/\, (1 + z^2/n)$$

with `z = 1.959964`. Used everywhere a hit rate is gated, because it penalises small samples automatically: an anchor with 12 hits needs a much higher point estimate to clear the same gate as one with 400.

C.3 Expected value and the gate

$$EV = p_WLB \cdot R - 1, \text{ gate: } EV > 0.02$$

where `R` is the contract payout and `p_WLB` the Wilson bound of the *inbound* hit rate for that contract family.

C.4 Profit factor

$$PF = \sum pnl_i\ [pnl_i > 0] \ / \ |\sum pnl_i\ [pnl_i < 0]|$$

Computed on the friction-tier PnL column, not on gross sigma.

C.5 Sharpe, Sortino, Calmar

$$Sharpe = mean(pnl) \ / \ std(pnl) \cdot \sqrt{N}\ Sortino = mean(pnl) \ / \ std(min(pnl, 0)) \cdot \sqrt{N}\ Calmar = total_return \ / \ |max_drawdown|$$

Per-trade series, annualised by trade count rather than by calendar time, because the corpus is not uniformly sampled in wall-clock terms.

C.6 Kelly and the stake clamp

$$f^* = (p_WLB \cdot R - 1) \ / \ (R - 1)\ stake = clip(0.25 \cdot f^* \cdot balance, 0.50, 0.01 \cdot balance)$$

`f*` is set to zero when `p_WLB · R ≤ 1`, which is why the portfolio reports a Kelly fraction of zero.

C.7 Winsorised sigma

$$\sigma = 1.4826 \cdot median(|\Delta mid| \text{ capped at } 6.984375 \cdot median(|\Delta mid|))$$

over the trailing 1,800 ticks, expressed in bps of price. The 1.4826 factor converts a median absolute deviation to a normal-consistent standard deviation; the cap is set at six median absolute tick moves so that Jump-10's discrete 10-point jumps do not dominate.

C.8 Monte Carlo

5,000 block-bootstrap runs with block length 50 trades, sampling with replacement from the realised trade sequence, re-compounding equity from zero and reporting the 95th/99th percentile worst drawdown, the median profit factor and P(final > 0).

C.9 Whitening

$$z = \Lambda^{-1/2}\, U^T\, (x - \mu)$$

with `μ` the column means, `U` the eigenvectors of the covariance and `Λ` the eigenvalues, keeping the 132 components after dropping three constant columns and removing the first (70.3% variance) component.

C.10 Anchor orthogonality

Two anchors are considered the same anomaly when `|cos(z_i, z_j)| ≥ 0.82` in the 132-d space. Suppression keeps the anchor with the higher inbound Wilson bound. The gatekeeper asserts the surviving maximum is below the threshold.

Appendix D - glossary

Term	Definition as used in this project
Cell	150 consecutive ticks. The unit of everything: one cell, one possible trade.
Quartile (Q1-Q4)	37-38 tick sub-sequences of a cell. Q1+Q2 are observable; Q3+Q4 are forward.
Observable half	Ticks 0-74. The only data available at decision time.
Forward half	Ticks 75-149. Where every outcome is measured.
Hub	An inbound cell with an unusually large number of similar peers (top 5,000 by anomaly z-score).
Covariant edge	$\|r\| \geq 0.85$ between the observable halves of two cells.
Peer count	Number of cells matching an anchor above the similarity threshold.
Trigger	An outbound cell that matches an anchor and therefore generates a candidate trade.
Lock correlation	Correlation between the forward halves of an anchor and a peer: does the future continue the same way?
Protected extremum	The forward path never breaks the far barrier; the quantity ONETOUCH/NOTOUCH monetise.
Sigma	Causal cell volatility in bps, from the winsorised MAD estimator over the trailing 1,800 ticks.
Contract auction	Choosing, per anchor, the contract family with the highest gated EV.
Friction tier	A cost level (0.00/0.03/0.06/0.10 sigma) subtracted from the realised excursion.
Deduplication	Keeping only the highest-EV candidate per outbound cell.
Selection artefact	An apparent edge produced by taking a maximum over noisy estimates rather than by the data.
Wilson bound	Lower confidence bound on a binomial rate, used as the gate input instead of the point estimate.
Live parity	Recall of the live matcher against the backtest's trigger set. Currently 0.240.
Gatekeeper	An executable checklist over shipped artefacts; 10 checks in Phase 2, 18 in Phase 3.
Fable	The name for the Phase 3 sigma-survival framework and its seven live guards.

Appendix E - artifact inventory

> SHA-256 checksums for every artifact too large to ship. A reader with the source repository and these hashes can verify that a rebuilt artifact matches the one this report describes.

File	Bytes	MB	SHA-256
`phase1\final\jump10_phase1_fourier_functions.npz`	3,570,676,618	3,405.26	`F8229AEE44A562787173FF98D1C676FE79C0D440C561854A1DFD699ED0579F71`
`phase1\final\jump10_phase1_cell_catalog.parquet`	54,740,937	52.21	`A87F86F47C20810247EAF2F437D1DD2C794D2D798DEF1D36D79508904D2137F9`
`phase1\final\jump10_phase1_inspection.xlsx`	44,883,300	42.80	`34EDEA9761CF2BDBDC255DCC762473D424AB84DC184B626CF3B9B9D82AAFE742`
`phase2\jump10_phase2_subzones.npz`	2,586,971,229	2,467.13	`29075F0184E1C33030A1F6ADD0699A704A6FE45949662675DC4603B0640BA6E3`
`phase2\jump10_phase2_subzones_raw.npz`	2,578,717,567	2,459.26	`1E8A6318BEB856F0BAD7C187A43D1944F39C73D034BA6EBCCCB66CF6DBDBF945`
`phase2\czet_train_cache.npz`	690,489,542	658.50	`274FB2A8C0F1F95158A1C3D654B2D5221AA3D4712BCD816497CF278588506984`
`phase2\jump10_phase2_scan.npz`	645,560,588	615.65	`89D845FA760422A255477DD721FC1457011AFFAECD6FBDE77F2AD0437D3F8453`
`phase3\phase3_inputs.npz`	666,841,402	635.95	`CB2AF7A0B67D3196BDC1C17F5ACE966068126166A5D44AB512A34CE6BB764344`
`phase3\_live_feat.npy`	302,679,680	288.66	`FD7AF3797ED67D863BEF2F68F9FF0F3CC79B24381879035BF8E0B0A6DCE8639A`
`phase3\phase3_catalog_times.npy`	4,729,496	4.51	`854EB55799FA9EEF9C5424E7E7684F5892B4378A530B1076C4C49034692BB984`

E.2 Shipped in the repository

Path	Role
`README.md`	Top-level findings and reproduction commands.
`docs/PHASE3_BRIEF_REDACTED.md`	The Phase 3 specification, credentials stripped.
`docs/DATA_MANIFEST.json`	The checksum table above, machine-readable.
`docs/PROJECT_DOCUMENTATION.pdf`	This report.
`tools/build_docs.py` + `docs_part2..5.py`	The generator that produced this report.
`phase1/scripts/*.py`	Corpus construction and verification.
`phase2/*.py`	Graph, vault, CZET, audits, reports, gatekeeper.
`phase3/*.py`	Engine, tester, verifier, diagnostic, bridges.
`phase3/*.json, *.xlsx, *.parquet`	All numbers quoted in this report.

E.3 Build provenance

This document was generated on 2026-10-27 15:17 UTC from the working tree at commit `b71de45` (Phase 3) plus the uncommitted documentation files. It contains 26 top-level sections and is rendered from HTML by Chrome headless print-to-PDF.